
Docker e Containers

Do zero absoluto ao estado da arte

Curso completo: o que são containers, como usar, como funcionam por dentro, quando não usar, a economia do ecossistema e a fronteira de pesquisa. Versões de referência: Docker Engine 29.7.1 · Compose v2 · containerd 2.x.

Sumário

Mapa do assunto	xv
O que é este material	xv
O que você saberá ao final	xv
Roteiro de leitura	xvi
Arquivos	xvi
Status por bloco	xviii
Aviso de validade	xviii
Autoteste do mapa	xviii
I Porta de entrada	1
1 Docker e containers para quem nunca ouviu falar	3
1.1 A pergunta que dá origem a tudo	3
1.2 A analogia do contêiner de navio	3
1.3 Então o que é, tecnicamente, um container?	4
1.4 E o Docker, o que é?	4
1.5 Os três substantivos que você precisa separar	5
1.6 Container não é máquina virtual	5
1.7 Por que isso pegou tanto?	6
1.8 E onde isso não serve?	6
1.9 O vocabulário mínimo, tudo definido	7
1.10 O que fazer agora	7
1.11 Autoteste	7
2 Pré-requisitos	9
2.1 Conhecimento	9
2.2 Ambiente	10
2.3 Decisão que você precisa tomar antes de instalar	11
2.4 Tempo realista de estudo	11
2.5 Rota de resgate — falta um pré-requisito, e agora?	12
2.6 Checklist antes de seguir	12
2.7 Autoteste	12

3	Manual de instalação	15
3.1	Leia isto antes de qualquer coisa	15
3.2	Alternativa sem instalar nada	16
3.3	Linux — família Debian/Ubuntu	16
3.4	Linux — família Fedora/RHEL/Rocky	19
3.5	Linux — pós-instalação obrigatória	19
3.6	macOS — Intel e Apple Silicon	21
3.7	Windows — WSL2 (recomendado) e nativo	22
3.8	Git em todos os sistemas	24
3.9	Editor e extensões	24
3.10	Conta no Docker Hub e login	25
3.11	PATH e variáveis de ambiente	25
3.12	Permissões e o problema do sudo	26
3.13	Rede corporativa: proxy, certificado e registry espelhado	27
3.14	Convivência de versões na mesma máquina	28
3.15	Reprodutibilidade	29
3.16	Atualizar e voltar atrás	29
3.17	Desinstalar por completo	30
3.18	Solução de problemas — erros literais	31
3.19	Checklist “ambiente pronto”	32
3.20	Autoteste	33
4	Como começar — do ambiente pronto ao primeiro resultado	35
4.1	Passo 1 — Rodar algo de verdade em 10 segundos	35
4.2	Passo 2 — O ciclo de trabalho do dia a dia	36
4.3	Passo 3 — Construir a sua primeira imagem	37
4.4	Passo 4 — Dados que sobrevivem: volumes	38
4.5	Passo 5 — Vários containers juntos: Compose	40
4.6	Os cinco primeiros erros de uso (não de instalação)	41
4.7	Limpeza ao terminar	42
4.8	Para onde ir agora	42
4.9	Autoteste	42
5	Manual de uso — referência consultável	45
5.1	Índice	45
5.2	Rodar um container	45
5.3	Inspecionar e diagnosticar	47
5.4	Entrar e depurar	48
5.5	Imagens	48

5.6	Construir imagens	49
5.7	Instruções do Dockerfile	50
5.8	Volumes e dados	51
5.9	Redes	52
5.10	Registries	53
5.11	Compose	53
5.12	Limites de recurso	55
5.13	Limpeza e espaço em disco	55
5.14	Contexts e daemon remoto	56
5.15	Formatação de saída (Go templates)	56
5.16	Obsoleto — o que não usar mais	56
5.17	Atalhos que só quem usa há anos conhece	57
5.18	Autoteste	58
6	Exemplos — 14 receitas completas e executáveis	59
6.1	Banco de dados descartável para testar uma consulta	59
6.2	Rodar uma ferramenta sem instalá-la	60
6.3	Servir um site estático	60
6.4	Dockerfile Node com multi-stage e cache eficiente	61
6.5	Python com uv e imagem enxuta	63
6.6	Go: imagem de 8 MB com scratch	63
6.7	Ambiente de desenvolvimento com recarga automática	64
6.8	Stack completa: API + Postgres + Redis + nginx	65
6.9	Migração de banco antes de subir a API	67
6.10	Segredo no build sem vaziar na imagem	68
6.11	Build multi-arquitetura (amd64 + arm64)	69
6.12	PRODUÇÃO — pipeline de CI completo no GitHub Actions	70
6.13	PRODUÇÃO — backup automatizado de Postgres com retenção	72
6.14	PRODUÇÃO — proxy reverso com TLS automático para vários serviços	73
6.15	Autoteste	75
7	Projeto-modelo — Mural de Recados	77
7.1	O que este projeto ensina	77
7.2	Pré-requisitos	77
7.3	Como rodar — comandos exatos	78
7.4	Estrutura de pastas	78
7.5	O que cada decisão de projeto ensina	79
7.6	Laboratórios com este projeto	80
7.7	Próximos passos — como evoluir este projeto	82

7.8	Verificação feita neste material	83
7.9	Autoteste	83
II	Núcleo	85
8	Fundamentos	87
8.1	A definição formal de container	87
8.2	O modelo mental correto	87
8.3	Os quatro substantivos e como eles se relacionam	88
8.4	Imagem: pilha de camadas endereçadas por conteúdo	88
8.5	Namespaces: o isolamento de visão	89
8.6	cgroups: a limitação de consumo	90
8.7	Capabilities, seccomp e LSM: a redução de privilégio	90
8.8	A arquitetura do Docker: quem faz o quê	91
8.9	OCI: o padrão que impede o aprisionamento	92
8.10	As seis coisas que um container não é	92
8.11	O ciclo de vida completo de um container	92
8.12	Os cinco modelos mentais que valem mais que qualquer comando	93
8.13	Os cinco porquês aplicados: por que containers iniciam tão rápido?	93
8.14	Vocabulário consolidado	94
8.15	Autoteste	94
9	História	97
9.1	Linha do tempo em uma tela	97
9.2	Ato 1 — A pré-história (1979-2012): a ideia existia, faltava a embalagem	97
9.3	Ato 2 — 2013: o Docker aparece	98
9.4	Ato 3 — 2014-2017: orquestração e a guerra dos padrões	99
9.5	Ato 4 — 2018-2022: comoditização, dessacralização e o susto	100
9.6	Ato 5 — 2021-2026: o modelo de negócio e a maturidade	101
9.7	O que a história ensina, em quatro lições	102
9.8	Autoteste	102
10	Imagens e camadas — o formato por dentro	105
10.1	O que uma imagem é, literalmente	105
10.2	Endereçamento por conteúdo: por que o digest é imutável e a tag não	106
10.3	Como as camadas viram um sistema de arquivos: OverlayFS	106
10.4	O cache de build: a regra e as cinco consequências	107
10.5	Por que sua imagem está gorda — e como emagrecer	109
10.6	Segredos vazam em camadas — a demonstração	110

10.7	Imagens multi-arquitetura	111
10.8	O armazenamento de imagens: overlay2 e o containerd image store . . .	111
10.9	Metadados e proveniência	112
10.10	Autoteste	112
11	Isolamento	115
11.1	Os sete namespaces, um a um	115
11.2	Construindo um container à mão, sem Docker	117
11.3	cgroups v2 na prática	118
11.4	Capabilities: o que o container pode fazer	119
11.5	seccomp	120
11.6	Rootless: o modelo mais seguro	120
11.7	O que o isolamento NÃO protege	121
11.8	Depuração de isolamento — a caixa de ferramentas	121
11.9	Autoteste	122
12	Runtime e arquitetura	123
12.1	As camadas, e por que existem tantas	123
12.2	O que acontece, passo a passo, num <code>docker run -d -p 8080:80 nginx:alpine</code>	124
12.3	O <code>config.json</code> da OCI — o contrato	125
12.4	Os runtimes de baixo nível, comparados	125
12.5	BuildKit: por que o build ficou outro	126
12.6	Drivers plugáveis: log, storage, rede	127
12.7	A API do daemon	127
12.8	Docker vs. containerd vs. Podman vs. CRI-O	128
12.9	Onde as coisas ficam no disco	129
12.10	Autoteste	129
13	Armazenamento e volumes — onde os dados realmente vivem	131
13.1	As quatro formas de armazenamento, comparadas	131
13.2	Volumes nomeados	132
13.3	Bind mounts e o problema de permissão	133
13.4	Desempenho: onde o I/O dói	134
13.5	Backup, restauração e migração	135
13.6	Volume drivers e armazenamento em rede	136
13.7	Diagnóstico — quando o disco enche	136
13.8	Recuperação de desastre — os três casos que acontecem	137
13.9	Autoteste	137

14 Redes — como containers falam entre si e com o mundo	139
14.1 O que o Docker cria por baixo	139
14.2 Os drivers de rede	139
14.3 DNS interno: a regra que resolve tudo	141
14.4 Publicação de portas — como funciona de verdade	141
14.5 Como um container alcança o host	142
14.6 Segmentação de rede como camada de segurança	142
14.7 Redes overlay (multi-host)	143
14.8 Depuração de rede — o procedimento	144
14.9 Configurações de rede no daemon	144
14.10 Boas práticas, condensadas	145
14.11 Autoteste	145
15 Dockerfile e build — do texto à imagem	147
15.1 A anatomia de um Dockerfile de produção	147
15.2 As armadilhas de escrita, uma a uma	147
15.3 O cache, na prática	150
15.4 .dockerignore — o arquivo que quase ninguém escreve direito	151
15.5 Multi-stage: cinco padrões úteis	151
15.6 Escolha da imagem base — o quadro de decisão	153
15.7 Reprodutibilidade	153
15.8 Linting e verificação	153
15.9 Erros de build e o que significam	154
15.10 Checklist de Dockerfile de produção	154
15.11 Autoteste	155
16 Compose e aplicações multi-container	157
16.1 O que o Compose faz por você	157
16.2 Compose v1 × v2, e o version: que morreu	157
16.3 Anatomia de um compose.yaml completo	158
16.4 depends_on — as três condições	159
16.5 Variáveis, .env e interpolação	159
16.6 Sobreposição de arquivos: base + ambiente	160
16.7 Perfis	161
16.8 docker compose watch — desenvolvimento sem bind mount de tudo	162
16.9 Escala e as suas limitações	162
16.10 Compose em produção — quando faz sentido	163
16.11 Segredos no Compose	164
16.12 Comandos e diagnóstico	164
16.13 Autoteste	164

17 Registries, distribuição e cadeia de suprimentos	167
17.1 Como um registry funciona	167
17.2 Os registries, comparados	167
17.3 Autenticação	168
17.4 Estratégia de tags	169
17.5 Cadeia de suprimentos: o assunto de 2026	170
17.6 Registry próprio	171
17.7 Transferir imagens sem registry	172
17.8 Boas práticas de distribuição	173
17.9 Autoteste	173
18 Segurança de containers	175
18.1 O modelo de ameaça, explicitado	175
18.2 As dez medidas, ordenadas por custo-benefício	175
18.3 A demonstração que convence: escape com socket montado	177
18.4 As configurações que anulam o isolamento	178
18.5 Ferramentas de auditoria	178
18.6 Isolamento reforçado, quando o padrão não basta	179
18.7 Rootless como padrão de servidor	179
18.8 Segredos em tempo de execução	180
18.9 Endurecimento do host	180
18.10 Resposta a incidentes	181
18.11 Checklist de produção	181
18.12 Autoteste	182
19 Observabilidade e operação	185
19.1 Logs	185
19.2 Métricas	187
19.3 Healthchecks	188
19.4 Encerramento gracioso e deploy sem queda	189
19.5 Diagnóstico de produção — o procedimento	189
19.6 Atualização de containers	190
19.7 Backup operacional	191
19.8 Uma stack de observabilidade completa	192
19.9 Rotina operacional	193
19.10 Autoteste	193

20 Orquestração — quando um servidor não basta	195
20.1 Os problemas que a orquestração resolve	195
20.2 As opções, honestamente comparadas	195
20.3 Swarm em 20 minutos	196
20.4 Kubernetes — o mínimo para atravessar a ponte	197
20.5 O caminho de migração, na ordem que funciona	199
20.6 Sinais de que chegou a hora (e de que não chegou)	199
20.7 Alternativas que dispensam orquestrador	200
20.8 Autoteste	200
21 Teoria avançada — limites, provas e resultados formais	201
21.1 Agendamento de containers é NP-difícil	201
21.2 Isolamento de desempenho: o resultado negativo	202
21.3 O throttling de CFS: um problema quantificável	202
21.4 Limites de isolamento de segurança	203
21.5 Custo teórico do copy-on-write	204
21.6 Deduplicação: o ganho é combinatório	204
21.7 Tempo de partida: a análise	205
21.8 O modelo formal da OCI Runtime Spec	205
21.9 Problemas em aberto	206
21.10 Leituras primárias	206
21.11 Autoteste	207
22 Estado da arte — agosto de 2026	209
22.1 O panorama em uma tabela	209
22.2 O que a Docker Engine 29 mudou	209
22.3 WebAssembly: a aposta que amadureceu parcialmente	210
22.4 Cadeia de suprimentos: de boa prática a obrigação legal	210
22.5 Isolamento: o retorno da máquina virtual	211
22.6 IA mudou os requisitos de container	211
22.7 eBPF: a camada que está reescrevendo a rede e a observabilidade	212
22.8 Debates abertos, com os dois lados	213
22.9 Previsões para 2027-2028 — e o que as invalidaria	213
22.10 O que não mudou, e provavelmente não vai mudar	214
22.11 Autoteste	214

III Prática e erros	217
23 Prática — laboratórios progressivos	219
23.1 Lab 1 — Sobrevivência (30 min)	219
23.2 Lab 2 — A primeira imagem sua (45 min)	219
23.3 Lab 3 — Persistência (45 min)	220
23.4 Lab 4 — Multi-stage e o tamanho da imagem (1 h)	221
23.5 Lab 5 — Compose com dependências (1 h)	221
23.6 Lab 6 — Depuração de rede (Linux, 1 h)	221
23.7 Lab 7 — Segurança: endurecer um container (1 h)	222
23.8 Lab 8 — Segredo que não vaza (Linux, 45 min)	223
23.9 Lab 9 — Limites e OOM (Linux, 45 min)	223
23.10 Lab 10 — Do zero à produção (projeto integrador, 3 h)	224
23.11 Autoavaliação final	224
24 Armadilhas, mitos e más práticas	227
24.1 Parte 1 — Os oito erros que travam o iniciante	227
24.2 Parte 2 — Os mitos, e a realidade	228
24.3 Parte 3 — Más práticas que persistem (e por quê)	229
24.4 Parte 4 — Armadilhas de plataforma	230
24.5 Parte 5 — Referência rápida de diagnóstico	230
24.6 Parte 6 — O anti-checklist	231
24.7 Autoteste	231
IV Economia e ecossistema	233
25 Custos e licenças	235
25.1 A resposta curta	235
25.2 Docker Engine — o que é grátis	235
25.3 Docker Desktop — onde entra a cobrança	235
25.4 Docker Hub — armazenamento e o limite de pull	237
25.5 Registries de nuvem — o custo por GB e por egress	237
25.6 O custo de rodar containers na nuvem	238
25.7 Imagens base endurecidas — o custo novo de 2026	238
25.8 O custo que ninguém coloca na planilha	239
25.9 Cenários de custo, do zero ao corporativo	239
25.10 Como não gastar à toa	240
25.11 Autoteste	240

26 Cursos gratuitos e certificações	241
26.1 Português (Brasil e Portugal)	241
26.2 Inglês	242
26.3 Francês	242
26.4 Certificações — o mapa honesto	243
26.5 Trilha de estudo sugerida, do zero ao empregável	244
26.6 Certificadores gratuitos — existe algo de valor real?	245
26.7 Autoteste	245
 V Fontes	 247
27 Bibliografia comentada	249
27.1 Para aprender Docker	249
27.2 Para ir além do Docker (containers e Kubernetes)	250
27.3 Fundamentos de sistemas (o “por baixo” dos containers)	250
27.4 Para arquitetura e produção	251
27.5 O que é legalmente gratuito	252
27.6 Recomendação por perfil	252
27.7 O que não recomendo	252
27.8 Autoteste	252
 28 Referências — specs, docs, código, pessoas	 255
28.1 Especificações (as fontes de verdade)	255
28.2 Documentação oficial	255
28.3 Código-fonte (para quem quer o fundo do poço)	255
28.4 Ferramentas mencionadas neste material	256
28.5 Artigos acadêmicos (fundamentos e teoria)	256
28.6 Páginas de manual do Linux (fonte primária, gratuita)	257
28.7 Blogs e fontes secundárias (marcados como tais)	257
28.8 Pessoas a seguir	257
28.9 Comunidades	258
28.10 Como este material foi verificado	258
28.11 Autoteste	258
 29 Glossário — Docker e containers	 261

A	Código-fonte do projeto modelo	267
A.1	07-projeto-modelo/.dockerignore	267
A.2	07-projeto-modelo/.env.example	268
A.3	07-projeto-modelo/app/package.json	268
A.4	07-projeto-modelo/app/src/config.js	268
A.5	07-projeto-modelo/app/src/log.js	269
A.6	07-projeto-modelo/app/src/repositorio.js	270
A.7	07-projeto-modelo/app/src/rotas.js	272
A.8	07-projeto-modelo/app/src/server.js	274
A.9	07-projeto-modelo/app/test/repositorio.test.js	275
A.10	07-projeto-modelo/app/test/rotas.test.js	277
A.11	07-projeto-modelo/compose.yaml	279
A.12	07-projeto-modelo/Dockerfile	281
A.13	07-projeto-modelo/Makefile	282
A.14	07-projeto-modelo/nginx/default.conf	283

Mapa do assunto

Nível: do zero absoluto ao de pesquisa · Última atualização: 11/08/2026 · Versões de referência: Docker Engine 29.7.1 · Compose v2 · containerd 2.x

O que é este material

Um curso completo sobre **Docker e containers**: o que são, como usar, como funcionam por dentro, quando **não** usar, a economia do ecossistema e a fronteira de pesquisa em agosto de 2026.

Ele responde, na ordem em que as perguntas aparecem na vida real:

1. **O que é isso e por que existe?** → [01](#) (cap. 1) e [11](#) (cap. 9)
2. **Como começo hoje, sem quebrar nada?** → [02](#) (cap. 2) a [07](#) (cap. 7)
3. **Como funciona por dentro, e onde estão os limites?** → Bloco B
4. **Como opero isso em produção sem me arrepender?** → [20](#) (cap. 18), [21](#) (cap. 19)
5. **Quanto custa e onde estudo mais?** → Blocos D e E

Container **não** é máquina virtual, **não** exige microserviços e **não** é seguro por padrão. Este material insiste nesses três pontos porque são a origem da maior parte dos erros.

O que você saberá ao final

- Explicar a um leigo o que é um container e por que "na minha máquina funciona" deixou de ser problema.
- Instalar todo o ferramental, em qualquer SO, e sair do zero a uma aplicação rodando.
- Escrever um Dockerfile de produção: multi-stage, não-root, cache eficiente, imagem enxuta, sinais tratados, healthcheck honesto.
- Operar aplicações multi-container com Compose: redes, volumes, dependências, segredos.
- Explicar namespaces, cgroups e OverlayFS a ponto de depurar sem o docker.
- Distribuir imagens com segurança: registries, digest, SBOM, assinatura, escaneamento.
- Endurecer containers e saber quando o isolamento padrão **não** basta (micro-VM, gVisor).
- Observar e operar: logs, métricas, throttling, encerramento gracioso, backup testado.
- Decidir com fundamento quando (e quando não) partir para orquestração.
- Estimar custo de verdade — licença, pulls, egress, e o custo de gente.

Roteiro de leitura

Caminho rápido (um fim de semana, "quero entender e mexer")

01 → 02 → 03 → 04 → 06 → 07-projeto-modelo/ → 75

Caminho do desenvolvedor

01 → 03 → 04 → 10 → 12 → 15 → 16 → 17 → 18 → 07-projeto-modelo/ → 70 → 75

Caminho de quem vai operar em produção

01 → 10 → 17 → 18 → 19 → 20 → 21 → 25 → 70 → 75

Caminho de arquiteto / pesquisador

todo o Bloco B em ordem, com peso em 13 → 14 → 60 → 65, depois 95

Caminho de quem decide (compra, arquitetura, contratação)

01 → 11 → 25 → 80 → 65 → 75

Arquivos

BLOCO A · Porta de entrada (01-09)

Arquivo	Nível	Conteúdo
cap. 1	iniciante	O que é, para que serve, por que existe. Zero jargão.
cap. 2	iniciante	O que saber e ter antes. Tempo realista. Rota de resgate.
cap. 3	iniciante	Manual de campo: Engine/Desktop, Compose, Git por SO. Proxy, PATH, desinstalar, erros.
cap. 4	iniciante	Do ambiente pronto à primeira imagem, volume e Compose rodando.
cap. 5	intermediário	Referência consultável de comandos, flags e padrões.
cap. 6	intermediário	14 receitas completas, do trivial ao CI/CD e proxy com TLS.
cap. 7	intermediário	App completo (mural de recados) que roda de verdade, com 10 laboratórios.

BLOCO B · Núcleo (10-69)

Arquivo	Nível	Conteúdo
cap. 8	iniciante→interm.	Definição formal, os quatro substantivos, cinco modelos mentais, os cinco porquês.
cap. 9	iniciante→interm.	1979→2026: chroot, cgroups, Docker, a guerra dos orquestradores, o dockershim.
cap. 10	interm.→avançado	Formato OCI, camadas, cache de build, tamanho, vazamento de segredo.
cap. 11	avançado	Container à mão sem Docker; os 7 namespaces, cgroups v2, capabilities, seccomp.

Arquivo	Nível	Conteúdo
cap. 12	avançado	CLI→dockerd→containerd→shim→runc→processo. OCI config, BuildKit, drivers.
cap. 13	interm.→avançado	Volumes, bind mounts, permissões (UID/GID, SELinux), backup, desempenho.
cap. 14	interm.→avançado	Bridge, host, macvlan, DNS interno, publicação de porta, o firewall furado.
cap. 15	interm.→avançado	Cada instrução, cache, multi-stage, escolha de base, linting, erros de build.
cap. 16	intermediário	depends_on, variáveis, sobreposição, perfis, watch, escala, Compose em produção.
cap. 17	interm.→avançado	Registries, tags vs digest, cadeia de suprimentos (SBOM, assinatura), registry próprio.
cap. 18	avançado	Modelo de ameaça, as 10 medidas, o que anula o isolamento, incidentes.
cap. 19	interm.→avançado	Logs, métricas, healthchecks, encerramento gracioso, diagnóstico, backup.
cap. 20	interm.→avançado	Quando um servidor não basta. Swarm, Kubernetes, a ponte, quando não ir.
cap. 21	pesquisa	NP-dificuldade do agendamento, isolamento de desempenho, throttling, provas.
cap. 22	pesquisa	Agosto/2026: Engine 29, Wasm, cadeia de suprimentos legal, IA, eBPF.

BLOCO C · Prática e erros (70-79)

Arquivo	Nível	Conteúdo
cap. 23	todos	10 laboratórios progressivos com critério de aprovação.
cap. 24	todos	Os 8 erros de iniciante, os mitos, as más práticas e por que persistem.

BLOCO D · Economia e ecossistema (80-89)

Arquivo	Nível	Conteúdo
cap. 25	todos	Engine grátis, Desktop pago, limites do Hub, egress, o custo de gente.
cap. 26	todos	Cursos grátis PT/EN/FR e o mapa das certificações (DCA, KCNA, CKA).

BLOCO E · Fontes (90-99)

Arquivo	Nível	Conteúdo
cap. 27	todos	Livros com edição, nível e o que envelheceu.
cap. 28	todos	Specs OCI, docs, código-fonte, papers, pessoas, como o material foi verificado.
cap. 29	todos	~90 termos definidos.

Status por bloco

Bloco	Status	Observação
A · Porta de entrada	✓	6 documentos + projeto-modelo executável (testes rodam e passam)
B · Núcleo	✓	15 documentos, fundamentos → arquitetura interna → teoria → estado da arte
C · Prática e erros	✓	10 laboratórios + catálogo de armadilhas
D · Economia	✓	Preços e cursos consultados em 11/08/2026
E · Fontes	✓	Bibliografia, referências e glossário

Legenda: ✓ completo · ● parcial · □ pendente

Aviso de validade

Docker e o ecossistema de containers mudam rápido. Este material foi escrito sobre:

- **Docker Engine:** 29.7.1 (04/08/2026) · **Compose:** v2 · **containerd:** série 2.x
- **Data das consultas de preço, versão e curso:** 11/08/2026
- **Câmbio para ordens de grandeza:** US\$ 1 ≈ R\$ 5,40

O que envelhece mais rápido, em ordem: [65-estado-da-arte](#) (cap. 22) e [80-custos-e-licencas](#) (cap. 25) (meses), [03-instalacao](#) (cap. 3) e [85-cursos-e-certificacoes](#) (cap. 26) (releases / ~1 ano). O núcleo conceitual ([10](#) (cap. 8), [13](#) (cap. 11), [14](#) (cap. 12), [60](#) (cap. 21)) envelhece em década — porque namespaces, cgroups e o formato OCI são estáveis.

Nota de verificação: o projeto-modelo teve sua suíte de testes **executada e aprovada** (22 testes, Node v24.18.0) e a aplicação foi rodada fora de container; o docker build/compose up não puderam ser executados no ambiente de escrita (sem socket do daemon). Detalhes no [README do projeto](#) (cap. 7) e em [cap. 28](#).

Autoteste do mapa

1. Container é uma máquina virtual? Qual é a única diferença estrutural, e o que decorre dela?
2. Qual arquivo você leria primeiro para justificar (ou desaconselhar) adotar Kubernetes?
3. Qual bloco você consulta se seu container está reiniciando em laço em produção?
4. Onde estão os limites teóricos (o que **não** dá para fazer com containers)?
5. Por que o núcleo conceitual envelhece em década e o estado da arte em meses?

PARTE I

Porta de entrada

O que é, o que ter antes, como instalar e como começar

Docker e containers para quem nunca ouviu falar

Nível: iniciante · Sem jargão · Última atualização: 11/08/2026

1.1 A pergunta que dá origem a tudo

Um programa nunca roda sozinho. Ele depende de outras coisas: uma versão específica de uma linguagem, um punhado de bibliotecas, arquivos de configuração, variáveis de ambiente, um banco de dados do outro lado da rede. Esse conjunto de dependências é invisível quando tudo funciona e é a origem de quase todo sofrimento quando não funciona.

A frase que resume o problema é conhecida e antiga:

“Na minha máquina funciona.”

Ela não é preguiça nem má-fé. É uma constatação verdadeira: o programa realmente funciona na máquina de quem escreveu — porque *aquela* máquina tem exatamente as versões, os caminhos e as configurações de que ele precisa. Em outra máquina, uma dessas coisas é diferente, e o programa quebra.

Docker existe para resolver isso de um jeito específico: **em vez de tentar deixar todas as máquinas iguais, você empacota o programa junto com tudo de que ele precisa e leva o pacote inteiro para onde quiser.**

1.2 A analogia do contêiner de navio

Antes dos anos 1950, carregar um navio era um pesadelo artesanal. Cada mercadoria tinha formato próprio: sacas de café, barris, engradados, caixas soltas. Estivadores empilhavam tudo à mão. Carregar e descarregar um navio levava dias, quebrava carga, e cada porto tinha seu jeito.

Aí veio Malcom McLean com uma ideia entediante e revolucionária: **uma caixa metálica de tamanho padronizado**. Não interessa o que tem dentro — café, geladeiras, parafusos. Por fora, todo contêiner é igual. E porque é igual por fora, o guindaste do porto de Santos é igual ao de Roterdã, o encaixe do caminhão é igual ao do vagão de trem, e o navio é projetado para encaixar contêineres, não para acomodar sacas.

O ganho não veio da caixa. Veio da **interface padronizada** entre a carga e o mundo.

Um *container* de software é a mesma ideia:

Mundo físico	Mundo do software
A carga (café, geladeira)	Sua aplicação e suas bibliotecas
A caixa metálica padronizada	A imagem do container
O guindaste, o caminhão, o navio	Docker, Kubernetes, o servidor na nuvem
“Cabe em qualquer porto”	“Roda em qualquer máquina com um runtime de container”

E o mesmo alerta vale: **um contêiner não protege a carga de tudo**. Se o navio afundar, ele afunda junto. Voltaremos a isso quando falarmos de segurança e isolamento.

1.3 Então o que é, tecnicamente, um container?

Um container é **um processo comum do sistema operacional que foi enganado**.

Não há mágica, não há máquina virtual, não há emulação. Quando você roda um container, o sistema operacional inicia um processo normal — como um navegador ou um editor de texto — e então mente para ele sobre três coisas:

1. **O que ele enxerga.** O processo acha que o sistema de arquivos inteiro é uma pastinha que você preparou. Acha que só existem ele e seus filhos rodando na máquina. Acha que tem uma placa de rede só dele, com IP próprio. Nada disso é verdade; é uma visão recortada.
2. **Quanto ele pode consumir.** Você diz “esse processo usa no máximo 512 MB de memória e meio núcleo de CPU”, e o sistema operacional obriga.
3. **O que ele pode fazer.** Você tira dele permissões perigosas — mudar o relógio do sistema, carregar módulos do kernel, mexer em dispositivos.

O nome técnico da primeira mentira é **namespace**; da segunda, **cgroup**; da terceira, **capabilities** e **seccomp**. Todos os três são recursos do **kernel** do Linux — o núcleo do sistema operacional, o programa que fala com o hardware e arbitra quem usa o quê.

Definição informal: um container é um processo isolado por namespaces, limitado por cgroups e restringido por políticas de segurança, executando a partir de um sistema de arquivos empacotado (a imagem).

Isso tem uma consequência que muita gente demora a internalizar:

Não existe “dentro do container”. Do lado de fora, no `htop` do servidor, o processo do seu container aparece como qualquer outro processo, com PID normal. Ele só *acha* que está num mundo separado.

1.4 E o Docker, o que é?

Docker é a **ferramenta** que torna isso usável por gente normal.

Os recursos do kernel que citei existiam desde 2006–2008. Ninguém os usava diretamente porque era terrivelmente trabalhoso: você precisava montar o sistema de arquivos à mão, configurar namespaces com chamadas de sistema, escrever regras de rede. Era possível, mas era artesanal.

Em 2013, o Docker apareceu com três coisas juntas — e é a **combinação** que mudou o mundo, não cada peça isolada:

1. **Um formato de empacotamento com camadas** — a *imagem*, que você constrói uma vez e distribui.
2. **Um jeito de descrever a construção em texto** — o *Dockerfile*, versionável no Git ao lado do código.

3. Um lugar público para guardar e baixar imagens — o *registry* (Docker Hub), que fez `docker run nginx` funcionar sem você saber nada de `nginx`.

O comando ficou tão simples que virou trivial:

```
docker run -d -p 8080:80 nginx
```

bash

Uma linha, e você tem um servidor web funcionando em `http://localhost:8080`. Sem instalar `nginx`, sem configurar nada, sem sujar sua máquina. E quando terminar, você apaga o container e não sobra rastro.

1.5 Os três substantivos que você precisa separar

Este é o ponto onde quase todo iniciante se enrola. Preste atenção porque tudo depende disso.

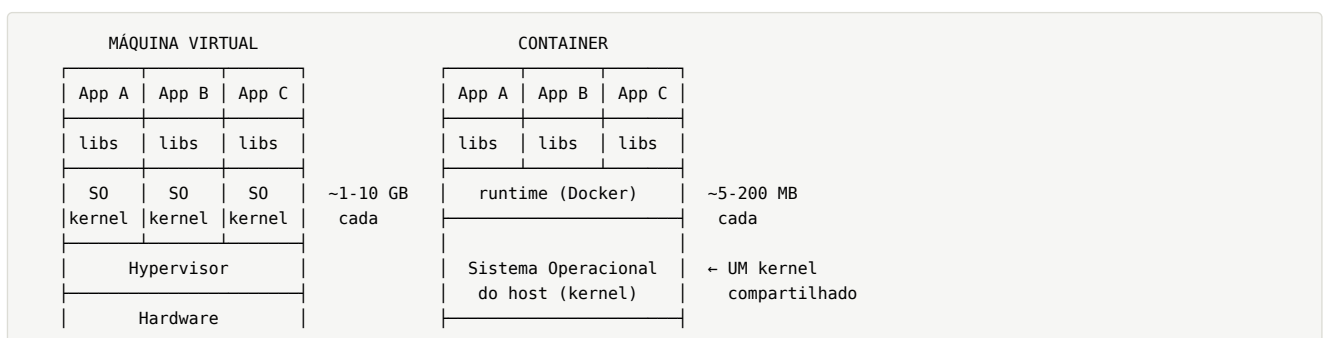
Termo	O que é	Analogia de programação	Analogia do dia a dia
Dockerfile	Um arquivo de texto com a receita de como montar a imagem	Código-fonte	A receita de bolo escrita
Imagem	O pacote pronto, imutável, em camadas	Binário compilado	O bolo assado e congelado
Container	Uma instância em execução de uma imagem	Processo rodando	A fatia servida no prato

Três consequências práticas dessa separação:

- **Uma imagem gera muitos containers.** Assim como um executável gera muitos processos. Você pode rodar 50 containers da mesma imagem `nginx` ao mesmo tempo, e eles não se conhecem.
- **A imagem não muda.** Nunca. É imutável por construção. Se você precisa de uma versão nova, constrói **outra** imagem. Isso é o que torna "voltar atrás" trivial: você aponta para a imagem antiga.
- **O container é descartável.** Se você editar um arquivo dentro de um container e depois apagar o container, a edição some. Isso é intencional, e é a causa do erro nº 1 de iniciante: perder dados de banco. (Solução: *volumes* — veja [cap. 13.](#))

1.6 Container não é máquina virtual

Essa é a comparação mais útil que existe, então vale um diagrama.



	Hardware
Inicia em: minutos Isola em: hardware virtual	Inicia em: milissegundos Isola em: recursos do kernel

A diferença essencial está numa linha só: **a VM traz seu próprio kernel; o container usa o kernel do host.**

Disso decorre tudo:

	Máquina virtual	Container
Tempo de partida	segundos a minutos	milissegundos
Tamanho típico	1-10 GB	5-500 MB
Quantos cabem numa máquina	dezenas	centenas a milhares
Isolamento	forte (barreira de hardware)	mais fraco (barreira de software)
Pode rodar outro SO?	sim (Windows sobre Linux)	não (mesmo kernel)
Uso típico	separar clientes, multi-inquilino hostil	separar aplicações da mesma organização

A troca é honesta e explícita: você ganha velocidade e densidade, e paga com isolamento mais fraco. Se um atacante escapar de um container, ele cai no host — não numa segunda barreira. Por isso provedores de nuvem que rodam código de terceiros desconhecidos (AWS Lambda, por exemplo) colocam containers **dentro** de micro-VMs. O detalhe está em [cap. 18](#).

1.7 Por que isso pegou tanto?

Três razões, em ordem de importância na minha experiência:

- 1. Acabou com a diferença entre desenvolvimento e produção.** Antes, “funciona no meu Ubuntu, quebra no RHEL do servidor” era rotina. Com container, o artefato que você testou é *bit a bit* o mesmo que roda em produção. Não é “parecido”: é o mesmo.
- 2. Fez o software virar um artefato distribuível.** `docker run postgres:16` sobe um Postgres funcionando em 10 segundos, sem ler manual. Isso transformou a economia de experimentar tecnologia. Você testa cinco bancos de dados numa tarde.
- 3. Tornou possível o resto da infraestrutura moderna.** Kubernetes, CI/CD moderno, microserviços e boa parte do que se chama de “cloud native” só existem porque há um artefato padronizado embaixo. Container é a peça de LEGO; o resto é o que se constrói com ela.

1.8 E onde isso não serve?

Honestidade profissional, porque a propaganda costuma omitir:

- **Se você tem uma aplicação só, num servidor só, e ela funciona** — containerizar adiciona complexidade e não resolve problema nenhum. Faça se quiser aprender, não porque “é o certo”.
- **Se você precisa de isolamento forte contra código hostil** — use VM ou micro-VM.

- **Se você precisa de um kernel diferente** — driver especial, versão específica, outro SO. Container não faz isso, por definição.
- **Desktop gráfico, áudio, USB, GPU exclusiva** — dá para fazer, mas dói. Frequentemente uma VM é menos sofrimento.
- **macOS e Windows** — containers Linux rodam ali dentro de uma VM escondida. Funciona bem, mas a performance de disco é notavelmente pior, e isso derruba projetos com muitos arquivos pequenos (`node_modules`, por exemplo).

Opinião profissional, não consenso: a maior parte das equipes pequenas que adotou microserviços em containers teria vivido melhor com um monólito bem feito, containerizado num único serviço. Container é ótimo; microserviço é uma decisão organizacional cara e independente.

1.9 O vocabulário mínimo, tudo definido

Termo	Definição de uma linha
Kernel	O núcleo do sistema operacional: fala com o hardware e arbitra recursos entre processos.
Processo	Um programa em execução, com seu espaço de memória.
Namespace	Recurso do kernel Linux que recorta o que um processo enxerga (arquivos, rede, outros processos).
cgroup	<i>control group</i> — recurso do kernel que limita quanto um processo pode consumir.
Imagem	Pacote imutável com o sistema de arquivos e os metadados da aplicação.
Camada (layer)	Um pedaço da imagem; imagens são pilhas de camadas reaproveitáveis.
Container	Instância em execução de uma imagem.
Dockerfile	Arquivo texto que descreve como construir uma imagem.
Registry	Servidor que armazena e distribui imagens (ex.: Docker Hub, GHCR).
Volume	Área de disco gerenciada pelo Docker que sobrevive à morte do container.
Docker Compose	Ferramenta que sobe vários containers juntos a partir de um arquivo YAML.
Orquestrador	Sistema que decide onde e quantos containers rodar num conjunto de máquinas (ex.: Kubernetes).
OCI	<i>Open Container Initiative</i> — o padrão aberto de formato de imagem e de runtime.

O glossário completo está em [cap. 29](#).

1.10 O que fazer agora

1. Se quer só experimentar sem instalar nada → há um caminho no [cap. 3](#), seção “Alternativa sem instalar nada”.
2. Se quer instalar → [cap. 2](#) e depois [cap. 3](#).
3. Se quer entender por dentro antes de mexer → [cap. 8](#).

1.11 Autoteste

1. Explique, sem usar a palavra “container”, o problema que o Docker resolve.

2. Qual é a única diferença estrutural entre uma VM e um container, da qual todas as outras decorrem?
3. Se uma imagem é imutável, como se corrige um bug numa aplicação containerizada?
4. Você edita um arquivo dentro de um container em execução e depois roda `docker rm`. O que acontece com a edição, e por quê?
5. Por que `docker run nginx` funciona mesmo numa máquina onde o `nginx` nunca foi instalado?
6. Cite duas situações em que usar container é a escolha errada, e justifique cada uma.
7. Namespace e cgroup resolvem problemas diferentes. Quais?

CAPÍTULO 2

Pré-requisitos

Nível: iniciante · Última atualização: 11/08/2026

O que você precisa **saber**, **ter** e **decidir** antes de abrir o [cap. 3](#).

2.1 Conhecimento

2.1.1 Indispensável

Pré-requisito	Por que é indispensável	Onde aprender
Usar um terminal — abrir, navegar (<code>cd</code> , <code>ls</code> , <code>pwd</code>), ler saída de erro	O Docker é essencialmente uma CLI. Sem terminal, não há Docker.	Linux Journey — Command Line · Curso em Vídeo — Linux (PT)
Caminhos de arquivo — absoluto vs. relativo, <code>.</code> , <code>..</code> , <code>~</code>	Volumes, <code>COPY</code> e <code>build context</code> são todos sobre caminhos. Erro de caminho é o erro nº 1.	Mesma fonte acima
Noção de processo — o que é um programa em execução, o que é PID	Um container é um processo. Sem esse modelo mental, tudo vira mágica.	cap. 8 cobre o necessário
Noção de porta de rede — o que é <code>localhost:8080</code> , cliente e servidor	Publicar porta (<code>-p</code>) é metade do uso cotidiano.	cap. 14 cobre o necessário
Editar um arquivo de texto e salvar	Dockerfile e Compose são arquivos de texto.	Qualquer editor: VS Code, nano, vim

2.1.2 Ajuda muito (mas dá para começar sem)

Pré-requisito	Onde isso aparece	Rota de resgate
Git	Você vai versionar Dockerfile e Compose junto do código; CI/CD parte daí	Aprenda o mínimo: <code>clone</code> , <code>add</code> , <code>commit</code> , <code>push</code>
YAML	<code>compose.yaml</code> é YAML. Indentação com espaços, nunca TAB	20 minutos em learnxinyminutes.com/yaml resolvem
Alguma linguagem de programação	Para construir sua própria imagem em vez de só usar as prontas	Não bloqueia: você pode passar semanas só rodando imagens de terceiros
Fundamentos de Linux — permissões, UID/GID, <code>/etc</code> , <code>/var</code>	O container roda Linux mesmo que seu desktop seja Windows. Problema de permissão em volume é 100% sobre UID/GID	Linux Journey · aparece explicado em cap. 13
Redes: DNS, NAT, sub-rede	Bridge networks, resolução de nome entre serviços	Explicado do zero em cap. 14
HTTP e APIs	A maioria dos exemplos sobre um serviço web	Não bloqueia

2.1.3 O que não é pré-requisito (apesar do que dizem)

- **Kubernetes.** É o contrário: containers vêm antes. Quem começa por Kubernetes aprende os dois pela metade.
- **Ser administrador de sistemas.** Docker foi feito para desenvolvedores.
- **Saber compilar kernel, escrever em Go, entender syscalls.** Isso é o Bloco B avançado, não a porta de entrada.

2.2 Ambiente

2.2.1 Hardware — mínimo real, não o do folheto

Recurso	Mínimo funcional	Confortável	Observação
CPU	2 núcleos, x86-64 ou ARM64	4+ núcleos	Build de imagem é o que mais consome
RAM	4 GB (Linux) · 8 GB (macOS/Windows)	16 GB	Em macOS/Windows a VM do Docker reserva RAM fixa
Disco livre	20 GB	60 GB+	Imagens acumulam rápido. É o recurso que acaba primeiro
Arquitetura	x86-64 (amd64) ou ARM64 (aarch64)	—	Apple Silicon é ARM64: nem toda imagem tem build ARM64
Virtualização	habilitada na BIOS/UEFI	—	Obrigatória em Windows e macOS; não é necessária em Linux

Cicatriz de campo: disco cheio por imagens e volumes órfãos é, disparado, o incidente mais comum em máquina de desenvolvedor. Reserve espaço e conheça `docker system df` antes de precisar dele às 3 da manhã.

2.2.2 Sistema operacional

SO	Situação	Recomendação
Linux (kernel ≥ 5.10 , recomendado ≥ 6.1)	Ambiente nativo. Sem VM no meio, performance total	Melhor experiência. É onde containers realmente rodam
Windows 10/11 com WSL2	Containers Linux rodam numa VM leve gerenciada pelo WSL2	Caminho recomendado no Windows. Requer WSL2, não WSL1
Windows nativo (Hyper-V)	Funciona, mas performance de disco pior e mais atrito	Só se WSL2 não for possível
macOS (Intel ou Apple Silicon)	Roda numa VM Linux escondida	Funciona bem. I/O de disco em bind mount é o ponto fraco
Windows containers	Containers que rodam Windows de verdade, só em host Windows	Nicho (.NET Framework legado). Não é o que este material cobre

Regra que economiza horas: container Linux só roda sobre kernel Linux. Em macOS e Windows há sempre uma VM Linux no meio — visível ou não. Isso não é detalhe: explica a lentidão de I/O e o consumo de RAM nessas plataformas.

2.2.3 Contas e serviços

Conta	Obrigatória?	Para quê	Custo
Docker Hub	Não para usar; sim na prática	Sem login: 10 <i>pulls</i> por hora por IP. Com conta grátis: 100/hora	Grátis (plano Personal)
Docker Desktop (licença)	Depende da empresa	Grátis para uso pessoal, educação, open source não comercial e empresas com < 250 funcionários e < US\$ 10 mi de receita anual	Ver cap. 25
GitHub	Não	GHCR (registry gratuito), Codespaces, CI	Grátis

Sem cartão de crédito em nenhum dos itens acima no plano gratuito (verificado em 11/08/2026).

2.3 Decisão que você precisa tomar antes de instalar

Existem dois caminhos, e escolher errado custa retrabalho:

	Docker Engine (CLI + daemon)	Docker Desktop (app gráfico)
Plataforma	Linux apenas	Linux, macOS, Windows
Interface	Só linha de comando	GUI + CLI
Licença	Apache 2.0, livre para tudo	Grátis com limites; paga em empresa grande
Consumo	Nenhum extra	VM dedicada, ~2 GB RAM em repouso
Recomendação	Servidor e desktop Linux	macOS e Windows

Alternativas legítimas ao Docker Desktop, se a licença for problema: **Podman Desktop**, **Rancher Desktop**, **colima** (macOS), **OrbStack** (macOS, pago, muito rápido). Comparadas em [cap. 25](#).

2.4 Tempo realista de estudo

Números honestos, assumindo estudo consistente e prática com as mãos. Se alguém prometer "Docker em 1 hora", entendeu-se docker run, não Docker.

Nível	O que você consegue fazer	Tempo realista
Sobrevivência	Rodar imagens prontas, publicar porta, ler logs, subir um Postgres	3-5 horas
Produtivo em dev	Escrever Dockerfile decente, usar Compose, montar volume, depurar container	15-25 horas (1 a 2 semanas)
Competente	Multi-stage, cache de build eficiente, redes, healthcheck, imagem enxuta e segura, CI	60-100 horas (2 a 3 meses)
Produção	Operar sob carga, limites de recurso, log e métrica, segurança, registry privado, multi-arch	200-400 horas (6 meses a 1 ano, com produção real)
Profundidade interna	Entender namespaces/cgroups/OverlayFS a ponto de depurar sem docker, escrever runtime, avaliar gVisor/Kata	500+ horas e trabalho no assunto

A cicatriz que ninguém conta: o salto difícil não é do 1 para o 2, é do 3 para o 4. Rodar container é fácil; operar container com estado, rede e segurança em produção é uma especialidade inteira.

2.5 Rota de resgate — falta um pré-requisito, e agora?

O que falta	Rota mais curta
Nunca usei terminal	Faça 2 horas de linha de comando básica antes . Insistir sem isso gera frustração falsa ("Docker é difícil" quando o difícil é o <code>cd</code>).
Máquina fraca / sem disco	Use ambiente na nuvem: Play with Docker (grátis, sessões de 4h) ou GitHub Codespaces (60h/mês grátis). Ver 03-instalacao.md (cap. 3).
Windows sem WSL2 / sem permissão de admin	Play with Docker ou Codespaces. Ou uma VM Linux num serviço de nuvem barato.
Empresa proíbe Docker Desktop	Instale Podman Desktop ou Rancher Desktop — mesmos comandos, licença livre. alias <code>docker=podman</code> cobre 90% do uso.
Não sei programar	Passe o Bloco A inteiro usando imagens prontas (nginx, postgres, redis). Só volte ao Dockerfile quando tiver algo seu para empacotar.
Não sei YAML	Comece só com <code>docker run</code> . Compose entra no cap. 4 , e o YAML necessário é trivial.
Não sei Linux	Não bloqueia o começo. Bloqueia no 75-armadilhas.md (permissão de volume, usuário não-root). Aprenda <code>chown</code> , <code>UID/GID</code> e permissões quando chegar lá.

2.6 Checklist antes de seguir

bash

```
# 1. Terminal abre e responde
echo "ok"
# esperado: ok

# 2. Você sabe onde está
pwd
# esperado: um caminho absoluto, ex.: /home/seu-usuario

# 3. Há disco livre suficiente (>= 20 GB)
df -h /
# esperado: coluna "Avail" com 20G ou mais

# 4. Arquitetura da máquina
uname -m
# esperado: x86_64 (amd64) ou aarch64/arm64

# 5. Só em Linux — versão do kernel (>= 5.10)
uname -r
# esperado: 6.8.0-... ou similar

# 6. Só em Linux — virtualização NÃO é necessária, mas confira o suporte a cgroup v2
stat -fc %T /sys/fs/cgroup/
# esperado: cgroup2fs (se vier "tmpfs", você está em cgroup v1 — funciona, mas é legado)
```

Passou nos seis? Vá para [cap. 3](#).

2.7 Autoteste

1. Por que "saber Kubernetes" **não** é pré-requisito para Docker — e por que a ordem inversa prejudica o aprendizado?
2. Qual é o recurso de hardware que costuma acabar primeiro numa máquina de desenvolvedor com Docker, e por quê?
3. Em qual dos três sistemas operacionais **não** existe uma VM entre você e o container? Que consequência prática isso tem?

4. Sua empresa tem 400 funcionários. Você pode usar Docker Desktop gratuitamente? E Docker Engine?
5. Quantos *pulls* por hora você consegue do Docker Hub sem fazer login? E com conta gratuita?
6. Você tem 3 horas por semana. Quanto tempo até conseguir escrever um Dockerfile decente com Compose?
7. Cite duas rotas de resgate para alguém que não pode instalar nada na própria máquina.

CAPÍTULO 3

Manual de instalação — passo a passo, por sistema operacional

Nível: iniciante · Manual de campo · Última atualização: 11/08/2026

Versões de referência deste manual (consultadas em **11/08/2026**):

- **Docker Engine: 29.7.1**, publicada em 04/08/2026. Série **29** é a atual.
- **Docker Compose**: plugin v2 (docker compose, com espaço). O binário legado docker-compose (com hífen, v1, Python) está **descontinuado** — não instale.
- **containerd**: série 2.x. Desde a Engine 29, o *containerd image store* é o **padrão em instalações novas**.
- **Versão mínima que ainda vale a pena usar**: Engine 24.x. Abaixo disso faltam BuildKit por padrão e correções de segurança relevantes.
- **Versão a evitar**: qualquer pacote docker.io de repositório antigo de distro que entregue Engine < 20.10, e o docker-compose v1.

Onde eu **não** consegui confirmar um número exato na data acima, o texto diz isso explicitamente em vez de inventar. Sempre confie na saída de `--version` da sua máquina.

3.1 Leia isto antes de qualquer coisa

Este manual instala **o conjunto inteiro**, não só o Docker:

1. **Docker Engine** (o daemon que roda containers) ou **Docker Desktop**
2. **Docker CLI** (o comando docker)
3. **Buildx** (motor de build moderno — plugin docker-buildx-plugin)
4. **Compose v2** (plugin docker-compose-plugin)
5. **Git** (você vai versionar Dockerfile e Compose)
6. **Editor + extensão** (VS Code + extensão Docker/Container Tools)
7. **WSL2** (só Windows)
8. **Conta no Docker Hub** (para não bater no limite de 10 *pulls*/hora)

Pule qualquer um desses e você trava mais adiante, em geral no pior momento.

3.1.1 Índice

- Alternativa sem instalar nada ← **comece por aqui se tiver pressa**
- Linux — Debian/Ubuntu
- Linux — Fedora/RHEL

- Linux — pós-instalação obrigatória
- macOS
- Windows
- Git
- Editor e extensões
- Conta no Docker Hub
- PATH e variáveis de ambiente
- Permissões e o problema do sudo
- Rede corporativa: proxy e certificado
- Convivência de versões
- Reprodutibilidade
- Atualizar e voltar atrás
- Desinstalar por completo
- Solução de problemas
- Checklist “ambiente pronto”

3.2 Alternativa sem instalar nada

Faça isto primeiro se você só quer começar hoje. Instalar leva 30–60 minutos e pode dar errado; estas opções levam 2 minutos e permitem seguir todo o [cap. 4](#) e boa parte do [cap. 6](#).

Opção	Link	O que dá	Limite
Play with Docker	labs.play-with-docker.com	Terminal com Docker real, no navegador. Cria até 5 nós — dá para praticar Swarm	Sessão de 4 horas , tudo é apagado depois. Exige conta Docker Hub
Killercoda	killercoda.com/docker	Cenários guiados com terminal real	Sessões curtas, ambiente efêmero
GitHub Co- despaces	github.com/codespaces	VM Linux completa com Docker, VS Code no navegador	Cota gratuita mensal na conta pessoal (verifique a sua; a cota já mudou várias vezes)
Google Cloud Shell	shell.cloud.google.com	Shell Linux com Docker, 5 GB persistentes	Requer conta Google; horas mensais limitadas
VM na nuvem	qualquer VPS	Ambiente definitivo, seu	Custa dinheiro (US\$ 4–6/mês numa VPS mínima)

Recomendação: comece no *Play with Docker* hoje, e instale localmente no fim de semana. É o que evita a desistência no primeiro dia.

3.3 Linux — família Debian/Ubuntu

Testado em Ubuntu 24.04 LTS e Debian 12, em 11/08/2026. Vale também para Linux Mint, Pop!_OS, Raspberry Pi OS (trocando ubuntu por debian na URL do repositório).

3.3.1 Métodos disponíveis e qual escolher

Método	Quando usar	Recomendado?
Repositório APT oficial da Docker	Praticamente sempre	✓ Sim — é este manual
<code>apt install docker.io</code> (repo da distro)	Nunca, se puder evitar	✗ Versão atrasada, Compose v2 ausente ou defasado
Script <code>get.docker.com</code>	Máquina descartável, laboratório, CI	⚠ Funciona, mas roda um script remoto como root e não é ideal em produção
Pacote <code>.deb</code> avulso	Máquina sem internet (<i>air-gapped</i>)	⚠ Só nesse caso; você atualiza tudo à mão
Docker Desktop for Linux	Se quiser a GUI	⚠ Adiciona uma VM desnecessária no Linux
Compilar do fonte (Moby)	Desenvolver o próprio Docker	✗ Não para uso

3.3.2 Passo 1 — remover pacotes conflitantes

```
bash
sudo apt remove docker.io docker-compose docker-compose-v2 docker-doc docker-buildx podman-docker containerd runc
```

O que faz: remove versões antigas ou concorrentes que sequestram o nome docker. É normal o apt dizer que alguns pacotes nem estavam instalados.

```
bash
dpkg -l | grep -E '^ii\s+(docker|containerd|runc)' || echo "limpo"
# esperado: limpo
```

Se a saída listar pacotes: remova-os explicitamente pelo nome antes de continuar. Manter dois Dockers na máquina é fonte garantida de confusão.

Atenção: este passo **não** apaga imagens, containers e volumes — eles ficam em `/var/lib/docker`. Para apagar de verdade, veja Desinstalar por completo.

3.3.3 Passo 2 — chave GPG do repositório oficial

```
bash
sudo apt update
sudo apt install -y ca-certificates curl
```

O que faz: garante que o sistema saiba validar certificados TLS e tenha o curl.

```
bash
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

O que faz: baixa a chave pública com que a Docker assina os pacotes, e a torna legível pelo apt. Sem isso, o apt recusa o repositório — corretamente.

Debian: troque ubuntu por debian na URL acima e no passo 3.

bash

```
ls -l /etc/apt/keyrings/docker.asc
# esperado: -rw-r--r-- 1 root root <alguns milhares de bytes> ... /etc/apt/keyrings/docker.asc
```

Se o arquivo tiver 0 byte: o download falhou (proxy, DNS, firewall). Veja Rede corporativa.

3.3.4 Passo 3 — adicionar o repositório

bash

```
sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update
```

O que faz: declara o repositório da Docker no formato **deb822** (.sources), que substituiu o antigo .list de uma linha. As substituições \$(...) preenchem automaticamente o codinome da sua distro e a arquitetura.

bash

```
apt-cache policy docker-ce | head -5
# esperado: linhas com "Candidato:" apontando para uma versão 5:29.x e o repositório download.docker.com
```

Se Candidato: vier (nenhum): o codinome da sua distro não existe no repositório (distro muito nova ou derivada não reconhecida). Solução: fixe manualmente um codinome suportado, ex. Suites: noble no Ubuntu 24.04.

3.3.5 Passo 4 — instalar

bash

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

O que faz: instala, respectivamente: o daemon, a CLI, o runtime containerd, o motor de build Buildx e o Compose v2. **Os cinco são necessários** — instalar só docker-ce deixa você sem docker compose e sem docker buildx.

bash

```
sudo docker version
# esperado: seções "Client:" e "Server: Docker Engine - Community", ambas com Version: 29.x.x
```

Se aparecer só "Client" e um erro de conexão: o daemon não está rodando — vá para o passo 5.

3.3.6 Passo 5 — daemon ativo

bash

```
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager
```

O que faz: habilita o serviço no boot e o inicia agora.

```
# esperado: "Active: active (running)" em verde
```

Se vier Failed to start: rode `sudo journalctl -u docker -n 50 --no-pager` e procure a mensagem real. As causas comuns estão na tabela de erros.

3.4 Linux — família Fedora/RHEL/Rocky

Testado em Fedora 42 e Rocky Linux 9, em 11/08/2026.

3.4.1 Passo 1 — remover conflitantes

```
sudo dnf remove -y docker docker-client docker-client-latest docker-common \
docker-latest docker-latest-logrotate docker-logrotate docker-selinux \
docker-engine-selinux docker-engine podman runc
```

bash

O que faz: limpa pacotes antigos. **Cuidado:** isso remove o podman, que vem instalado por padrão no Fedora. Se você usa Podman, pule-o da lista e veja Convivência de versões.

3.4.2 Passo 2 — adicionar o repositório

```
sudo dnf -y install dnf-plugins-core
sudo dnf config-manager addrepo --from-repofile=https://download.docker.com/linux/fedora/docker-ce.repo
```

bash

O que faz: registra o repositório oficial. **Em RHEL/Rocky/AlmaLinux**, troque fedora por rhel na URL. Em versões mais antigas do dnf, o subcomando é `sudo dnf config-manager --add-repo <url>` — se a primeira forma falhar, use a segunda.

```
dnf repolist | grep docker
# esperado: uma linha com "docker-ce-stable"
```

bash

3.4.3 Passo 3 — instalar e ativar

```
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

bash

```
sudo docker run --rm hello-world
# esperado: "Hello from Docker!" seguido de uma explicação de 4 parágrafos
```

bash

Se falhar com erro de SELinux: veja a tabela de erros. No Fedora/RHEL, SELinux está ativo e **vai** interferir em bind mounts — a solução é a flag `:z/:Z`, explicada em [cap. 13](#).

3.5 Linux — pós-instalação obrigatória

3.5.1 Rodar docker sem sudo

Por padrão, o socket do daemon (/var/run/docker.sock) pertence ao root. Digitar sudo a cada comando é insuportável, e — pior — arquivos criados por containers ficam com dono errado.

bash

```
sudo groupadd -f docker
sudo usermod -aG docker $USER
```

O que faz: cria o grupo docker (se não existir) e adiciona você a ele. Quem está nesse grupo pode falar com o socket.

bash

```
newgrp docker      # aplica o grupo na sessão atual, sem relogar
docker run --rm hello-world
# esperado: "Hello from Docker!" — SEM sudo
```

Se ainda pedir permissão: faça logout e login de novo (ou reinicie). Mudança de grupo só vale para sessões criadas depois dela; newgrp resolve apenas o terminal atual.

3.5.2 ⚠ O que ninguém te conta sobre o grupo docker

Estar no grupo docker é equivalente a ser root na máquina. Não é exagero retórico — é uma escalada de privilégio de uma linha:

bash

```
docker run -v /:/host -it alpine chroot /host sh # você é root no host
```

O daemon roda como root e faz o que você mandar. Por isso:

- Numa máquina pessoal, o grupo docker é aceitável e conveniente.
- Num **servidor compartilhado**, colocar alguém no grupo docker é dar root a essa pessoa. Use **rootless mode** (abaixo) ou sudo explícito e auditado.

3.5.3 Rootless mode (opcional, recomendado em servidor multiusuário)

Roda o daemon como seu usuário comum, sem privilégio de root.

bash

```
sudo apt install -y docker-ce-rootless-extras uidmap # Debian/Ubuntu
dockerd-rootless-setup.sh install
```

O que faz: instala o suporte a mapeamento de UID e configura um daemon pessoal na sua sessão.

bash

```
export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock
docker info | grep -i rootless
# esperado: uma linha contendo "rootless"
```

Adicione a linha `export DOCKER_HOST=...` ao seu `~/.bashrc` para ela persistir.

O que se perde no rootless: portas < 1024 exigem configuração extra, alguns drivers de rede e de storage não funcionam, e o desempenho de rede é levemente pior. **O que se ganha:** uma falha de escape de container não entrega a máquina.

3.5.4 Iniciar no boot

```
sudo systemctl enable docker.service containerd.service
systemctl is-enabled docker
# esperado: enabled
```

bash

3.6 macOS — Intel e Apple Silicon

Fato estrutural: o macOS não tem kernel Linux. **Todo** container Linux no macOS roda dentro de uma VM Linux. As opções abaixo diferem em quem gerencia essa VM, não em se ela existe.

3.6.1 Requisitos

- macOS 13 (Ventura) ou superior — as versões suportadas mudam a cada release do Docker Desktop
- 8 GB de RAM no mínimo (16 GB é o confortável)
- Apple Silicon (M1-M4+) ou Intel — o instalador é **diferente** para cada um

```
uname -m
# esperado: arm64 (Apple Silicon) ou x86_64 (Intel)
```

bash

3.6.2 Métodos, e qual escolher

Método	Licença	Recomendado para
Docker Desktop	Grátis com limites; paga em empresa grande	✓ Padrão, se a licença permitir
colima (Homebrew, livre)	Apache 2.0	✓ Quem quer só a CLI, sem GUI, sem licença
Podman Desktop	Apache 2.0	✓ Alternativa livre com GUI
Rancher Desktop	Apache 2.0	✓ Alternativa livre, traz Kubernetes junto
OrbStack	Proprietário, pago para uso comercial	⚠ O mais rápido do mercado; avalie se a velocidade justifica

3.6.3 Opção A — Docker Desktop

```
# 1) Instale o Homebrew, se ainda não tiver
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 2) Instale o Docker Desktop
brew install --cask docker
```

bash

O que faz: baixa e instala o app. Alternativamente, baixe o .dmg de docs.docker.com/desktop/install/mac-install — **atenção ao escolher o arquivo certo para Apple Silicon ou Intel.**

Abra o app **Docker** uma vez (ele precisa de autorização do sistema e instala um helper privilegiado). Espere o ícone da baleia na barra de menu ficar estável.

bash

```
docker version
# esperado: Client e Server, ambos com versão 2x.x
docker run --rm hello-world
# esperado: "Hello from Docker!"
```

Se der Cannot connect to the Docker daemon: o app não está rodando. Abra-o pelo Launchpad e aguarde.

Ajuste de recursos (Settings → Resources): o padrão costuma ser conservador. Em máquina de 16 GB, dar 6–8 GB e 4 CPUs à VM é razoável. Reduzir o *Virtual disk limit* não recupera espaço já usado — veja Desinstalar por completo.

Habilite o VirtioFS (Settings → General → *file sharing implementation*) se estiver disponível. É o que torna bind mounts toleráveis no macOS.

3.6.4 Opção B — colima (sem GUI, sem licença)

bash

```
brew install docker docker-compose docker-buildx colima
colima start --cpu 4 --memory 8 --disk 60 --vm-type vz --mount-type virtiofs
```

O que faz: instala a CLI (sem o Desktop) e sobe uma VM Linux enxuta com o daemon. `--vm-type vz` usa o framework de virtualização nativo da Apple; `virtiofs` é o modo de compartilhamento de arquivos mais rápido.

bash

```
docker context ls
# esperado: uma linha "colima" marcada com * (contexto ativo)
docker run --rm hello-world
# esperado: "Hello from Docker!"
```

Os plugins instalados pelo Homebrew precisam ser registrados para o docker enxergá-los:

bash

```
mkdir -p ~/.docker/cli-plugins
ln -sfn $(brew --prefix)/opt/docker-compose/bin/docker-compose ~/.docker/cli-plugins/docker-compose
ln -sfn $(brew --prefix)/opt/docker-buildx/bin/docker-buildx ~/.docker/cli-plugins/docker-buildx
docker compose version
# esperado: "Docker Compose version v2.x.x"
```

3.7 Windows — WSL2 (recomendado) e nativo

Recomendação e o porquê: use **WSL2**. Containers Linux no Windows sempre rodam numa VM; a diferença é que com WSL2 essa VM é integrada, compartilha memória dinamicamente com o Windows e dá I/O de disco muito melhor **desde que seus arquivos estejam dentro do sistema de arquivos do Linux**. Trabalhar em `/mnt/c/...` a partir do WSL é a causa nº 1 de "Docker no Windows é lento".

3.7.1 Passo 1 — habilitar o WSL2

Abra o **PowerShell como Administrador**:

PowerShell

```
wsl --install -d Ubuntu
```

O que faz: habilita os recursos de virtualização necessários, instala o WSL2 e a distro Ubuntu numa tacada. **Reinicie a máquina** quando pedir.

PowerShell

```
wsl --status
# esperado: "Versão padrão: 2"
wsl --list --verbose
# esperado: uma linha "Ubuntu    Running    2" - a coluna final PRECISA ser 2, não 1
```

Se a versão for 1: `wsl --set-version Ubuntu 2` e depois `wsl --set-default-version 2`. *Se der erro de virtualização:* entre na BIOS/UEFI e habilite **Intel VT-x** ou **AMD-V**. Em notebooks é comum vir desabilitado de fábrica.

PowerShell

```
wsl --update
```

O que faz: atualiza o kernel do WSL2. Kernel velho causa falhas obscuras no Docker.

3.7.2 Passo 2 — instalar o Docker Desktop

Baixe de docs.docker.com/desktop/install/windows-install e execute. Na tela de opções, marque **"Use WSL 2 instead of Hyper-V"**.

Depois de instalado: **Settings → Resources → WSL Integration** → habilite a integração com a distro Ubuntu. Sem isso, o comando docker não existe dentro do WSL.

3.7.3 Passo 3 — verificar de dentro do WSL

Abra o terminal **Ubuntu** (não o PowerShell):

bash

```
docker version
# esperado: Client e Server presentes, versão 2x.x
docker run --rm hello-world
# esperado: "Hello from Docker!"
```

Se docker: command not found dentro do WSL: a integração WSL não foi habilitada no Docker Desktop. Volte às Settings.

3.7.4 Passo 4 — trabalhar no lugar certo (isto é performance, não estética)

bash

```
# [ok] CERTO — sistema de arquivos do Linux, rápido
cd ~
mkdir -p ~/projetos && cd ~/projetos

# [x] ERRADO — disco do Windows via tradução de protocolo, 10x a 50x mais lento
cd /mnt/c/Users/SeuNome/projetos
```

No VS Code, instale a extensão **WSL** e abra a pasta com `code .` **de dentro do WSL**. A janela mostrará [WSL: Ubuntu] no canto inferior esquerdo — é assim que você confirma.

3.7.5 Alternativa: Docker Engine dentro do WSL, sem Docker Desktop

Se a licença do Docker Desktop for problema na sua empresa, você pode instalar o **Docker Engine nativamente dentro da distro WSL** seguindo a seção Debian/Ubuntu deste manual, de ponta a ponta. Funciona. Duas ressalvas:

- O daemon não inicia sozinho no boot do Windows. Adicione ao ~/.bashrc:

```
# inicia o daemon se não estiver rodando (WSL não usa systemd por padrão em versões antigas)
if ! service docker status > /dev/null 2>&1; then sudo service docker start > /dev/null 2>&1; fi
```

Em WSL com systemd=true habilitado no /etc/wsl.conf, use systemctl enable docker normalmente.

- Não há integração com o Docker do lado Windows — o que é irrelevante se você trabalha só no WSL.

3.7.6 Windows containers (nicho)

Containers que rodam **Windows de verdade** exigem host Windows e imagens base da Microsoft (mcr.microsoft.com/windows/servercore). São gigantes (GB), lentos para iniciar e o isolamento de versão de kernel é rígido. Só fazem sentido para .NET Framework legado que não migra para .NET moderno. Este material **não** cobre esse caminho.

3.8 Git em todos os sistemas

```
# Debian/Ubuntu
sudo apt install -y git
# Fedora/RHEL
sudo dnf install -y git
# macOS
brew install git
# Windows: já vem no WSL Ubuntu; para o lado Windows, use https://git-scm.com/download/win
```

```
git --version
# esperado: git version 2.4x.x ou superior
git config --global user.name "Seu Nome"
git config --global user.email "seu@email"
git config --global init.defaultBranch main
```

3.9 Editor e extensões

VS Code é o padrão de fato, e as extensões abaixo poupam tempo real:

Extensão	ID	O que faz
Container Tools / Docker	ms-azuretools.vscode-docker	Realce e autocompletar em Dockerfile e Compose, painel de containers, anexar terminal
Dev Containers	ms-vscode-remote.remote-containers	Abrir o projeto dentro de um container, com o editor junto
WSL	ms-vscode-remote.remote-wsl	Obrigatória no Windows

Extensão	ID	O que faz
YAML	redhat.vscode-yaml	Valida <code>compose.yaml</code> antes de você descobrir o erro no terminal
Hadolint	exiasr.hadolint	<i>Lint</i> er de Dockerfile — pega má prática enquanto você digita (exige o binário hadolint)

```
code --version
# esperado: número de versão em três linhas
```

bash

3.10 Conta no Docker Hub e login

Sem login você tem **10 pulls por hora, contados por IP** — e IP compartilhado (empresa, universidade, CGNAT de provedor) estoura esse número em minutos. Com conta gratuita, são **100 por hora**. Isso, sozinho, justifica criar a conta.

1. Crie a conta em hub.docker.com/signup (gratuita, sem cartão).
2. Gere um **Personal Access Token** em *Account Settings* → *Personal access tokens*. **Nunca use a senha da conta na CLI** — o token pode ser revogado sozinho e tem escopo limitado.

```
echo "SEU_TOKEN_AQUI" | docker login -u SEU_USUARIO --password-stdin
# esperado: Login Succeeded
```

bash

Por que --password-stdin: passar o segredo como argumento (-p senha) o grava no histórico do shell e o expõe na lista de processos.

```
docker system info | grep -i username
# esperado: Username: SEU_USUARIO
```

bash

Onde a credencial fica: por padrão, em `~/.docker/config.json`, **codificada em base64, não criptografada**. Em máquina compartilhada, instale um *credential helper* (docker-credential-pass no Linux, Keychain no macOS, wincred no Windows).

3.11 PATH e variáveis de ambiente

3.11.1 Conferir se o docker está no PATH

```
which docker          # Linux/macOS
# esperado: /usr/bin/docker ou /usr/local/bin/docker
```

bash

```
where.exe docker      # Windows
```

PowerShell

Se não retornar nada: o binário existe mas o shell não o encontra.

3.11.2 Corrigir, e em qual arquivo

Shell	Arquivo de perfil	Como saber qual é o seu
bash	~/.bashrc (interativo) e ~/.bash_profile (login)	echo \$SHELL → /bin/bash
zsh (padrão no macOS)	~/.zshrc	echo \$SHELL → /bin/zsh
fish	~/.config/fish/config.fish	echo \$SHELL → /usr/bin/fish
PowerShell	\$PROFILE (notepad \$PROFILE)	—

bash

```
echo 'export PATH="/usr/local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc      # <- ESTE passo é o que as pessoas esquecem
```

Por que “não pegou” antes de reabrir o terminal: o arquivo de perfil é lido **uma vez**, quando o shell inicia. Editá-lo não afeta shells já abertos. `source` relê o arquivo na sessão atual; abrir um terminal novo tem o mesmo efeito.

3.11.3 Variáveis que importam

Variável	Para que serve	Exemplo
DOCKER_HOST	Aponta para outro daemon (remoto, rootless, colima)	unix:///run/user/1000/docker.sock
DOCKER_CONTEXT	Seleciona um <i>context</i> nomeado (melhor que DOCKER_HOST)	docker context use remoto
DOCKER_BUILDKIT	Liga/desliga o BuildKit (já é padrão desde a Engine 23)	1
COMPOSE_PROJECT_NAME	Prefixo dos recursos criados pelo Compose	meuapp
COMPOSE_FILE	Arquivo Compose padrão	compose.yaml:compose.dev.yaml
DOCKER_DEFAULT_PLATFORM	Força a arquitetura das imagens	linux/amd64 (útil no Apple Silicon)

bash

```
docker context ls
# esperado: lista de contextos, com * no ativo
```

3.12 Permissões e o problema do sudo

3.12.1 Regra geral

Situação	Faça	Não faça
Rodar docker no seu desktop	Entre no grupo docker	sudo docker ... toda vez
Servidor com vários usuários	rootless mode ou sudo auditado	Colocar todo mundo no grupo docker
Container que escreve em bind mount	Rode com --user "\$(id -u):\$(id -g)"	Rodar como root e depois sudo chown
Instalar ferramentas	Pacote da distro ou gerenciador de versão	sudo npm install -g, sudo pip install

3.12.2 Por que sudo docker estraga arquivos

O daemon roda como root. Se o processo dentro do container roda como root e escreve num diretório do host montado por bind mount, os arquivos nascem **pertencendo ao root no host**. Aí você não consegue mais editá-los no seu editor e precisa de `sudo chown -R $USER:` para consertar. É desagradável e evitável:

```
docker run --rm -v "$PWD:/app" -w /app --user "$(id -u):$(id -g)" node:22-alpine npm init -y
ls -l package.json
# esperado: o dono é você, não root
```

bash

3.12.3 Por que sudo npm install -g e sudo pip install são problema

Não é etiqueta — é dano concreto:

1. **Corrompe a árvore de permissões:** parte dos arquivos passa a ser do root e o gerenciador de pacotes não consegue mais atualizá-los sem sudo, criando dependência permanente.
2. **Executa código arbitrário como root:** scripts de postinstall de um pacote qualquer rodam com privilégio total.
3. **Mistura com o gerenciador de pacotes do sistema,** que também escreve nos mesmos diretórios, e o próximo apt upgrade sobrescreve ou quebra sua instalação.

O caminho certo é usar um gerenciador de versão por usuário: `nvm`, `pyenv`, `mise`, `asdf`. Ou — apropriadamente — rodar a ferramenta **dentro de um container**.

3.13 Rede corporativa: proxy, certificado e registry espelhado

3.13.1 Proxy para o daemon

O daemon é um serviço do sistema; ele **não** lê as variáveis de proxy do seu shell.

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf > /dev/null <<'EOF'
[Service]
Environment="HTTP_PROXY=http://proxy.empresa.com:3128"
Environment="HTTPS_PROXY=http://proxy.empresa.com:3128"
Environment="NO_PROXY=localhost,127.0.0.1,.empresa.com,registry.interno"
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker
```

bash

```
sudo systemctl show --property=Environment docker
# esperado: a linha com HTTP_PROXY e HTTPS_PROXY que você definiu
```

bash

3.13.2 Proxy para a CLI e para os builds

bash

```
mkdir -p ~/.docker
cat > ~/.docker/config.json <<'EOF'
{
  "proxies": {
    "default": {
      "httpProxy": "http://proxy.empresa.com:3128",
      "httpsProxy": "http://proxy.empresa.com:3128",
      "noProxy": "localhost,127.0.0.1,empresa.com"
    }
  }
}
EOF
```

O que faz: injeta as variáveis de proxy dentro dos containers e dos builds automaticamente — sem isso, apt-get **dentro** do Dockerfile falha mesmo com o daemon configurado.

3.13.3 Certificado TLS interno (inspeção de tráfego / TLS interception)

Sintoma: x509: certificate signed by unknown authority.

bash

```
# Registry interno com certificado próprio
sudo mkdir -p /etc/docker/certs.d/registry.empresa.com:5000
sudo cp ca-empresa.crt /etc/docker/certs.d/registry.empresa.com:5000/ca.crt
sudo systemctl restart docker
```

E, para que os builds também confiem, copie a CA para dentro da imagem:

Dockerfile

```
COPY ca-empresa.crt /usr/local/share/ca-certificates/
RUN update-ca-certificates
```

3.13.4 Registry espelhado (e como escapar do limite do Docker Hub)

bash

```
sudo tee /etc/docker/daemon.json > /dev/null <<'EOF'
{
  "registry-mirrors": ["https://mirror.empresa.com"],
  "insecure-registries": []
}
EOF
sudo systemctl restart docker
docker system info | grep -A2 "Registry Mirrors"
# esperado: a URL do espelho listada
```

insecure-registries desliga a verificação TLS. Só use em rede isolada de laboratório, e saiba que está trocando segurança por conveniência.

3.14 Convivência de versões na mesma máquina

Não instale dois Docker Engines na mesma máquina. Ao contrário de Node ou Python, o Docker tem um daemon único, um socket único e um diretório de dados único; duas instalações brigam.

Como resolver os casos reais:

Necessidade	Solução
Testar uma versão diferente do Engine	Uma VM ou um servidor separado, acessado por <code>docker context</code>
Vários daemons na mesma máquina	<code>docker context + rootless</code> (um daemon por usuário)
Podman e Docker convivendo	Convivem sem conflito. podman usa socket próprio. Evite o pacote <code>podman-docker</code> , que cria um docker falso
Versões diferentes do Compose	Instale binários avulsos em <code>~/.docker/cli-plugins/</code> e alterne por symlink
Versões diferentes de linguagem/runtime	Não é problema de Docker: cada imagem traz a sua. É exatamente a vantagem — <code>node:20</code> e <code>node:22</code> coexistem sem conflito

bash

```
docker context create servidor --docker "host=ssh://usuario@servidor"
docker context use servidor
docker ps                # agora lista os containers do servidor remoto
docker context use default
```

3.15 Reprodutibilidade

Instalação reprodutível é o que separa “funcionou uma vez” de “funciona para a equipe inteira”.

Mecanismo	Arquivo	O que garante
Tag imutável por digest	FROM <code>node:22.4.0-alpine@sha256:abc123...</code>	O <i>mesmo byte</i> de imagem base, sempre. Tag pode ser reescrita; digest, não
Lockfile da linguagem	<code>package-lock.json</code> , <code>poetry.lock</code> , <code>go.sum</code> , <code>Cargo.lock</code>	Mesmas dependências
.dockerignore	<code>.dockerignore</code>	Contexto de build idêntico entre máquinas (e build mais rápido)
Versão da ferramenta	<code>.tool-versions</code> (mise/asdf)	Mesma versão de CLI auxiliar na equipe
Dev Container	<code>.devcontainer/devcontainer.json</code>	Ambiente de desenvolvimento inteiro versionado
Compose fixado	<code>image: postgres:16.3</code> (nunca <code>postgres:latest</code>)	Serviços de apoio iguais para todos

bash

```
# Descubra o digest de uma imagem que você já usa
docker image inspect --format '{{index .RepoDigests 0}}' node:22-alpine
# esperado: node@sha256:<64 caracteres hexadecimais>
```

latest é uma mentira útil. Não significa “a mais recente”: é apenas a tag padrão quando nenhuma é dada, e ela é reescrita a cada publicação. Em produção, latest é a causa de “ontem funcionava”.

3.16 Atualizar e voltar atrás

3.16.1 Atualizar

bash

```
# Debian/Ubuntu
sudo apt update && sudo apt upgrade docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
# Fedora/RHEL
sudo dnf upgrade docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
# macOS / Windows: o Docker Desktop avisa e atualiza sozinho
```

bash

```
docker version --format '{{.Server.Version}}'
# esperado: a nova versão
```

Antes de atualizar em servidor: containers em execução são **reiniciados** quando o daemon reinicia (a menos que live-restore esteja habilitado). Planeje janela. E leia as notas da versão: a Engine 29 mudou o *image store* padrão para containerd em instalações novas — o que **não** migra imagens existentes automaticamente numa atualização, mas muda o comportamento de quem reinstala do zero.

3.16.2 Voltar atrás

bash

```
apt-cache madison docker-ce
# esperado: lista de versões disponíveis, ex.: 5:29.7.1-1-ubuntu.24.04-noble

sudo apt install -y --allow-downgrades docker-ce=<VERSAO> docker-ce-cli=<VERSAO>
sudo apt-mark hold docker-ce docker-ce-cli # impede que o upgrade automático suba de novo
```

Para liberar depois: `sudo apt-mark unhold docker-ce docker-ce-cli`.

3.17 Desinstalar por completo

A remoção do pacote não apaga os dados. Imagens, containers, volumes e redes ficam em `/var/lib/docker` e podem ocupar dezenas de GB indefinidamente.

3.17.1 Linux

bash

```
# 1) Ver o que está ocupando espaço ANTES de decidir
docker system df
# esperado: tabela com Images / Containers / Local Volumes / Build Cache e o "RECLAIMABLE"

# 2) Remover os pacotes
sudo apt purge -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin \
  docker-compose-plugin docker-ce-rootless-extras
# (Fedora/RHEL: sudo dnf remove -y ...)
```

3) Apagar TODOS os dados – imagens, containers e VOLUMES. Irreversível.

```
sudo rm -rf /var/lib/docker /var/lib/containerd
```

4) Configuração e credenciais

```
sudo rm -rf /etc/docker /etc/apt/sources.list.d/docker.sources /etc/apt/keyrings/docker.asc
rm -rf ~/.docker
```

5) Grupo

```
sudo groupdel docker
```


⚠ O passo 3 **apaga volumes**. Se houver banco de dados ali, o dado morre. Faça backup antes: `docker run --rm -v meu_volume:/d -v "$PWD:/b" alpine tar czf /b/backup.tgz -C /d .`

3.17.2 macOS

```
# Docker Desktop: use "Troubleshoot → Uninstall" no próprio app, depois:
rm -rf ~/Library/Group\ Containers/group.com.docker
rm -rf ~/Library/Containers/com.docker.docker
rm -rf ~/.docker ~/Library/Application\ Support/Docker\ Desktop
rm -f ~/Library/Preferences/com.docker.docker.plist

# colima
colima delete
brew uninstall colima docker docker-compose docker-buildx
```

bash

3.17.3 Windows

Painel de Controle → Programas → desinstalar **Docker Desktop**. Depois, no PowerShell como Administrador:

```
wsl --unregister docker-desktop
Remove-Item -Recurse -Force "$env:APPDATA\Docker", "$env:LOCALAPPDATA\Docker", "$env:USERPROFILE\.docker"
```

PowerShell

3.17.4 Limpeza sem desinstalar (o caso comum: disco cheio)

```
docker system df          # o que ocupa espaço
docker system prune       # remove containers parados, redes órfãs e cache pendente
docker system prune -a    # + TODAS as imagens não usadas por nenhum container
docker system prune -a --volumes # + VOLUMES órfãos ⚠ APAGA DADOS
docker builder prune --keep-storage 10GB # limita o cache de build
```

bash

3.18 Solução de problemas — erros literais

Mensagem (literal)	Causa provável	Correção
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?	O daemon não está no ar, ou DOCKER_HOST aponta para o lugar errado	<code>sudo systemctl start docker</code> (Linux) · abrir o Docker Desktop (mac/Win) · <code>colima start</code> · conferir <code>echo \$DOCKER_HOST</code>
permission denied while trying to connect to the Docker daemon socket	Seu usuário não está no grupo docker, ou o grupo ainda não valeu nesta sessão	<code>sudo usermod -aG docker \$USER</code> e então logout/login (ou <code>newgrp docker</code>)
docker: command not found	Binário ausente ou fora do PATH; no Windows, integração WSL desligada	<code>which docker</code> · reinstalar · Docker Desktop → Settings → Resources → WSL Integration
toomanyrequests: You have reached your pull rate limit	Limite do Docker Hub: 10 pulls/h sem login, 100/h com conta grátis	<code>docker login</code> · usar espelho (registry-mirrors) · usar GHCR/Quay · plano pago

Mensagem (literal)	Causa provável	Correção
x509: certificate signed by unknown authority	Proxy corporativo com inspeção TLS, ou registry com certificado próprio	Instalar a CA em /etc/docker/certs.d/<host>/ca.crt e no sistema; reiniciar o daemon
no space left on device	/var/lib/docker cheio de imagens, volumes e cache de build	docker system df → docker system prune -a · docker builder prune · aumentar o disco
Error response from daemon: driver failed programming external connectivity ... address already in use	Outro processo já ocupa a porta do host	ss -tlnp grep :8080 para achar o culpado · trocar a porta do lado esquerdo do -p
WSL 2 installation is incomplete / Hardware assisted virtualization ... must be enabled	VT-x/AMD-V desabilitado na BIOS, ou kernel do WSL velho	Habilitar virtualização na BIOS/UEFI · wsl --update · reiniciar
exec /app/entrypoint.sh: no such file or directory (e o arquivo existe!)	Fim de linha CRLF do Windows, ou binário de arquitetura errada	dos2unix entrypoint.sh · .gitattributes com * text eol=lf · conferir --platform
exec format error	Imagem de outra arquitetura (amd64 num ARM, ou vice-versa)	docker run --platform linux/amd64 ... · construir multi-arch com buildx
Permission denied num arquivo dentro de bind mount, só em Fedora/RHEL	SELinux bloqueando o acesso do container	Use -v "\$PWD:/app:Z" (rótulo exclusivo) ou :z (compartilhado)
failed to solve: process "/bin/sh -c apt-get update" did not complete successfully	Sem rede/DNS dentro do build, quase sempre proxy não configurado para builds	Configurar proxies em ~/.docker/config.json · conferir DNS do daemon
manifest unknown / pull access denied	Nome ou tag errados, ou imagem privada sem login	Conferir o nome exato no registry · docker login
The container name "/x" is already in use by container ..	Já existe um container com esse nome, mesmo parado	docker rm x · ou use --rm para containers efêmeros

3.19 Checklist "ambiente pronto"

Rode tudo. Se um item falhar, resolva antes de ir para o [cap. 4](#).

bash

```

docker version                                # Client E Server, ambos 29.x (ou >= 24)
docker compose version                        # Docker Compose version v2.x.x
docker buildx version                         # github.com/docker/buildx v0.x.x
docker run --rm hello-world                  # "Hello from Docker!"
docker run --rm alpine ping -c1 1.1.1.1      # rede saindo do container: "1 packets received"
docker run --rm alpine nslookup docker.com   # DNS dentro do container resolve
docker run -d --rm --name t -p 8080:80 nginx:alpine && sleep 2 && curl -sI localhost:8080 | head -1 && docker stop t
                                                # "HTTP/1.1 200 OK" → publicação de porta funciona
docker volume create teste && docker run --rm -v teste:/d alpine sh -c 'echo ok > /d/f' \
&& docker run --rm -v teste:/d alpine cat /d/f && docker volume rm teste
                                                # "ok" → volumes funcionam
docker run --rm -v "$PWD:/w" -w /w alpine ls  # bind mount enxerga seus arquivos
git --version                                # 2.4x ou superior
docker system info | grep -i username         # você está logado no Hub
docker system df                              # sem erro, e você sabe quanto está ocupando

```

Dez linhas verdes = ambiente pronto. Siga para [cap. 4](#).

3.20 Autoteste

1. Por que instalar apenas `docker-ce` (sem os plugins) deixa o ambiente incompleto?
2. Você adicionou seu usuário ao grupo `docker` e continua tomando `permission denied`. Por quê?
3. Explique, em uma frase e com um comando, por que estar no grupo `docker` equivale a ter `root`.
4. Qual é a diferença prática entre `docker-compose` e `docker compose`, e qual usar?
5. No Windows, por que trabalhar em `/mnt/c/...` a partir do WSL degrada tanto o desempenho?
6. Seu build falha com `x509: certificate signed by unknown authority` só na rede da empresa. Quais dois lugares precisam receber a CA?
7. `sudo apt purge docker-ce` libera espaço em disco? Justifique.
8. Por que fixar `FROM node:22-alpine` ainda não é reprodutível, e o que torna?
9. Você recebe `exec format error` ao rodar uma imagem num Mac M3. O que aconteceu e como resolver de dois jeitos?

3.20.1 Fontes consultadas (11/08/2026)

- [Docker Docs — Install Docker Engine on Ubuntu](#) — comandos exatos do repositório APT em formato `deb`⁸²²
- [Docker Engine v29 Release — Docker Blog](#) e [Docker Engine 29 release notes](#) — `containerd` image store como padrão em instalações novas, backend `nftables` experimental
- [Linuxiac — Docker Engine 29: Containerd Becomes Default, Experimental nftables Support](#)
- [endoflife.date — Docker Engine](#) — versão 29.7.1 publicada em 04/08/2026
- [Docker Docs — Usage and rate limits](#) e [GitLab Support — Docker Hub rate limiting](#) — 10 *pulls*/hora sem autenticação, 100/hora com conta gratuita, desde 01/04/2025

CAPÍTULO 4

Como começar — do ambiente pronto ao primeiro resultado

Nível: iniciante · Tempo: 45–75 minutos · Última atualização: 11/08/2026

Este documento **assume o ambiente já instalado** pelo [cap. 3](#). Se `docker run --rm hello-world` não imprime “Hello from Docker!”, volte lá.

Ao final você terá: rodado um serviço pronto, construído a sua própria imagem, persistido dados num volume e subido dois containers conversando entre si com Compose. Nessa ordem, porque cada passo depende do anterior.

4.1 Passo 1 — Rodar algo de verdade em 10 segundos

```
docker run -d --name web -p 8080:80 nginx:alpine
```

bash

Anatomia do comando — cada pedaço importa:

Pedaço	Significado
<code>docker run</code>	crie um container a partir de uma imagem e inicie-o
<code>-d</code>	<i>detached</i> : rode em segundo plano e devolva o terminal
<code>--name web</code>	dê o nome web (sem isso o Docker sorteia algo como <code>nostalgic_hopper</code>)
<code>-p 8080:80</code>	publique a porta 80 do container na porta 8080 do host
<code>nginx:alpine</code>	imagem nginx, tag alpine (variante minúscula, ~10 MB)

Verificação:

```
curl -sI http://localhost:8080 | head -1
# esperado: HTTP/1.1 200 OK
```

bash

Ou abra `http://localhost:8080` no navegador: aparece a página “Welcome to nginx!”.

A ordem de `-p` é `host:container`, e trocá-la é o erro mais comum da semana 1.
`-p 8080:80` = “quem chegar na 8080 da minha máquina, mande para a 80 lá dentro”.
`-p 80:8080` faria o contrário e não funcionaria com o nginx, que escuta na 80.
Mnemônico: **de fora para dentro**, esquerda para direita.

```
docker ps
# esperado: uma linha com IMAGE nginx:alpine, STATUS "Up X seconds", PORTS "0.0.0.0:8080->80/tcp"
```

bash

4.1.1 O que acabou de acontecer, por dentro

1. O Docker procurou `nginx:alpine` **localmente**. Não achou.

2. Baixou do Docker Hub, camada por camada (as barras de progresso são as camadas).
3. Criou um sistema de arquivos empilhando essas camadas + uma camada de escrita vazia por cima.
4. Criou namespaces novos (rede, PID, mount...) e iniciou o processo nginx dentro deles.
5. Programou uma regra de encaminhamento (iptables/nftables) mandando a porta 8080 do host para o IP interno do container.

Cada uma dessas cinco etapas tem um capítulo no Bloco B.

4.2 Passo 2 — O ciclo de trabalho do dia a dia

Estes cinco comandos são 80% do seu uso cotidiano. Aprenda-os agora, não depois.

bash

```
docker ps                # o que está rodando
docker ps -a             # + o que está parado (aqui moram os containers que morreram)
docker logs -f web       # ver a saída do processo, acompanhando (-f = follow, Ctrl+C sai)
docker exec -it web sh   # abrir um shell DENTRO do container que já está rodando
docker stop web          # parar (SIGTERM, e SIGKILL após 10s)
docker rm web            # remover o container parado
```

Experimente o exec — é onde o modelo mental se consolida:

bash

```
docker exec -it web sh
```

sh

```
# você agora está dentro do container
hostname          # um ID hexadecimal, não o nome da sua máquina
ls /              # um sistema de arquivos Linux completo, minúsculo
ps aux            # SÓ o nginx e o seu shell. PID 1 é o nginx!
cat /etc/os-release # Alpine Linux, mesmo que seu host seja Ubuntu ou macOS
exit
```

ps aux mostrando apenas dois processos é a demonstração mais direta de namespace que existe. Sua máquina tem centenas de processos rodando; o container enxerga dois. Não é filtragem cosmética — é o kernel devolvendo uma visão diferente da tabela de processos.

E o detalhe mais consequente:

PID 1 é o seu processo. No Linux comum, PID 1 é o init/systemd, que adota processos órfãos e responde a sinais. Dentro do container, PID 1 é o seu app — e o kernel trata PID 1 de forma especial: **sinais que não têm tratador registrado são ignorados**. É por isso que alguns containers demoram exatos 10 segundos para parar: o SIGTERM do docker stop foi ignorado, e o Docker recorreu ao SIGKILL depois do timeout. A solução está em [cap. 15](#) (forma exec do CMD, --init).

Limpe antes de seguir:

bash

```
docker stop web && docker rm web
```

4.3 Passo 3 — Construir a sua primeira imagem

Rodar imagem dos outros é útil. Empacotar a sua é o que muda seu trabalho.

bash

```
mkdir -p ~/docker-primeiros-passos && cd ~/docker-primeiros-passos
```

app.js — um servidor HTTP mínimo, sem dependência externa:

JavaScript

```
// app.js - servidor HTTP com a biblioteca padrão do Node, sem npm install
const http = require('node:http');

const PORTA = process.env.PORT || 3000;
const NOME = process.env.NOME || 'mundo';

const servidor = http.createServer((req, res) => {
  if (req.url === '/saude') { // usado pelo HEALTHCHECK adiante
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ status: 'ok' }));
  }
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('Olá, ${NOME}! Rodando em ${require('node:os').hostname()}\n');
});

// Sem isto, o container ignora o SIGTERM e demora 10s para parar.
process.on('SIGTERM', () => {
  console.log('SIGTERM recebido, encerrando com elegância...');
  servidor.close(() => process.exit(0));
});

servidor.listen(PORTA, '0.0.0.0', () => console.log(`ouvindo na porta ${PORTA}`));
```

'0.0.0.0' e não '127.0.0.1'. Dentro do container, 127.0.0.1 significa "só eu mesmo". Se o app escutar apenas ali, o -p do host não alcança nada e você vê "connection refused" com o container aparentemente saudável. **Esta é a armadilha nº 2 de iniciante** — a nº 1 é inverter a ordem do -p.

Dockerfile — a receita:

Dockerfile

```
# syntax=docker/dockerfile:1
# ^ ativa os recursos modernos do BuildKit; mantenha sempre esta linha

FROM node:22-alpine
# imagem base: Node 22 sobre Alpine Linux (~50 MB contra ~400 MB da variante padrão)

WORKDIR /app
# cria e entra em /app; todos os comandos seguintes rodam a partir daqui

COPY app.js .
# copia do CONTEXTO DE BUILD (a pasta atual, no host) para /app dentro da imagem

USER node
# não rode como root. A imagem oficial do Node já traz o usuário 'node' (UID 1000)

EXPOSE 3000
# DOCUMENTAÇÃO apenas - não publica porta nenhuma. Quem publica é o -p do run

ENV PORT=3000
# valor padrão, sobrescrevível com -e PORT=... no run

CMD ["node", "app.js"]
# comando padrão, na forma EXEC (lista JSON). Nunca use a forma shell aqui - veja abaixo
```

Construa:

bash

```
docker build -t meu-app:1.0 .
```

O que faz: lê o Dockerfile da pasta atual, envia a pasta (o *build context*) ao daemon e produz uma imagem chamada meu-app com a tag 1.0. **O . final é o contexto, não é decoração** — é a pasta cujo conteúdo o COPY pode enxergar.

bash

```
docker images meu-app
# esperado: REPOSITORY meu-app, TAG 1.0, SIZE ~140MB
```

Rode:

bash

```
docker run -d --name app -p 3000:3000 -e NOME="Roni" meu-app:1.0
curl http://localhost:3000
# esperado: Olá, Roni! Rodando em <id hexadecimal>
curl http://localhost:3000/saude
# esperado: {"status":"ok"}
```

Pare e observe a diferença que o tratador de SIGTERM faz:

bash

```
time docker stop app
# esperado: menos de 1 segundo (sem o handler, seriam exatos 10s)
docker rm app
```

4.3.1 Forma exec vs. forma shell — a diferença que custa caro

Dockerfile

```
CMD ["node", "app.js"]      # [ok] forma EXEC: o node vira PID 1 e recebe os sinais
CMD node app.js            # [x] forma SHELL: vira /bin/sh -c "node app.js"
                           #   o PID 1 é o sh, que NÃO repassa SIGTERM ao filho
```

Consequência real: com a forma shell, docker stop sempre demora 10 segundos e mata seu processo à força — sem *graceful shutdown*, sem fechar conexões de banco, sem terminar a requisição em curso. Em produção, isso é erro 502 no usuário a cada deploy.

4.4 Passo 4 — Dados que sobrevivem: volumes

Prove primeiro que o container é descartável:

bash

```
docker run -it --name teste alpine sh
```

sh

```
echo "informação importantíssima" > /dados.txt
cat /dados.txt
exit
```

bash

```
docker rm teste
docker run --rm alpine cat /dados.txt
# esperado: cat: can't open '/dados.txt': No such file or directory
```


O arquivo morreu com o container. Isso é **projeto**, não defeito: a camada de escrita do container é efêmera por definição.

Agora com volume:

```
docker volume create meus-dados
docker run --rm -v meus-dados:/dados alpine sh -c 'echo "agora sobrevive" > /dados/f.txt'
docker run --rm -v meus-dados:/dados alpine cat /dados/f.txt
# esperado: agora sobrevive
```

bash

Dois containers diferentes, ambos já destruídos, e o dado continua lá.

4.4.1 Volume nomeado vs. bind mount — a distinção que organiza tudo

	Volume nomeado	Bind mount
Sintaxe	-v meus-dados:/dados	-v "\$PWD/src:/app/src" (caminho absoluto)
Onde fica	Gerenciado pelo Docker (/var/lib/docker/volumes/)	Numa pasta sua, que você escolhe
Para que serve	Estado de produção: banco de dados, uploads, cache	Desenvolvimento: editar código no host e ver dentro do container
Portabilidade	Funciona igual em qualquer host	Depende do caminho existir naquela máquina
Desempenho em macOS/Windows	Rápido (fica dentro da VM)	Lento (atravessa a fronteira host↔VM)
Permissões	Docker ajusta na criação	Aqui mora a dor: UID do container ≠ UID do host

Use o bind mount agora, para o ciclo de desenvolvimento com recarga:

```
cd ~/docker-primeiros-passos
docker run -d --name dev -p 3000:3000 -v "$PWD:/app" -w /app node:22-alpine \
sh -c "node --watch app.js"
```

bash

O que faz: monta a pasta atual em /app, define /app como diretório de trabalho e roda o Node com --watch, que reinicia sozinho a cada alteração de arquivo. **Nenhuma imagem foi construída** — o código vem do host em tempo real.

Edite o app.js (mude o texto do "Olá") e salve. Depois:

```
docker logs dev          # deve mostrar o Node reiniciando
curl http://localhost:3000 # o texto novo aparece, sem rebuild
docker rm -f dev
```

bash

A regra que organiza a cabeça: bind mount para **código durante o desenvolvimento**; volume nomeado para **dados que precisam viver**. Nunca coloque banco de dados de produção em bind mount, e nunca dependa de bind mount para entregar código em produção — em produção, o código vai **dentro** da imagem.

4.5 Passo 5 — Vários containers juntos: Compose

Aplicação real quase nunca é um container só. Fazer isso com `docker run` exige criar rede à mão, lembrar a ordem, gerenciar nomes. Compose descreve tudo num arquivo.

`compose.yaml` na mesma pasta:

YAML

```
# compose.yaml — não precisa de "version:", ele é obsoleto na especificação atual
services:

  app:
    build: .                # constrói a partir do Dockerfile desta pasta
    ports:
      - "3000:3000"        # host:container
    environment:
      NOME: "mundo containerizado"
      REDIS_URL: "redis://cache:6379" # 'cache' é o NOME DO SERVIÇO abaixo
    depends_on:
      cache:
        condition: service_healthy # só sobe quando o redis estiver realmente pronto
    restart: unless-stopped

  cache:
    image: redis:7-alpine
    volumes:
      - dados-redis:/data      # o estado do redis num volume nomeado
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

volumes:
  dados-redis:                # declara o volume nomeado usado acima
```

bash

```
docker compose up -d
# esperado: linhas "Container <pasta>-cache-1 Healthy" e "<pasta>-app-1 Started"
```

bash

```
docker compose ps
# esperado: dois serviços, ambos "running"; o cache com "(healthy)"
curl http://localhost:3000
# esperado: Olá, mundo containerizado! ...
```

4.5.1 As três coisas que o Compose fez por você

1. **Criou uma rede privada** para o projeto. Os dois containers estão nela e em nenhuma outra.
2. **Deu DNS interno**: dentro do container `app`, o nome `cache` resolve para o IP do Redis. Você **nunca** usa IP no Compose — usa o nome do serviço.
3. **Respeitou a ordem** graças ao `depends_on` + `healthcheck`. Sem o `condition: service_healthy`, o `depends_on` só garante que o container *iniciou*, não que o serviço *está pronto* — distinção que causa muita falha intermitente.

Prove o DNS interno:

bash

```
docker compose exec app sh -c "nslookup cache"
# esperado: um endereço 172.x.x.x — o IP do container do Redis na rede do projeto
```

Comandos do dia a dia com Compose:

```
docker compose up -d      # sobe tudo em segundo plano
docker compose logs -f app # logs de um serviço
docker compose logs -f    # logs de todos, entrelaçados e coloridos
docker compose ps         # o que está de pé
docker compose exec app sh # shell num serviço
docker compose restart app # reinicia um serviço
docker compose up -d --build # reconstrói a imagem e sobe
docker compose down       # derruba tudo e remove a rede (VOLUMES ficam)
docker compose down -v    # △ derruba e APAGA OS VOLUMES – perde os dados
```

bash

4.6 Os cinco primeiros erros de uso (não de instalação)

Instalação já foi tratada no 03 (cap. 3). Estes são os erros do **uso**, na ordem em que aparecem na vida real:

4.6.1 -p invertido → "connection refused"

```
docker run -p 80:8080 nginx # [x] nginx escuta na 80, não na 8080
docker run -p 8080:80 nginx # [ok]
```

bash

Diagnóstico: `docker port <container>` mostra o mapeamento real; `docker logs` mostra em que porta o app disse que subiu.

4.6.2 App escutando em 127.0.0.1 dentro do container → "connection refused" com container saudável

O container tem loopback próprio. 127.0.0.1 lá dentro não é a sua máquina. **Correção:** faça o app escutar em 0.0.0.0. Isso vale para Flask (`--host=0.0.0.0`), Vite (`--host`), Rails (`-b 0.0.0.0`), dotnet (`ASPNETCORE_URLS=http://0.0.0.0:8080`) e todos os outros.

4.6.3 Editar arquivo dentro do container e perder tudo

```
docker exec -it app vi /app/config.json # a edição morre com o container
```

bash

Correção: edite no host e reconstrua a imagem, ou use bind mount durante o desenvolvimento. Container **não é servidor**; é artefato descartável.

4.6.4 docker compose down -v num container de banco

O `-v` apaga volumes. Some o banco inteiro, sem confirmação. **Correção:** `docker compose down` (sem `-v`) para o dia a dia. Reserve `-v` para quando você *quer* zerar. E faça backup:

```
docker run --rm -v dados-redis:/d -v "$PWD:/b" alpine tar czf /b/backup.tgz -C /d .
```

bash

4.6.5 Container "sai imediatamente" (Exited (0)) e você não entende

bash

```
docker run -d ubuntu          # sai na hora
docker ps -a                 # STATUS: Exited (0) 2 seconds ago
```

Causa: um container vive exatamente enquanto o processo PID 1 viver. A imagem ubuntu tem CMD ["bash"]; sem terminal interativo, o bash lê EOF e termina — e o container com ele. **Correção:** dê a ele um processo que fique de pé (nginx, seu app) ou use -it para dar um terminal:

bash

```
docker run -it ubuntu bash    # [ok] agora tem terminal, e o bash fica
```

Diagnóstico geral de container que morre: docker logs <nome> (funciona mesmo com o container parado) e docker inspect --format '{{.State.ExitCode}} {{.State.Error}}' <nome>.

4.7 Limpeza ao terminar

bash

```
docker compose down
docker rm -f $(docker ps -aq) 2>/dev/null || true # remove todos os containers
docker system df               # veja o que sobrou de espaço
```

4.8 Para onde ir agora

Se você quer...	Vá para
Uma referência de comandos para consultar	cap. 5
Mais receitas prontas, do trivial ao de produção	cap. 6
Uma aplicação completa que roda de verdade	cap. 7
Entender o que acontece por baixo	cap. 8
Exercícios com critério de aprovação	cap. 23

4.9 Autoteste

1. Em -p 3000:80, qual número é o do host e qual é o do container? Como você lembra disso?
2. Você roda docker run -d ubuntu e o container morre imediatamente. Explique por quê, em termos de PID 1.
3. Qual é a diferença entre docker stop e docker kill, e em quantos segundos um vira o outro?
4. Seu app responde em localhost:5000 quando roda direto na máquina, mas dá "connection refused" quando containerizado, e o container está de pé. Qual é a hipótese nº 1?
5. Por que CMD node app.js e CMD ["node", "app.js"] produzem comportamentos diferentes ao parar o container?
6. Quando usar volume nomeado e quando usar bind mount? Dê um exemplo de cada.

7. No Compose, como o container app encontra o container cache sem saber o IP dele?
8. Qual é a diferença entre `docker compose down` e `docker compose down -v`, e por que a segunda merece medo?
9. `EXPOSE 3000` no Dockerfile publica a porta? Se não, o que ele faz?

CAPÍTULO 5

Manual de uso — referência consultável

Nível: intermediário · Organizado por tarefa, não por ordem alfabética · Última atualização: 11/08/2026

Referência de bolso. Não se lê de ponta a ponta — se consulta. Cada seção responde a uma pergunta do tipo “como eu faço X”.

5.1 Índice

- 1. Rodar um container
- 2. Inspecionar e diagnosticar
- 3. Entrar e depurar
- 4. Imagens
- 5. Construir imagens
- 6. Instruções do Dockerfile
- 7. Volumes e dados
- 8. Redes
- 9. Registries
- 10. Compose
- 11. Limites de recurso
- 12. Limpeza e espaço em disco
- 13. Contexts e daemon remoto
- 14. Formatação de saída
- 15. Obsoleto — o que não usar mais
- 16. Atalhos que só quem usa há anos conhece

5.2 Rodar um container

bash

```
docker run [OPÇÕES] IMAGEM [COMANDO] [ARG...]
```

5.2.1 Flags essenciais

Flag	O que faz	Quando usar
-d, --detach	Roda em segundo plano	Serviços de longa duração
-it	-i mantém stdin aberto, -t aloca TTY	Sempre que quiser um shell interativo

Flag	O que faz	Quando usar
--rm	Remove o container ao terminar	Sempre , em containers efêmeros e de teste
--name NOME	Nome fixo	Sempre que precisar referenciá-lo depois
-p HOST:CONTAINER	Publica porta	Serviço acessível de fora
-P	Publica todas as portas de EXPOSE em portas altas aleatórias	Testes rápidos
-e VAR=valor	Variável de ambiente	Configuração
--env-file arq	Lê variáveis de um arquivo	Muitas variáveis; evita expor no histórico
-v NOME:/cam	Volume nomeado	Dados persistentes
-v /host:/cam	Bind mount (caminho absoluto)	Código em desenvolvimento
--mount type=...	Sintaxe explícita e verbosa	Scripts e produção; erra menos
-w /cam	Diretório de trabalho	Evita cd no comando
-u UID:GID	Usuário do processo	Corrigir dono de arquivos em bind mount
--network NOME	Conecta a uma rede	Comunicação entre containers
--restart POLÍTICA	no · on-failure[:N] · always · unless-stopped	Serviços que devem voltar sozinhos
--init	Injeta um init (tini) como PID 1	App que não trata sinais nem colhe zumbis
--platform linux/amd64	Força a arquitetura	Imagem sem build para ARM
--pull always	Rebaixa a imagem local e busca do registry	CI, garantir versão fresca
--entrypoint CMD	Substitui o ENTRYPOINT da imagem	Depurar imagem que não abre shell
--health-cmd, --health-interval	Healthcheck sem alterar a imagem	Ajuste pontual

5.2.2 Diferença run × create × start

bash

```
docker create --name x nginx # cria, NÃO inicia
docker start x # inicia um container existente
docker run --name x nginx # equivale a create + start
docker start -ai x # inicia e anexa (interativo)
```

5.2.3 Política de reinício — a tabela que resolve a dúvida

Política	Reinicia se o processo falhar?	Reinicia no boot da máquina?	Reinicia após docker stop manual?
no (padrão)	não	não	não
on-failure[:5]	sim, se exit ≠ 0, até 5 vezes	sim	não
always	sim	sim	sim (surpresa comum)
unless-stopped	sim	sim, exceto se você parou antes	não

Recomendação: unless-stopped para quase tudo. always só quando o container realmente deve voltar mesmo após parada manual — o que raramente é o desejado.

5.3 Inspecionar e diagnosticar

Comando	Para quê
<code>docker ps</code>	Containers rodando
<code>docker ps -a</code>	Todos, inclusive parados — onde estão os que morreram
<code>docker ps -q</code>	Só os IDs (para encadear em outros comandos)
<code>docker ps --filter "status=exited"</code>	Filtrar por estado
<code>docker logs NOME</code>	Saída padrão e de erro do PID 1
<code>docker logs -f --tail 100 NOME</code>	Acompanhar, começando pelas últimas 100 linhas
<code>docker logs --since 10m NOME</code>	Só os últimos 10 minutos
<code>docker logs -t NOME</code>	Com timestamp
<code>docker inspect NOME</code>	Tudo sobre o container, em JSON
<code>docker stats</code>	CPU, memória, rede e I/O ao vivo
<code>docker stats --no-stream</code>	Uma foto, sem ficar atualizando
<code>docker top NOME</code>	Processos do container, vistos do host
<code>docker port NOME</code>	Mapeamento real de portas
<code>docker diff NOME</code>	Arquivos alterados na camada de escrita (A=add, C=change, D=delete)
<code>docker events</code>	Fluxo de eventos do daemon ao vivo
<code>docker system info</code>	Configuração do daemon: driver de storage, cgroup, runtime

5.3.1 Perguntas de diagnóstico e a resposta em uma linha

bash

```
# Por que o container morreu?
docker inspect --format '{{.State.ExitCode}} {{.State.Error}} {{.State.OOMKilled}}' NOME
# OOMKilled=true → estourou o limite de memória

# Qual é o IP interno dele?
docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' NOME

# Que comando ele está rodando de fato?
docker inspect --format '{{.Config.Entrypoint}} {{.Config.Cmd}}' NOME

# Quais variáveis de ambiente ele recebeu?
docker inspect --format '{{range .Config.Env}}{{println .}}{{end}}' NOME

# O que está montado nele?
docker inspect --format '{{range .Mounts}}{{.Type}} {{.Source}} -> {{.Destination}}{{println}}{{end}}' NOME

# O healthcheck está falhando por quê?
docker inspect --format '{{json .State.Health}}' NOME | jq
```

5.3.2 Códigos de saída que dizem alguma coisa

Código	Significado
0	Terminou normalmente. Se era para ser um serviço, o processo estava errado
1	Erro genérico da aplicação — leia os logs
125	Erro do próprio Docker (flag inválida, por exemplo)
126	O comando existe mas não é executável (falta bit de execução)
127	Comando não encontrado (típico de erro de caminho ou binário ausente)
137	SIGKILL (128+9) — quase sempre OOM : confira OOMKilled
143	SIGTERM (128+15) — parada normal via <code>docker stop</code>

5.4 Entrar e depurar

bash

```
docker exec -it NOME sh          # shell num container rodando (funciona em Alpine)
docker exec -it NOME bash        # se a imagem tiver bash
docker exec -u root -it NOME sh  # entrar como root, mesmo que o container rode sem privilégio
docker exec -it NOME env         # ver as variáveis efetivas
docker attach NOME               # anexa ao PID 1 (△ Ctrl+C mata o container; saia com Ctrl-P Ctrl-Q)
```

5.4.1 Quando a imagem não tem shell (distroless, scratch)

bash

```
# Opção 1 – anexar um container de depuração ao namespace do alvo
docker run -it --rm --pid container:NOME --network container:NOME \
  --cap-add SYS_PTRACE nicolaka/netshoot

# Opção 2 – docker debug (Docker Desktop, assinatura paga)
docker debug NOME

# Opção 3 – copiar o sistema de arquivos para fora e olhar
docker export NOME | tar -tv | head -50
```

nicolaka/netshoot é a caixa de ferramentas de rede de fato: traz dig, curl, tcpdump, ss, iperf, nmap. Vale memorizar o nome.

5.4.2 Copiar arquivos

bash

```
docker cp NOME:/app/log.txt ./      # de dentro para fora
docker cp ./config.json NOME:/app/  # de fora para dentro (não persiste após rm!)
```

5.5 Imagens

Comando	Para quê
docker images / docker image ls	Listar
docker images -a	Incluir camadas intermediárias
docker pull IMG:TAG	Baixar
docker push IMG:TAG	Enviar ao registry
docker tag ORIGEM DESTINO	Criar outro nome para a mesma imagem (não copia nada)
docker rmi IMG	Remover
docker image inspect IMG	Metadados completos
docker history IMG	Camadas, tamanho e comando de cada uma — o mapa do que engordou
docker save IMG -o img.tar	Exportar para arquivo (máquina sem rede)
docker load -i img.tar	Importar
docker image prune -a	Remover imagens não usadas

5.5.1 Anatomia do nome de uma imagem

```
registry.exemplo.com:5000/organizacao/nome:tag@sha256:abc...
└── registry ─┬── namespace ┬── nome ┬── tag ┬── digest ─┘
              (padrão: docker.io) (padrão: library) (padrão: latest)
```

Portanto nginx é açúcar sintático para docker.io/library/nginx:latest. Saber disso resolve metade das confusões com registry privado.

```
# Ver o digest imutável de uma imagem
docker image inspect --format '{{index .RepoDigests 0}}' nginx:alpine
```

bash

5.6 Construir imagens

```
docker build -t nome:tag .
docker buildx build -t nome:tag . # mesma coisa, com o motor moderno explícito
```

bash

Flag	O que faz
-t nome:tag	Nomeia. Pode repetir para múltiplas tags
-f caminho/Dockerfile	Dockerfile fora do padrão
--build-arg CHAVE=valor	Passa um ARG para o build (⚠ fica no histórico da imagem — nunca use para segredo)
--target ESTÁGIO	Para num estágio do multi-stage (ex.: --target dev)
--no-cache	Ignora todo o cache
--pull	Rebusca a imagem base
--platform linux/amd64, linux/arm64	Build multi-arquitetura
--progress=plain	Log completo do build, sem a UI que colapsa linhas — essencial para depurar
--secret id=x,src=arq	Segredo montado só durante um RUN, sem ir para a imagem
--ssh default	Repassa o agente SSH para clonar repositório privado
--cache-from / --cache-to	Cache externo (registry), o que faz CI ficar rápido
--output type=local,dest=./ out	Extraí artefatos do build sem gerar imagem

5.6.1 Multi-arquitetura, na prática

```
docker buildx create --name multi --driver docker-container --use # uma vez só
docker buildx build --platform linux/amd64,linux/arm64 \
-t usuario/app:1.0 --push .
```

bash

Multi-arch **exige** --push: o *manifest list* que agrupa as arquiteturas não cabe no armazenamento local antigo. Com o containerd image store (padrão desde a Engine 29), o --load de múltiplas plataformas passa a ser possível.

5.6.2 Cache de build em CI

bash

```
docker buildx build \
--cache-from type=registry,ref=usuario/app:cache \
--cache-to type=registry,ref=usuario/app:cache,mode=max \
-t usuario/app:1.0 --push .
```

5.7 Instruções do Dockerfile

Instrução	O que faz	Cuidado
FROM img:tag AS nome	Imagem base; inicia um estágio	Sempre fixe a tag; AS habilita multi-stage
WORKDIR /cam	Define e cria o diretório de trabalho	Use em vez de RUN cd, que não persiste
COPY origem destino	Copia do contexto de build	Prefira a ADD
COPY --from=estagio a b	Copia de outro estágio	O coração do multi-stage
COPY --chown=user:grp a b	Copia já com o dono certo	Evita uma camada extra de chown
ADD	COPY + extrai tar + baixa URL	Evite: comportamento implícito surpreende
RUN cmd	Executa no build, cria camada	Encadeie com && para reduzir camadas
RUN --mount=type=cache,target=/cam	Cache persistente entre builds	Ouro para npm, pip, apt, go mod
RUN --mount=type=secret,id=x	Segredo temporário	O jeito certo de usar credencial no build
CMD ["a", "b"]	Comando padrão, sobrecrevível	Forma exec (lista JSON), sempre
ENTRYPOINT ["a"]	Executável fixo; o CMD vira argumento	Para imagens que se comportam como um comando
ENV K=V	Variável de ambiente na imagem e nos containers	Fica visível no inspect — não coloque segredo
ARG K=V	Variável só durante o build	Fica no histórico; não é segredo
EXPOSE 3000	Documenta a porta	Não publica nada
USER node	Troca o usuário	Sempre, antes do CMD
VOLUME /dados	Marca um caminho como volume anônimo	Use com parcimônia: cria volumes órfãos
HEALTHCHECK CMD ...	Como saber se está saudável	Habilita condition: service_healthy no Compose
LABEL k=v	Metadados	Use o padrão OCI (org.opencontainers.image.*)
ONBUILD	Dispara instruções na imagem filha	Confuso; evite
STOPSIGNAL SIGQUIT	Sinal de parada	nginx, por exemplo, quer SIGQUIT para sair rápido
SHELL ["pwsh", "-c"]	Troca o shell da forma shell	Raro fora de Windows

5.7.1 CMD × ENTRYPOINT — a tabela que encerra a dúvida

Dockerfile	docker run img executa	docker run img echo oi executa
CMD ["node", "app.js"]	node app.js	echo oi (substitui)
ENTRYPOINT ["node"]	node	node echo oi (acrescenta)
ENTRYPOINT ["node"] + CMD ["app.js"]	node app.js	node echo oi

Regra prática: **ENTRYPOINT = o que a imagem é; CMD = o argumento *padrão***. Uma imagem de ffmpeg deveria ter ENTRYPOINT ["ffmpeg"]; uma imagem de aplicação, só CMD.

5.8 Volumes e dados

```
docker volume create nome
docker volume ls
docker volume inspect nome          # mostra o Mountpoint no host
docker volume rm nome
docker volume prune                 # remove os órfãos △ apaga dados
docker volume ls -f dangling=true   # listar órfãos ANTES de apagar
```

bash

5.8.1 As três formas de montar

```
-v nome:/cam                        # volume nomeado
-v /host/abs:/cam                   # bind mount (o caminho DEVE ser absoluto)
--mount type=volume,source=nome,target=/cam # sintaxe explícita, equivalente à 1ª
--mount type=bind,source=/host/abs,target=/cam,readonly
--mount type=tmpfs,target=/tmp      # em RAM, some ao parar; ideal para segredo
```

bash

5.8.2 Modificadores

Sufixo	Efeito
:ro	Somente leitura — use sempre que o container não precise escrever
:rw	Leitura e escrita (padrão)
:z	Rótulo SELinux compartilhado entre containers (Fedora/RHEL)
:Z	Rótulo SELinux exclusivo deste container
:delegated /	Dicas de consistência no macOS (obsoletas com VirtioFS)
:cached	

5.8.3 Backup e restauração — a receita padrão

```
# Backup
docker run --rm -v MEU_VOLUME:/dados:ro -v "$PWD:/backup" alpine \
  tar czf /backup/volume-$(date +%F).tgz -C /dados .

# Restauração
docker run --rm -v MEU_VOLUME:/dados -v "$PWD:/backup" alpine \
  sh -c "rm -rf /dados/* && tar xzf /backup/volume-2026-08-11.tgz -C /dados"

# Backup lógico de banco (melhor que copiar arquivo do Postgres a quente)
docker compose exec -T db pg_dump -U usuario base | gzip > dump-$(date +%F).sql.gz
```

bash

Nunca faça backup de um volume de Postgres/MySQL copiando os arquivos com o banco rodando. Você captura um estado inconsistente. Use `pg_dump/mysql_dump`, ou pare o container antes.

5.9 Redes

bash

```
docker network ls
docker network create minha-rede
docker network create --driver bridge --subnet 172.28.0.0/16 rede-fixa
docker network inspect minha-rede      # quem está conectado, com quais IPs
docker network connect minha-rede NOME # conecta um container já rodando
docker network disconnect minha-rede NOME
docker network prune
```

5.9.1 Drivers

Driver	O que faz	Quando usar
bridge	Rede virtual isolada, com NAT (padrão)	Praticamente sempre
host	Sem isolamento de rede: usa a pilha do host direto	Desempenho máximo, ou serviço que precisa de todas as portas. ⚠ Só Linux, e -p deixa de funcionar
none	Sem rede	Processamento isolado, batch sem rede
overlay	Rede entre múltiplos hosts	Swarm, cluster
macvlan	O container recebe um MAC e um IP na sua LAN	Aparelhos que precisam ser vistos na rede física (Home Assistant, Pi-hole)

5.9.2 As regras de DNS que você precisa saber

1. Na **rede bridge padrão** (a que você usa sem `--network`), **não há resolução por nome**. Containers só se acham por IP. É legado, e é a causa de “por que ping outro-container não funciona”.
2. Em **qualquer rede criada por você** — inclusive as que o Compose cria — o Docker embute um servidor DNS em `127.0.0.11` e **o nome do container/serviço resolve automaticamente**.
3. `host.docker.internal` resolve para o host, a partir do container. Funciona por padrão no Docker Desktop; no Linux, exige `--add-host=host.docker.internal:host-gateway`.

bash

```
docker run -d --network minha-rede --name api nginx
docker run --rm --network minha-rede alpine ping -c1 api # [ok] funciona
docker run --rm alpine ping -c1 api                     # [x] rede padrão: não resolve
```

5.9.3 Publicação de porta com cuidado

bash

```
-p 8080:80          # escuta em TODAS as interfaces — visível na sua LAN
-p 127.0.0.1:8080:80 # [ok] só na máquina local
-p 8080:80/udp       # UDP
-p 8080-8090:8080-8090 # faixa
```

Cicatriz de campo: -p 5432:5432 num Postgres de desenvolvimento expõe o banco para toda a rede local, e o Docker escreve regras de iptables que **passam por cima** do seu ufw. Um banco com senha fraca em rede de coworking é comprometido em horas. Use -p 127.0.0.1:5432:5432.

5.10 Registries

bash

```
echo "$TOKEN" | docker login -u USUARIO --password-stdin
echo "$TOKEN" | docker login ghcr.io -u USUARIO --password-stdin
docker logout [registry]

docker tag app:1.0 ghcr.io/usuario/app:1.0
docker push ghcr.io/usuario/app:1.0
docker pull ghcr.io/usuario/app:1.0
```

Registry	Endereço	Gratuito para
Docker Hub	docker.io	Público ilimitado; 1 repositório privado no plano gratuito
GitHub Container Registry	ghcr.io	Público ilimitado; privado dentro da cota do GitHub
Quay.io	quay.io	Público ilimitado
GitLab	registry.gitlab.com	Dentro da cota do projeto
AWS ECR / Google AR / Azure ACR	vários	Pago por armazenamento e tráfego
Registry local	registry:2	Você mesmo hospeda

bash

```
# Registry local em 30 segundos (laboratório)
docker run -d -p 5000:5000 --name registry --restart unless-stopped registry:2
docker tag app:1.0 localhost:5000/app:1.0
docker push localhost:5000/app:1.0
```

5.11 Compose

Arquivo padrão: **compose.yaml** (nomes aceitos, em ordem de precedência: compose.yaml, compose.yml, docker-compose.yaml, docker-compose.yml).

Comando	O que faz
docker compose up -d	Sobe tudo em segundo plano
docker compose up -d --build	Reconstrói as imagens antes
docker compose up -d --force-recreate	Recria containers mesmo sem mudança
docker compose down	Para e remove containers e redes (mantém volumes)
docker compose down -v	⚠ + remove volumes
docker compose down --rm all	+ remove imagens
docker compose ps	Estado dos serviços
docker compose logs -f [serviço]	Logs
docker compose exec serviço sh	Shell num serviço rodando
docker compose run --rm serviço cmd	Container novo e efêmero (migrações, testes)
docker compose build [serviço]	Só constrói
docker compose pull	Baixa as imagens declaradas

Comando	O que faz
<code>docker compose restart [serviço]</code>	Reinicia
<code>docker compose stop / start</code>	Para/inicia sem remover
<code>docker compose config</code>	Mostra o YAML final resolvido — o melhor depurador de Compose
<code>docker compose top</code>	Processos de todos os serviços
<code>docker compose watch</code>	Sincroniza arquivos e reconstrói ao salvar (dev)
<code>docker compose -f a.yaml -f b.yaml up</code>	Sobrepõe arquivos (base + ambiente)
<code>docker compose --profile dev up</code>	Sobe só os serviços do perfil

5.11.1 exec × run — a confusão clássica

bash

```
docker compose exec db psql -U app      # usa o container QUE JÁ ESTÁ RODANDO
docker compose run --rm db psql -U app  # cria um container NOVO (e sobe as dependências)
```

Use `exec` para inspecionar o que está no ar; `run --rm` para tarefas pontuais (migração, seed, teste), porque não interfere no serviço em execução.

5.11.2 Campos de `compose.yaml` que valem memorizar

YAML

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
      target: production      # estágio do multi-stage
      args:                  # ARGs do build
        NODE_VERSION: "22"
    image: usuario/api:1.0    # nome da imagem construída
    command: ["node", "server.js"]
    entrypoint: ["/entrypoint.sh"]
    ports: ["127.0.0.1:3000:3000"]
    environment:
      NODE_ENV: production
    env_file: [.env]
    volumes:
      - ./src:/app/src:ro
      - dados:/app/uploads
    depends_on:
      db: { condition: service_healthy }
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/saude"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 40s      # carência inicial: falhas aqui não contam
    restart: unless-stopped
    deploy:
      resources:
        limits: { cpus: "1.0", memory: 512M }
        reservations: { memory: 256M }
    networks: [internal]
    profiles: [prod]
    logging:
      driver: json-file
      options: { max-size: "10m", max-file: "3" }

volumes:
  dados:

networks:
```



```
interna:
internal: true          # sem saída para a internet
```

5.12 Limites de recurso

Flag	Efeito	Observação
-m 512m / --memory	Teto de RAM	Ultrapassar = processo morto pelo OOM killer, exit 137
--memory-reservation 256m	Limite flexível, sob pressão	
--memory-swap 1g	Memória + swap somados	--memory-swap = --memory desliga swap
--oom-kill-disable	Não mata no OOM	⚠ Congela o container. Quase nunca é o que se quer
--cpus 1.5	1,5 núcleo	A forma moderna e legível
--cpu-shares 512	Peso relativo (padrão 1024)	Só vale sob disputa
--cpuset-cpus 0,1	Fixa em núcleos específicos	Latência previsível
--pids-limit 200	Máximo de processos	Defesa barata contra fork bomb
--ulimit nofile=65535: 65535	Descritores de arquivo	Servidores com muitas conexões
--device-read-bps /dev/sda: 10mb	Limita I/O de disco	Exige cgroup v2
--blkio-weight 500	Peso relativo de I/O	

bash

```
docker run -d --name api -m 512m --cpus 1.0 --pids-limit 200 minha-api:1.0
docker stats --no-stream api
# esperado: MEM USAGE / LIMIT mostrando ".../ 512MiB"
```

Regra de produção: todo container deve ter limite de memória. Sem limite, um vazamento numa aplicação derruba o host inteiro e leva junto todos os outros containers. O limite transforma "servidor caiu" em "um container reiniciou".

5.13 Limpeza e espaço em disco

bash

```
docker system df          # panorama; a coluna RECLAIMABLE é a que importa
docker system df -v       # detalhado, por imagem e volume

docker container prune     # containers parados
docker image prune         # imagens dangling (<none>)
docker image prune -a      # + todas as não usadas por container
docker volume prune       # △ volumes órfãos = perda de dados
docker network prune
docker builder prune      # cache de build (costuma ser o maior vilão)
docker builder prune --keep-storage 10GB

docker system prune       # tudo acima, exceto volumes e imagens usadas
docker system prune -a --volumes # △△ arrasa quarteirão

# Filtrar por idade (mais seguro que sair podando tudo)
docker image prune -a --filter "until=168h" # imagens com mais de 7 dias
```

5.13.1 Limitar log — o vazamento silencioso

Por padrão, o driver json-file **não tem limite de tamanho**. Um container falante enche o disco em semanas. Corrija globalmente:

JSON

```
// /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

bash

```
sudo systemctl restart docker # vale para containers criados DEPOIS disso
```

5.14 Contexts e daemon remoto

bash

```
docker context ls
docker context create prod --docker "host=ssh://usuario@servidor.exemplo.com"
docker context use prod
docker ps # lista os containers do servidor remoto
docker context use default
docker --context prod ps # uso pontual, sem trocar o padrão
```

Melhor que DOCKER_HOST porque é nomeado, persistente e aparece no ls. Sobre SSH, não precisa expor o socket do daemon na rede — o que seria entregar root remoto.

5.15 Formatação de saída (Go templates)

bash

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
docker images --format "{{.Repository}}:{{.Tag}}\t{{.Size}}"
docker ps --format json | jq -r '.Names'
docker inspect --format '{{.NetworkSettings.IPAddress}}' NOME
docker ps -a --filter "status=exited" --format "{{.ID}}" | xargs -r docker rm
```

Alias que vale a pena guardar no ~/.bashrc:

bash

```
alias dps='docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"'
alias dclean='docker system prune -f && docker builder prune -f'
alias dsh='f(){ docker exec -it "$1" sh; }; f'
alias dlog='f(){ docker logs -f --tail 100 "$1"; }; f'
```

5.16 Obsoleto — o que não usar mais

Obsoleto	Substituto	Desde
docker-compose (v1, Python, com hífen)	docker compose (v2, plugin Go)	v1 sem suporte desde jul/2023
version: "3.8" no topo do Compose	Nada — o campo é ignorado e gera aviso	Compose Specification
links: entre serviços	Redes + DNS automático	Docker 1.9

Obsoleto	Substituto	Desde
<code>docker run --link</code>	<code>docker network create</code>	Docker 1.9
MAINTAINER no Dockerfile	<code>LABEL org.opencontainers.image.authors="..."</code>	Docker 1.13
ADD para copiar arquivo local	COPY	Sempre foi a recomendação
<code>docker build</code> legado (sem BuildKit)	BuildKit/buildx (padrão desde a Engine 23)	2023
<code>docker rmi/docker rm</code> soltos	<code>docker image rm / docker container rm</code>	Docker 1.13 (os antigos continuam funcionando)
Docker Swarm clássico (standalone)	Swarm mode embutido, ou Kubernetes	2016
latest em produção	Tag semântica + digest	Sempre
Segredo via <code>--build-arg</code>	<code>--mount=type=secret</code>	BuildKit

5.17 Atalhos que só quem usa há anos conhece

bash

```
# Prefixo de ID basta, se for único
docker stop 3f2 # em vez do ID completo de 64 caracteres

# O último container criado
docker logs $(docker ps -lq)
docker rm -f $(docker ps -lq)

# Rodar um comando pontual sem sujar a máquina
docker run --rm -it -v "$PWD:/w" -w /w python:3.13-alpine python script.py
docker run --rm -v "$PWD:/w" -w /w node:22-alpine npx prettier --write .

# Um Postgres descartável para 5 minutos de teste
docker run --rm -d --name pg -e POSTGRES_PASSWORD=x -p 5432:5432 postgres:16
docker exec -it pg psql -U postgres

# Ver o Dockerfile aproximado de uma imagem alheia
docker history --no-trunc --format "{{.CreatedBy}}" IMAGE | tac

# Descobrir por que a imagem está gorda
docker history IMAGE # ou a ferramenta 'dive', que é muito melhor
docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock wagooodman/dive IMAGE

# Entrar no namespace de rede de outro container (depurar rede sem sujar a imagem)
docker run --rm -it --network container:NOME nicolaka/netshoot

# Parar tudo, de uma vez
docker stop $(docker ps -q)

# Achar o container que está comendo a CPU
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}" | sort -k2 -hr

# Ver o processo do container no HOST (prova de que é só um processo)
docker inspect --format '{{.State.Pid}}' NOME
sudo ls -l /proc/$(docker inspect --format '{{.State.Pid}}' NOME)/ns/

# Copiar uma imagem entre máquinas sem registry
docker save img:tag | ssh usuario@destino 'docker load'

# Aquecer o cache de build em CI sem construir
docker buildx build --cache-to type=inline -t app:cache --push .
```

5.18 Autoteste

1. Qual é a diferença entre `--restart always` e `--restart unless-stopped`? Em que cenário ela aparece?
2. Um container saiu com código 137. Qual é a causa mais provável e qual comando confirma?
3. Por que `docker run --link` está obsoleto e o que o substituiu?
4. `ENTRYPOINT ["python"] + CMD ["app.py"]`: o que executa `docker run img teste.py`?
5. Você precisa de um token do npm durante o build. Por que `--build-arg` é errado e o que usar?
6. Na rede bridge **padrão**, `ping` outro-container funciona? E numa rede criada por você?
7. Qual é a diferença entre `docker compose exec` e `docker compose run`, e quando cada um?
8. Qual comando mostra o YAML final do Compose com todas as sobreposições resolvidas?
9. Por que `-p 5432:5432` num Postgres de desenvolvimento é perigoso, e qual é a forma segura?
10. Cite três comandos que apagam dados sem pedir confirmação.

CAPÍTULO 6

Exemplos — 14 receitas completas e executáveis

Nível: intermediário · Todo código é completo – nada de "...". Última atualização: 11/08/2026

Cada exemplo tem: **problema** → **solução** → **explicação**. Os quatro últimos são casos reais de produção, não didáticos.

#	Exemplo	Nível
1	Banco descartável em 30 segundos	trivial
2	Rodar ferramenta sem instalar	trivial
3	Site estático com nginx	trivial
4	Node multi-stage com cache	básico
5	Python enxuto	básico
6	Go em imagem de 8 MB	intermediário
7	Dev com recarga automática	intermediário
8	Stack completa com Compose	intermediário
9	Migração antes do boot	intermediário
10	Segredo no build	avançado
11	Multi-arch amd64 + arm64	avançado
12	Produção: CI/CD completo	avançado
13	Produção: backup com retenção	avançado
14	Produção: proxy reverso com TLS	avançado

6.1 Banco de dados descartável para testar uma consulta

Problema: você quer testar uma query em Postgres 16, mas não quer instalar Postgres na sua máquina nem sujar o banco de desenvolvimento.

```
docker run --rm -d --name pg-teste \
-e POSTGRES_PASSWORD=segredo \
-e POSTGRES_DB=laboratorio \
-p 127.0.0.1:5432:5432 \
postgres:16-alpine

# Espere ficar pronto (o Postgres reinicia uma vez durante a inicialização)
until docker exec pg-teste pg_isready -U postgres > /dev/null 2>&1; do sleep 1; done
echo "pronto"

docker exec -it pg-teste psql -U postgres -d laboratorio -c "
CREATE TABLE pedidos (id serial PRIMARY KEY, cliente text, valor numeric);
INSERT INTO pedidos (cliente, valor) VALUES ('ana', 100), ('bruno', 250), ('ana', 75);
SELECT cliente, sum(valor) AS total FROM pedidos GROUP BY cliente ORDER BY total DESC;
"

docker stop pg-teste # --rm remove sozinho; não sobra nada
```

Saída esperada:

```

cliente | total
-----+-----
bruno   |    250
ana     |    175

```

Explicação. `--rm` faz o container se autodestruir ao parar. Como **não** há volume, os dados morrem junto — que é exatamente o desejado num laboratório. 127.0.0.1:5432:5432 publica a porta apenas no loopback: sem isso, o banco ficaria visível para toda a rede local. O laço com `pg_isready` existe porque a imagem do Postgres reinicia internamente durante a primeira inicialização; conectar cedo demais dá “connection refused” enganoso.

6.2 Rodar uma ferramenta sem instalá-la

Problema: você precisa de `jq`, `imagemagick`, uma versão específica do Node, ou o `awscli` — e não quer poluir a máquina.

bash

```

# jq, sem instalar jq
echo '{"nome":"docker","ano":2013}' | docker run --rm -i ghcr.io/jqlang/jq '.nome'
# esperado: "docker"

# Redimensionar imagens com ImageMagick, sem instalar ImageMagick
docker run --rm -v "$PWD:/img" -w /img dpokidov/imagick \
  -resize 800x600 foto.jpg foto-menor.jpg

# Node 18 numa máquina onde só existe o Node 22
docker run --rm -it -v "$PWD:/app" -w /app node:18-alpine npm test

# Python 3.11 com as dependências do projeto, sem virtualenv
docker run --rm -it -v "$PWD:/app" -w /app python:3.11-slim \
  sh -c "pip install -q -r requirements.txt && python main.py"

# Escanear vulnerabilidades sem instalar o Trivy
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image minha-app:1.0

```

Explicação. Este é o uso do Docker que muda o dia a dia mais rápido do que qualquer outro: a máquina fica limpa e cada ferramenta vem na versão exata. O padrão é sempre o mesmo: `--rm + -v "$PWD:/app" + -w /app`.

Armadilha: arquivos criados por esses containers pertencem ao **root** do host, porque o processo lá dentro roda como root. Corrija com `--user "$(id -u):$(id -g)"`:

bash

```

docker run --rm --user "$(id -u):$(id -g)" -v "$PWD:/app" -w /app node:22-alpine npm init -y
ls -l package.json      # dono: você, não root

```

6.3 Servir um site estático

Problema: publicar uma pasta com HTML/CSS/JS.

site/index.html:

HTML

```

<!doctype html>
<html lang="pt-BR">
  <head><meta charset="utf-8"><title>Meu site</title></head>
  <body><h1>Funcionando em container</h1></body>

```

</html>

Modo rápido (desenvolvimento, sem construir imagem):

bash

```
docker run --rm -d --name site -p 8080:80 -v "$PWD/site:/usr/share/nginx/html:ro" nginx:alpine
curl -s localhost:8080 | grep h1
# esperado: <h1>Funcionando em container</h1>
docker stop site
```

Modo imagem (o que vai para produção):

Dockerfile:

Dockerfile

```
# syntax=docker/dockerfile:1
FROM nginx:1.27-alpine
COPY site/ /usr/share/nginx/html/
COPY nginx.conf /etc/nginx/conf.d/default.conf
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost/ || exit 1
```

nginx.conf:

nginx

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;

    # SPA: qualquer rota desconhecida devolve o index.html
    location / { try_files $uri $uri/ /index.html; }

    # Assets com hash no nome podem ser cacheados para sempre
    location ~* \.(js|css|png|jpg|svg|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

bash

```
docker build -t meu-site:1.0 .
docker run --rm -d --name site -p 8080:80 meu-site:1.0
curl -sI localhost:8080 | head -1 # esperado: HTTP/1.1 200 OK
docker stop site
```

Explicação. O bind mount com `:ro` serve para desenvolvimento — o conteúdo vem do host e você vê a mudança na hora. A imagem serve para produção: o conteúdo está **dentro** dela, e por isso é reproduzível e distribuível. Nunca dependa de bind mount em produção.

6.4 Dockerfile Node com multi-stage e cache eficiente

Problema: o build demora 4 minutos porque reinstala todas as dependências a cada mudança de uma linha de código. E a imagem final tem 1,2 GB.

Dockerfile

```
# syntax=docker/dockerfile:1

# ----- Estágio 1: dependências de produção -----
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm \
    npm ci --omit=dev

# ----- Estágio 2: build (precisa das devDependencies) -----
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm \
    npm ci
COPY . .
RUN npm run build

# ----- Estágio 3: imagem final -----
FROM node:22-alpine AS production
ENV NODE_ENV=production
WORKDIR /app

RUN apk add --no-cache tini          # init de verdade: repassa sinais e colhe zumbis

COPY --from=deps --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist
COPY --chown=node:node package.json ./

USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s \
    CMD wget -qO- http://localhost:3000/saude || exit 1

ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "dist/server.js"]
```

.dockerignore — tão importante quanto o Dockerfile:

```
node_modules
dist
.git
.env
*.log
coverage
.vscode
Dockerfile
.dockerignore
README.md
```

bash

```
docker build -t api:1.0 .
docker images api:1.0
# esperado: SIZE em torno de 150-200MB, contra ~1.2GB sem multi-stage
```

Explicação — as quatro decisões que importam:

1. **COPY package*.json antes de COPY . .** O cache do Docker invalida uma camada e **todas as seguintes**. Copiando só o manifesto primeiro, o `npm ci` só refaz quando as dependências mudam de verdade. Sem isso, cada `git commit` custa 4 minutos.
2. **--mount=type=cache** mantém o cache do `npm` **entre builds diferentes**, sem que ele entre na imagem. É a diferença entre baixar da rede e ler do disco.
3. **Multi-stage**. As `devDependencies`, o compilador TypeScript e o código-fonte ficam no estágio de build e **não vão** para a imagem final — nem em camada escondida.

Menos tamanho e menos superfície de ataque.

4. **USER node + tini.** Não rodar como root é higiene básica; o tini garante que SIGTERM chegue ao processo e que processos zumbis sejam colhidos.

6.5 Python com uv e imagem enxuta

Problema: imagem Python de 1 GB e pip install lento a cada build.

Dockerfile

```
# syntax=docker/dockerfile:1

FROM python:3.13-slim AS base
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1

# ----- build -----
FROM base AS build
COPY --from=ghcr.io/astral-sh/uv:latest /usr/local/bin/uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync --frozen --no-dev

# ----- runtime -----
FROM base AS runtime
RUN useradd --create-home --uid 1000 app
WORKDIR /app
COPY --from=build --chown=app:app /app/.venv /app/.venv
COPY --chown=app:app src/ ./src/
ENV PATH="/app/.venv/bin:$PATH"
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD python -c \
    "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/saude').status==200 else 1)"
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Explicação.

- PYTHONUNBUFFERED=1 é **obrigatório** em container: sem ele, o Python bufferiza a saída e seus print/logs só aparecem quando o buffer enche — ou nunca, se o processo morre. Causa clássica de “meu container não tem logs”.
- python:3.13-slim (~120 MB) contra python:3.13 (~1 GB). Só use a variante completa se precisar compilar extensões C **em tempo de execução** — o que quase nunca é o caso.
- uv (escrito em Rust) resolve e instala dependências uma ordem de grandeza mais rápido que o pip. Copiado direto de uma imagem oficial, sem instalação.
- --host 0.0.0.0 no uvicorn: com o padrão 127.0.0.1, nada de fora do container o alcança.

6.6 Go: imagem de 8 MB com scratch

Problema: você quer a menor imagem possível e a menor superfície de ataque possível.

Dockerfile

```
# syntax=docker/dockerfile:1

FROM golang:1.23-alpine AS build
```

```

WORKDIR /src
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download
COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o /app ./cmd/servidor

FROM scratch
# scratch é literalmente vazio: sem shell, sem libc, sem /etc/passwd
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=build /app /app
USER 65534:65534      # nobody — precisa ser numérico: não há /etc/passwd para resolver nome
EXPOSE 8080
ENTRYPOINT ["/app"]

```

bash

```

docker build -t servidor-go:1.0 .
docker images servidor-go:1.0
# esperado: SIZE entre 8MB e 15MB

```

Explicação.

- `CGO_ENABLED=0` produz um binário **estático**, sem depender da `libc` — pré-requisito para rodar em `scratch`.
- `-ldflags="-s -w"` remove tabela de símbolos e informação de debug: 20-30% menor.
- A cópia dos **certificados raiz** é obrigatória se o programa fizer HTTPS. Esquecer isso gera `x509: certificate signed by unknown authority` para qualquer site — erro que confunde porque parece problema de rede.
- `USER 65534:65534` precisa ser numérico, porque não existe `/etc/passwd` para traduzir `nobody`.

Trade-off honesto: em `scratch` você não tem shell, então `docker exec -it x sh` não funciona e depurar em produção fica difícil. Alternativa equilibrada: `gcr.io/distroless/static-debian12` — quase tão pequena, com certificados e usuário `nonroot` já prontos, e uma variante `:debug` com shell para emergências.

6.7 Ambiente de desenvolvimento com recarga automática

Problema: você quer editar o código no seu editor e ver a mudança no container imediatamente, sem reconstruir a imagem.

`compose.yaml`:

YAML

```

services:
  api:
    build:
      context: .
      target: dev          # estágio dedicado, com as devDependencies
    ports:
      - "127.0.0.1:3000:3000"
      - "127.0.0.1:9229:9229" # porta do depurador do Node
    environment:
      NODE_ENV: development
    volumes:
      - ./src:/app/src          # código do host, ao vivo
      - ./package.json:/app/package.json:ro
      - /app/node_modules       # △ volume anônimo: PROTEGE os node_modules da imagem
    command: ["node", "--watch", "--inspect=0.0.0.0:9229", "src/server.js"]

```

Estágio dev no Dockerfile:

Dockerfile

```
FROM node:22-alpine AS dev
WORKDIR /app
COPY package*.json ./
RUN npm ci                # inclui devDependencies
COPY . .
CMD ["node", "--watch", "src/server.js"]
```

bash

```
docker compose up
# edite src/server.js e salve → o Node reinicia sozinho, sem rebuild
```

Explicação — a linha - /app/node_modules é a mais importante e a menos óbvia.

Sem ela, se você tiver uma pasta node_modules no host (ou nenhuma), o bind mount de ./ a cobriria e apagaria a que foi instalada na imagem — quebrando o container com Cannot find module. Declarar /app/node_modules como **volume anônimo** faz o Docker montar algo por cima daquele caminho, protegendo o conteúdo vindo da imagem.

Isso importa especialmente quando o host e o container têm arquiteturas ou sistemas operacionais diferentes: pacotes com binários nativos (bcrypt, sharp, sqlite3) compilados no macOS **não funcionam** dentro do container Linux.

Alternativa moderna: docker compose watch faz sincronização seletiva e declarativa, sem bind mount de tudo:

YAML

```
develop:
  watch:
    - action: sync          # copia o arquivo para dentro
      path: ./src
      target: /app/src
    - action: rebuild       # mudou dependência → reconstrói a imagem
      path: package.json
```

6.8 Stack completa: API + Postgres + Redis + nginx

Problema: subir uma aplicação inteira com um comando, com ordem de inicialização correta, rede isolada e o banco inacessível de fora.

compose.yaml:

YAML

```
services:

  proxy:
    image: nginx:1.27-alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      api: { condition: service_healthy }
    networks: [publica, interna]
    restart: unless-stopped

  api:
    build: .
    environment:
      DATABASE_URL: postgres://app:${DB_SENHA:?defina DB_SENHA no .env}@db:5432/app
      REDIS_URL: redis://cache:6379
```

```

    NODE_ENV: production
  depends_on:
    db: { condition: service_healthy }
    cache: { condition: service_healthy }
  healthcheck:
    test: ["CMD", "wget", "-q0-", "http://localhost:3000/saude"]
    interval: 10s
    timeout: 3s
    retries: 5
    start_period: 20s
  networks: [interna]          # NÃO está na rede pública: só o proxy a alcança
  restart: unless-stopped
  deploy:
    resources:
      limits: { memory: 512M, cpus: "1.0" }

db:
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: app
    POSTGRES_PASSWORD: ${DB_SENHA:?defina DB_SENHA no .env}
    POSTGRES_DB: app
  volumes:
    - dados-db:/var/lib/postgresql/data
    - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U app -d app"]
    interval: 5s
    timeout: 3s
    retries: 10
  networks: [interna]
  restart: unless-stopped
  # SEM 'ports:' – o banco NÃO é acessível de fora da rede do Compose

cache:
  image: redis:7-alpine
  command: ["redis-server", "--maxmemory", "256mb", "--maxmemory-policy", "allkeys-lru"]
  volumes:
    - dados-redis:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 3s
    retries: 5
  networks: [interna]
  restart: unless-stopped

volumes:
  dados-db:
  dados-redis:

networks:
  publica:
  interna:
    internal: true              # esta rede não tem rota para a internet

```

nginx.conf:

nginx

```

upstream api { server api:3000; }

server {
  listen 80;

  location / {
    proxy_pass http://api;
    proxy_set_header Host          $host;
    proxy_set_header X-Real-IP     $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}

```

```
location /saude { access_log off; proxy_pass http://api/saude; }
}
```

.env (e .env no .gitignore):

```
DB_SENHA=troque-esta-senha
```

bash

```
docker compose up -d
docker compose ps      # todos "running", db e cache "(healthy)"
curl -s localhost/saude # esperado: {"status":"ok"}
docker compose down     # volumes preservados
```

bash

Explicação — as cinco decisões de arquitetura:

1. **Duas redes.** publica só tem o proxy; interna (com `internal: true`) não tem rota para a internet. Se a API for comprometida, o atacante não consegue nem exfiltrar dados por HTTP.
2. **O banco não tem ports:.** Só quem está na rede interna o alcança. Precisa acessar de fora para depurar? Use `docker compose exec db psql -U app`.
3. **condition: service_healthy** em vez de `depends_on` simples. Sem isso, a API tenta conectar num Postgres que ainda está inicializando e morre no primeiro segundo.
4. `${DB_SENHA:?mensagem}` faz o Compose **falhar imediatamente** se a variável não estiver definida, em vez de subir com senha vazia. Muito melhor que descobrir isso em produção.
5. **start_period** no healthcheck da API: durante a carência, falhas não contam como unhealthy. Sem isso, aplicações com boot lento entram em laço de reinício.

6.9 Migração de banco antes de subir a API

Problema: o esquema do banco precisa estar atualizado **antes** de a API começar a responder, e a migração não pode rodar duas vezes em paralelo.

```
services:
  migracao:
    build: .
    command: ["npm", "run", "migrate:up"]
    environment:
      DATABASE_URL: postgres://app:${DB_SENHA}@db:5432/app
    depends_on:
      db: { condition: service_healthy }
    restart: "no" # roda uma vez e termina; NÃO reinicie
    networks: [interna]

  api:
    build: .
    depends_on:
      migracao: { condition: service_completed_successfully }
      db: { condition: service_healthy }
    networks: [interna]
```

YAML

bash

```
docker compose up -d
# ordem garantida: db saudável → migração termina com exit 0 → api sobe
docker compose logs migracao
```

Explicação. `service_completed_successfully` é a condição menos conhecida e mais útil do `depends_on`: o serviço dependente só sobe se o container de migração terminar com **código de saída 0**. Se a migração falhar, a API não sobe — que é o comportamento correto, porque uma API contra esquema errado corrompe dados em silêncio.

Anti-padrão comum a evitar: rodar a migração no `ENTRYPOINT` da própria API. Com duas réplicas, duas migrações concorrentes brigam pelo mesmo lock. Migração é um passo separado, e idealmente um passo do *deploy*, não do *boot*.

6.10 Segredo no build sem vaziar na imagem

Problema: o build precisa de um token (npm privado, repositório Git privado) e esse token **não pode** ficar na imagem.

✗ **O jeito errado, que quase todo mundo faz primeiro:**

Dockerfile

```
ARG NPM_TOKEN
RUN echo "registry.npmjs.org/:_authToken=${NPM_TOKEN}" > .npmrc && npm ci && rm .npmrc
```

Apagar o arquivo **não adianta**: a camada anterior, que o continha, permanece na imagem. Prove:

bash

```
docker history --no-trunc IMAGE | grep -i token # o token está lá
docker save IMAGE | tar -x -O | grep -a _authToken # e no conteúdo das camadas, também
```

✓ **O jeito certo — `--mount=type=secret`:**

Dockerfile

```
# syntax=docker/dockerfile:1
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./

RUN --mount=type=secret,id=npmc,target=/root/.npmrc,mode=0400 \
    --mount=type=cache,target=/root/.npm \
    npm ci --omit=dev

COPY . .
RUN npm run build
```

bash

```
docker buildx build --secret id=npmc,src=$HOME/.npmrc -t app:1.0 .

# Prova de que não vazou:
docker history --no-trunc app:1.0 | grep -ci token # esperado: 0
```

Para clonar um repositório Git privado:

Dockerfile

```
RUN --mount=type=ssh \
    git clone git@github.com:org/repo-privado.git /app/vendor
```

bash

```
docker buildx build --ssh default -t app:1.0 .
```

Explicação. O `--mount=type=secret` monta o arquivo num tmpfs que existe **só durante aquele RUN**, e nunca vira camada. Já `ARG/--build-arg` fica gravado no histórico da imagem e é recuperável por qualquer um que baixe a imagem — inclusive de um registry público. Isto é uma das causas mais frequentes de vazamento de credencial em imagens públicas.

Regra: `ARG` para configuração (versão, arquitetura); `--mount=type=secret` para segredo. **Nunca** o contrário.

6.11 Build multi-arquitetura (amd64 + arm64)

Problema: a equipe usa MacBook com Apple Silicon (ARM64) e o servidor é x86-64. Uma imagem só precisa servir aos dois.

bash

```
# Uma vez: cria um builder capaz de emular outras arquiteturas
docker buildx create --name multi --driver docker-container --bootstrap --use
docker buildx inspect --bootstrap | grep Platforms
# esperado: linux/amd64, linux/arm64, ... (a emulação vem via QEMU/binfmt)

docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t usuario/app:1.0 \
  --push .

# Verifique o manifest list gerado
docker buildx imagetools inspect usuario/app:1.0
# esperado: duas entradas, uma por plataforma
```

Para builds nativos rápidos, use `TARGETARCH` e evite a emulação na etapa cara:

Dockerfile

```
# syntax=docker/dockerfile:1
FROM --platform=$BUILDPLATFORM golang:1.23-alpine AS build
ARG TARGETOS TARGETARCH
WORKDIR /src
COPY . .
# compila NATIVAMENTE para o alvo – cross-compilação, sem QEMU
RUN CGO_ENABLED=0 GOOS=$TARGETOS GOARCH=$TARGETARCH go build -o /app ./cmd/servidor

FROM alpine:3.20
COPY --from=build /app /app
ENTRYPOINT ["/app"]
```

Explicação. `--platform=$BUILDPLATFORM` no estágio de build significa “compile na arquitetura **da máquina que está construindo**”, e `GOARCH=$TARGETARCH` gera o binário para a arquitetura **de destino**. Isso troca emulação QEMU (lenta, às vezes 10× mais) por cross-compilação nativa. Linguagens que cross-compilam bem (Go, Rust, Zig) ganham muito com esse padrão; linguagens interpretadas, nem tanto.

Armadilha: builds ARM64 emulados de projetos Node/Python com dependências nativas podem levar **20+ minutos** ou simplesmente falhar. Se puder, use runners nativos de cada arquitetura no CI e junte os manifests no fim (`docker buildx imagetools create`).

6.12 PRODUÇÃO — pipeline de CI completo no GitHub Actions

Problema real: a cada push na main, construir a imagem multi-arch, escanear vulnerabilidades, gerar SBOM, assinar, publicar no GHCR e fazer o deploy — falhando o pipeline se houver vulnerabilidade crítica.

.github/workflows/build.yml:

YAML

```
name: build-e-publica

on:
  push:
    branches: [main]
    tags: ['v*']
  pull_request:

env:
  REGISTRY: ghcr.io
  IMAGE: ${ github.repository }

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
      id-token: write          # necessário para assinatura keyless com cosign
      security-events: write  # para enviar o relatório ao GitHub Security

    steps:
      - uses: actions/checkout@v4

      - name: Configura QEMU (emulação para arm64)
        uses: docker/setup-qemu-action@v3

      - name: Configura Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login no GHCR
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Calcula tags e labels
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${ env.REGISTRY }/${ env.IMAGE }
          tags: |
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=sha,format=long
            type=ref,event=branch

      - name: Constrói e publica
        id: build
        uses: docker/build-push-action@v6
        with:
          context: .
          platforms: linux/amd64,linux/arm64
          push: ${ github.event_name != 'pull_request' }
          tags: ${ steps.meta.outputs.tags }
          labels: ${ steps.meta.outputs.labels }
          cache-from: type=gha
          cache-to: type=gha,mode=max
          provenance: true          # atestado SLSA de proveniência
```



```

    sbom: true          # lista de materiais de software

- name: Escaneia vulnerabilidades
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.REGISTRY }}/${ env.IMAGEM }}:sha-${ github.sha }}
    format: sarif
    output: trivy.sarif
    severity: CRITICAL,HIGH
    exit-code: '1'      # falha o pipeline se houver CRITICAL ou HIGH
    ignore-unfixed: true # não falhe por CVE sem correção disponível

- name: Envia o relatório para o GitHub Security
  if: always()
  uses: github/codeql-action/upload-sarif@v3
  with: { sarif_file: trivy.sarif }

- name: Instala cosign
  if: github.event_name != 'pull_request'
  uses: sigstore/cosign-installer@v3

- name: Assina a imagem (keyless, via OIDC)
  if: github.event_name != 'pull_request'
  run: |
    cosign sign --yes \
      ${ env.REGISTRY }}/${ env.IMAGEM }}@${ steps.build.outputs.digest }}

```

Deploy no servidor, verificando a assinatura antes de subir:

```

#!/usr/bin/env bash
# deploy.sh — executado no servidor
set -euo pipefail

IMAGE="ghcr.io/org/app"
DIGEST="$1"          # passado pelo pipeline

cosign verify "${IMAGE}@${DIGEST}" \
  --certificate-identity-regexp "https://github.com/org/app/.github/workflows/.*" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com" > /dev/null

echo "IMAGE=${IMAGE}@${DIGEST}" > /opt/app/.env
docker compose -f /opt/app/compose.yaml pull
docker compose -f /opt/app/compose.yaml up -d --wait # --wait espera ficar healthy
docker image prune -af --filter "until=168h"

```

Explicação — por que cada peça está aí:

- **cache-from/to: type=gha** usa o cache do GitHub Actions entre execuções. Build de 6 minutos cai para menos de 1 em mudanças pequenas.
- **ignore-unfixed: true** no Trivy é pragmatismo: falhar o pipeline por uma CVE sem correção publicada apenas ensina a equipe a ignorar o scanner.
- **provenance e sbom** geram atestados anexados à imagem no registry — cada vez mais exigidos por política de conformidade (e pelo *EU Cyber Resilience Act*).
- **Assinatura keyless com cosign** usa OIDC do próprio GitHub: **não há chave privada para vazar**. A verificação no deploy garante que a imagem foi construída por aquele workflow daquele repositório, e não empurrada à mão por alguém.
- **Deploy por digest, não por tag**. Tag é mutável; digest é o hash do conteúdo. Deploy por tag é como fazer deploy de “a versão que estiver lá quando o servidor puxar”.
- **--wait** no `compose up` faz o comando retornar somente quando os healthchecks passarem — transformando um deploy silenciosamente quebrado em um deploy que falha alto.

6.13 PRODUÇÃO — backup automatizado de Postgres com retenção

Problema real: backup diário do banco, com retenção de 7 diários + 4 semanais, enviado para armazenamento externo, com verificação de que o dump não está corrompido.

compose.yaml (trecho):

```
backup:
  image: postgres:16-alpine
  depends_on:
    db: { condition: service_healthy }
  environment:
    PGPASSWORD: ${DB_SENHA:?}
  volumes:
    - ./backups:/backups
    - ./backup.sh:/backup.sh:ro
  entrypoint: ["/bin/sh", "/backup.sh"]
  networks: [internal]
  restart: unless-stopped
```

YAML

backup.sh:

```
#!/bin/sh
# Backup do Postgres com retenção. Roda em laço; sem cron, sem daemon extra.
set -eu

DESTINO=/backups
RETENCAO_DIARIA=7
RETENCAO_SEMANAL=4
INTERVALO=86400      # 24h

mkdir -p "$DESTINO/diario" "$DESTINO/semanal"

fazer_backup() {
  carimbo=$(date +%Y-%m-%d_%H%M%S)
  arquivo="$DESTINO/diario/app_${carimbo}.sql.gz"

  echo "[$(date -Iseconds)] iniciando backup -> $arquivo"

  # -Fc seria o formato custom; usamos SQL puro comprimido por ser mais portátil
  if ! pg_dump -h db -U app -d app --no-owner --no-acl | gzip -9 > "$arquivo.parcial"; then
    echo "[$(date -Iseconds)] ERRO: pg_dump falhou" >&2
    rm -f "$arquivo.parcial"
    return 1
  fi

  # Verificação: o gzip está íntegro e o dump não está vazio?
  if ! gzip -t "$arquivo.parcial"; then
    echo "[$(date -Iseconds)] ERRO: arquivo corrompido" >&2
    rm -f "$arquivo.parcial"
    return 1
  fi

  tamanho=$(wc -c < "$arquivo.parcial")
  if [ "$tamanho" -lt 1024 ]; then
    echo "[$(date -Iseconds)] ERRO: dump suspeito de vazio (${tamanho} bytes)" >&2
    rm -f "$arquivo.parcial"
    return 1
  fi

  # Só agora vira definitivo — evita backup pela metade sendo tomado por válido
  mv "$arquivo.parcial" "$arquivo"
  echo "[$(date -Iseconds)] ok: $(du -h "$arquivo" | cut -f1)"

  # Domingo vira backup semanal
  if [ "$(date +%u)" = "7" ]; then
    cp "$arquivo" "$DESTINO/semanal/"
```

sh

```
fi

# Retenção
ls -lt "$DESTINO/diario"/*.sql.gz 2>/dev/null | tail -n +${(RETENCAO_DIARIA + 1)} | xargs -r rm -f
ls -lt "$DESTINO/semanal"/*.sql.gz 2>/dev/null | tail -n +${(RETENCAO_SEMANAL + 1)} | xargs -r rm -f
}

# Encerramento limpo quando o container for parado
trap 'echo "encerrando"; exit 0' TERM INT

while true; do
    fazer_backup || echo "[$(date -Iseconds)] backup falhou; tentarei de novo no próximo ciclo" >&2
    sleep "$INTERVALO" &
    wait $!          # 'wait' permite que o trap interrompa o sleep imediatamente
done
```

Teste de restauração — um backup nunca testado não é um backup:

```
# Sobe um Postgres limpo e restaura o dump mais recente
docker run --rm -d --name pg-restauracao -e POSTGRES_PASSWORD=x -e POSTGRES_DB=app postgres:16-alpine
until docker exec pg-restauracao pg_isready -U postgres >/dev/null 2>&1; do sleep 1; done

gunzip -c ./backups/diario/$(ls -lt ./backups/diario | head -1) \
| docker exec -i pg-restauracao psql -U postgres -d app

docker exec pg-restauracao psql -U postgres -d app -c "\dt"          # as tabelas voltaram?
docker exec pg-restauracao psql -U postgres -d app -c "SELECT count(*) FROM pedidos;"
docker stop pg-restauracao
```

Explicação — as decisões que separam backup de teatro de backup:

1. **Escreve em .parcial e só depois renomeia.** Se o processo morrer no meio, não fica um arquivo truncado parecendo válido. `mv` no mesmo sistema de arquivos é atômico.
2. **Verifica integridade e tamanho mínimo.** Um `pg_dump` que falha por autenticação pode produzir um `.gz` válido de 20 bytes. Sem a checagem de tamanho, o alarme só toca no dia do desastre.
3. **pg_dump, não cópia de arquivo.** Copiar `/var/lib/postgresql/data` com o banco rodando captura um estado inconsistente que pode não restaurar.
4. **trap + wait \$!** fazem o `docker stop` funcionar em menos de um segundo, em vez de esperar o `sleep` de 24h ser morto à força.
5. **O teste de restauração é parte do procedimento,** não uma boa intenção. A regra 3-2-1 (3 cópias, 2 mídias, 1 fora do local) exige ainda enviar esses arquivos para fora do servidor — `rclone`, `restic` ou `aws s3 sync` num segundo container.

6.14 PRODUÇÃO — proxy reverso com TLS automático para vários serviços

Problema real: vários serviços em um servidor, cada um num subdomínio, todos com HTTPS válido e renovação automática, sem editar configuração de `nginx` a cada serviço novo.

`compose.yaml`:

```
services:

  caddy:
```

YAML

```

image: caddy:2-alpine
ports:
  - "80:80"
  - "443:443"
  - "443:443/udp"          # HTTP/3
volumes:
  - ./Caddyfile:/etc/caddy/Caddyfile:ro
  - caddy-dados:/data      # ⚠ os certificados vivem AQUI. Perder = re-emitir tudo
  - caddy-config:/config
networks: [borda]
restart: unless-stopped

app:
image: ghcr.io/org/app:1.4.2
environment:
  DATABASE_URL: postgres://app:${DB_SENHA:?}@db:5432/app
depends_on:
  db: { condition: service_healthy }
networks: [borda, interna]
restart: unless-stopped
deploy:
  replicas: 2              # o Caddy balanceia entre as réplicas via DNS
resources:
  limits: { memory: 512M }

painel:
image: grafana/grafana:11.1.0
environment:
  GF_SERVER_ROOT_URL: https://painel.exemplo.com
  GF_AUTH_ANONYMOUS_ENABLED: "false"
volumes:
  - grafana-dados:/var/lib/grafana
networks: [borda]
restart: unless-stopped

db:
image: postgres:16-alpine
environment:
  POSTGRES_USER: app
  POSTGRES_PASSWORD: ${DB_SENHA:?}
  POSTGRES_DB: app
volumes:
  - dados-db:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U app"]
  interval: 5s
  retries: 10
networks: [interna]
restart: unless-stopped

volumes:
  caddy-dados:
  caddy-config:
  grafana-dados:
  dados-db:

networks:
  borda:
  interna:
    internal: true

```

Caddyfile:

Caddyfile

```

{
  email admin@exemplo.com    # usado pelo Let's Encrypt para avisos de expiração
}

app.exemplo.com {
  reverse_proxy app:3000 {
    health_uri      /saude
  }
}

```

```

    health_interval 10s
    lb_policy      round_robin
}
encode gzip zstd
header {
    Strict-Transport-Security "max-age=31536000; includeSubDomains"
    X-Content-Type-Options   "nosniff"
    X-Frame-Options          "DENY"
    Referrer-Policy          "strict-origin-when-cross-origin"
    -Server
}
log {
    output file /data/access.log {
        roll_size 10mb
        roll_keep 5
    }
}
}

painel.exemplo.com {
    # Restringe o painel à VPN/rede interna, mesmo com certificado público
    @externo not remote_ip 10.0.0.0/8 192.168.0.0/16
    respond @externo "acesso restrito" 403
    reverse_proxy painel:3000
}

```

bash

```

docker compose up -d
docker compose logs caddy | grep -i certificate
# esperado: linhas indicando a obtenção do certificado junto ao Let's Encrypt
curl -sI https://app.exemplo.com | head -1    # HTTP/2 200

```

Explicação.

- **Caddy em vez de nginx + certbot** elimina a peça mais frágil da operação: o certificado é obtido e renovado automaticamente via ACME, sem cron, sem hook, sem “esqueci de renovar”. *Opinião profissional:* para 90% dos casos de proxy reverso caseiro ou de pequena empresa, Caddy custa menos operação que nginx. nginx ainda ganha em ajuste fino e em carga muito alta.
- **caddy-dados é o volume mais crítico da stack.** É onde ficam as chaves e certificados. Recriar o container sem esse volume força re-emissão, e o Let’s Encrypt tem limite de 5 certificados idênticos por semana — dá para se auto-bloquear.
- **replicas: 2 + DNS.** O Docker resolve app para os IPs das duas réplicas; o Caddy balanceia com verificação de saúde e tira do rodízio a réplica que falhar.
- **443:443/udp** habilita HTTP/3 (QUIC), que roda sobre UDP. Esquecer essa linha faz o HTTP/3 simplesmente não funcionar, sem erro visível.
- **Rede interna: true** para o banco: nem o Caddy o alcança. Só a app está nas duas redes.

6.15 Autoteste

1. Por que copiar package.json antes de COPY . . reduz o tempo de build em 90%?
2. O que a linha - /app/node_modules num Compose faz, e o que quebra sem ela?
3. Você usou --build-arg TOKEN=xyz. Como provar em dois comandos que o token vazou na imagem?
4. Qual é a diferença entre depends_on: condition: service_started, service_healthy e service_completed_successfully?

5. Uma imagem scratch com um binário Go falha ao chamar uma API HTTPS. O que faltou copiar?
6. Por que PYTHONUNBUFFERED=1 é praticamente obrigatório em container Python?
7. No exemplo 13, por que o dump é escrito em .parcial e só depois renomeado?
8. Por que fazer deploy por digest é diferente de fazer deploy por tag, e por que isso importa?
9. No exemplo 14, qual volume é o mais crítico e o que acontece se você o perder?
10. --platform=\$BUILDPLATFORM no estágio de build: o que ele evita, e para quais linguagens esse ganho é maior?

Projeto-modelo — Mural de Recados

Nível: intermediário · Última atualização: 11/08/2026

Uma aplicação **pequena porém inteira** que roda de verdade: API HTTP + página web, persistência em volume, proxy reverso, testes, healthcheck, encerramento gracioso, limites de recurso e endurecimento de segurança.

Não é um trecho de código. É o esqueleto do que você encontraria numa implantação real, reduzido ao mínimo que ainda ensina.

7.1 O que este projeto ensina

Conceito	Onde está no código
Multi-stage build com 4 estágios	Dockerfile (Dockerfile)
Cache de camadas e .dockerignore	.dockerignore (.dockerignore)
Container sem root, com USER	Dockerfile, estágio producao
tini como PID 1 e repasse de sinais	Dockerfile + server.js
Encerramento gracioso no SIGTERM	app/src/server.js (app/src/server.js)
Configuração por ambiente, falhando rápido	app/src/config.js (app/src/config.js)
Log estruturado em stdout	app/src/log.js (app/src/log.js)
Volume nomeado e escrita atômica	app/src/repositorio.js (app/src/repositorio.js)
Healthcheck que exercita a dependência real	Dockerfile + app/src/rotas.js
Liveness × readiness (/vivo × /saude)	app/src/rotas.js
Redes separadas, uma delas internal	compose.yaml (compose.yaml)
depends_on: condition: service_healthy	compose.yaml
Perfis do Compose (prod / dev / testes)	compose.yaml
Limites de memória, CPU e capabilities	compose.yaml
Sistema de arquivos raiz somente leitura + tmpfs	compose.yaml
Proxy reverso e DNS por nome de serviço	nginx/default.conf (nginx/default.conf)
Testes rodando dentro do build	Dockerfile, estágio testes
Backup e restauração de volume	Makefile (Makefile)

7.2 Pré-requisitos

- Docker Engine ≥ **24** (testado com a série 29) — veja [cap. 3](#)
- docker compose **v2** (com espaço, não hífen)
- make (opcional — todos os comandos têm equivalente direto abaixo)

- **Nenhuma dependência de npm.** O projeto usa só a biblioteca padrão do Node 22+, então o build não precisa de rede para instalar pacotes e é reproduzível em qualquer lugar.

7.3 Como rodar — comandos exatos

bash

```
cd homelab/learn-process/docker/07-projeto-modelo

# 1. Configuração
cp .env.example .env

# 2. Subir (constrói, sobe e ESPERA os healthchecks passarem)
docker compose up -d --build --wait
#   equivalente: make subir

# 3. Verificar
curl -s http://localhost:8080/saude
# esperado: {"status":"ok","recados":0,"ambiente":"production"}

# 4. Usar a API
curl -s -X POST http://localhost:8080/api/recados \
  -H 'Content-Type: application/json' \
  -d '{"autor":"Roni","texto":"primeiro recado containerizado"}'
# esperado: 201 com o recado, incluindo id (UUID) e criadoEm (ISO-8601)

curl -s http://localhost:8080/api/recados
# esperado: {"total":1,"recados":[{"...}]}
```

```
# 5. Abrir a página no navegador
#   http://localhost:8080

# 6. Derrubar (os dados PERMANECEM no volume)
docker compose down
#   equivalente: make derrubar
```

7.3.1 Outros modos

bash

```
make testes    # docker compose --profile testes run --rm testes
make dev       # docker compose --profile dev up --build → http://localhost:3000, com recarga
make logs      # docker compose logs -f
make shell     # docker compose exec api sh
make backup    # salva o volume em ./backups/
make limpar    # △ docker compose down -v – APAGA os recados
make           # lista todos os alvos
```

7.4 Estrutura de pastas

```
07-projeto-modelo/
├─ compose.yaml      # a stack: proxy + api, mais os perfis dev e testes
├─ Dockerfile        # 4 estágios: base → dev / testes / producao
├─ .dockerignore     # o que NÃO entra no contexto de build (inclusive segredos)
├─ .env.example       # modelo da configuração; copie para .env
├─ Makefile          # atalhos legíveis para os comandos longos
├─
├─ nginx/
│   └─ default.conf   # proxy reverso: única porta exposta ao host
├─
├─ app/
│   └─ package.json   # sem dependências – de propósito
└─ src/
```



```

| └─ server.js      # PID 1: sobe, trata sinais, encerra com elegância
| └─ config.js     # ambiente → configuração validada, falhando rápido
| └─ log.js        # log JSON de uma linha, em stdout
| └─ repositório.js # persistência atômica em volume + fila de escrita
| └─ rotas.js       # roteamento, validação e tratamento de erro
└─ test/
    └─ repositório.test.js # 12 testes de unidade
    └─ rotas.test.js      # 10 testes de integração HTTP

```

7.5 O que cada decisão de projeto ensina

7.5.1 Quatro estágios no Dockerfile, não um

base concentra o que é comum; dev traz recarga automática; testes **roda a suíte durante o build** — se um teste falhar, a imagem não é produzida; produção leva só o necessário para executar.

O que se aprende: multi-stage não serve só para “diminuir a imagem”. Serve para separar propósitos. O estágio testes transforma o build num portão de qualidade que funciona igual na sua máquina e no CI, sem duplicar configuração.

7.5.2 tini como PID 1

Sem um init de verdade, o processo Node vira PID 1 e o kernel trata PID 1 de forma especial: sinais sem tratador registrado são **ignorados**, e processos órfãos nunca são colhidos.

O que se aprende: por que tantos containers demoram exatos 10 segundos para parar. O tini custa 10 KB e resolve os dois problemas.

7.5.3 SIGTERM tratado explicitamente no server.js

docker stop manda SIGTERM, espera (por padrão) 10 segundos e manda SIGKILL. Quem não trata SIGTERM perde toda requisição em curso a cada deploy — que o usuário vê como erro 502.

O que se aprende: o encerramento gracioso tem um **prazo próprio** (prazoEncerramentoMs = 8000) menor que o do Docker. Se as conexões não fecharem a tempo, o processo sai por conta própria em vez de ser morto à força.

7.5.4 Escrita atômica no repositório.js

Grava em recados.json.tmp e só então rename. No mesmo sistema de arquivos, rename é atômico no Linux: ou está o arquivo antigo, ou o novo, nunca metade.

O que se aprende: container **morre a qualquer momento** — por deploy, por OOM, por reagendamento. Código que assume “vou terminar de escrever” produz corrupção. Esta é a diferença entre um exemplo de tutorial e código que aguenta um kill -9.

7.5.5 Fila de escrita serializando as gravações

Duas requisições simultâneas leem o mesmo estado, cada uma acrescenta o seu recado, e uma sobrescreve a outra — *lost update*. A fila (#enfileirar) elimina isso. Há um teste específico para essa condição de corrida.

O que se aprende: concorrência não some porque o app está em container. Ela só fica mais visível, porque containers são replicados.

7.5.6 Healthcheck que escreve no disco

/saude chama `repositorio.verificarSaude()`, que grava um arquivo-sonda. Um healthcheck que devolve `{"status": "ok"}` sem tocar na dependência crítica declara “saudável” um container com o volume cheio ou somente leitura.

O que se aprende: healthcheck ruim é pior que nenhum, porque dá falsa confiança. E a distinção entre /vivo (liveness: “reinicie-me”) e /saude (readiness: “não me mande tráfego”) é o que evita reinícios em cascata quando uma dependência oscila.

7.5.7 Duas redes, uma delas internal: true

proxy está em borda e em interna. api está **só** em interna, que não tem rota para a internet. Não existe ports: no serviço api.

O que se aprende: o modelo de rede é a sua primeira camada de defesa. Se a api for comprometida, o atacante não consegue nem exfiltrar dados por HTTP, porque a rede não tem saída.

7.5.8 read_only: true, cap_drop: ALL, no-new-privileges

O sistema de arquivos raiz é somente leitura (com um /tmp em RAM), o container não tem nenhuma *capability* do kernel, e nenhum binário setuid pode elevar privilégio.

O que se aprende: essas três linhas eliminam classes inteiras de exploração e custam praticamente nada. A maioria das aplicações web funciona assim sem nenhuma alteração — só não se faz por desconhecimento.

7.5.9 \${NOME_DO_MURAL: ?mensagem} no Compose

Se a variável não existir, o `docker compose up` **aborta com uma mensagem explícita**, em vez de subir com valor vazio.

O que se aprende: falhar cedo e alto. O orquestrador sabe lidar com “não subiu”; ele não sabe lidar com “subiu errado”.

7.5.10 Zero dependências de npm

Não há `node_modules`, `package-lock.json` nem `npm ci` no build.

O que se aprende: é uma escolha **deliberada** deste material didático — o build fica reproduzível sem rede e o Dockerfile fica legível sem o ruído de lockfile. Em um projeto real você **terá** dependências, e aí valem os padrões do [06-exemplos.md](#) (cap. 6): `COPY package*.json` antes de `COPY . .`, `npm ci` (não `npm install`) e `RUN --mount=type=cache`.

7.6 Laboratórios com este projeto

Faça-os na ordem. Cada um demonstra na prática algo que só se entende mexendo.

7.6.1 Lab 1 — O container é descartável

bash

```
docker compose up -d --wait
curl -X POST localhost:8080/api/recados -H 'Content-Type: application/json' \
  -d '{"autor":"eu","texto":"sobrevivo?"}'
docker compose down          # remove os containers
docker compose up -d --wait  # recria do zero
curl -s localhost:8080/api/recados
```

Esperado: o recado continua lá. **Por quê:** ele está no volume mural_dados-mural, não na camada de escrita do container.

7.6.2 Lab 2 — E o volume, não

bash

```
docker compose down -v      # o -v apaga o volume
docker compose up -d --wait
curl -s localhost:8080/api/recados
```

Esperado: {"total":0,"recados":[]}. **Lição:** down -v não pede confirmação. Em produção, esse comando já apagou muito banco de dados.

7.6.3 Lab 3 — Falhar rápido

bash

```
NOME_DO_MURAL= docker compose up api
```

Esperado: o Compose aborta antes de criar o container, citando a variável pelo nome. Agora remova o :? do compose.yaml e repita: o container sobe e **morre** no boot, com o erro vindo do config.js. Compare as duas mensagens — a primeira é muito melhor.

7.6.4 Lab 4 — Encerramento gracioso

bash

```
docker compose up -d --wait
time docker compose stop api
```

Esperado: menos de 1 segundo. Agora troque no Dockerfile a linha CMD ["node", "src/server.js"] por CMD node src/server.js (forma shell), reconstrua e repita. **Esperado:** exatos 10 segundos, porque o /bin/sh intermediário não repassa o SIGTERM.

7.6.5 Lab 5 — Limite de memória e OOM

bash

```
docker compose up -d --wait
docker stats --no-stream $(docker compose ps -q api)
# MEM USAGE / LIMIT mostra ".../ 256MiB"

# Force o estouro:
docker compose exec api node -e "const a=[]; while(true) a.push(Buffer.alloc(10*1024*1024))"
docker inspect --format '{{.State.OOMKilled}} {{.State.ExitCode}}' $(docker compose ps -q api)
```

Esperado: true 137. **Lição:** 137 = 128 + 9 (SIGKILL) é quase sempre OOM. E o limite converteu "o servidor caiu" em "um container reiniciou".

7.6.6 Lab 6 — Isolamento de rede

bash

```
docker compose exec api wget -qO- --timeout=3 https://example.com || echo "sem saída — correto"
docker compose exec proxy wget -qO- --timeout=3 http://api:3000/saude
```

Esperado: a api não alcança a internet (rede internal); o proxy alcança a api pelo **nome do serviço**, resolvido pelo DNS interno do Docker em 127.0.0.11.

7.6.7 Lab 7 — Sistema de arquivos somente leitura

bash

```
docker compose exec api sh -c 'echo teste > /app/arquivo.txt' # deve falhar
docker compose exec api sh -c 'echo teste > /tmp/arquivo.txt' # deve funcionar (tmpfs)
docker compose exec api sh -c 'echo teste > /dados/arquivo.txt' # deve funcionar (volume)
```

Lição: `read_only: true` não impede o app de trabalhar — só o obriga a declarar onde escreve.

7.6.8 Lab 8 — O build como portão de qualidade

Introduza um bug em `app/src/repositorio.js` (por exemplo, remova o `.trim()` da validação de autor) e rode:

bash

```
docker build --target testes -t mural:teste .
```

Esperado: o build **falha** no estágio testes, e nenhuma imagem de produção é gerada.

7.6.9 Lab 9 — Backup e restauração

bash

```
make backup
make limpar # apaga tudo
docker compose up -d --wait
make restaurar
curl -s localhost:8080/api/recados
```

Esperado: os recados voltam. **Lição:** um backup nunca restaurado não é um backup.

7.6.10 Lab 10 — Onde o tamanho está

bash

```
make tamanho
docker history mural-de-recados:1.0
```

Identifique qual camada é a maior. **Esperado:** a base `node:22-alpine`. **Pergunta para levar adiante:** como chegar perto de 10 MB? (Resposta: `gcr.io/distroless/nodejs22`, ou trocar de linguagem — veja o [exemplo 6](#) (cap. 6).)

7.7 Próximos passos — como evoluir este projeto

Na ordem em que fazem sentido:

1. **Trocar o arquivo JSON por Postgres.** Acrescente um serviço db com healthcheck, um serviço migracao com `service_completed_successfully`, e um driver no `package.json`. É quando o `.dockerignore`, o `npm ci` e o cache de build passam a importar de verdade.
2. **Publicar a imagem** no GHCR e fazer deploy por *digest*, com o pipeline do [exemplo 12](#) (cap. 6).
3. **Escanear e assinar:** `trivy image` e `cosign sign`.
4. **Métricas:** expor `/metricas` em formato Prometheus e acrescentar Prometheus + Grafana.
5. **Escalar:** `docker compose up -d --scale api=3` e observar o nginx balancear entre as réplicas — momento em que o estado em arquivo local mostra a sua limitação, e por isso mesmo ensina por que serviços replicados guardam estado fora do container.

7.8 Verificação feita neste material

Sejamos precisos sobre o que foi testado e o que não foi:

- ✓ **A suíte de testes foi executada e passou:** 22 testes, 2 suítes, em Node v24.18.0, em 11/08/2026 — 12 testes de unidade do repositório e 10 de integração HTTP.
- ✓ **A aplicação foi executada fora de container** e verificada de ponta a ponta: `/saude`, `POST /api/recados`, `GET /api/recados`, a página HTML e o encerramento gracioso via SIGTERM (menos de 1 segundo, com a linha encerrado com elegância no log).
- ⚠ **O `docker build` e o `docker compose up` não puderam ser executados no ambiente em que este material foi escrito** (sem acesso ao socket do daemon). O `Dockerfile` e o `compose.yaml` seguem as práticas verificadas na documentação oficial na data acima, mas **rode-os você mesmo** e trate qualquer divergência como um exercício legítimo do Lab 8. O material não afirma o que não verificou.

7.9 Autoteste

1. Por que o estágio testes existe se os testes já rodam com `npm test`?
2. O que acontece se você remover `tini` do `Dockerfile` e parar o container?
3. Por que o `repositorio.js` grava em `.tmp` antes de renomear?
4. O serviço `api` não tem `ports`. Como o navegador chega até ele?
5. Qual é a diferença entre `/vivo` e `/saude`, e qual sonda o orquestrador usa para reiniciar?
6. Por que `read_only: true` exige um `tmpfs` em `/tmp`?
7. `docker compose down` e `docker compose down -v`: qual apaga os recados?
8. Se você rodar `--scale api=3`, o que quebra neste projeto e por quê?
9. Por que o `mkdir /dados && chown node:node /dados` no `Dockerfile` é necessário mesmo com o volume sendo criado pelo Compose?

PARTE II

Núcleo

Fundamentos, arquitetura interna, teoria e estado da arte

CAPÍTULO 8

Fundamentos — vocabulário, modelos mentais e definições formais

Nível: iniciante → intermediário · Última atualização: 11/08/2026

Este é o arquivo que transforma "eu sei rodar docker run" em "eu sei o que está acontecendo". Tudo aqui é definido antes de ser usado.

8.1 A definição formal de container

Container é um ou mais processos executando em um sistema operacional hospedeiro, sob um conjunto de restrições impostas pelo kernel, das quais três eixos são essenciais: **isolamento de visão** (namespaces), **limitação de consumo** (cgroups) e **redução de privilégio** (capabilities, seccomp, LSM), executando a partir de um **sistema de arquivos raiz próprio**, montado a partir de uma imagem.

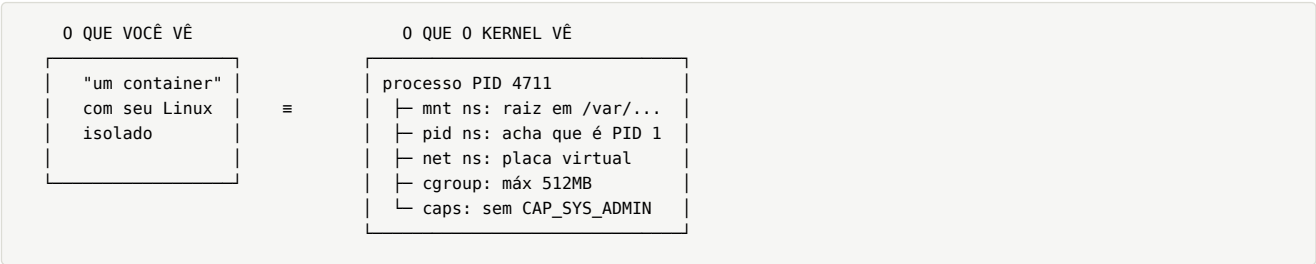
Cada elemento dessa definição responde a uma pergunta:

Elemento	Pergunta que responde
"um ou mais processos"	O container é processo. Não é máquina, não é servidor
"sistema operacional hospedeiro"	O kernel é compartilhado . Daí toda a economia e todo o risco
"isolamento de visão"	O que o processo enxerga (PIDs, arquivos, rede, hostname)
"limitação de consumo"	Quanto ele pode usar (CPU, RAM, I/O, número de processos)
"redução de privilégio"	O que ele pode fazer (chamadas de sistema, operações privilegiadas)
"sistema de arquivos raiz próprio"	De onde vêm os arquivos que ele vê como /

O que a definição NÃO diz, e é o erro conceitual mais comum: ela não diz "máquina virtual leve". Não há virtualização. Não há kernel próprio. Não há hardware emulado.

8.2 O modelo mental correto

Guarde esta imagem:



Prove você mesmo, em Linux:

bash

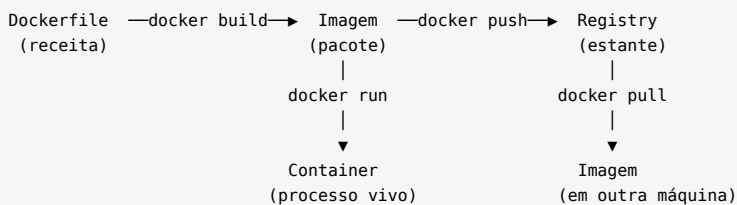
```
docker run -d --name prova nginx:alpine
PID=$(docker inspect --format '{{.State.Pid}}' prova)

ps -p $PID -o pid,ppid,cmd          # o processo do container, visto do host
sudo ls -l /proc/$PID/ns/           # os namespaces dele
sudo cat /proc/$PID/cgroup           # o cgroup ao qual pertence
sudo ls /proc/$PID/root/             # o sistema de arquivos que ELE vê como /

docker rm -f prova
```

Você acabou de olhar “dentro” do container **sem usar o Docker**. Isso não é curiosidade: é a prova de que não há mágica, e é como se depura quando o docker exec não funciona.

8.3 Os quatro substantivos e como eles se relacionam

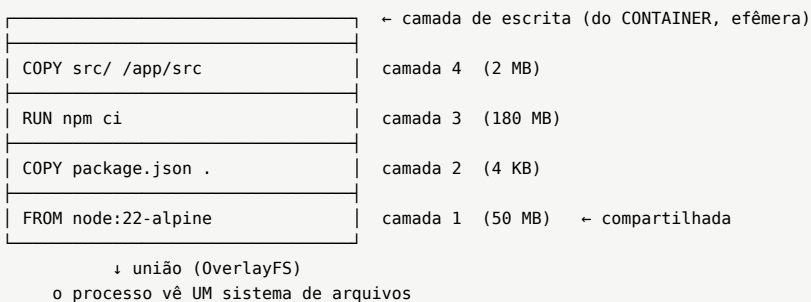


Substantivo	Natureza	Vive onde	Some quando
Dockerfile	Texto	No seu repositório Git	Você apaga o arquivo
Imagem	Dado imutável, endereçado por conteúdo	Disco local + registry	Você faz docker rmi
Container	Processo + camada de escrita	Memória + disco local	Você faz docker rm
Volume	Dado mutável	Disco, fora das camadas	Você faz docker volume rm

A regra que evita perda de dados: só o **volume** é feito para durar. Todo o resto é descartável por projeto.

8.4 Imagem: pilha de camadas endereçadas por conteúdo

Uma imagem não é um arquivo único. É uma **pilha ordenada de camadas (layers)**, cada uma sendo um conjunto de alterações no sistema de arquivos, mais um **manifesto** que descreve a pilha e a configuração.



Três propriedades decorrem disso, e todas têm consequência prática:

(a) Camadas são identificadas pelo hash do próprio conteúdo. Duas imagens que usam `node:22-alpine` **compartilham fisicamente** aquela camada no disco. Baixar a segunda imagem custa quase nada. É por isso que padronizar a imagem base numa equipe economiza gigabytes.

(b) Camadas são somente leitura; só a do container é escrevível. Quando o processo escreve em um arquivo que veio de uma camada inferior, o sistema faz **copy-on-write**: copia o arquivo para a camada superior e escreve na cópia. Arquivo grande = primeira escrita cara.

(c) Apagar não diminui. `RUN rm -rf /cache` cria uma camada que **marca** o arquivo como apagado (um *whiteout*), mas a camada anterior, com o arquivo, continua na imagem. Por isso:

Dockerfile

```
# [x] a imagem continua carregando o cache do apt
RUN apt-get update && apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# [ok] tudo numa camada só: o apagado nunca chega a existir numa camada final
RUN apt-get update && apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

E é por isso que **segredo apagado num RUN posterior continua recuperável** — o assunto do [exemplo 10](#) (cap. 6).

Detalhes completos em [cap. 10](#).

8.5 Namespaces: o isolamento de visão

Um **namespace** é um recurso do kernel Linux que faz um conjunto de processos ter uma visão própria de um recurso global do sistema.

Namespace	Isola	Efeito visível	Ano no kernel
mnt	Pontos de montagem	O container tem seu próprio /	2002 (2.4.19)
uts	Hostname e domínio	hostname devolve o ID do container	2006 (2.6.19)
ipc	Memória compartilhada, filas	Processos não veem IPC de fora	2006 (2.6.19)
pid	Tabela de processos	O app é PID 1; não enxerga o host	2008 (2.6.24)
net	Pilha de rede inteira	IP, rotas, iptables e portas próprios	2009 (2.6.29)
user	Mapeamento de UID/GID	Root dentro ≠ root fora — base do rootless	2013 (3.8)
cgroup	Visão da hierarquia de cgroups	Não vê os limites dos outros	2016 (4.6)
time	Relógios monotônico e de boot	Raro; usado em migração de processos	2020 (5.6)

A chamada de sistema que os cria é `clone()` com as flags `CLONE_NEW*`, ou `unshare()`.

Você pode criar um container “à mão”, sem Docker, para ver que não há truque:

bash

```
sudo unshare --pid --fork --mount-proc --uts --net --ipc bash
# você está num "container" agora:
hostname container-manual
```

```
ps aux      # só o bash e o ps
ip addr     # só o loopback, sem rede
exit
```

Isso é 80% do que o `docker run` faz. Os outros 20% são a imagem, os `cgroups` e a rede. Aprofundamento: [cap. 11](#).

8.6 cgroups: a limitação de consumo

cgroup = *control group*. Agrupa processos e aplica limites e contabilidade sobre recursos.

Controlador	Limita	Flag do Docker
memory	RAM e swap	<code>-m 512m, --memory-swap</code>
cpu	Tempo de CPU	<code>--cpus, --cpu-shares</code>
cpuset	Quais núcleos	<code>--cpuset-cpus</code>
io	Vazão de disco	<code>--device-read-bps</code>
pids	Número de processos	<code>--pids-limit</code>

cgroup v1 vs v2: a v2 unifica todos os controladores numa hierarquia única, com semântica consistente. É o padrão em distros modernas desde ~2021 e é o que você deve usar. Alguns recursos (limites de I/O confiáveis, contabilidade de memória precisa) **só funcionam bem na v2**.

bash

```
stat -fc %T /sys/fs/cgroup/
# cgroup2fs → v2 (bom) · tmpfs → v1 (legado)
```

8.6.1 A consequência que morde: o OOM killer

Quando um container excede o limite de memória, o kernel **mata** o processo — não devolve erro de alocação, não avisa o app. Você vê apenas exit code 137.

bash

```
docker inspect --format '{{.State.OOMKilled}} {{.State.ExitCode}}' NOME
# true 137 → confirmado
```

E um detalhe que causa horas de confusão: **muitas runtimes de linguagem não enxergam o limite do cgroup**, e sim a RAM total do host. Uma JVM antiga num container de 512 MB podia dimensionar seu heap para 8 GB e ser morta imediatamente. Java 10+ e .NET Core 3+ passaram a respeitar `cgroups`; Node e Python ainda exigem configuração explícita (`--max-old-space-size`, por exemplo).

8.7 Capabilities, seccomp e LSM: a redução de privilégio

Historicamente, o Linux tinha dois estados: `root` (pode tudo) e `não-root` (não pode quase nada). As **capabilities** quebraram o poder do `root` em ~40 permissões independentes.

Capability	Permite	O Docker dá por padrão?
CAP_NET_BIND_SERVICE	Escutar em porta < 1024	Sim
CAP_CHOWN	Mudar dono de arquivo	Sim
CAP_SETUID / CAP_SETGID	Trocar de usuário	Sim
CAP_NET_RAW	Pacotes crus (ping, ARP spoofing)	Sim (discutível)
CAP_SYS_ADMIN	"O novo root": montar, namespaces, quase tudo	Não
CAP_SYS_PTRACE	Depurar outros processos	Não
CAP_SYS_MODULE	Carregar módulos do kernel	Não

O Docker mantém 14 capabilities por padrão, de ~40. A boa prática é ir mais longe:

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE minha-app
```

bash

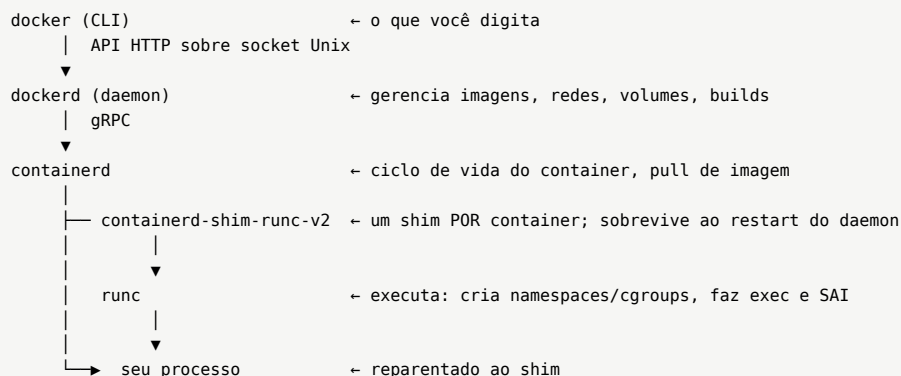
seccomp (*secure computing mode*) filtra **chamadas de sistema**. O perfil padrão do Docker bloqueia cerca de 40 das ~350 syscalls — as historicamente exploráveis (kexec_load, mount, ptrace em certos contextos). Rodar com `--security-opt seccomp=unconfined` remove esse filtro e é um retrocesso relevante de segurança.

LSM (*Linux Security Modules*): AppArmor (Debian/Ubuntu) e SELinux (Fedora/RHEL) aplicam uma camada adicional, de controle de acesso obrigatório. É o SELinux que exige `:z/:Z` em bind mounts no Fedora.

Aprofundamento: [cap. 18](#).

8.8 A arquitetura do Docker: quem faz o quê

Muita gente acha que "Docker" é um programa. São pelo menos quatro, em camadas:



Por que essa separação importa, em três consequências:

1. **runc executa e sai.** Ele não fica rodando. Depois do exec, o processo do container é filho do *shim*, não do daemon.
2. **Por isso você pode reiniciar o dockerd sem matar os containers** (com live-restore habilitado). O shim mantém tudo de pé.
3. **containerd e runc são padrões da indústria**, doados à CNCF. O Kubernetes fala diretamente com o containerd, **sem Docker no meio** — foi isso, e não mais que isso, o "Kubernetes está removendo o Docker" de 2020, que gerou pânico desproporcional.

Aprofundamento: [cap. 12](#).

8.9 OCI: o padrão que impede o aprisionamento

A **Open Container Initiative**, criada em 2015 sob a Linux Foundation, mantém três especificações:

Especificação	Define
Image Spec	O formato da imagem: manifesto, camadas, configuração, digests
Runtime Spec	O formato do <i>bundle</i> e como executá-lo (config.json + rootfs)
Distribution Spec	A API HTTP de um registry (push, pull, autenticação)

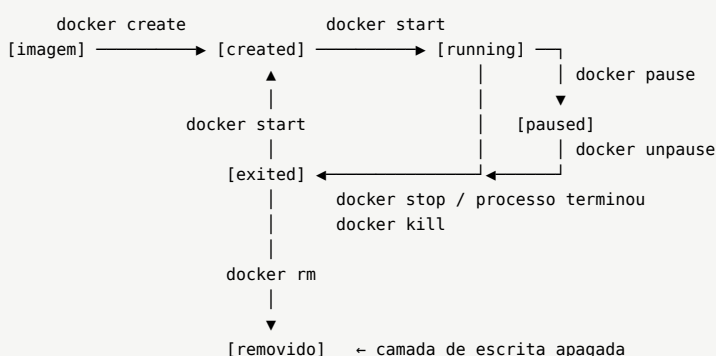
A consequência prática é a mais importante do ecossistema: **imagem construída com Docker roda em Podman, containerd, CRI-O, Kubernetes ou AWS Fargate**. Você não está preso a um fornecedor. Foi isso que transformou “container” de produto em infraestrutura comum.

8.10 As seis coisas que um container não é

Corrigir esses mal-entendidos economiza semanas:

Mito	Realidade
“É uma VM leve”	Não há virtualização nem kernel próprio. É um processo restrito
“É seguro por padrão”	O padrão é razoável, não é isolamento forte. Kernel compartilhado = superfície compartilhada
“É imutável”	A imagem é imutável. O container tem uma camada de escrita mutável
“Container é para microserviço”	Container é embalagem. Microserviço é decisão organizacional. São independentes
“Docker é o container”	Docker é uma implementação. O padrão é OCI; há várias implementações
“Container não tem estado”	Ele pode ter estado — em volume. O que ele não deve ter é estado na camada de escrita

8.11 O ciclo de vida completo de um container



docker run = create + start. Um container parado ainda ocupa disco (a camada de escrita e os logs). É por isso que `docker ps -a` costuma revelar dezenas de containers esquecidos consumindo espaço.

Estado restarting: se a política de reinício estiver ativa e o processo falhar repetidamente, o container fica em laço. O Docker aplica *backoff* exponencial (100 ms, 200 ms, 400 ms...) para não derrubar a máquina. `docker ps` mostrando `Restarting (1) 3 seconds ago` é o sintoma clássico de app que morre no boot — e `docker logs` mostra o porquê.

8.12 Os cinco modelos mentais que valem mais que qualquer comando

1. **"Container é processo."** Toda dúvida sobre ciclo de vida se resolve perguntando: *o que o PID 1 está fazendo?* Container "não sobe" = processo saiu. Container "não morre" = processo ignora sinal.
2. **"A imagem é o artefato; o container é a execução."** Deploy é trocar qual imagem executa. Rollback é apontar para a imagem anterior. Se você está fazendo ssh no servidor e editando arquivo dentro de um container, o modelo está errado.
3. **"Camada é cache."** A ordem das instruções no Dockerfile determina o tempo de build. O que muda pouco vai em cima; o que muda a cada commit vai embaixo.
4. **"Tudo é efêmero, exceto o que você declarou que não é."** Se um dado importa, ele está em volume — ou está por um fio.
5. **"O kernel é compartilhado."** Toda decisão de segurança em container decorre desta frase. Se o isolamento precisa ser forte, é preciso outra barreira (VM, micro-VM, gVisor, Kata).

8.13 Os cinco porquês aplicados: por que containers iniciam tão rápido?

Seguindo a regra dos cinco porquês do preset, até uma parada legítima:

1. **Por que um container inicia em milissegundos e uma VM em segundos?** Porque o container não faz boot. É um `fork` + `exec` de um processo, com algumas chamadas de configuração antes.
2. **Por que a VM precisa dar boot?** Porque ela tem um kernel próprio, que precisa inicializar hardware (virtual), montar sistemas de arquivos, iniciar o `init` e uma árvore de serviços.
3. **Por que o container não precisa de kernel próprio?** Porque namespaces e cgroups permitem que o **mesmo** kernel apresente visões e limites diferentes para conjuntos de processos distintos. O isolamento é feito na estrutura de dados do kernel, não em hardware.
4. **Por que o kernel foi construído com essa capacidade?** Por uma linhagem de necessidades reais: chroot (1979, isolar visão do sistema de arquivos em testes de compilação), FreeBSD Jails (2000, hospedagem compartilhada), Solaris Zones (2004), e sobretudo o **Process Containers do Google** (2006, Paul Menage e Rohit Seth), renomeado para cgroups e mesclado no kernel 2.6.24 em 2008. O Google precisava rodar milhares de trabalhos por máquina com isolamento de desempenho — e VMs eram caras demais para essa densidade.
5. **Por que VMs eram caras demais para essa densidade?** Porque cada VM duplica o kernel, o `init`, os daemons e o cache de página em memória. Com milhares de trabalhos por máquina, essa duplicação consome uma fração dominante da RAM e do CPU.

Aqui a cadeia chega a um trade-off econômico explícito: a densidade que o Google precisava só era alcançável abrindo mão do isolamento por hardware — decisão consciente, tomada num ambiente onde todo o código executado era da própria empresa, e portanto o modelo de ameaça era diferente do de um provedor de nuvem pública.

E é exatamente por isso que provedores que rodam **código de terceiros desconhecidos** (AWS Lambda, Cloudflare Workers, Fly.io) reintroduziram uma barreira: micro-VMs (Firecracker), sandbox de syscalls (gVisor) ou isolados de V8. A escolha do Google não era universal — era apropriada ao contexto dele.

8.14 Vocabulário consolidado

Termo	Definição
Imagem	Pilha imutável de camadas + configuração, identificada por digest
Camada (<i>layer</i>)	Conjunto de alterações no sistema de arquivos, endereçado por hash de conteúdo
Manifesto	Documento JSON que lista as camadas e a configuração de uma imagem
Manifest list / índice	Manifesto que aponta para vários manifestos, um por arquitetura (multi-arch)
Digest	sha256:... do conteúdo. Imutável, ao contrário da tag
Tag	Rótulo humano e mutável apontando para um digest
Container	Processo(s) isolado(s) executando a partir de uma imagem
Camada de escrita	Camada mutável e efêmera, exclusiva de cada container
Copy-on-write	Estratégia de copiar o arquivo para a camada superior na primeira escrita
Volume	Armazenamento gerenciado, fora das camadas, que sobrevive ao container
Bind mount	Diretório do host montado dentro do container
Registry	Servidor que armazena e distribui imagens
Repositório	Conjunto de tags de uma mesma imagem num registry
Namespace	Recurso do kernel que isola a visão de um recurso global
cgroup	Recurso do kernel que limita e contabiliza consumo
Capability	Fração do poder do root, concedida individualmente
seccomp	Filtro de chamadas de sistema
runtime	Programa que executa o container (runc, crun, youki, gVisor, kata)
shim	Processo intermediário que mantém o container vivo entre o daemon e o runtime
OCI	Padrão aberto de imagem, runtime e distribuição
Orquestrador	Sistema que agenda containers num conjunto de máquinas

Lista completa em [cap. 29](#).

8.15 Autoteste

1. Escreva a definição formal de container sem consultar, e explique cada uma das cinco partes.
2. Por que `RUN rm -rf /cache` numa linha separada não reduz o tamanho da imagem?
3. Qual comando prova, sem usar o Docker, que um container é apenas um processo do host?
4. Diferencie namespace de cgroup em uma frase cada, com um exemplo de problema que cada um resolve.

5. Seu container Java num limite de 512 MB é morto com exit 137. Cite duas causas possíveis.
6. Por que runc não aparece no ps de um container em execução?
7. O que exatamente aconteceu em 2020 quando "o Kubernetes removeu o Docker"?
8. Cite as três especificações da OCI e o que cada uma torna possível.
9. Percorra os cinco porquês de "por que containers iniciam rápido" até a parada legítima. Que tipo de parada é (lei física, decisão histórica, trade-off econômico ou convenção)?
10. Um container está Restarting (1) 3 seconds ago. Qual é o diagnóstico e qual comando você roda primeiro?

CAPÍTULO 9

História — de onde vieram os containers e que problema resolveram

Nível: iniciante → intermediário · Última atualização: 11/08/2026

Este arquivo existe porque quase toda decisão estranha do Docker tem explicação histórica. Saber a história é o que separa "isso é assim" de "isso é assim **porque**".

9.1 Linha do tempo em uma tela

1979	chroot	isolar a visão do sistema de arquivos
1982	chroot no BSD	idem, difundido
2000	FreeBSD Jails	isolamento de processo + rede, para hospedagem
2004	Solaris Zones	zonas com recursos controlados
2005	OpenVZ	containers em Linux, com kernel modificado
2006	Process Containers (Google)	→ renomeado para cgroups
2008	cgroups no kernel 2.6.24	limitação de recursos entra oficialmente no Linux
2008	LXC	primeiro conjunto completo em Linux puro
2011	Warden (Cloud Foundry)	containers como plataforma
2013	Docker 0.1 (março)	★ o ponto de virada
2013	Docker Hub	distribuição pública de imagens
2014	Docker 1.0 · Kubernetes	produção + orquestração
2015	OCI · CNCF	padronização, fim do risco de aprisionamento
2015	runC doado	runtime de referência
2016	containerd	runtime de alto nível
2017	Moby · Kubernetes vence	Docker vira upstream; K8s vence a guerra dos orquestradores
2018	Podman 1.0 · gVisor	alternativa sem daemon; sandbox de syscalls
2019	Mirantis compra Docker EE	Docker Inc. foca em desenvolvedores
2020	"K8s remove o Docker"	dockershim descontinuado – pânico desproporcional
2021	Docker Desktop passa a cobrar de empresas grandes	
2022	dockershim removido de fato (K8s 1.24) · Wasm entra na conversa	
2023	BuildKit padrão · Docker Scout · Wasm no Docker Desktop	
2025	Docker Hub aperta limites de pull (abril) · Docker Model Runner	
2026	Engine 29: containerd image store por padrão, nftables experimental	

9.2 Ato 1 — A pré-história (1979-2012): a ideia existia, faltava a embalagem

9.2.1 chroot (1979)

Bill Joy acrescenta ao Unix V7 uma chamada que muda o diretório raiz de um processo. Motivo original: **testar builds e instalações num ambiente limpo**, sem sujar o sistema.

Isolava só o sistema de arquivos. E era escapável: um processo root com um descritor de diretório aberto antes do chroot conseguia sair. A lição que ficou — **isolamento parcial não é isolamento** — voltaria a ser aprendida várias vezes.

9.2.2 FreeBSD Jails (2000)

Poul-Henning Kamp implementa *jails* para um provedor de hospedagem que precisava separar clientes na mesma máquina. Acrescenta ao chroot o isolamento de processos, de usuários e de rede (cada jail com seu IP).

Aqui já está quase tudo que chamamos de container. **Faltava o que faltaria por mais 13 anos: um jeito de empacotar e distribuir o conteúdo do jail.**

9.2.3 Solaris Zones (2004) e OpenVZ (2005)

Zones acrescentam controle fino de recursos e uma noção de “marca” (rodar Linux dentro de uma zone Solaris). OpenVZ traz containers para o Linux, mas com um **kernel modificado** — o que impediu a adoção ampla, porque exigia trocar o kernel da distribuição.

9.2.4 cgroups (2006–2008) — a peça que faltava no Linux

Paul Menage e Rohit Seth, no Google, escrevem os *Process Containers*. O Google rodava milhares de trabalhos por máquina no Borg e precisava garantir que um trabalho não roubasse CPU e memória dos outros. Renomeado para **cgroups**, entra no kernel 2.6.24 em janeiro de 2008.

Combinado com os namespaces, que vinham sendo adicionados desde 2002, **o Linux passou a ter tudo de que um container precisa** — em kernel padrão, sem patch.

9.2.5 LXC (2008)

Primeiro conjunto de ferramentas a juntar namespaces + cgroups em Linux puro. Funcionava. Era usado por gente que sabia o que estava fazendo. E era **difícil**: configuração extensa, nenhum formato de distribuição, cada instalação era artesanal.

A lição central da pré-história: de 2008 a 2013, todos os recursos técnicos do container já existiam no kernel Linux padrão. O que não existia era **experiência de uso**. A revolução de 2013 não foi tecnológica — foi de embalagem.

9.3 Ato 2 — 2013: o Docker aparece

9.3.1 O contexto

A dotCloud era uma PaaS francesa (Solomon Hykes, Sebastien Pahl, Kamel Founadi) que usava LXC internamente para isolar aplicações de clientes. Havia construído ferramental interno para tornar isso viável. O negócio de PaaS não decolava.

Em **março de 2013**, na PyCon, Hykes faz um *lightning talk* de 5 minutos e libera o ferramental interno como código aberto. Em **outubro**, a empresa se renomeia para Docker, Inc.

9.3.2 O que o Docker realmente inventou

Nenhuma das peças isoladas era nova. **A combinação era.**

Peça	Já existia?	O que o Docker fez
Isolamento de processo	Sim (LXC)	Usou o LXC no começo; depois escreveu o libcontainer
Sistema de arquivos em camadas	Sim (AUFS, UnionFS)	Fez dele o formato de distribuição
Empacotamento	Não, nesse formato	Criou a imagem : unidade portátil e endereçada por conteúdo
Descrição do build	Não	Criou o Dockerfile : build versionável em texto
Distribuição pública	Não	Criou o Docker Hub : docker run redis funciona sem saber nada de Redis
Experiência de uso	Não	docker run em uma linha, com saída legível

Opinião profissional, não consenso: a peça mais subestimada é o **Docker Hub**. O formato de imagem sem um lugar público para buscá-las teria sido mais uma ferramenta de nicho. O que mudou o comportamento de milhões de desenvolvedores foi poder subir qualquer tecnologia com uma linha, sem ler manual. Foi um efeito de rede, não uma inovação de kernel.

9.3.3 Por que pegou tão rápido

Três problemas convergiam na época:

1. **A explosão da complexidade de dependências.** Aplicações web modernas tinham dezenas de dependências com versões conflitantes. “Funciona na minha máquina” havia deixado de ser piada e virado custo.
2. **A nuvem tinha virado commodity, mas a implantação não.** Ninguém queria repetir Chef e Puppet convergindo servidores por dez minutos.
3. **A pressão por velocidade de entrega.** Deploy semanal virou deploy diário, e o gargalo era a diferença entre ambientes.

O Docker atacou os três com o mesmo artefato.

9.4 Ato 3 — 2014-2017: orquestração e a guerra dos padrões

Uma vez que rodar um container ficou trivial, a pergunta seguinte apareceu imediatamente: **e mil containers, em cem máquinas?**

Ano	Evento
2014	Google libera o Kubernetes , destilado de 15 anos de Borg
2014	CoreOS lança o rkt , criticando o daemon monolítico e o modelo de segurança do Docker
2015	Docker responde com o Swarm mode , integrado ao Engine
2015	Apache Mesos /Marathon disputa o mesmo espaço
2015	OCI é criada; Docker doa o runC
2015	CNCF é criada; Kubernetes é o projeto fundador
2016	containerd é extraído do Docker e doado
2017	AWS lança EKS; Docker Inc. anuncia suporte a Kubernetes no Desktop

Em 2017 a disputa estava decidida: Kubernetes venceu a orquestração; Docker permaneceu como a ferramenta de desenvolvimento e o formato de imagem. rkt foi arquivado em 2020. Mesos foi para a manutenção mínima em 2021.

9.4.1 Por que o Kubernetes venceu

Cinco razões, em ordem de peso na minha leitura:

1. **Governança neutra.** Sob a CNCF, nenhum fornecedor controlava o projeto. Swarm era da Docker Inc.; adotá-lo era apostar numa empresa.
2. **A herança do Borg.** O Kubernetes já nasceu com respostas para problemas que só aparecem em escala real — e que quem estava começando ainda nem conhecia.
3. **Extensibilidade.** CRDs e operadores permitiram que o ecossistema estendesse a plataforma sem esperar o projeto principal.
4. **Todos os provedores de nuvem o adotaram,** porque nenhum queria a plataforma do concorrente.
5. **Modelo declarativo.** Descrever o estado desejado envelheceu melhor que emitir comandos.

E o que o Swarm fazia melhor — e ainda faz: **ser simples**. Para 3 servidores e 10 serviços, Swarm continua sendo tecnicamente adequado e muito mais barato de operar. Perdeu por dinâmica de ecossistema, não por mérito técnico isolado.

9.5 Ato 4 — 2018-2022: comoditização, dessacralização e o susto

9.5.1 O Docker deixa de ser insubstituível

Ano	Evento	Consequência
2018	Podman 1.0 (Red Hat)	Sem daemon, rootless por padrão, CLI compatível
2018	gVisor (Google)	Sandbox de syscalls em espaço de usuário
2018	Kata Containers	Containers dentro de micro-VMs
2018	BuildKit	Build paralelo, cache melhor, segredos
2019	Mirantis compra a Docker Enterprise	Docker Inc. fica com o produto de desenvolvedor
2019	Buildah, Skopeo, crun	O ferramental se fragmenta em peças pequenas

9.5.2 O episódio do dockershim (2020) — e o pânico que ele causou

Em dezembro de 2020, o Kubernetes anunciou a descontinuação do **dockershim**, o adaptador que permitia ao kubelet falar com o Docker Engine. Manchetes anunciaram que “o Kubernetes está removendo o Docker”.

O que realmente aconteceu: o Kubernetes definiu, em 2016, uma interface chamada **CRI** (*Container Runtime Interface*). containerd e CRI-O a implementam nativamente. O Docker Engine, não — ele é anterior à CRI e tem funcionalidades que o Kubernetes não usa (build, rede própria, Compose). O dockershim era código de adaptação que o time do Kubernetes mantinha de graça, dentro do próprio projeto, para um único fornecedor. Foi removido em 2022 (K8s 1.24).

O que mudou para você: essencialmente nada.

- As imagens continuam sendo as mesmas — são OCI, não “imagens Docker”.
- O docker build continua produzindo imagens que rodam em qualquer Kubernetes.
- Só quem operava clusters precisou trocar o runtime do nó para containerd — e o containerd já era o que o Docker usava por baixo.

A lição real do episódio: o Docker havia se tornado sinônimo de container no vocabulário popular, e essa confusão custou semanas de pânico à indústria inteira. Precisão de vocabulário tem valor operacional.

9.6 Ato 5 — 2021-2026: o modelo de negócio e a maturidade

9.6.1 A mudança de licença (2021)

Em agosto de 2021, a Docker Inc. alterou os termos do **Docker Desktop**: passou a exigir assinatura paga para empresas com mais de 250 funcionários **ou** mais de US\$ 10 milhões de receita anual.

Reação previsível e legítima: empresas migraram para Podman Desktop, Rancher Desktop, colima, OrbStack. Também previsível e legítimo: a Docker Inc. precisava de receita — o Docker Engine é Apache 2.0 e sempre foi gratuito; manter um app de desktop para macOS e Windows custa dinheiro.

Ponto que costuma se perder no debate: a mudança **nunca** afetou o Docker Engine no Linux, que continua livre. Muita gente migrou de ferramenta sem precisar.

9.6.2 Os limites do Docker Hub (2025)

Em **1º de abril de 2025**, entrou em vigor a política atual de limites de *pull*:

Situação	Limite
Sem autenticação	10 pulls/hora , por endereço IP
Conta gratuita autenticada	100 pulls/hora
Pro, Team, Business	Ilimitado, sob uso justo

Consequência prática mais dolorosa: pipelines de CI em IP compartilhado quebraram em massa. A resposta da indústria foi acelerar a migração para GHCR, ECR Public e espelhos internos. *Interpretação:* o Docker Hub foi, por uma década, uma infraestrutura pública crítica financiada por uma única empresa privada. Isso não era sustentável, e a correção veio de uma vez.

9.6.3 Onde estamos em agosto de 2026

- **Docker Engine 29** (29.7.1, de 04/08/2026): o **containerd image store** virou o padrão em instalações novas, e há suporte experimental a **nftables** como backend de firewall, em substituição ao iptables legado.
- **containerd 2.x** e **CRI-O** dominam os clusters; **Podman 5** e alternativas avançam nas máquinas de desenvolvimento.

- **OCI é o padrão inquestionado.** O registry deixou de ser “lugar de imagem” e virou armazenamento de artefatos genéricos: SBOMs, assinaturas, políticas, gráficos Helm, modelos de IA.
- A conversa se deslocou de “como rodo containers” para **cadeia de suprimentos** (proveniência, assinatura, SBOM), **isolamento mais forte** (micro-VMs, gVisor) e **runtimes alternativos** (WebAssembly).

Detalhes em [cap. 22](#).

9.7 O que a história ensina, em quatro lições

1. **A tecnologia raramente vence pela tecnologia.** Containers existiam havia 13 anos quando o Docker apareceu. Ganhou quem resolveu a experiência de uso e a distribuição.
2. **Padrão aberto é o que preserva o investimento.** A OCI é a razão pela qual sua imagem de 2016 ainda roda hoje, em runtimes que nem existiam então. Sem ela, teríamos formatos incompatíveis por fornecedor.
3. **Quem controla o formato não controla o mercado.** O Docker criou o formato e perdeu a orquestração para um projeto de governança neutra. Foi doar o runC e o containerd — e não retê-los — o que manteve o formato relevante.
4. **Infraestrutura pública gratuita tem um dono, e ele tem contas a pagar.** Docker Hub, npm, PyPI: quando a conta chega, os termos mudam. Planeje espelho e alternativa **antes** de o pipeline quebrar.

9.8 Autoteste

1. Todos os recursos de kernel para containers existiam desde 2008. Por que a adoção massiva só veio em 2013-2015?
2. Qual das invenções do Docker você considera a mais decisiva, e por quê? Defenda a escolha.
3. O que exatamente era o dockershim, por que foi removido, e o que mudou na sua vida?
4. Cite três razões pelas quais o Kubernetes venceu o Swarm, e uma coisa em que o Swarm é melhor.
5. A mudança de licença de 2021 afetou o Docker Engine no Linux? Justifique.
6. Que problema de negócio o Google estava resolvendo quando criou os cgroups em 2006?
7. Por que a criação da OCI em 2015 foi, no longo prazo, mais importante para você do que qualquer versão do Docker?
8. Que mudança de 01/04/2025 quebrou pipelines de CI no mundo inteiro, e qual foi a resposta da indústria?
9. O que mudou na Docker Engine 29 em relação ao armazenamento de imagens, e por que só afeta instalações novas?

9.8.1 Fontes consultadas (11/08/2026)

- [Docker \(software\) — Wikipedia](#) — cronologia geral

- [endoflife.date](#) — Docker Engine — Engine 29.7.1, 04/08/2026
- [Docker Engine v29 Release](#) — [Docker Blog](#) e [Linuxiac](#) — containerd image store padrão, nftables experimental
- [GitLab Support](#) — [Docker Hub rate limiting](#) — limites vigentes desde 01/04/2025

CAPÍTULO 10

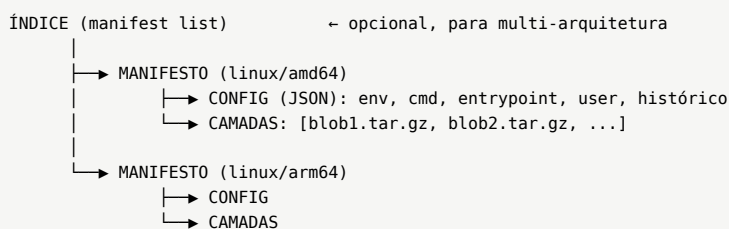
Imagens e camadas — o formato por dentro

Nível: intermediário → avançado · Última atualização: 11/08/2026

Se você entender este arquivo, para de errar em cache de build, tamanho de imagem e vazamento de segredo. É onde a maior parte do dinheiro e do tempo é perdida por desconhecimento.

10.1 O que uma imagem é, literalmente

Uma imagem OCI é composta de três tipos de objeto, todos identificados por hash SHA-256 do próprio conteúdo:



Veja de verdade:

```
docker manifest inspect nginx:alpine
# ou, sem depender de login:
docker buildx imagetools inspect nginx:alpine
```

bash

O manifesto é um JSON pequeno assim:

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "config": {
    "mediaType": "application/vnd.oci.image.config.v1+json",
    "digest": "sha256:9b1e2f...",
    "size": 7023
  },
  "layers": [
    { "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:alb2c3...", "size": 3401819 },
    { "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:d4e5f6...", "size": 1204 }
  ]
}
```

JSON

Cada camada é literalmente um arquivo .tar.gz com as alterações do sistema de arquivos daquele passo. Nada mais exótico que isso.

bash

```
# Extraia uma imagem e olhe por dentro
docker save alpine:3.20 -o alpine.tar
mkdir -p /tmp/alpine && tar -xf alpine.tar -C /tmp/alpine
ls /tmp/alpine
# esperado: blobs/, index.json, oci-layout (formato OCI moderno)
cat /tmp/alpine/index.json | jq
```

10.2 Endereçamento por conteúdo: por que o digest é imutável e a tag não

O identificador real de uma imagem é o **digest**: o SHA-256 do manifesto.

```
nginx:alpine ← tag: MUTÁVEL
nginx@sha256:1a2b3c...64 caracteres hexadecimais ← digest: IMUTÁVEL
```

- **Tag** é um ponteiro nomeado. Quem tem permissão de push pode reapontá-la a qualquer momento. `nginx:alpine` hoje e amanhã podem ser imagens completamente diferentes.
- **Digest** é derivado do conteúdo. Se o conteúdo mudar, o digest muda. Não há como reapontar um digest para outro conteúdo sem quebrar o SHA-256 — o que é a garantia de integridade.

bash

```
docker pull nginx:alpine
docker image inspect --format '{{index .RepoDigests 0}}' nginx:alpine
# nginx@sha256:...

# Em produção, referencie assim:
docker run nginx@sha256:1a2b3c...
```

Dockerfile

```
# E no Dockerfile, para builds realmente reproduzíveis:
FROM node:22.4.0-alpine@sha256:1a2b3c...
```

Isto é a base de tudo em cadeia de suprimentos. Assinatura, atestado de proveniência e SBOM referenciam o **digest**, nunca a tag. Uma política que diz “só rode imagens assinadas” é inútil se o deploy for por tag.

10.3 Como as camadas viram um sistema de arquivos: OverlayFS

O driver de armazenamento padrão hoje é o **overlay2**, que usa o OverlayFS do kernel.

```
Camada superior (upperdir) ← escrita do container, efêmera
┌──────────────────────────┴──────────────────────────┐
lowerdir 3           │
lowerdir 2           │└─ somente leitura, compartilhadas entre containers
lowerdir 1           │
└──────────────────────────┴──────────────────────────┘
merged                ← o que o processo enxerga como /
```

Três regras governam a união:

1. **Arquivo presente em várias camadas:** vence o da camada **mais alta**.
2. **Escrita em arquivo de camada inferior:** *copy-on-write* — o arquivo inteiro é copiado para a camada superior e a escrita ocorre na cópia.
3. **Remoção:** cria-se um **whiteout** (um arquivo de dispositivo de caractere 0:0 com o mesmo nome) na camada superior, que esconde o original. **O original continua ocupando espaço.**

bash

```
# Veja a estrutura real
docker run -d --name teste alpine sleep 3600
docker inspect --format '{{json .GraphDriver.Data}}' teste | jq
# esperado: LowerDir, UpperDir, MergedDir, WorkDir com caminhos em /var/lib/docker/overlay2/

# Prove o copy-on-write
docker exec teste sh -c 'echo alterado >> /etc/hostname'
docker diff teste
# esperado: C /etc e C /etc/hostname (C = changed)
docker rm -f teste
```

10.3.1 As consequências de desempenho que você vai encontrar

Situação	Efeito	Mitigação
Primeira escrita em arquivo grande	Copia o arquivo inteiro para a camada superior	Coloque dados mutáveis em volume , não na camada de escrita
Muitas camadas (>50)	Cada <code>open()</code> percorre a pilha; latência cresce	Consolide RUNs; use multi-stage
Banco de dados na camada de escrita	Escrita aleatória sofre com CoW	Sempre use volume para banco
Muitos arquivos pequenos	page cache compartilhado ajuda, mas metadados pesam	Menos camadas; base menor

10.4 O cache de build: a regra e as cinco consequências

A regra, em uma frase: o Docker calcula uma chave de cache por instrução; se a chave bate e **todas as instruções anteriores também bateram**, a camada é reaproveitada. Uma invalidação invalida tudo dali para baixo.

Como a chave é calculada:

Instrução	Chave de cache baseada em
FROM	Digest da imagem base
RUN	O texto literal do comando — não o resultado dele
COPY / ADD	Checksum do conteúdo dos arquivos copiados + destino
ENV, ARG, LABEL, WORKDIR	O texto da instrução

10.4.1 Consequência 1 — ordene do menos volátil ao mais volátil

Dockerfile

```
# [x] 4 minutos a cada mudança de uma linha de código
COPY . .
RUN npm ci

# [ok] o npm ci só refaz quando as dependências mudam
COPY package.json package-lock.json ./
```

```
RUN npm ci
COPY . .
```

10.4.2 Consequência 2 — RUN apt-get update sozinho é uma armadilha

Dockerfile

```
# [x] o 'update' fica em cache por semanas; o 'install' pega índice velho
RUN apt-get update
RUN apt-get install -y curl

# [ok] mesma camada: se um roda, o outro roda
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

Isso tem nome: **cache busting**. O apt-get update cacheado aponta para espelhos cujos pacotes já mudaram, e o install falha com 404 Not Found — erro que confunde porque “funcionava ontem”.

10.4.3 Consequência 3 — .dockerignore faz parte da chave de cache

Se node_modules não estiver no .dockerignore, qualquer alteração ali invalida o COPY . . — e você não vai entender por quê.

10.4.4 Consequência 4 — o cache é local por padrão

Em CI, cada execução começa em uma máquina limpa e o cache não existe. Solução: cache externo.

bash

```
docker buildx build \
  --cache-from type=registry,ref=usuario/app:cache \
  --cache-to type=registry,ref=usuario/app:cache,mode=max \
  -t usuario/app:1.0 --push .
```

mode=max exporta o cache de **todos os estágios** (inclusive os intermediários do multi-stage); mode=min, só o do estágio final. Em multi-stage, mode=max é quase sempre o que você quer.

10.4.5 Consequência 5 — cache montado é diferente de camada cacheada

Dockerfile

```
RUN --mount=type=cache,target=/root/.npm npm ci
```

Esse cache **persiste entre builds diferentes e não entra na imagem**. É o mecanismo certo para caches de gerenciador de pacotes (~/.npm, ~/.cache/pip, /go/pkg/mod, ~/.cargo/registry, /var/cache/apt). A camada continua sendo recalculada, mas o download não.

bash

```
docker builder prune          # limpa esses caches quando o disco apertar
docker builder du             # veja quanto ocupam
```

10.5 Por que sua imagem está gorda — e como emagrecer

10.5.1 Diagnóstico primeiro

bash

```
docker images --format "table {{.Repository}}:{{.Tag}}\t{{.Size}}"
docker history minha-app:1.0          # camada a camada
docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock \
wagoodman/dive minha-app:1.0          # a melhor ferramenta: mostra o desperdício por camada
```

O dive calcula um "score de eficiência" e lista arquivos que são adicionados e depois removidos ou sobrescritos — exatamente o desperdício que o `docker history` não mostra.

10.5.2 As técnicas, em ordem de retorno sobre o esforço

#	Técnica	Redução típica
1	Multi-stage: compilar em um estágio, copiar só o artefato	60–90%
2	Base menor: alpine, slim, distroless	50–80%
3	.dockerignore correto	variável, às vezes enorme
4	Consolidar RUN e limpar cache na mesma camada	5–30%
5	<code>--no-install-recommends (apt)</code> / <code>--no-cache (apk)</code>	10–40%
6	Remover ferramentas de build da imagem final	incluído no (1)
7	<code>strip/ldflags="-s -w"</code> em binários compilados	20–30% do binário

10.5.3 Comparação de bases (ordens de grandeza; confira na sua máquina)

Base	Tamanho aprox.	Tem shell?	Gerenciador de pacotes	Quando usar
scratch	0	não	não	Binário estático (Go, Rust)
gcr.io/distroless/static	~2 MB	não (há variante <code>:debug</code>)	não	Binário estático, com certificados e usuário nonroot
alpine:3.20	~8 MB	sim (ash)	apk	Padrão para a maioria
debian:12-slim	~75 MB	sim	apt	Quando precisa de glibc
ubuntu:24.04	~78 MB	sim	apt	Familiaridade
node:22-alpine	~50 MB	sim	apk + npm	Node enxuto
node:22	~400 MB	sim	apt + npm	Só se precisar de toolchain

10.5.4 ⚠ A ressalva importante sobre Alpine

Alpine usa **musl libc** em vez de **glibc**. Isso causa três problemas reais:

1. **Pacotes Python com rodas binárias (manylinux) não funcionam.** O pip cai para compilar do fonte, o que exige toolchain e faz o build durar 10 minutos em vez de 20 segundos. Para Python, `python:3.13-slim` costuma ser a escolha melhor.
2. **Diferenças de desempenho no alocador de memória** já foram medidas como significativas em cargas com muita alocação. Menos crítico nas versões recentes do musl, mas ainda real.

3. **Resolução de DNS** tem diferenças históricas de comportamento (uso de search, TCP como fallback) que causam falhas intermitentes difíceis de diagnosticar.

Recomendação: Alpine para Go, Rust, C compilado estaticamente e Node (que traz seu próprio runtime). slim (Debian) para Python, Ruby e qualquer coisa com extensões nativas.

10.5.5 O padrão multi-stage, resumido

Dockerfile

```
FROM node:22 AS build          # 400 MB de toolchain, tudo bem
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS producao # 50 MB
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node
CMD ["node", "dist/main.js"]
```

O estágio de build **não vai** para o registry. Ele existe só durante o docker build.

10.6 Segredos vazam em camadas — a demonstração

Esta seção existe porque é o erro com maior consequência e menor visibilidade.

Dockerfile

```
# [x] ERRAO
ARG NPM_TOKEN
RUN echo "//registry.npmjs.org/:_authToken=${NPM_TOKEN}" > ~/.npmrc \
    && npm ci \
    && rm ~/.npmrc          # o rm NÃO adianta
```

Prove o vazamento por três caminhos independentes:

bash

```
# 1) O ARG fica no histórico e na config da imagem
docker history --no-trunc IMAGE | grep -i token
docker image inspect IMAGE --format '{{json .Config.Env}}'

# 2) O arquivo está no tar de alguma camada
docker save IMAGE -o img.tar && tar -xf img.tar -C /tmp/img
grep -ra "_authToken" /tmp/img/blobs | head

# 3) Um container montado numa camada intermediária o encontra
docker run --rm IMAGE cat /root/.npmrc 2>/dev/null # pode não estar na final...
# ...mas a camada anterior, sim — e ela está no registry.
```

Isso vale para **qualquer** segredo: chave SSH, token de nuvem, senha de banco, certificado. Uma imagem publicada num registry público com uma dessas camadas é uma credencial vazada.

O jeito certo:

Dockerfile

```
# syntax=docker/dockerfile:1
RUN --mount=type=secret,id=npnrc,target=/root/.npmrc,mode=0400 npm ci
```

bash

```
docker buildx build --secret id=npnrc,src=$HOME/.npmrc -t app:1.0 .
```

O arquivo é montado num tmpfs que existe apenas durante aquele RUN e **nunca vira camada**.

Regra final: ARG para configuração não sensível (versão, arquitetura, flags); --mount=type=secret para segredo. Sem exceção.

10.7 Imagens multi-arquitetura

Um **índice** (antes chamado *manifest list*) aponta para vários manifestos, um por os/arquitetura. Quando você faz docker pull, o cliente informa sua plataforma e o registry devolve o manifesto certo.

bash

```
docker buildx imagetools inspect alpine:3.20
# esperado: entradas para linux/amd64, linux/arm64, linux/arm/v7, linux/386, ...
```

bash

```
docker buildx create --name multi --driver docker-container --bootstrap --use
docker buildx build --platform linux/amd64,linux/arm64 -t usuario/app:1.0 --push .
```

Erros clássicos aqui:

- exec format error = você está rodando uma imagem de outra arquitetura. Confirme com `docker image inspect --format '{{.Architecture}}'`.
- Build ARM64 emulado por QEMU pode levar **10x mais tempo** ou falhar em dependências nativas. Use `--platform=$BUILDPLATFORM` + cross-compilação quando a linguagem permitir, ou runners nativos por arquitetura no CI.
- Multi-arch tradicionalmente exigia `--push` (o armazenamento local legado não guardava índices com múltiplas plataformas). Com o **containerd image store** — padrão em instalações novas desde a Engine 29 — isso deixa de ser limitação.

10.8 O armazenamento de imagens: overlay2 e o containerd image store

Historicamente, o Docker mantinha seu próprio banco de imagens em `/var/lib/docker`, com o driver overlay2. A partir da **Engine 29**, instalações novas passam a usar o **containerd image store** por padrão.

	Store legado do Docker	containerd image store
Quem gerencia	dockerd	containerd
Multi-plataforma local	limitado	nativo
Compartilhamento com Kubernetes/nerdctl	não	sim

	Store legado do Docker	containerd image store
Snapshotters plugáveis (stargz, overlaybd)	não	sim
Migração ao atualizar	—	não é automática: vale para instalações novas

bash

```
docker system info | grep -i -A2 "storage driver"
# esperado no padrão novo: menção a containerd snapshotter
```

Por que isso importa: o containerd é o mesmo runtime que o Kubernetes usa. Unificar o armazenamento aproxima o que você constrói na máquina do que roda no cluster, e destrava recursos como *lazy pulling* (iniciar o container antes de baixar a imagem inteira, via stargz/eStargz) — relevante quando a imagem tem gigabytes, como as de IA.

10.9 Metadados e proveniência

Dockerfile

```
LABEL org.opencontainers.image.title="minha-api" \
org.opencontainers.image.description="API de pedidos" \
org.opencontainers.image.version="1.4.2" \
org.opencontainers.image.revision="$GIT_SHA" \
org.opencontainers.image.source="https://github.com/org/repo" \
org.opencontainers.image.licenses="MIT"
```

E os artefatos modernos que acompanham a imagem no registry:

Artefato	O que é	Ferramenta
SBOM	Lista de todos os componentes e versões	docker buildx build --sbom=true, syft
Proveniência (SLSA)	Como, onde e a partir de quê a imagem foi construída	--provenance=true
Assinatura	Prova criptográfica de quem publicou	cosign sign
Atestado de vulnerabilidade	Resultado de escaneamento anexado	trivy, gype, Docker Scout

bash

```
docker buildx imagetools inspect usuario/app:1.0 --format '{{json .SBOM}}'
cosign verify usuario/app:1.0 --certificate-identity-regexp '.*' \
--certificate-oidc-issuer https://token.actions.githubusercontent.com
```

Esses artefatos ficam **no registry, ao lado da imagem**, referenciados pelo digest. É o registry deixando de ser depósito de imagem e virando depósito de artefatos OCI.

10.10 Autoteste

1. Qual é a diferença entre tag e digest, e por que política de segurança referencia digest?
2. Explique copy-on-write e diga por que colocar um banco de dados na camada de escrita é ruim.

3. Por que `RUN apt-get update` numa camada e `RUN apt-get install` em outra causa erro 404 semanas depois?
4. O que a chave de cache de um `RUN` leva em conta — o comando ou o resultado dele? Qual é a consequência?
5. Cite três problemas concretos de usar Alpine, e para quais linguagens você o evitaria.
6. Você usou `--build-arg TOKEN=xyz` e depois `rm`. Descreva três formas independentes de recuperar o token da imagem.
7. Qual é a diferença entre `mode=min` e `mode=max` no cache de registry, e qual usar em multi-stage?
8. O que é um whiteout no OverlayFS e por que ele explica "apaguei e a imagem não diminuiu"?
9. O que muda com o containerd image store, e por que uma atualização de versão não migra automaticamente?
10. Como você provaria, em um comando, que duas imagens diferentes compartilham a mesma camada base no disco?

CAPÍTULO 11

Isolamento — namespaces, cgroups e o que o kernel realmente faz

Nível: avançado · Última atualização: 11/08/2026

Aqui não há caixa-preta. Vamos construir um container à mão, sem Docker, e depois desmontar o que o Docker faz por cima disso.

Os comandos deste arquivo exigem **Linux** e, em alguns casos, `sudo`. Em macOS/Windows, rode-os dentro de uma VM Linux ou no WSL2.

11.1 Os sete namespaces, um a um

Um namespace faz um conjunto de processos ter uma **visão própria** de um recurso global do kernel. Não há cópia do recurso: há uma indireção na estrutura de dados do kernel.

```
# Todo processo tem seus namespaces expostos como links simbólicos
ls -l /proc/self/ns/
# esperado: cgroup, ipc, mnt, net, pid, pid_for_children, time, user, uts
# cada um aponta para algo como 'net:[4026531840]' — o número é o ID do namespace
```

bash

Dois processos com o **mesmo número** compartilham aquele namespace.

11.1.1 mnt — pontos de montagem (Linux 2.4.19, 2002)

Dá ao processo sua própria tabela de montagens. É o que torna possível o container ter / próprio.

```
sudo unshare --mount bash
mount -t tmpfs tmpfs /mnt      # só este shell enxerga essa montagem
findmnt /mnt
exit
findmnt /mnt                  # não existe fora
```

bash

A sutileza que causa bug real: montagens têm **propagação** (shared, private, slave). Se o Docker montar algo como shared, montagens feitas dentro do container vazam para o host. É por isso que `--privileged` combinado com montagens é tão perigoso, e por que `/sys/fs/cgroup` costuma ser montado como somente leitura.

11.1.2 uts — hostname (2.6.19, 2006)

```
sudo unshare --uts bash
hostname meu-container
hostname      # meu-container
exit
hostname      # o do host, intacto
```

bash

“UTS” vem de *Unix Time-Sharing System* — um nome herdado da estrutura `utsname`, que também carrega o nome do sistema. É uma **convenção arbitrária** de nomenclatura, e vale dizer isso em vez de inventar um significado.

11.1.3 ipc — comunicação entre processos (2.6.19, 2006)

Isola memória compartilhada System V, filas de mensagens e semáforos. É o motivo pelo qual dois containers não conseguem, por padrão, se comunicar por memória compartilhada — e por que `--ipc=host` existe para os casos em que isso é necessário (PyTorch com múltiplos DataLoader workers, por exemplo, que estoura o `/dev/shm` padrão de 64 MB).

11.1.4 pid — tabela de processos (2.6.24, 2008)

O mais didático de todos.

bash

```
sudo unshare --pid --fork --mount-proc bash
ps aux          # apenas bash e ps
echo $$         # 1 - este shell é o PID 1
exit
```

`--mount-proc` é necessário porque o `ps` lê `/proc`; sem remontar `/proc` no novo namespace, você continuaria vendo os processos do host.

Três propriedades do PID namespace que geram comportamento surpreendente:

1. **É hierárquico.** O processo tem PID 1 dentro e um PID normal fora. O host vê tudo; o container não vê o host.
2. **PID 1 tem tratamento especial do kernel:** sinais sem tratador registrado são **descartados**. É por isso que `docker stop` demora 10 s em containers que não tratam `SIGTERM`.
3. **Se o PID 1 morre, o kernel mata todos os processos daquele namespace.** É o que faz o container terminar por inteiro quando o processo principal termina.

11.1.5 net — pilha de rede (2.6.29, 2009)

Cada namespace de rede tem interfaces, endereços, tabelas de roteamento, regras de firewall e portas próprios.

bash

```
sudo ip netns add teste
sudo ip netns exec teste ip addr      # só 'lo', e DOWN
sudo ip netns exec teste ip link set lo up
sudo ip netns del teste
```

Consequência que confunde todo iniciante: 127.0.0.1 dentro do container **não é** o 127.0.0.1 do host. São loopbacks diferentes. E dois containers podem ambos escutar na porta 80 sem conflito, porque cada um tem sua própria tabela de portas.

11.1.6 user — mapeamento de UID/GID (3.8, 2013)

O mais poderoso e o mais tardio. Permite que o UID 0 (root) **dentro** do namespace corresponda a um UID sem privilégio **fora**.

bash

```
unshare --user --map-root-user bash    # NÃO precisa de sudo!
id                                     # uid=0(root)
cat /proc/self/uid_map                 # 0 1000 1 → root dentro = uid 1000 fora
touch /etc/teste                       # Permission denied: você não é root de verdade
exit
```

É a base do **rootless containers** (Podman por padrão, Docker rootless mode). Também é a base do `--userns-remap` do Docker.

Por que só chegou em 2013, cinco anos depois dos outros? Porque tocar no modelo de credenciais do kernel é a mudança mais arriscada possível — e, de fato, o user namespace foi fonte de uma sequência de CVEs de escalada de privilégio nos anos seguintes. Algumas distros chegaram a desabilitá-lo para usuários comuns por precaução.

11.1.7 cgroup (4.6, 2016) e time (5.6, 2020)

O namespace de cgroup esconde a hierarquia real, para o container não descobrir o caminho do seu cgroup no host. O de tempo permite deslocar `CLOCK_MONOTONIC` e `CLOCK_BOOTTIME` — usado sobretudo em *checkpoint/restore* (CRIU), quando um processo migra de máquina.

11.2 Construindo um container à mão, sem Docker

Este exercício vale mais que dez artigos. Em uma máquina Linux:

bash

```
# 1) Um sistema de arquivos raiz. Pegamos o do Alpine, via Docker, só para não baixar à mão.
mkdir -p /tmp/meurootfs
docker export $(docker create alpine:3.20) | tar -x -C /tmp/meurootfs
ls /tmp/meurootfs          # bin dev etc home lib ... — um Linux completo, ~8 MB

# 2) Crie os namespaces e troque a raiz
sudo unshare --mount --uts --ipc --pid --fork --net --mount-proc=/tmp/meurootfs/proc \
  chroot /tmp/meurootfs /bin/sh

# --- você está "dentro" ---
hostname container-artesanal
ps aux                     # só sh e ps
ls /                       # o rootfs do Alpine
cat /etc/os-release        # Alpine Linux
ip addr                    # só lo — sem rede, porque não configuramos o par veth
exit
```

Você acabou de criar um container com **um comando do coreutils**. O que o Docker acrescenta:

O Docker faz	O que fizemos acima
Baixa e monta camadas via OverlayFS	Extraímos um tar
Cria par veth, bridge, NAT, DNS	Nada (<code>--net</code> isolou, mas não conectou)
Aplica cgroups	Nada
Aplica seccomp, capabilities, AppArmor	Nada (herdamos root pleno — perigoso)
Gerencia ciclo de vida, logs, restart	Nada
Distribui a imagem por registry	Nada

A conclusão importante: o isolamento é do **kernel**. O Docker é a ergonomia, a distribuição e as políticas de segurança padrão. Isso explica por que Podman, containerd e CRI-O produzem containers equivalentes: todos chamam o mesmo kernel.

11.3 cgroups v2 na prática

bash

```
stat -fc %T /sys/fs/cgroup/      # cgroup2fs = v2
```

Crie e use um cgroup manualmente:

bash

```
sudo mkdir /sys/fs/cgroup/meu-teste

# Habilite os controladores no pai (na v2, é preciso delegar explicitamente)
cat /sys/fs/cgroup/cgroup.controllers      # o que está disponível
echo "+memory +cpu +pids" | sudo tee /sys/fs/cgroup/cgroup.subtree_control

# Limite: 100 MB de memória, 50% de um núcleo, 20 processos
echo 100000000 | sudo tee /sys/fs/cgroup/meu-teste/memory.max
echo "50000 100000" | sudo tee /sys/fs/cgroup/meu-teste/cpu.max # 50ms a cada 100ms
echo 20 | sudo tee /sys/fs/cgroup/meu-teste/pids.max

# Coloque um shell dentro e observe
echo $$ | sudo tee /sys/fs/cgroup/meu-teste/cgroup.procs
cat /sys/fs/cgroup/meu-teste/memory.current      # uso atual, em bytes

# Estoure o limite (o kernel mata o processo)
python3 -c "a=[]
while True: a.append(' '*10_000_000)"
# esperado: Killed
dmesg | tail -5      # a mensagem do OOM killer, com o cgroup nomeado

sudo rmdir /sys/fs/cgroup/meu-teste
```

11.3.1 Os arquivos que valem conhecer (cgroup v2)

Arquivo	O que faz
memory.max	Teto rígido. Ultrapassar = OOM kill
memory.high	Teto flexível : acima dele o kernel aplica pressão e throttling, sem matar
memory.current	Uso atual
memory.events	Contadores: high, max, oom, oom_kill
cpu.max	"QUOTA PERÍODO" em microssegundos. "max 100000" = sem limite
cpu.stat	nr_throttled, throttled_usec — a métrica que revela throttling de CPU
pids.max	Máximo de processos/threads
io.max	Limites de IOPS e vazão por dispositivo
cgroup.procs	Os PIDs no grupo

11.3.2 memory.high vs memory.max — a diferença que o Docker esconde

-m 512m no Docker define memory.max: ao ultrapassar, o processo **morre**. --memory-reservation mapeia para algo próximo de memory.high: o kernel aplica pressão de recuperação de memória, degradando desempenho em vez de matar.

Opinião profissional: usar memory.high um pouco abaixo de memory.max produz um comportamento muito mais operável — o serviço fica lento e emite sinal antes de morrer, dando tempo de alertar. Poucos usam porque a exposição pelo Docker é indireta.

11.3.3 O throttling de CPU que ninguém vê

bash

```
docker run -d --name t --cpus 0.5 alpine sh -c 'while ;; do ;; done'
sleep 5
cat /sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}' t).scope/cpu.stat
# nr_throttled > 0 e throttled_usec crescendo → o processo está sendo estrangulado
docker rm -f t
```

Este é um dos diagnósticos mais valiosos de produção. Latência alta com CPU aparentemente ociosa costuma ser throttling: o container consome sua cota em 20 ms do período de 100 ms e fica 80 ms parado. Aplicações com muitas threads (JVM, Go com GOMAXPROCS alto) atingem a cota mais rápido do que o esperado. A correção pode ser aumentar a cota **ou** reduzir o paralelismo interno.

11.4 Capabilities: o que o container pode fazer

bash

```
# 0 que o Docker concede por padrão
docker run --rm alpine sh -c 'apk add -q libcap 2>/dev/null; capsh --print' 2>/dev/null | head -3
# ou, de forma mais direta:
docker run --rm alpine grep CapEff /proc/1/status
# CapEff: 00000000a80425fb → uma máscara de bits das capabilities efetivas
```

As 14 concedidas por padrão pelo Docker:

CHOWN, DAC_OVERRIDE, FSETID, FOWNER, MKNOD, NET_RAW, SETGID, SETUID, SETFCAP, SETPCAP, NET_BIND_SERVICE, SYS_CHROOT, KILL, AUDIT_WRITE.

Duas dessas são discutíveis:

- **NET_RAW** permite construir pacotes crus: ARP spoofing e varredura de rede a partir do container. Muitos guias de segurança recomendam removê-la. O motivo de ela estar no padrão é histórico: fazer ping funcionar sem configuração extra. É um **trade-off explícito de conveniência sobre segurança**.
- **DAC_OVERRIDE** permite ignorar permissões de arquivo — o que reduz o benefício de rodar como não-root em vários cenários.

O caminho recomendado:

bash

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE minha-app
```

E se o app escuta em porta > 1024 (o que deveria), nem essa é necessária: `--cap-drop=ALL` puro.

11.4.1 --privileged: o que ele realmente faz

bash

```
docker run --privileged ...
```

Concede **todas** as capabilities, desabilita seccomp e AppArmor, e dá acesso a todos os dispositivos do host. **É equivalente a rodar como root no host, sem barreira.** Prova:

bash

```
docker run --rm --privileged alpine sh -c 'mkdir -p /h && mount /dev/sda1 /h && ls /h'
# o disco do host, montado dentro do container
```

Casos em que às vezes se justifica: Docker-in-Docker, ferramentas que manipulam o kernel, alguns testes de sistema. Em **todos** eles existe uma alternativa mais restrita: `--cap-add` específico, `--device` específico, ou sysbox/kaniko no lugar de DinD.

Regra: `--privileged` num docker run de produção é um incidente de segurança esperando data.

11.5 seccomp

O perfil padrão do Docker bloqueia cerca de 40 das ~350 chamadas de sistema do Linux, entre elas `kexec_load` (trocar o kernel), `open_by_handle_at` (base do exploit "Shocker", de 2014), `init_module`, `bpf` e `clock_settime`.

bash

```
# Sem seccomp, a syscall passa; com o perfil padrão, é bloqueada
docker run --rm --security-opt seccomp=unconfined alpine sh -c 'echo teste'

# Perfil customizado (allow-list)
docker run --security-opt seccomp=/caminho/perfil.json minha-app
```

Ferramentas como `oci-seccomp-bpf-hook` e `strace` permitem **gerar** um perfil observando o que o app realmente chama. Isso é trabalho de segurança madura, não de primeiro dia — mas é o que reduz a superfície de kernel de 350 syscalls para as 40 que seu app usa.

11.6 Rootless: o modelo mais seguro

No modo tradicional, o `dockerd` roda como root. Quem tem acesso ao socket tem root no host (demonstrado em [03-instalacao.md](#) (cap. 3)).

No **rootless**, o daemon e os containers rodam como seu usuário, usando `user namespace` + `slirp4netns`/pasta para rede.

bash

```
dockerd-rootless-setup.sh install
export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock
docker info | grep -i rootless
```

Ganho

Escape de container não entrega root do host
Vários usuários, daemons isolados
Não requer privilégio para instalar

—

Perda

Portas < 1024 exigem configuração extra
Rede mais lenta (userspace networking)
Alguns drivers de storage e rede indisponíveis
`--net=host`, cgroup limitado sem delegação

O **Podman** é **rootless por padrão** e sem daemon — cada `podman run` é um `fork/exec` direto, com um `common` por container. É a razão principal de sua adoção em ambientes regulados.

11.7 O que o isolamento NÃO protege

Honestidade sobre o modelo de ameaça:

Vetor	Container protege?	Observação
App consome toda a RAM	✓ (com cgroup)	Só se você definiu o limite
App vê processos do host	✓	PID namespace
App lê arquivos do host	✓	Salvo bind mount, --privileged, socket montado
Vulnerabilidade no kernel	✗	Kernel é compartilhado. Um bug em syscall afeta o host
Canal lateral de CPU (Spectre, L1TF)	✗	Mitigado por microcódigo e flags do kernel, não pelo container
Exaustão de recurso não limitada por cgroup	⚠	Descritores de arquivo, portas, inodes, entropia
--privileged ou socket do Docker montado	✗	Você entregou a máquina voluntariamente
Imagem base maliciosa	✗	É um problema de cadeia de suprimentos, não de isolamento

Quando é preciso mais que container:

Tecnologia	Como funciona	Custo
gVisor (Google)	Um kernel em espaço de usuário, em Go, intercepta as syscalls. O kernel real vê pouquíssimas chamadas	10-30% de sobrecarga em I/O; algumas syscalls não suportadas
Kata Containers	Cada container numa micro-VM leve, com kernel próprio	~100-200 ms extras no start, mais RAM
Firecracker (AWS)	Micro-VM minimalista; base do Lambda e do Fargate	~125 ms de boot, footprint mínimo
Sandbox de V8 (Cloudflare Workers)	Isolados de JavaScript, sem processo por tenant	Só serve para JS/Wasm

Todos são **drop-in** via --runtime ou RuntimeClass no Kubernetes:

```
docker run --runtime=runc alpine # gVisor
```

bash

11.8 Depuração de isolamento — a caixa de ferramentas

```
# Em quais namespaces o container está?
PID=$(docker inspect -f '{{.State.Pid}}' NOME)
sudo ls -l /proc/$PID/ns/

# Entrar nos namespaces de um container sem usar 'docker exec'
sudo nsenter -t $PID -n -p -m -u ip addr

# Qual é o cgroup dele, e quanto está consumindo?
sudo cat /proc/$PID/cgroup
CG=/sys/fs/cgroup/(sudo cat /proc/$PID/cgroup | cut -d: -f3)
cat $CG/memory.current $CG/memory.max
cat $CG/cpu.stat | grep throttled
```

bash

```
# Quais capabilities ele tem?
sudo grep Cap /proc/$PID/status
sudo capsh --decode=$(sudo grep CapEff /proc/$PID/status | awk '{print $2}')

# seccomp está ativo?
sudo grep Seccomp /proc/$PID/status
# 0 = desligado · 2 = filtro ativo

# 0 que o container enxerga como raiz?
sudo ls /proc/$PID/root/
```

nsenter é a ferramenta que salva quando a imagem não tem shell: você entra nos namespaces do alvo usando o binário **do host**.

11.9 Autoteste

1. Liste os sete namespaces e diga, para cada um, um sintoma observável de que ele está ativo.
2. Por que unshare --pid --fork precisa também de --mount-proc para o ps fazer sentido?
3. Escreva a sequência de comandos que cria um container à mão, sem Docker.
4. Por que o user namespace só chegou ao kernel em 2013, cinco anos depois do PID namespace?
5. Qual é a diferença entre memory.max e memory.high, e por que a segunda é mais operável?
6. Você tem latência alta e a CPU parece ociosa. Qual arquivo de cgroup você lê, e o que procura?
7. Cite duas capabilities concedidas por padrão pelo Docker que são discutíveis, e por quê.
8. O que --privileged faz exatamente? Demonstre o risco em um comando.
9. Cite três coisas contra as quais o isolamento por namespace **não** protege.
10. Quando você escolheria gVisor em vez de runc, e o que pagaria por isso?

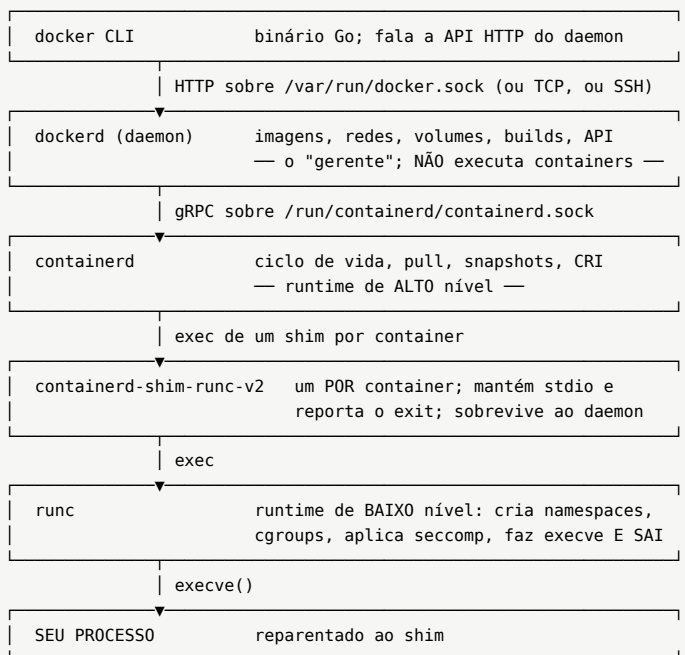
CAPÍTULO 12

Runtime e arquitetura — quem faz o quê, da CLI ao processo

Nível: avançado · Última atualização: 11/08/2026

Rastrear um `docker run` do teclado até a chamada `execve()` é o que dissolve as últimas caixas-pretas.

12.1 As camadas, e por que existem tantas



Veja a árvore real:

```
bash
docker run -d --name arv nginx:alpine
PID=$(docker inspect -f '{{.State.Pid}}' arv)
ps -o pid,ppid,cmd --forest -p $PID $(ps -o ppid= -p $PID)
pstree -p $(pgrep -f containerd-shim | head -1)
docker rm -f arv
```

Note o que **não** aparece: o `runc`. Ele já terminou.

12.1.1 Por que essa separação — as cinco razões

1. **runc executa e sai.** É um programa de vida curta que faz `clone()`, configura tudo e `execve()`. Depois, some. Só o shim permanece.

2. **O shim mantém o container vivo se o daemon reiniciar.** Com live-restore ativado, `systemctl restart docker` **não derruba os containers** — o shim segura o `stdio` e a contabilidade do `exit code`, e o daemon reconecta.
3. **Padronização.** `runc` implementa a OCI Runtime Spec; `containerd` implementa a CRI do Kubernetes. Ambos podem ser trocados por implementações alternativas.
4. **Responsabilidade única.** `containerd` não sabe construir imagem, não sabe `Compose`, não tem opinião sobre rede. Isso o tornou adotável por Kubernetes, `nerdctl`, Rancher e AWS.
5. **Modelo de segurança.** Menos código rodando como `root` permanentemente.

bash

```
# Habilite live-restore e comprove
sudo tee /etc/docker/daemon.json >/dev/null <<'EOF'
{ "live-restore": true }
EOF
sudo systemctl restart docker

docker run -d --name sobrevivente nginx:alpine
sudo systemctl restart docker
docker ps      # o container continua "Up" — não reiniciou
docker rm -f sobrevivente
```

Limitação de live-restore: não funciona em Swarm mode, e uma mudança de configuração de rede do daemon ainda exige derrubar os containers.

12.2 O que acontece, passo a passo, num `docker run -d -p 8080:80 nginx:alpine`

#	Quem	O que faz
1	CLI	Traduz as flags e faz <code>POST /containers/create</code> no socket
2	<code>dockerd</code>	Resolve <code>nginx:alpine</code> → <code>docker.io/library/nginx:alpine</code>
3	<code>dockerd</code>	Não achou local: pede à <code>containerd</code> para baixar
4	<code>containerd</code>	Autentica no registry, baixa manifesto , depois cada camada ausente
5	<code>containerd</code>	Descompacta as camadas em <i>snapshots</i> (overlay2 ou o snapshotter do <code>containerd</code>)
6	<code>dockerd</code>	Cria o registro do container, aloca IP na bridge, prepara o volume/rootfs
7	<code>dockerd</code>	Escreve as regras de NAT (iptables ou nftables) para o <code>-p 8080:80</code>
8	<code>dockerd</code>	<code>POST /containers/{id}/start</code> → chama a <code>containerd</code>
9	<code>containerd</code>	Monta o rootfs (merged do OverlayFS) e gera o <code>config.json</code> da OCI
10	<code>containerd</code>	Faz exec de <code>containerd-shim-runc-v2</code>
11	shim	Faz exec de <code>runc create</code> e <code>runc start</code>
12	runc	<code>clone()</code> com as flags de namespace; entra no cgroup; aplica capabilities, <code>sec-comp</code> , <code>AppArmor</code> ; faz <code>pivot_root</code> ; <code>execve("/docker-entrypoint.sh")</code>
13	runc	Termina. O processo do <code>nginx</code> é reparentado ao shim
14	shim	Captura <code>stdout/stderr</code> e os entrega ao driver de log do daemon

Você pode observar boa parte disso ao vivo:

bash

```
docker events & # eventos do daemon
sudo journalctl -u docker -f & # log do daemon
docker run --rm alpine echo oi
kill %1 %2
```

12.3 O config.json da OCI — o contrato

Todo container é, para o runc, um **bundle**: um diretório com config.json + rootfs/.

bash

```
# Veja o config.json real que o Docker gerou para um container em execução
docker run -d --name espiar --memory 256m --cap-drop=ALL nginx:alpine
ID=$(docker inspect -f '{{.Id}}' espiar)
sudo cat /run/containerd/io.containerd.runtime.v2.task/moby/$ID/config.json | jq '{process: .process.args, user:
↳ .process.user, caps: .process.capabilities.effective, ns: [.linux.namespaces[].type], mem: .linux.resources.memory}'
docker rm -f espiar
```

Estrutura essencial:

JSON

```
{
  "ociVersion": "1.2.0",
  "process": {
    "terminal": false,
    "user": { "uid": 101, "gid": 101 },
    "args": ["nginx", "-g", "daemon off;"],
    "env": ["PATH=/usr/local/sbin:...", "NGINX_VERSION=1.27.0"],
    "cwd": "/",
    "capabilities": { "effective": ["CAP_CHOWN", "CAP_NET_BIND_SERVICE"] },
    "noNewPrivileges": true
  },
  "root": { "path": "rootfs", "readonly": false },
  "hostname": "alb2c3d4",
  "mounts": [ { "destination": "/proc", "type": "proc", "source": "proc" } ],
  "linux": {
    "resources": {
      "memory": { "limit": 268435456 },
      "cpu": { "quota": 50000, "period": 100000 },
      "pids": { "limit": 200 }
    },
    "namespaces": [
      { "type": "pid" }, { "type": "network" }, { "type": "ipc" },
      { "type": "uts" }, { "type": "mount" }, { "type": "cgroup" }
    ],
    "seccomp": { "defaultAction": "SCMP_ACT_ERRNO", "syscalls": [ /* ... */ ] }
  }
}
```

Este arquivo é o contrato da OCI Runtime Spec. Qualquer runtime que o entenda pode executar o container: runc, crun, youki, runsc (gVisor), kata-runtime. Trocar o runtime é trocar um binário.

12.4 Os runtimes de baixo nível, comparados

Runtime	Linguagem Isolamento		Característica
runc	Go	namespaces + cgroups	Referência da OCI. O padrão em todo lugar
crun	C	namespaces + cgroups	~2× mais rápido para iniciar, memória menor. Padrão no Podman em Fedora

Runtime	Linguagem	Isolamento	Característica
youki	Rust	namespaces + cgroups	Segurança de memória por construção; em amadurecimento
runsc (gVisor)	Go	kernel em espaço de usuário	Intercepta syscalls; superfície de kernel drasticamente menor
kata-runtime	Go	micro-VM com kernel próprio	Isolamento de VM com interface de container
runwasi / wasmedge	Rust	sandbox Wasm	Executa módulos WebAssembly em vez de processos Linux

bash

```
# Instalar e usar um runtime alternativo
sudo tee -a /etc/docker/daemon.json >/dev/null <<'EOF'
{ "runtimes": { "crun": { "path": "/usr/bin/crun" } } }
EOF
sudo systemctl restart docker
docker run --rm --runtime=crun alpine echo "rodou com crun"
```

Opinião profissional: crun é uma troca de baixo risco e ganho mensurável quando você inicia muitos containers curtos (CI, funções, testes). Para carga estável de longa duração, a diferença some.

12.5 BuildKit: por que o build ficou outro

Até a Engine 23, docker build usava o construtor legado: sequencial, cache frágil, sem segredos. O **BuildKit** é padrão desde então.

	Construtor legado	BuildKit
Execução	Sequencial, instrução a instrução	Grafo de dependências ; estágios independentes em paralelo
Cache	Por camada, local	Por conteúdo, exportável para registry
Segredos	Não há	--mount=type=secret, --mount=type=ssh
Cache de gerenciador de pacotes	Não	--mount=type=cache
Multi-plataforma	Precária	Nativa
Saída	Só imagem	Imagem, tar, arquivos locais, cache
Frontend	Fixo	Plugável (# syntax=)

A linha # syntax=docker/dockerfile:1 no topo do Dockerfile **baixa o frontend mais recente**, o que dá acesso a recursos novos sem atualizar o Docker. É por isso que ela aparece em todos os exemplos deste material.

bash

```
docker buildx build --progress=plain -t app . # log completo, essencial para depurar
docker buildx du # quanto o cache ocupa
docker buildx prune --filter until=72h # limpeza seletiva
```

12.5.1 Drivers do buildx

Driver	Onde constrói	Quando usar
docker	Dentro do daemon local	Padrão; limitado em multi-plataforma
docker-container	Container BuildKit dedicado	Multi-arch, cache exportável

Driver	Onde constrói	Quando usar
kubernetes	Pods num cluster	Builds distribuídos em equipe
remote	BuildKit já em execução	Builder compartilhado

12.6 Drivers plugáveis: log, storage, rede

O daemon delega três subsistemas a drivers.

12.6.1 Log

```
docker system info | grep "Logging Driver"
```

bash

Driver	Uso	Ressalva
json-file	Padrão	Sem limite de tamanho por padrão — enche disco
local	Formato binário, mais eficiente, com rotação por padrão	docker logs funciona; ferramentas externas, não
journald	systemd	Integra com journalctl
syslog, fluentd, gelf, awslogs	Agregação central	docker logs deixa de funcionar
none	Descarta	Quando o app já envia para outro lugar

```
// /etc/docker/daemon.json – o ajuste que todo servidor deveria ter
{ "log-driver": "json-file", "log-opts": { "max-size": "10m", "max-file": "3" } }
```

JSON

12.6.2 Storage

Driver	Situação
overlay2	Padrão há anos; maduro e rápido
containerd snapshotter	Padrão em instalações novas desde a Engine 29
btrfs, zfs	Quando o sistema de arquivos do host é esse; snapshot nativo
devicemapper	Obsoleto ; removido
vfs	Sem CoW: cópia integral. Só para testes; enorme e lento

12.6.3 Rede

bridge, host, none, overlay, macvlan, ipvlan, mais plugins (Weave, Calico). Detalhes em [cap. 14](#).

12.7 A API do daemon

Tudo que a CLI faz é HTTP. Você pode falar direto:

bash

```
curl -s --unix-socket /var/run/docker.sock http://localhost/version | jq
curl -s --unix-socket /var/run/docker.sock http://localhost/containers/json | jq '.[].Names'
curl -s --unix-socket /var/run/docker.sock http://localhost/info | jq '.ServerVersion, .Driver'

# Criar e iniciar um container só com curl
curl -s -X POST --unix-socket /var/run/docker.sock \
  -H "Content-Type: application/json" \
  -d '{"Image":"alpine","Cmd":["echo","via API"]}' \
  http://localhost/containers/create
```

12.7.1 ⚠ Por que montar /var/run/docker.sock num container é perigoso

Ferramentas como Portainer, Watchtower e agentes de CI pedem esse socket. Quem o tem, tem **root no host**:

bash

```
# De DENTRO de um container com o socket montado:
curl -s -X POST --unix-socket /var/run/docker.sock \
  -H "Content-Type: application/json" \
  -d '{"Image":"alpine","Cmd":["chroot","/host","sh"],"HostConfig":{"Binds":["/:/host"],"Privileged":true}}' \
  http://localhost/containers/create
```

Mitigações reais: um **proxy de socket** com allow-list de endpoints (tecnativa/docker-socket-proxy), a **API por TLS** com certificado de cliente, ou **rootless mode** — que reduz o dano a "root do seu usuário".

12.8 Docker vs. containerd vs. Podman vs. CRI-O

	Docker	containerd	Podman	CRI-O
Daemon	sim (dockerd)	sim	não	sim (mínimo)
Root necessário	por padrão, sim	sim	não	sim
Constrói imagem	sim (Build-Kit)	não	sim (Buildah embutido)	não
Compose	sim	não	sim (podman compose, pods)	não
CRI para Kubernetes	não	sim	não	sim
CLI	docker	ctr (baixo nível), nerdctl	podman (compatível com docker)	crictrl
Uso típico	Desktop, dev	Nós de cluster	Dev e produção em ambiente regulado	Nós de cluster (OpenShift)

bash

```
# Podman é praticamente drop-in
alias docker=podman
podman run -d -p 8080:80 nginx:alpine
podman generate systemd --name web > web.service # gera unidade systemd: ótimo em servidor
```

Diferença conceitual que importa: o Podman tem **pods** (grupo de containers compartilhando namespace de rede), tomando emprestado o conceito do Kubernetes. Isso facilita levar uma stack local para um cluster.

Recomendação prática: em desktop Linux e em servidores de organizações com restrição de licença, Podman é a escolha racional. Em desktop macOS/Windows, o Docker

Desktop ainda tem a melhor integração. Em nós de Kubernetes, a pergunta nem se coloca: é containerd ou CRI-O.

12.9 Onde as coisas ficam no disco

bash

```
docker system info | grep "Docker Root Dir"      # normalmente /var/lib/docker
sudo du -sh /var/lib/docker/*
```

Caminho	Conteúdo
/var/lib/docker/overlay2/	Camadas de imagem e de container — quase sempre o maior
/var/lib/docker/volumes/	Volumes nomeados
/var/lib/docker/containers/<id>/	Metadados e logs (<id>-json.log)
/var/lib/docker/image/	Índice de imagens do store legado
/var/lib/docker/buildkit/	Cache de build
/run/containerd/	Sockets e bundles em execução (efêmero)
/etc/docker/daemon.json	Configuração do daemon

bash

```
# Achar o log gigante que encheu o disco
sudo find /var/lib/docker/containers -name "*.json.log" -size +100M -exec ls -lh {} \;
```

Mover o diretório de dados (por exemplo, para um disco maior):

bash

```
sudo systemctl stop docker
sudo rsync -aP /var/lib/docker/ /mnt/dados/docker/
echo '{ "data-root": "/mnt/dados/docker" }' | sudo tee /etc/docker/daemon.json
sudo systemctl start docker
docker system info | grep "Docker Root Dir"
```

12.10 Autoteste

1. Desenhe de memória a cadeia CLI → processo, nomeando os cinco componentes.
2. Por que runc não aparece no ps de um container em execução?
3. O que live-restore permite, e em que situação ele não funciona?
4. O que é um *bundle* OCI e quais são suas duas partes?
5. Cite três runtimes de baixo nível alternativos ao runc e o que cada um oferece de diferente.
6. Liste quatro capacidades do BuildKit que o construtor legado não tinha.
7. Por que o driver de log json-file sem max-size é um risco operacional?
8. Escreva o comando curl que lista os containers pelo socket do daemon.
9. Explique, com um exemplo concreto, por que montar /var/run/docker.sock equivale a dar root.
10. Você precisa liberar espaço urgente. Quais três diretórios sob /var/lib/docker você inspeciona primeiro, e em que ordem?

Armazenamento e volumes — onde os dados realmente vivem

Nível: intermediário → avançado · Última atualização: 11/08/2026

Perda de dados é o incidente mais caro do mundo dos containers, e quase sempre tem a mesma causa: alguém achou que o container guardava o dado. Este arquivo elimina essa possibilidade.

13.1 As quatro formas de armazenamento, comparadas

	Camada de escrita	Volume nomeado	Bind mount	tmpfs
Onde fica	/var/lib/docker/overlay2/	/var/lib/docker/volumes/	Onde você mandar	RAM
Sobrevive a docker rm?	✗	✓	✓	✗
Sobrevive a reboot?	✗	✓	✓	✗
Gerenciado pelo Docker?	sim	sim	não	sim
Desempenho em Linux	CoW, mais lento	nativo	nativo	RAM (o mais rápido)
Desempenho dentro da VM em macOS/Win		dentro da VM (rápido)	atravessa a fronteira (lento)	RAM
Backup	difícil	fácil e padronizado	trivial (é uma pasta sua)	não se aplica
Uso correto	nada que importe	estado de produção	código em desenvolvimento	segredo, cache, /tmp

bash

```
-v nome:/caminho # volume nomeado
-v /caminho/absoluto/host:/caminho # bind mount
--mount type=volume,source=nome,target=/cam # o mesmo do primeiro, explícito
--mount type=bind,source=/abs,target=/cam,readonly
--mount type=tmpfs,target=/tmp,tmpfs-size=64m
```

-v vs --mount: funcionalmente equivalentes para os casos comuns. O -v é conciso e **cria o diretório do host se não existir** (fonte de erros silenciosos: você digita o caminho errado e ganha uma pasta vazia em vez de um erro). O --mount é verboso, explícito e **falha** se a origem não existir. Use -v no terminal, --mount em script e produção.

13.2 Volumes nomeados

bash

```
docker volume create dados-app
docker volume ls
docker volume inspect dados-app
# esperado: "Mountpoint": "/var/lib/docker/volumes/dados-app/_data"
docker volume rm dados-app
```

13.2.1 A regra de inicialização que salva e que surpreende

Quando um volume nomeado vazio é montado sobre um caminho que já existe na imagem, o Docker copia o conteúdo E as permissões daquele caminho para dentro do volume.

bash

```
docker run --rm -v v-teste:/etc/nginx nginx:alpine ls /etc/nginx
docker run --rm -v v-teste:/x alpine ls /x
# esperado: nginx.conf, conf.d/, mime.types... — vieram da imagem
docker volume rm v-teste
```

Isso tem três consequências:

1. **É como um container sem root consegue escrever num volume.** Se a imagem tem `RUN mkdir /dados && chown node:node /dados`, o volume nasce pertencendo ao `node`. Sem isso, nasceria do `root` e o app tomaria EACCES.
2. **Só acontece se o volume estiver vazio.** Na segunda execução, o conteúdo do volume vence. É por isso que “editei a config na imagem e nada mudou”: o volume antigo tem a versão velha.
3. **Bind mount NÃO faz isso.** Um bind mount de uma pasta vazia **esconde** o conteúdo da imagem. Essa assimetria é a origem de metade da confusão sobre volumes.

13.2.2 Volumes anônimos

bash

```
docker run -v /dados alpine          # sem nome → o Docker sorteia um hash
docker volume ls -f dangling=true    # eles se acumulam aqui
```

Também são criados implicitamente pela instrução `VOLUME` no Dockerfile — o que é uma armadilha das piores, porque acontece sem você pedir:

Dockerfile

```
VOLUME /var/lib/postgresql/data      # a imagem oficial do Postgres faz isso
```

Efeito: **todo** `docker run postgres` sem `-v` explícito cria um volume anônimo que nunca é removido por `docker rm`. Depois de meses, `docker volume ls` mostra centenas deles ocupando dezenas de GB.

bash

```
docker rm -v CONTAINER                # o -v remove os volumes ANÔNIMOS junto
docker volume prune                    # remove todos os órfãos ▲ verifique antes
docker volume ls -f dangling=true      # ← rode ISTO antes daquilo
```

Recomendação: evite `VOLUME` no seu Dockerfile. Declare o volume no `compose.yaml`, onde ele é visível e nomeado.

13.3 Bind mounts e o problema de permissão

O bind mount é simples até o momento em que os UIDs não batem — e aí consome uma tarde.

13.3.1 Por que acontece

O kernel não conhece nomes de usuário. Ele conhece **números**. Se dentro do container o app roda como UID 1000 chamado node, e no host o UID 1000 é você, funciona por coincidência. Se o container roda como UID 999 (postgres, por exemplo) e o host não tem esse UID, os arquivos aparecem com dono numérico e ninguém consegue editá-los.

```
# Demonstre o problema
mkdir -p /tmp/perm && cd /tmp/perm
docker run --rm -v "$PWD:/w" -w /w alpine touch arquivo-root
ls -l arquivo-root
# esperado: -rw-r--r-- 1 root root ... - você não consegue apagar sem sudo
```

bash

13.3.2 As quatro soluções, em ordem de qualidade

1. Rode o container com o seu UID (melhor para desenvolvimento):

```
docker run --rm --user "${id -u}:${id -g}" -v "$PWD:/w" -w /w alpine touch arquivo-meu
ls -l arquivo-meu # dono: você
```

bash

Ressalva: o UID pode não existir no /etc/passwd do container, e alguns programas reclamam ("whoami: unknown uid"). Contorne com --user "\${id -u}:\${id -g}" -e HOME=/tmp.

2. Crie na imagem um usuário com o UID certo (melhor para equipe):

```
ARG UID=1000
ARG GID=1000
RUN addgroup -g $GID app && adduser -D -u $UID -G app app
USER app
```

Dockerfile

```
docker build --build-arg UID=$(id -u) --build-arg GID=$(id -g) -t app .
```

bash

3. chown no entrypoint (o que as imagens oficiais fazem): Um entrypoint que roda como root, ajusta o dono e então faz su-exec/gosu para o usuário final. Funciona, mas exige começar como root.

4. User namespace remap (mais robusto, mais complexo):

```
// /etc/docker/daemon.json
{ "userns-remap": "default" }
```

JSON

O root do container passa a mapear para um UID sem privilégio no host. Muda o modelo de permissão de todos os volumes existentes — planeje a migração.

13.3.3 SELinux (Fedora, RHEL, Rocky)

bash

```
docker run -v "$PWD:/app" alpine ls /app # Permission denied, mesmo com dono correto
docker run -v "$PWD:/app:Z" alpine ls /app # [ok]
```

Sufixo	Efeito
:z (minúsculo)	Rótulo compartilhado : vários containers podem acessar
:Z (maiúsculo)	Rótulo exclusivo deste container

⚠ **Cuidado sério:** :Z em um diretório do sistema (como /home ou /var) **relabela recursivamente** aquele diretório e pode quebrar o host. Use apenas em pastas de projeto.

13.4 Desempenho: onde o I/O dói

13.4.1 Linux

Volumes e bind mounts vão direto ao sistema de arquivos do host — desempenho nativo. A camada de escrita (OverlayFS) é mais lenta por causa do copy-on-write, sobretudo na primeira escrita de arquivos grandes.

Regra: **carga de escrita intensa nunca fica na camada de escrita**. Banco de dados, índices, uploads e caches vão para volume.

13.4.2 macOS e Windows — a fronteira que custa caro

Todo container Linux roda numa VM. Bind mounts precisam atravessar host ↔ VM:

Mecanismo	Plataforma	Desempenho relativo
VirtioFS	macOS (Docker Desktop recente)	Bom; use este
gRPC-FUSE	macOS (legado)	Ruim
osxfs	macOS (histórico)	Muito ruim
9p	WSL2 acessando /mnt/c	Péssimo
ext4 nativo	WSL2 dentro de ~	Nativo

Números que aparecem na prática: `npm install` numa pasta bind-montada do /mnt/c no WSL2 podia levar **10 a 50x** o tempo do mesmo comando dentro do sistema de arquivos do Linux.

As correções, em ordem:

1. **No Windows, mantenha o projeto em ~ dentro do WSL**, nunca em /mnt/c.
2. **No macOS, habilite VirtioFS** nas configurações do Docker Desktop.
3. **Coloque as pastas pesadas em volume nomeado**, não em bind mount:

YAML

```
volumes:
  - ./src:/app/src # código: bind mount (pequeno, precisa ser editável)
  - node_modules:/app/node_modules # dependências: volume (grande, não precisa ser editável)
```


4. Use `docker compose watch` com `action: sync`, que copia arquivos em vez de montar.

13.5 Backup, restauração e migração

13.5.1 O padrão que funciona para qualquer volume

bash

```
# Backup
docker run --rm \
  -v MEU_VOLUME:/dados:ro \
  -v "$PWD:/backup" \
  alpine tar czf /backup/backup-$(date +%F).tgz -C /dados .

# Restauração
docker run --rm \
  -v MEU_VOLUME:/dados \
  -v "$PWD:/backup" \
  alpine sh -c "rm -rf /dados/* /dados/..?* 2>/dev/null; tar xzf /backup/backup-2026-08-11.tgz -C /dados"

# Migrar entre máquinas
docker run --rm -v MEU_VOLUME:/d alpine tar cz -C /d . | ssh destino \
  'docker run --rm -i -v MEU_VOLUME:/d alpine tar xz -C /d'
```

13.5.2 Banco de dados: não copie os arquivos a quente

Copiar `/var/lib/postgresql/data` com o Postgres em execução captura páginas em estados inconsistentes — o dump pode restaurar, pode não restaurar, ou pode restaurar corrompido. Use a ferramenta lógica do próprio banco:

bash

```
# Postgres
docker compose exec -T db pg_dump -U app --no-owner app | gzip > dump-$(date +%F).sql.gz
gunzip -c dump-2026-08-11.sql.gz | docker compose exec -T db psql -U app app

# MySQL / MariaDB
docker compose exec -T db mysqldump -u root -p"$SENHA" --single-transaction app | gzip > dump.sql.gz

# MongoDB
docker compose exec -T db mongodump --archive --gzip > dump.gz

# Redis (o próprio Redis já grava snapshot; force um antes de copiar)
docker compose exec redis redis-cli BGSAVE
```

13.5.3 A regra 3-2-1, aplicada a containers

3 cópias, em **2** mídias diferentes, **1** fora do local.

YAML

```
backup:
  image: restic/restic:latest
  environment:
    RESTIC_REPOSITORY: s3:s3.amazonaws.com/meu-bucket/backups
    RESTIC_PASSWORD: ${RESTIC_SENHA:?}
    AWS_ACCESS_KEY_ID: ${AWS_KEY:?}
    AWS_SECRET_ACCESS_KEY: ${AWS_SECRET:?}
  volumes:
    - dados-db:/dados:ro
  command: ["backup", "/dados", "--tag", "diario"]
```

E teste a restauração. Um backup nunca restaurado não é um backup — é uma esperança. Coloque o teste de restauração no calendário, trimestral no mínimo.

13.6 Volume drivers e armazenamento em rede

Volumes locais servem a uma máquina. Em cluster, o volume precisa seguir o container.

bash

```
# NFS, sem plugin externo – o driver 'local' sabe fazer isso
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.10,rw,nfsvers=4 \
  --opt device=:/export/dados \
  dados-nfs

# CIFS/SMB
docker volume create --driver local \
  --opt type=cifs \
  --opt o=username=usuario,password=senha,uid=1000 \
  --opt device=//servidor/compartilhamento \
  dados-smb
```

Solução	Modelo	Observação
local + NFS	Rede	Simples; latência e travamento de bloqueio (lock) são os riscos
CSI (Container Storage Interface)	Padrão do Kubernetes	O caminho moderno; EBS, GCE PD, Ceph, Longhorn
Ceph / Longhorn / OpenEBS	Replicado	Para cluster próprio
EBS / Persistent Disk / Azure Disk	Nuvem	Preso a uma zona de disponibilidade

Advertência que vale a tarde perdida: bancos de dados relacionais sobre NFS são uma fonte clássica de corrupção e de travamento por problemas de lock e semântica de fsync. Se precisar de banco replicado, use replicação **do banco** (streaming replication, réplicas), não armazenamento compartilhado.

13.7 Diagnóstico — quando o disco enche

bash

```
docker system df          # panorama
docker system df -v       # detalhado, por imagem, container e volume

# Os maiores volumes
sudo du -sh /var/lib/docker/volumes/* | sort -hr | head -10

# Os maiores logs de container
sudo find /var/lib/docker/containers -name "*.json.log" -exec du -h {} + | sort -hr | head

# Cache de build
docker builder du
docker buildx du --verbose

# Órfãos (verifique ANTES de podar)
docker volume ls -f dangling=true
docker ps -a --filter status=exited --format "{{.Names}}\t{{.Status}}"
```

A ordem segura de limpeza:

bash

```
docker container prune          # 1. containers parados
docker builder prune --keep-storage 10GB # 2. cache de build (costuma ser o maior)
docker image prune -a --filter "until=168h" # 3. imagens sem uso há mais de 7 dias
docker volume ls -f dangling=true # 4. LISTE os volumes órfãos...
docker volume rm <um por um>      # ...e remova o que reconhecer
```

Nunca comece por `docker system prune -a --volumes`. Ele é a última linha, não a primeira, e apaga dados sem perguntar.

13.8 Recuperação de desastre — os três casos que acontecem

13.8.1 "Apaguei o container e perdi o banco"

Se havia volume, o dado está lá:

bash

```
docker volume ls
docker run --rm -v NOME_DO_VOLUME:/d alpine ls -la /d
# recrie o container apontando para o mesmo volume
```

Se **não** havia volume, o dado estava na camada de escrita e foi removido com o `docker rm`. Não há recuperação prática.

13.8.2 "Rodei `docker compose down -v`"

Os volumes do projeto foram removidos. Recuperação só a partir de backup. É por isso que a seção 5 existe.

13.8.3 "O volume existe mas o container não sobe por permissão"

bash

```
docker run --rm -v NOME:/d alpine ls -ln /d      # veja os UIDs numéricos
docker run --rm -v NOME:/d alpine chown -R 1000:1000 /d # ajuste para o UID do app
```

13.9 Autoteste

1. Quais das quatro formas de armazenamento sobrevivem a `docker rm`? E a um reboot?
2. Descreva a regra de inicialização de volume nomeado e diga por que ela **não** vale para bind mount.
3. Você alterou um arquivo de configuração na imagem e nada mudou no container. Qual é a hipótese nº 1?
4. Por que `VOLUME` no Dockerfile é considerado uma armadilha? O que ele provoca ao longo dos meses?
5. Um arquivo criado por um container aparece com dono `root` no host. Explique a causa em termos de UID e dê duas soluções.
6. Qual é a diferença entre `:z` e `:Z` no SELinux, e qual perigo o `:Z` traz?
7. Por que `npm install` é dramaticamente mais lento em `/mnt/c` no WSL2, e qual é a correção?

8. Por que copiar `/var/lib/postgresql/data` a quente é perigoso, e o que fazer em vez disso?
9. Escreva o comando de backup genérico de um volume, explicando cada `-v`.
10. Liste, em ordem, os quatro passos seguros de limpeza quando o disco enche.

CAPÍTULO 14

Redes — como containers falam entre si e com o mundo

Nível: intermediário → avançado · Última atualização: 11/08/2026

Rede é onde a maioria dos problemas “estranhos” de container mora. A boa notícia: o modelo é pequeno e, uma vez entendido, não surpreende mais.

14.1 O que o Docker cria por baixo

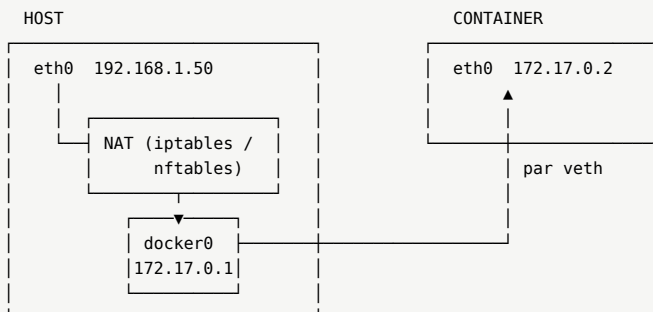
Ao instalar, o Docker cria uma bridge chamada `docker0` (uma placa de rede virtual, como um switch de software) e uma sub-rede privada, tipicamente `172.17.0.0/16`.

bash

```
ip addr show docker0
# esperado: inet 172.17.0.1/16 – este é o gateway dos containers na bridge padrão

docker network ls
# esperado: bridge, host, none (as três criadas na instalação)
```

Ao subir um container, acontece o seguinte:



Um **par veth** é como um cabo virtual com duas pontas: uma dentro do namespace de rede do container (aparece como `eth0`), outra no host (`veth1a2b3c`), plugada na bridge.

bash

```
docker run -d --name r nginx:alpine
docker exec r ip addr show eth0      # o IP interno
ip link | grep veth                  # a outra ponta, no host
docker exec r ip route                # default via 172.17.0.1
docker rm -f r
```

14.2 Os drivers de rede

Driver	O que faz	Quando usar
bridge	Rede virtual isolada com NAT (padrão)	95% dos casos

Driver	O que faz	Quando usar
host	Sem namespace de rede: o container usa a pilha do host	Máximo desempenho, muitas portas. ⚠ Só Linux
none	Só loopback	Processamento sem rede
overlay	Rede entre vários hosts (VXLAN)	Swarm, cluster
macvlan	O container ganha MAC e IP na sua LAN física	Aparelhos que precisam ser vistos na rede (Pi-hole, Home Assistant)
ipvlan	Como macvlan, compartilhando o MAC	Ambientes que limitam MACs por porta

14.2.1 host — o que se ganha e o que se perde

bash

```
docker run -d --network host nginx:alpine
curl localhost:80 # funciona — o nginx escutou direto na porta 80 do host
```

Ganho: sem NAT, sem tradução; latência e vazão praticamente nativas. Serviços que abrem centenas de portas (SIP, jogos, alguns protocolos P2P) ficam viáveis.

Perda: **nenhum isolamento de rede**; o -p deixa de existir (e é ignorado com um aviso); dois containers não podem escutar na mesma porta; e no macOS/Windows o "host" é a VM, não a sua máquina — o que faz o comportamento diferir da expectativa.

14.2.2 macvlan — o container como um aparelho na sua LAN

bash

```
docker network create -d macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 \
  lan

docker run -d --network lan --ip 192.168.1.90 --name pihole pihole/pihole
```

O container recebe 192.168.1.90 e responde a outros aparelhos da rede como se fosse um dispositivo físico.

A pegadinha clássica: por padrão, o host não consegue falar com o container **macvlan** (e vice-versa), porque o tráfego não passa pela bridge do kernel. A correção é criar uma interface macvlan também no host:

bash

```
sudo ip link add macvlan-host link eth0 type macvlan mode bridge
sudo ip addr add 192.168.1.200/32 dev macvlan-host
sudo ip link set macvlan-host up
sudo ip route add 192.168.1.90/32 dev macvlan-host
```

E mais duas: muitas redes Wi-Fi não permitem múltiplos MACs por associação (macvlan não funciona em Wi-Fi na maioria dos casos), e o servidor DHCP da sua rede não conhece esses IPs — reserve-os fora da faixa DHCP.

14.3 DNS interno: a regra que resolve tudo

Na bridge padrão (bridge), NÃO há resolução por nome. Em qualquer rede criada por você — inclusive as do Compose — há.

```
bash
# Bridge padrão: não resolve
docker run -d --name a nginx:alpine
docker run --rm alpine ping -c1 a
# esperado: bad address 'a'

# Rede criada: resolve
docker network create minha
docker run -d --network minha --name b nginx:alpine
docker run --rm --network minha alpine ping -c1 b
# esperado: 64 bytes from 172.x.x.x

docker rm -f a b && docker network rm minha
```

Por dentro: o Docker embute um resolvidor em 127.0.0.11 dentro do namespace de rede do container, e o `/etc/resolv.conf` gerado aponta para ele.

```
bash
docker run --rm --network minha alpine cat /etc/resolv.conf
# esperado: nameserver 127.0.0.11
```

Por que a bridge padrão não tem DNS? Compatibilidade histórica. Ela é anterior às redes definidas pelo usuário e ao mecanismo de `--link` (hoje obsoleto). Mudar o comportamento quebraria configurações antigas. É uma **decisão histórica documentada**, não uma limitação técnica — e a recomendação oficial há anos é: crie sempre uma rede.

14.3.1 Aliases e múltiplas redes

```
bash
docker network create app
docker run -d --network app --network-alias banco --name postgres-prod postgres:16
docker run --rm --network app alpine nslookup banco # resolve pelo alias
```

No Compose, o **nome do serviço** é o nome DNS. Se houver réplicas, o DNS devolve **todos** os IPs, e o cliente (ou o nginx) faz o balanceamento.

14.4 Publicação de portas — como funciona de verdade

```
bash
docker run -d -p 8080:80 nginx:alpine
```

O Docker escreve regras de NAT:

```
bash
sudo iptables -t nat -L DOCKER -n | grep 8080
# esperado: DNAT ... tcp dpt:8080 to:172.17.0.2:80
```

Formas de publicar:

bash

```
-p 8080:80          # TODAS as interfaces do host — visível na LAN
-p 127.0.0.1:8080:80 # [ok] só no loopback
-p 192.168.1.50:8080:80 # numa interface específica
-p 8080:80/udp       # UDP
-p 8000-8010:8000-8010 # faixa
-P                  # publica tudo do EXPOSE em portas altas aleatórias
```

14.4.1 ⚠ O Docker fura o seu firewall — e isso surpreende quase todo mundo

As regras do Docker vão para a cadeia DOCKER da tabela nat, que é processada **antes** das regras de INPUT que o ufw gerencia. Resultado: ufw deny 5432 **não** impede o acesso a um container publicado com -p 5432:5432.

bash

```
sudo ufw status      # diz que a porta está bloqueada
nmap -p 5432 IP_DA_MAQUINA # de outra máquina: a porta está aberta
```

As três correções, em ordem de qualidade:

1. **Publique só no loopback:** -p 127.0.0.1:5432:5432. Simples e eficaz.
2. **Use a cadeia DOCKER-USER**, que é avaliada antes das regras do Docker:

bash

```
sudo iptables -I DOCKER-USER -i eth0 ! -s 10.0.0.0/8 -j DROP
```

3. **Não publique.** Serviços internos (banco, cache) não precisam de ports: — o proxy fala com eles pela rede do Compose.

Na Engine 29 há suporte **experimental** a nftables como backend, ajustável com "firewall-backend": "nftables" no daemon.json. O comportamento é funcionalmente equivalente, mas **quem usa a cadeia DOCKER-USER do iptables precisa revisar as regras**.

14.5 Como um container alcança o host

Cenário	Como
Docker Desktop (macOS/Win)	host.docker.internal — já funciona
Linux	Adicione --add-host=host.docker.internal:host-gateway
Linux, alternativa	Use o IP do gateway: normalmente 172.17.0.1
Compose	extra_hosts: ["host.docker.internal:host-gateway"]

bash

```
docker run --rm --add-host=host.docker.internal:host-gateway alpine \
ping -c1 host.docker.internal
```

Caso típico: o container precisa falar com um banco que roda direto na máquina do desenvolvedor.

14.6 Segmentação de rede como camada de segurança

Este é o uso mais subaproveitado do modelo de rede.

YAML

```
services:
  proxy:
    networks: [publica, interna]    # ponte entre os dois mundos
    ports: ["443:443"]

  api:
    networks: [interna]             # não alcança a internet
    # sem ports:

  db:
    networks: [interna]
    # sem ports:

networks:
  publica:
  interna:
    internal: true                 # ← sem rota de saída para a internet
```

O que `internal: true` garante:

bash

```
docker compose exec api wget -q0- --timeout=3 https://example.com
# esperado: falha — não há rota
docker compose exec api wget -q0- http://db:5432
# esperado: alcança (mesma rede)
```

Por que isso importa: se a API for comprometida por uma vulnerabilidade de dependência, o atacante não consegue baixar ferramentas, nem exfiltrar dados por HTTP, nem contatar um servidor de comando e controle. Custa **uma linha** de YAML e elimina o passo mais comum de uma cadeia de ataque.

Opinião profissional: de todas as medidas de segurança em container, `internal: true` é a de melhor relação custo-benefício, e a menos usada.

14.7 Redes overlay (multi-host)

Conectam containers em máquinas diferentes, encapsulando o tráfego em **VXLAN** (UDP 4789).

bash

```
docker swarm init
docker network create -d overlay --attachable minha-overlay
docker service create --name web --network minha-overlay --replicas 3 nginx:alpine
```

Portas necessárias entre os nós: TCP 2377 (gerência do cluster), TCP/UDP 7946 (descoberta), UDP 4789 (VXLAN).

Custo real: encapsulamento VXLAN acrescenta cabeçalho e reduz o MTU efetivo. **Problemas de MTU são a causa nº 1 de "funciona com pacote pequeno e trava com pacote grande"** em overlay — tipicamente um TLS handshake que completa e uma transferência que congela.

bash

```
docker network create -d overlay --opt com.docker.network.driver.mtu=1450 rede
```

Criptografia opcional (IPsec), com custo de CPU:

bash

```
docker network create -d overlay --opt encrypted rede-segura
```

14.8 Depuração de rede — o procedimento

Use nicolaka/netshoot, que traz dig, curl, tcpdump, ss, nmap, iperf, mtr.

bash

```
# Entrar no NAMESPACE DE REDE do container alvo, sem alterar a imagem dele
docker run --rm -it --network container:MEU_CONTAINER nicolaka/netshoot
# lá dentro:
ss -tlnp          # o que está escutando
dig outro-servico  # o DNS interno resolve?
curl -v http://outro-servico:3000/saude
tcpdump -i any port 5432 -nn
```

14.8.1 A árvore de diagnóstico

```
"não consigo conectar"
├─ De FORA para o container?
│   ├── docker ps mostra "0.0.0.0:8080->80/tcp"? → se não, faltou -p
│   ├── docker logs: o app diz em que porta subiu?
│   ├── 0 app escuta em 0.0.0.0 ou em 127.0.0.1? → 127.0.0.1 é o erro nº 1
│   │   ├── docker exec X ss -tlnp → veja o endereço de bind
│   │   └─ Outro processo já usa a porta do host? → ss -tlnp | grep :8080
├─ Entre containers?
│   ├── Estão na MESMA rede? → docker network inspect REDE
│   ├── Está na bridge padrão? → não há DNS ali; crie uma rede
│   ├── Usou o nome do SERVIÇO (não do container, não o IP)?
│   └─ 0 alvo já subiu? → depends_on com condition: service_healthy
└─ Do container para a internet?
    ├── A rede é `internal: true`? → é o comportamento esperado
    ├── DNS resolve? → docker exec X nslookup github.com
    ├── Rota existe? → docker exec X ip route
    └─ Proxy corporativo configurado? → ver 03-instalacao.md
```

14.8.2 Os erros e o que significam

Erro	Causa quase sempre
connection refused	O alvo não está escutando naquela porta/endereço. App em 127.0.0.1, ou ainda subindo
connection timed out	Firewall, rota ausente, ou o alvo está em outra rede
bad address 'nome' / Name does not resolve	Bridge padrão (sem DNS), ou nome errado, ou o alvo ainda não existe
no route to host	Redes diferentes, ou internal: true
address already in use	Outro processo do host ocupa a porta
TLS handshake trava em overlay	MTU

14.9 Configurações de rede no daemon

JSON

```
// /etc/docker/daemon.json
{
  "bip": "172.30.0.1/16",
  "default-address-pools": [
    { "base": "172.31.0.0/16", "size": 24 }
  ],
  "dns": ["1.1.1.1", "8.8.8.8"],
```

```

"mtu": 1450,
"ipv6": true,
"fixed-cidr-v6": "fd00::/80",
"iptables": true,
"userland-proxy": false
}

```

Chave	Para que serve
bip	Muda a sub-rede da docker0 — necessário quando 172.17.x colide com a rede da empresa/VPN
default-address-pools	Faixas para as redes criadas depois; evita esgotamento e colisão
dns	Servidores DNS injetados nos containers
mtu	Essencial sob VPN, overlay ou nuvem com MTU reduzido
userland-proxy	false usa só NAT (mais rápido, menos memória); true mantém o docker-proxy
iptables: false	Desliga a manipulação de firewall pelo Docker. ⚠️ Você passa a ser responsável por tudo

Colisão de sub-rede é um problema real e frequente: se a VPN corporativa usa 172.17.0.0/16, todos os containers deixam de alcançar a rede da empresa e o sintoma parece aleatório. Mudar bip e default-address-pools resolve.

14.10 Boas práticas, condensadas

1. **Sempre crie uma rede** (ou use o Compose, que cria uma). Nunca dependa da bridge padrão.
2. **Uma rede por aplicação**, no mínimo. Idealmente, separe borda de interna.
3. **internal: true** para tudo que não precisa de internet.
4. **Publique apenas o necessário**, e prefira 127.0.0.1:porta:porta.
5. **Nunca use IP de container** em configuração: use o nome do serviço.
6. **-p ignora o ufw**. Trate isso como fato, não como bug.
7. **Ajuste o MTU** em VPN, overlay e nuvem, antes de perder um dia com "trava em arquivo grande".
8. **Mude a faixa padrão** se ela colidir com sua rede corporativa.

14.11 Autoteste

1. Explique o que é um par veth e onde ficam suas duas pontas.
2. Por que ping outro-container falha na bridge padrão e funciona numa rede criada por você? Qual é a razão histórica disso?
3. Em -p 127.0.0.1:8080:80, o que cada um dos três campos significa?
4. Por que ufw deny 5432 não protege um container publicado com -p 5432:5432? Dê duas correções.
5. O que internal: true garante, e por que é a medida de segurança de melhor custo-benefício?
6. Um container macvlan não é alcançável a partir do host. Por quê, e como resolver?

7. Um TLS handshake completa mas a transferência trava numa rede overlay. Qual é a suspeita nº 1?
8. Qual comando entra no namespace de rede de um container sem modificar a imagem dele?
9. `connection refused` versus `connection timed out`: o que cada um sugere?
10. Sua VPN corporativa usa 172.17.0.0/16. O que quebra e qual chave do `daemon.json` corrige?

CAPÍTULO 15

Dockerfile e build — do texto à imagem

Nível: intermediário → avançado · Última atualização: 11/08/2026

O Dockerfile parece simples e é onde mais se erra. Este arquivo é o guia de escrita, com o porquê de cada regra.

15.1 A anatomia de um Dockerfile de produção

Dockerfile

```
# syntax=docker/dockerfile:1
# ↑ Frontend do BuildKit. Baixa a versão mais recente do parser, dando acesso a recursos novos
# sem atualizar o Docker. Coloque SEMPRE, e sempre na primeira linha.

ARG NODE_VERSION=22.4.0
# ↑ ARG antes do primeiro FROM é "global": só pode ser usado em linhas FROM.

FROM node:${NODE_VERSION}-alpine AS build
WORKDIR /app

# Menos volátil primeiro: manifesto de dependências
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci

# Mais volátil depois: o código
COPY . .
RUN npm run build

FROM node:${NODE_VERSION}-alpine AS producao
# ↑ ARG global precisa ser redeclarado dentro do estágio para valer nas outras instruções

ENV NODE_ENV=production
WORKDIR /app

RUN apk add --no-cache tini

COPY --from=build --chown=node:node /app/dist ./dist
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --chown=node:node package.json ./

USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --start-period=15s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/saude || exit 1
STOPSIGNAL SIGTERM
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "dist/server.js"]
```

15.2 As armadilhas de escrita, uma a uma

15.2.1 ADD vs COPY

ADD faz três coisas: copia, **extraí automaticamente arquivos tar** e **baixa URLs**. Essa implicitude é o problema:

Dockerfile

```
ADD app.tar.gz /app/      # extrai — pode ser o que você quer, ou uma surpresa
COPY app.tar.gz /app/     # copia o arquivo, ponto
```

Um `ADD` arquivo.tar /dest que você achava que copiava passa a extrair quando alguém renomeia o arquivo. **Use `COPY` sempre**, salvo o caso legítimo de extrair um tar local — e aí escreva um comentário explicando.

Para baixar URL, `ADD` não valida checksum por padrão e cria uma camada difícil de auditar:

Dockerfile

```
# [ok] explícito, verificável, e limpa na mesma camada
RUN curl -fsSL -o /tmp/x.tgz https://exemplo.com/x.tgz \
&& echo "abc123... /tmp/x.tgz" | sha256sum -c - \
&& tar xzf /tmp/x.tgz -C /opt \
&& rm /tmp/x.tgz
```

15.2.2 ARG vs ENV

	ARG	ENV
Existe durante	Só o build	Build e execução
Visível no container	não	sim
No docker inspect	não diretamente	sim
No histórico da imagem	sim	sim
Serve para segredo?	NÃO	NÃO

Dockerfile

```
ARG VERSAO=1.0           # configuração de build
ENV APP_VERSAO=$VERSAO   # promovido para o runtime, se você quiser
```

Ambos ficam registrados. Segredo → `--mount=type=secret`.

15.2.3 CMD vs ENTRYPOINT, e as formas exec/shell

Dockerfile

```
CMD ["node", "app.js"]    # [ok] forma EXEC: o node vira PID 1
CMD node app.js           # [x] forma SHELL: vira ["/bin/sh", "-c", "node app.js"]
```

Na forma shell, o `/bin/sh` é o PID 1 e **não repassa sinais**. `docker stop` demora 10 s e mata o processo à força. Em produção, isso é 502 no usuário a cada deploy.

Dockerfile	docker run img	docker run img echo oi
CMD ["node", "app.js"]	node app.js	echo oi (substitui)
ENTRYPOINT ["node"]	node	node echo oi (acrescenta)
ENTRYPOINT ["node"] + CMD ["app.js"]	node app.js	node echo oi

Regra: `ENTRYPOINT` = o que a imagem **é**; `CMD` = o argumento **padrão**.

Quando precisar de expansão de variável no comando, use um `entrypoint` script — não a forma shell:

bash

```
#!/bin/sh
set -e
# Aqui a expansão funciona, e o 'exec' faz o app HERDAR o PID 1.
exec node --max-old-space-size="${HEAP_MB:-512}" dist/server.js
```

O exec é essencial: sem ele, o shell continua como PID 1 e o problema dos sinais volta.

15.2.4 EXPOSE não expõe nada

É documentação (e afeta -P e alguns orquestradores). Quem publica porta é -p no run ou ports: no Compose. Nunca conte com EXPOSE para acessibilidade.

15.2.5 WORKDIR, não RUN cd

Dockerfile

```
RUN cd /app && npm ci      # [x] o cd não persiste para a próxima instrução
WORKDIR /app              # [ok] persiste, e cria o diretório se não existir
RUN npm ci
```

15.2.6 USER — sempre, e antes do CMD

Dockerfile

```
# Alpine
RUN addgroup -S app && adduser -S -G app app
# Debian/Ubuntu
RUN groupadd -r app && useradd -r -g app app

USER app
```

Não rodar como root não impede toda exploração, mas elimina uma classe inteira: escrita em caminhos do sistema, uso de capabilities herdadas, escrita no volume com dono errado.

15.2.7 HEALTHCHECK que vale a pena

Dockerfile

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=15s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/saude || exit 1
```

Parâmetro	Significado
interval	De quanto em quanto tempo testar
timeout	Quanto esperar por resposta
start-period	Carência inicial: falhas aqui não contam
retries	Falhas consecutivas até marcar unhealthy

Sem start-period, aplicação de boot lento entra em laço de reinício. E o teste deve **exercitar a dependência real** (banco, disco), não devolver 200 fixo — healthcheck otimista é pior que nenhum.

15.3 O cache, na prática

Ver [cap. 10](#) para o mecanismo. Aqui, os padrões.

15.3.1 A ordem canônica

Dockerfile

```
FROM base                                # 1. muda raramente
RUN apk add --no-cache ferramentas        # 2. muda raramente
COPY package.json package-lock.json ./   # 3. muda quando dependências mudam
RUN npm ci                               # 4. idem
COPY . .                                  # 5. muda a cada commit
RUN npm run build                         # 6. idem
```

15.3.2 Cache montado por ecossistema

Dockerfile

```
# Node
RUN --mount=type=cache,target=/root/.npm npm ci

# Python (pip)
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt

# Python (uv)
RUN --mount=type=cache,target=/root/.cache/uv uv sync --frozen

# Go
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build go build ./...

# Rust
RUN --mount=type=cache,target=/usr/local/cargo/registry \
    --mount=type=cache,target=/app/target cargo build --release

# Maven
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests

# apt (exige desligar a limpeza automática do apt)
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    --mount=type=cache,target=/var/lib/apt,sharing=locked \
    rm -f /etc/apt/apt.conf.d/docker-clean && apt-get update \
    && apt-get install -y --no-install-recommends curl
```

Atenção com Rust: o cache de `/app/target` acelera muito, **mas o binário fica no cache, não na camada**. É preciso copiá-lo para fora dentro do mesmo RUN: `cargo build --release && cp target/release/app /app/bin`.

15.3.3 Cache em CI

bash

```
# GitHub Actions
--cache-from type=gha --cache-to type=gha,mode=max
# Registry (funciona em qualquer CI)
--cache-from type=registry,ref=usuario/app:cache \
--cache-to type=registry,ref=usuario/app:cache,mode=max
```


15.4 .dockerignore — o arquivo que quase ninguém escreve direito

Ele determina **o que é enviado ao daemon** como contexto de build. Sem ele:

- o build fica lento (enviar node_modules e .git custa segundos ou minutos);
- o cache é invalidado por arquivo irrelevante;
- **segredos entram na imagem** via COPY . ..

```
.git
.github
node_modules
dist
build
coverage
*.log
.env
.env.*
!.env.example
*.pem
*.key
.npmrc
.netrc
.DS_Store
.vscode
.idea
Dockerfile*
compose*.yaml
README.md
docs/
```

Verifique o tamanho real do que está sendo enviado:

```
docker build --progress=plain -t x . 2>&1 | grep -i "transferring context"
# esperado: alguns MB. Se aparecer "500MB", falta .dockerignore
```

bash

15.5 Multi-stage: cinco padrões úteis

15.5.1 (a) Compilar e descartar o toolchain

```
FROM golang:1.23-alpine AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /app ./cmd/api

FROM gcr.io/distroless/static-debian12
COPY --from=build /app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

Dockerfile

15.5.2 (b) Separar dependências de produção e de desenvolvimento

Dockerfile

```
FROM node:22-alpine AS deps-prod
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:22-alpine AS build
COPY package*.json ./
RUN npm ci                # com devDependencies
COPY . .
RUN npm run build

FROM node:22-alpine
COPY --from=deps-prod /node_modules ./node_modules
COPY --from=build /dist ./dist
```

15.5.3 (c) Testes como portão de qualidade

Dockerfile

```
FROM build AS testes
RUN npm run lint && npm test
# `docker build --target testes .` falha se algo quebrar
```

15.5.4 (d) Estágios paralelos

O BuildKit executa estágios independentes **ao mesmo tempo**:

Dockerfile

```
FROM node:22 AS frontend
RUN npm run build:web

FROM golang:1.23 AS backend
RUN go build -o /api

FROM alpine
COPY --from=frontend /app/dist /www
COPY --from=backend /api /api
```

Os dois primeiros estágios rodam em paralelo, sem que você faça nada.

15.5.5 (e) Base comum

Dockerfile

```
FROM python:3.13-slim AS base
ENV PYTHONUNBUFFERED=1
RUN useradd -m app

FROM base AS dev
RUN pip install -r requirements-dev.txt
FROM base AS producao
RUN pip install -r requirements.txt
```

15.6 Escolha da imagem base — o quadro de decisão

```

Seu app compila para binário estático (Go, Rust, Zig, C estático)?
├─ SIM → distroless/static ou scratch          (2-10 MB)
└─ NÃO
   └─ Precisa de glibc? (Python com wheels, extensões nativas, Oracle client)
      ├── SIM → debian:12-slim ou distroless com runtime (~80 MB)
      └── NÃO → alpine (~8 MB)
   └─ É uma linguagem com runtime oficial? (Node, Python, Java, Ruby)
      └─ Use a imagem oficial na variante -alpine ou -slim

```

Ressalvas por linguagem:

Linguagem	Recomendação	Motivo
Go / Rust	scratch ou distroless/static	Binário estático; nada mais é necessário
Node	node:22-alpine	O runtime vem junto; musl não é problema
Python	python:3.13-slim (não alpine)	Wheels manylinux exigem glibc; alpine força compilar do fonte
Java	eclipse-temurin:21-jre-alpine ou imagem com jlink	Use o JRE , não o JDK; jlink gera runtime sob medida
Ruby / PHP	-slim	Extensões nativas, mesma razão do Python
.NET	mcr.microsoft.com/dotnet/aspnet:8.0-alpine	Runtime, não SDK

15.7 Reprodutibilidade

Um build reproduzível produz o **mesmo digest** a partir da mesma entrada. Na prática, isso é difícil — mas dá para chegar perto:

Fonte de variação	Como controlar
Tag da imagem base	Fixe com digest: FROM node:22.4.0-alpine@sha256:...
Versões de pacote	apt-get install curl=8.5.0-2 · lockfiles
Timestamps nas camadas	SOURCE_DATE_EPOCH (suportado pelo BuildKit)
Ordem de arquivos	Naturalmente estável no COPY
Downloads da internet	Verifique checksum; prefira espelho interno
latest em qualquer lugar	Nunca use

bash

```

SOURCE_DATE_EPOCH=$(git log -1 --pretty=%ct) \
docker buildx build --output type=image,rewrite-timestamp=true -t app:1.0 .

```

15.8 Linting e verificação

bash

```

# hadolint — o linter de Dockerfile
docker run --rm -i hadolint/hadolint < Dockerfile

# BuildKit tem verificações embutidas
docker buildx build --check .

# Escanear a imagem resultante
docker scout cves minha-app:1.0

```

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy image minha-app:1.0
```

As advertências do hadolint que mais valem a pena:

Código	O que diz
DL3006	Fixe a tag da imagem base
DL3008	Fixe a versão dos pacotes do apt
DL3009	Limpe o cache do apt na mesma camada
DL3015	Use --no-install-recommends
DL3025	Use a forma exec em CMD/ENTRYPOINT
DL3002	Não termine com USER root

15.9 Erros de build e o que significam

Mensagem	Causa	Correção
COPY failed: file not found in build context	O arquivo está fora do contexto ou no .dockerignore	Confira o . do docker build e o ignore
failed to solve: process "/bin/sh -c apt-get update" did not complete	Sem rede/DNS/proxy no build	Configure proxies em ~/.docker/config.json
404 Not Found em apt-get install	Índice do apt-get update veio do cache	update && install na mesma camada
exec format error ao rodar	Arquitetura errada	--platform, ou build multi-arch
no such file or directory para um arquivo que existe	CRLF no script, ou binário dinâmico em scratch	dos2unix; CGO_ENABLED=0
unable to prepare context: path not found	Caminho do contexto errado	Verifique o último argumento do build
Error: EACCES: permission denied no npm ci	USER trocado antes do COPY	Troque de usuário depois das cópias
Build "trava" no transferring context	Contexto enorme	.dockerignore

Depure com log completo:

```
docker buildx build --progress=plain --no-cache -t x . 2>&1 | tee build.log
```

bash

E entre no estágio que falhou:

```
docker build --target build -t depurar .
docker run --rm -it depurar sh
```

bash

15.10 Checklist de Dockerfile de produção

- ☐ # syntax=docker/dockerfile:1 na primeira linha
- ☐ Imagem base fixada por tag específica (idealmente com digest)
- ☐ Multi-stage: nenhum toolchain na imagem final
- ☐ .dockerignore presente e cobrindo .git, node_modules, .env, chaves
- ☐ COPY do manifesto de dependências antes do COPY do código

- ☐ `--mount=type=cache` nos gerenciadores de pacote
- ☐ `--mount=type=secret` para qualquer credencial; nenhum ARG com segredo
- ☐ USER não-root antes do CMD
- ☐ Forma **exec** em CMD e ENTRYPOINT
- ☐ tini ou `--init` se o app não trata sinais
- ☐ HEALTHCHECK que exercita a dependência real, com `start-period`
- ☐ Cache de gerenciador de pacotes limpo **na mesma camada**
- ☐ LABEL `org.opencontainers.image.*` preenchidos
- ☐ Imagem escaneada (trivy/scout) no CI
- ☐ SBOM e proveniência gerados no build de produção

15.11 Autoteste

1. Por que a linha `# syntax=docker/dockerfile:1` importa mesmo com o Docker atualizado?
2. Quais três comportamentos o ADD tem que o COPY não tem, e por que isso é um problema?
3. Escreva a tabela CMD/ENTRYPOINT de memória, com os três casos.
4. Por que exec é obrigatório na última linha de um entrypoint script?
5. ARG e ENV: qual existe em tempo de execução, e qual dos dois serve para segredo?
6. Onde entra `--mount=type=cache` e onde entra `--mount=type=secret`? Dê um exemplo de cada.
7. Por que Alpine é má escolha para Python e boa para Go?
8. O build "trava" em transferring context. Qual é o diagnóstico e a correção?
9. Como fazer o build falhar se um teste quebrar, sem duplicar configuração de CI?
10. Liste cinco itens do checklist que você aplicaria hoje a um Dockerfile que já tem em uso.

Compose e aplicações multi-container

Nível: intermediário · Última atualização: 11/08/2026

Compose é o que transforma "sei rodar um container" em "sei rodar uma aplicação". É também a ferramenta mais usada e menos estudada do ecossistema.

16.1 O que o Compose faz por você

A partir de um `compose.yaml`, ele:

- 1. **Cria uma rede** dedicada ao projeto, com DNS por nome de serviço.
- 2. **Constrói ou baixa** as imagens declaradas.
- 3. **Cria volumes** nomeados, prefixados pelo nome do projeto.
- 4. **Inicia os serviços na ordem** definida por `depends_on`.
- 5. **Rotula tudo** (`com.docker.compose.project=...`), o que permite operar o conjunto como uma unidade.

bash

```
docker compose up -d
docker network ls | grep $(basename $PWD) # rede criada automaticamente
docker volume ls | grep $(basename $PWD) # volumes prefixados
```

O **nome do projeto** vem, em ordem de precedência: da flag `-p`, da chave `name:` no arquivo, da variável `COMPOSE_PROJECT_NAME`, ou do **nome da pasta**. Duas pastas com o mesmo nome em máquinas diferentes geram recursos com o mesmo nome — fonte de confusão em CI. Declare `name:` explicitamente.

16.2 Compose v1 x v2, e o `version:` que morreu

	v1 (docker-compose)	v2 (docker compose)
Implementação	Python, binário separado	Go, plugin da CLI
Suporte	encerrado em julho de 2023	atual
Comando	<code>docker-compose</code> (hífen)	<code>docker compose</code> (espaço)
Especificação	<code>version: "3.8"</code> no topo	Compose Specification — sem <code>version:</code>

YAML

```
version: "3.8" # [x] obsoleto: ignorado, e gera aviso
services:     # [ok] comece direto aqui
```

Se você encontrar tutorial com `docker-compose` e `version:`, ele é de antes de 2023. O conteúdo pode continuar válido, mas verifique tudo o que for sintaxe.

16.3 Anatomia de um compose.yaml completo

YAML

```

name: minha-aplicacao           # nome do projeto, explícito

x-comum: &comum                 # âncora YAML: bloco reutilizável
  restart: unless-stopped
  logging:
    driver: json-file
    options: { max-size: "10m", max-file: "3" }
  networks: [interna]

services:

  api:
    <<: *comum                  # herda o bloco acima
    build:
      context: .
      dockerfile: Dockerfile
      target: producao
      args:
        NODE_VERSION: "22"
      cache_from:
        - type=registry,ref=ghcr.io/org/api:cache
    image: ghcr.io/org/api:${TAG:-dev}
    environment:
      DATABASE_URL: postgres://app:${DB_SENHA:?defina DB_SENHA}@db:5432/app
      NODE_ENV: production
    env_file:
      - path: .env
        required: false
    depends_on:
      db: { condition: service_healthy }
      migracao: { condition: service_completed_successfully }
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://127.0.0.1:3000/saude"]
      interval: 15s
      timeout: 3s
      retries: 3
      start_period: 20s
    deploy:
      replicas: 2
      resources:
        limits: { cpus: "1.0", memory: 512M }
        reservations: { memory: 128M }
    stop_grace_period: 30s
    security_opt: [no-new-privileges:true]
    cap_drop: [ALL]
    read_only: true
    tmpfs: [/tmp]
    networks: [interna, borda]

  migracao:
    <<: *comum
    image: ghcr.io/org/api:${TAG:-dev}
    command: ["npm", "run", "migrate:up"]
    depends_on:
      db: { condition: service_healthy }
    restart: "no"                # roda uma vez e termina

  db:
    <<: *comum
    image: postgres:16.3-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${DB_SENHA:?}
      POSTGRES_DB: app
    volumes:
      - dados-db:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/10-init.sql:ro
    healthcheck:

```



```

test: ["CMD-SHELL", "pg_isready -U app -d app"]
interval: 5s
timeout: 3s
retries: 10
start_period: 10s
shm_size: 256mb          # o Postgres usa memória compartilhada; o padrão de 64MB aperta

volumes:
  dados-db:

networks:
  borda:
  interna:
    internal: true

```

16.4 depends_on — as três condições

Condição	Espera até	Uso
service_started (padrão)	O container iniciar	Quase nunca é suficiente
service_healthy	O healthcheck passar	O que você quer em bancos e APIs
service_completed_successfully	O container sair com código 0	Migrações, seeds, tarefas de preparo

YAML

```

depends_on:
  db:      { condition: service_healthy }
  migracao: { condition: service_completed_successfully }

```

`service_started` é insuficiente porque “o container iniciou” não significa “o serviço aceita conexão”. A imagem oficial do Postgres, por exemplo, **reinicia internamente** durante a primeira inicialização — conectar cedo dá “connection refused” enganoso.

Se o serviço não tem healthcheck, ou você quer robustez independente da ordem, faça o próprio app tentar de novo:

JavaScript

```

async function conectarComEspera(tentativas = 10) {
  for (let i = 1; i <= tentativas; i++) {
    try { return await conectar(); }
    catch (e) {
      const espera = Math.min(1000 * 2 ** i, 30000); // backoff exponencial com teto
      console.log(`tentativa ${i} falhou, aguardando ${espera}ms`);
      await new Promise(r => setTimeout(r, espera));
    }
  }
  throw new Error('banco indisponível após todas as tentativas');
}

```

Opinião profissional: `depends_on` resolve a ordem no **boot**; a lógica de reconexão no app resolve o resto da vida do sistema (banco reiniciado, rede oscilando, failover). Em produção, **os dois** são necessários — `depends_on` sozinho é insuficiente e reconexão sozinha atrasa o boot.

16.5 Variáveis, .env e interpolação

Ordem de precedência (do mais forte para o mais fraco):

1. Variável do ambiente do shell
2. --env-file na linha de comando
3. .env no diretório do projeto
4. Valor padrão na interpolação (\${VAR:-padrao})

Sintaxe	Comportamento
\${VAR}	Vazio se indefinida
\${VAR:-padrao}	padrao se indefinida ou vazia
\${VAR-padrao}	padrao só se indefinida
\${VAR:?mensagem}	Aborta com a mensagem se indefinida ou vazia
\${VAR:+valor}	valor só se VAR estiver definida

Distinção que confunde: o arquivo .env alimenta a **interpolação do YAML**; env_file: injeta variáveis **dentro do container**. São coisas diferentes, e usar uma esperando a outra é erro comum.

YAML

```
services:
  api:
    environment:
      URL: ${API_URL}      # ← vem do .env do projeto (interpolação)
    env_file:
      - .env.aplicacao     # ← vai para dentro do container
```

bash

```
docker compose config    # ← O MELHOR DEPURADOR: mostra o YAML final resolvido
```

16.6 Sobreposição de arquivos: base + ambiente

compose.yaml (base):

YAML

```
services:
  api:
    image: ghcr.io/org/api:${TAG:-latest}
    environment:
      NODE_ENV: production
```

compose.override.yaml (**carregado automaticamente**, se existir):

YAML

```
services:
  api:
    build: .
    environment:
      NODE_ENV: development
    volumes:
      - ./src:/app/src
    ports:
      - "3000:3000"
```

compose.prod.yaml (explícito):

YAML

```
services:
  api:
    deploy:
      replicas: 3
      resources:
        limits: { memory: 1G }
```

bash

```
docker compose up -d # base + override (dev)
docker compose -f compose.yaml -f compose.prod.yaml up -d # base + prod
```

As regras de mesclagem, que causam surpresa:

Tipo	Comportamento
Escalares (image, command)	O último substitui
Mapas (environment, labels)	Mesclados chave a chave
Listas (ports, volumes)	Concatenadas — e é aqui que dá problema

Concatenação de listas significa que você **não consegue remover** uma porta ou um volume por sobreposição, só acrescentar. Para casos assim, use `!reset` (Compose recente) ou arquivos separados sem base comum:

YAML

```
services:
  api:
    ports: !reset []
```

16.7 Perfis

YAML

```
services:
  api:
    # sem profiles → sempre sobe

  depurador:
    profiles: [debug]
    image: nicolaka/netshoot
    command: sleep infinity

  ferramentas:
    profiles: [tools, debug] # participa de dois perfis
    image: minha-app:1.0
```

bash

```
docker compose up -d # só a api
docker compose --profile debug up -d # api + depurador + ferramentas
COMPOSE_PROFILES=debug,tools docker compose up -d
```

Uso típico: um único arquivo servindo a dev, testes, ferramentas de administração e produção, sem multiplicar arquivos.

16.8 docker compose watch — desenvolvimento sem bind mount de tudo

YAML

```
services:
  api:
    build: .
    develop:
      watch:
        - action: sync          # copia o arquivo para dentro do container
          path: ./src
          target: /app/src
          ignore: ["**/*.test.js"]
        - action: rebuild       # mudou dependência → reconstrói a imagem
          path: package.json
        - action: sync+restart   # copia e reinicia o processo
          path: ./config
          target: /app/config
```

bash

```
docker compose watch
```

Vantagem sobre bind mount: **muito mais rápido em macOS e Windows** (copia arquivos em vez de montar um sistema de arquivos através da fronteira da VM) e permite reagir de formas diferentes conforme o arquivo alterado.

16.9 Escala e as suas limitações

bash

```
docker compose up -d --scale api=3
docker compose ps
# minha-aplicacao-api-1, -api-2, -api-3
```

O DNS interno devolve os três IPs para o nome `api`, e um proxy à frente pode balancear.

O que quebra ao escalar, e por quê:

Problema	Causa	Solução
ports: "3000:3000" falha na 2ª réplica	Porta do host já ocupada	Remova ports:; use proxy. Ou faixa: "3000-3002:3000"
Estado divergente entre réplicas	Cada uma tem sua camada de escrita	Estado em banco ou volume compartilhado
Migração roda 3 vezes	Cada réplica executa o entry-point	Serviço de migração separado
Sessão perdida a cada requisição	Sessão em memória do processo	Sessão em Redis
Cron duplicado	Todas as réplicas têm o mesmo agendador	Serviço dedicado, ou lock distribuído

O limite honesto do Compose: ele opera **um host**. Sem reagendamento em falha de máquina, sem *rolling update* de verdade, sem balanceamento nativo entre nós. Para isso existem Swarm e Kubernetes — [cap. 20](#).

16.10 Compose em produção — quando faz sentido

*Opinião profissional, e há discordância legítima: **Compose em produção é uma escolha respeitável para um servidor só.*** Homelab, aplicação interna, produto pequeno, SaaS em estágio inicial — Kubernetes ali é complexidade sem retorno.

O que fazer para tornar isso sério:

```
services:
  api:
    image: ghcr.io/org/api@sha256:abc... # por DIGEST, nunca por tag
    restart: unless-stopped
    healthcheck: { test: [...], start_period: 20s }
    deploy:
      resources:
        limits: { memory: 512M, cpus: "1.0" }
    logging:
      driver: json-file
      options: { max-size: "10m", max-file: "3" }
    security_opt: [no-new-privileges:true]
    cap_drop: [ALL]
    read_only: true
    tmpfs: [/tmp]
```

YAML

E o deploy:

```
docker compose pull
docker compose up -d --wait # --wait retorna só quando os healthchecks passarem
docker image prune -af --filter "until=168h"
```

bash

Deixe o systemd cuidar do boot:

```
# /etc/systemd/system/minha-app.service
[Unit]
Description=Minha aplicação
Requires=docker.service
After=docker.service network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/minha-app
ExecStart=/usr/bin/docker compose up -d --wait
ExecStop=/usr/bin/docker compose down
TimeoutStartSec=300

[Install]
WantedBy=multi-user.target
```

INI

```
sudo systemctl enable --now minha-app
```

bash

Quando sair do Compose: mais de um servidor; necessidade de deploy sem interrupção com rollback automático; escala automática; equipes múltiplas dividindo infraestrutura. Antes disso, Kubernetes costuma custar mais do que entrega.

16.11 Segredos no Compose

YAML

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_senha # ← as imagens oficiais aceitam _FILE
    secrets: [db_senha]

secrets:
  db_senha:
    file: ./segredos/db_senha.txt # em Swarm: external: true
```

O segredo é montado em `/run/secrets/db_senha` num **tmpfs** (RAM). Vantagem sobre `environment::` variáveis de ambiente aparecem em `docker inspect`, em `/proc/PID/environ`, em logs de crash e são herdadas por processos filhos. Arquivo em tmpfs, não.

Fora do Swarm, secrets do Compose é essencialmente um bind mount conveniente — não há criptografia em repouso nem distribuição segura. Para isso, use o gerenciador de segredos da plataforma (Vault, AWS Secrets Manager, SOPS + age).

16.12 Comandos e diagnóstico

bash

```
docker compose config          # YAML final resolvido – comece SEMPRE por aqui
docker compose config --services # lista os serviços
docker compose ps -a          # inclui os que saíram
docker compose logs -f --tail 100 api
docker compose exec api sh     # no container em execução
docker compose run --rm api npm test # container novo e efêmero
docker compose top
docker compose events
docker compose down --remove-orphans # remove containers de serviços que você apagou do YAML
docker compose kill -s SIGHUP api    # envia um sinal específico
```

Sintoma	Diagnóstico
"Minha variável não chegou"	<code>docker compose config</code> mostra o valor resolvido
"O serviço sobe e morre"	<code>docker compose logs SERVIÇO</code> (funciona com o container parado)
"Um serviço não enxerga o outro"	<code>docker compose exec a ping b</code> ; confira se estão na mesma rede
"Sobrou container de um serviço que apaguei"	<code>docker compose down --remove-orphans</code>
"Altere o YAML e nada mudou"	<code>docker compose up -d --force-recreate</code>
"A imagem não foi reconstruída"	<code>docker compose up -d --build</code>

16.13 Autoteste

1. Cite três coisas que o Compose faz automaticamente e que você teria de fazer à mão com `docker run`.
2. Por que `version: "3.8"` não deve mais aparecer, e o que o substituiu?
3. Explique as três condições de `depends_on`, com um exemplo de uso para cada.

4. Qual é a diferença entre o arquivo `.env` e a chave `env_file`?
5. `${DB_SENHA:?erro}` e `${DB_SENHA:-padrao}`: o que cada um faz quando a variável falta?
6. Como listas e mapas se comportam na sobreposição de arquivos, e qual das duas causa problema?
7. Você roda `--scale api=3` e a segunda réplica falha. Qual é a causa mais provável?
8. Quando `docker compose watch` é preferível a `bind mount`, e por quê?
9. Por que `secrets` é melhor que `environment` para uma senha? Cite dois vazamentos que ele evita.
10. Em que ponto `Compose` em produção deixa de ser adequado? Dê três critérios objetivos.

CAPÍTULO 17

Registries, distribuição e cadeia de suprimentos

Nível: intermediário → avançado · Última atualização: 11/08/2026

Onde as imagens moram, como viajam e como você garante que a que roda em produção é a que você construiu.

17.1 Como um registry funciona

Um registry OCI é uma API HTTP com um punhado de endpoints. Você pode falar com ele por curl.

bash

```
# Token anônimo para uma imagem pública do Docker Hub
TOKEN=$(curl -s "https://auth.docker.io/token?service=registry.docker.io&scope=repository:library/alpine:pull" | jq -r .token)

# 1) Listar tags
curl -s -H "Authorization: Bearer $TOKEN" \
  https://registry-1.docker.io/v2/library/alpine/tags/list | jq

# 2) Buscar o manifesto (o índice multi-arch)
curl -s -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/vnd.oci.image.index.v1+json" \
  https://registry-1.docker.io/v2/library/alpine/manifests/3.20 | jq

# 3) Baixar uma camada (blob), por digest
curl -sL -H "Authorization: Bearer $TOKEN" \
  https://registry-1.docker.io/v2/library/alpine/blobs/sha256:<digest> -o camada.tar.gz
```

O protocolo (OCI Distribution Spec) tem essencialmente três recursos:

Caminho	O que é
/v2/<nome>/tags/list	Tags do repositório
/v2/<nome>/manifests/<ref>	Manifesto, por tag ou digest
/v2/<nome>/blobs/<digest>	Um blob: camada ou config

Por que um pull costuma ser rápido: o cliente pede o manifesto, compara os digests das camadas com o que já tem no disco e **baixa somente o que falta**. Se você já tem node:22-alpine, uma imagem sua baseada nela transfere só as suas camadas.

17.2 Os registries, comparados

Registry	Endereço	Grátis para	Observações
Docker Hub	docker.io	Público ilimitado; 1 repositório privado no plano gratuito	Limite de pull: 10/h sem login, 100/h com conta gratuita
GHCR	ghcr.io	Público ilimitado; privado na cota do GitHub	Integração direta com Actions via GITHUB_TOKEN

Registry	Endereço	Grátis para	Observações
Quay.io	quay.io	Público ilimitado	Escaneamento embutido (Clair)
GitLab Registry	registry.gitlab.com	Na cota do projeto	Integrado ao CI do GitLab
AWS ECR	*.dkr.ecr*.amazonaws.com	Camada gratuita limitada	Cobra armazenamento e transferência
Google Artifact Registry	*-docker.pkg.dev	Camada gratuita limitada	Substituiu o GCR
Azure ACR	*.azurecr.io	Não	Três níveis de serviço
Harbor	auto-hospedado	Livre (Apache 2.0)	Replicação, políticas, escaneamento, quarentena
registry:2	auto-hospedado	Livre	Mínimo, sem interface
Zot	auto-hospedado	Livre	Registry OCI puro, leve

17.2.1 O limite do Docker Hub e o que fazer

Desde **1º de abril de 2025**:

Situação	Limite
Sem autenticação	10 pulls/hora , por IP
Conta gratuita autenticada	100 pulls/hora
Pro / Team / Business	Ilimitado, sob uso justo

O impacto real é em CI: um runner em IP compartilhado consome os 10 pulls em minutos e o pipeline quebra com toomanyrequests.

As quatro mitigações, em ordem de eficácia:

```
# 1) Autentique no CI (mais barato e imediato)
echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USER" --password-stdin

# 2) Espelho puxador (pull-through cache) na sua rede
docker run -d --name espelho -p 5000:5000 \
  -e REGISTRY_PROXY_REMOTEURL=https://registry-1.docker.io \
  -v espelho-dados:/var/lib/registry \
  registry:2
# e no /etc/docker/daemon.json:
# { "registry-mirrors": ["http://espelho.interno:5000"] }

# 3) Copie as imagens de que depende para o SEU registry
skopeo copy docker://docker.io/library/postgres:16 docker://ghcr.io/minhaorg/postgres:16

# 4) Use registries alternativos
#   public.ecr.aws/docker/library/postgres:16   · quay.io/...   · ghcr.io/...
```

bash

17.3 Autenticação

```
echo "$TOKEN" | docker login -u USUARIO --password-stdin
echo "$TOKEN" | docker login ghcr.io -u USUARIO --password-stdin
aws ecr get-login-password --region us-east-1 | docker login --password-stdin \
  --username AWS 123456789.dkr.ecr.us-east-1.amazonaws.com
gcloud auth configure-docker us-central1-docker.pkg.dev
```

bash

Nunca use a senha da conta. Use *Personal Access Token* com escopo mínimo — ele pode ser revogado isoladamente e não dá acesso ao restante da conta.

17.3.1 Onde a credencial fica, e o risco

```
cat ~/.docker/config.json
# { "auths": { "https://index.docker.io/v1/": { "auth": "dXNlcjp0b2tldg==" } } }
echo "dXNlcjp0b2tldg==" | base64 -d
# user:token ← base64 NÃO é criptografia
```

bash

Instale um *credential helper*:

Sistema	Helper
Linux	docker-credential-pass (GPG) ou docker-credential-secretservice
macOS	docker-credential-osxkeychain (padrão com o Desktop)
Windows	docker-credential-wincred

```
// ~/.docker/config.json
{ "credsStore": "pass" }
```

JSON

17.4 Estratégia de tags

A regra: latest não significa “mais recente”. É apenas a tag padrão quando nenhuma é dada, e é reescrita a cada publicação. Em produção, é a causa de “ontem funcionava”.

A estratégia que funciona:

```
IMG=ghcr.io/org/app
SHA=$(git rev-parse --short HEAD)
VER=$(cat VERSION)          # ex.: 1.4.2

docker build \
  -t $IMG:$VER \              # imutável, semântico → o que produção referencia
  -t $IMG:1.4 \               # minor móvel → recebe correções de patch
  -t $IMG:1 \                 # major móvel → recebe features compatíveis
  -t $IMG:sha-$SHA \          # rastreável ao commit → o que o CI usa
  -t $IMG:latest \            # conveniência de dev → NUNCA em produção
  .
```

bash

E o deploy sempre por digest:

```
DIGEST=$(docker buildx imagetools inspect $IMG:$VER --format '{{.Manifest.Digest}}')
docker run $IMG@$DIGEST
```

bash

Por quê: tag é ponteiro mutável; digest é o conteúdo. Fazer deploy por tag é fazer deploy do “que estiver lá quando o servidor puxar” — e é assim que um ataque de reescrita de tag chega à produção.

Automatize com `docker/metadata-action` (exemplo completo em [06-exemplos.md](#) (cap. 6)).

17.5 Cadeia de suprimentos: o assunto de 2026

A pergunta deixou de ser “esta imagem funciona?” e passou a ser **“esta imagem é a que eu acho que é, construída a partir do que eu acho, sem componente vulnerável conhecido?”**.

17.5.1 As quatro perguntas e as quatro respostas

Pergunta	Artefato	Ferramenta
O que tem dentro?	SBOM (<i>Software Bill of Materials</i>)	syft, docker buildx --sbom
Tem vulnerabilidade conhecida?	Relatório de escaneamento	trivy, gype, docker scout
Quem construiu, a partir de quê?	Atestado de proveniência (SLSA)	--provenance=true
É mesmo de quem diz ser?	Assinatura	cosign

17.5.2 SBOM

bash

```
docker buildx build --sbom=true --provenance=true -t org/app:1.0 --push .
docker buildx imagetools inspect org/app:1.0 --format '{{json .SBOM}}' | jq '.SPDX.packages | length'

# Ou gere separadamente
syft org/app:1.0 -o spdx-json > sbom.json
syft org/app:1.0 -o cyclonedx-json > sbom-cdx.json
```

Formatos: **SPDX** (Linux Foundation, ISO) e **CycloneDX** (OWASP). Ambos são aceitos; escolha um e seja consistente.

Por que importa: quando sai um Log4Shell, a pergunta “quais das nossas 400 imagens têm log4j 2.14?” precisa ser respondida em minutos, não em semanas. Com SBOM, é uma consulta.

17.5.3 Escaneamento

bash

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed org/app:1.0
gype org/app:1.0
docker scout cves org/app:1.0
docker scout recommendations org/app:1.0 # sugere base menos vulnerável
```

Sobre --ignore-unfixed: falhar o pipeline por uma CVE que **não tem correção publicada** só ensina a equipe a ignorar o scanner. Falhe pelo que é corrigível; monitore o resto.

Sobre falsos positivos: scanners olham versões de pacote, não uso. Uma CVE numa biblioteca que seu código nunca chama continua sendo reportada. Use .trivyignore com **justificativa e data de revisão** — nunca um ignore silencioso e eterno.

17.5.4 Assinatura com cosign (keyless)

bash

```
# Assina usando OIDC – SEM chave privada para vazar
cosign sign --yes ghcr.io/org/app@sha256:abc...

# Verifica: exige que tenha sido construída por AQUELE workflow daquele repositório
cosign verify ghcr.io/org/app@sha256:abc... \
  --certificate-identity-regexp "https://github.com/org/app/.github/workflows/.*" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com"
```

O modelo *keyless* usa o Fulcio (CA de curta duração) e o Rekor (log público de transparência, somente-anexação) do projeto Sigstore. Você troca “guardar uma chave privada para sempre” por “provar identidade no momento da assinatura” — que é um problema muito mais fácil de operar bem.

17.5.5 Aplicação de política

De nada adianta assinar se ninguém verifica. Em Kubernetes, um *admission controller*:

YAML

```
# Kyverno
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: exigir-assinatura
spec:
  validationFailureAction: Enforce
  rules:
    - name: verificar-imagens
      match:
        resources: { kinds: [Pod] }
      verifyImages:
        - imageReferences: ["ghcr.io/org/*"]
          attestors:
            - entries:
                - keyless:
                    subject: "https://github.com/org/*"
                    issuer: "https://token.actions.githubusercontent.com"
```

Alternativas: Sigstore Policy Controller, OPA Gatekeeper, Notary v2/Notation.

17.6 Registry próprio

17.6.1 Mínimo, para laboratório

bash

```
docker run -d --name registry -p 5000:5000 --restart unless-stopped \
  -v registry-dados:/var/lib/registry registry:2

docker tag app:1.0 localhost:5000/app:1.0
docker push localhost:5000/app:1.0
curl -s localhost:5000/v2/_catalog | jq
```

Sem TLS, só funciona via localhost ou com *insecure-registries* no daemon — que desliga a verificação e só é aceitável em rede isolada de laboratório.

17.6.2 Com TLS e autenticação

YAML

```
services:
  registry:
    image: registry:2
    environment:
      REGISTRY_AUTH: htpasswd
      REGISTRY_AUTH_HTPASSWD_REALM: "Registry"
      REGISTRY_AUTH_HTPASSWD_PATH: /auth/htpasswd
      REGISTRY_STORAGE_DELETE_ENABLED: "true" # sem isto, não dá para apagar nada
    volumes:
      - ./auth:/auth:ro
      - registry-dados:/var/lib/registry
    networks: [interna]

  proxy:
    image: caddy:2-alpine # termina o TLS com certificado automático
    ports: ["443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-dados:/data
    networks: [interna, borda]
```

bash

```
docker run --rm --entrypoint htpasswd httpd:2 -Bbn usuario senha > auth/htpasswd
```

17.6.3 Harbor, quando a organização cresce

Harbor acrescenta o que falta no registry:2: interface web, projetos com controle de acesso, replicação entre regiões, escaneamento com quarentena automática, políticas de retenção, assinatura, proxy cache e quotas. É o padrão de fato para registry auto-hospedado corporativo.

17.6.4 Retenção — o custo que ninguém prevê

Registries crescem sem parar. Cada build de CI deixa uma imagem.

bash

```
# Apagar por digest (exige REGISTRY_STORAGE_DELETE_ENABLED=true)
curl -X DELETE https://registry/v2/app/manifests/sha256:abc...
# E depois a coleta de lixo, que só roda com o registry parado ou em modo somente leitura:
docker exec registry bin/registry garbage-collect /etc/docker/registry/config.yml
```

Política sensata: manter todas as tags semânticas, os últimos N builds por branch, e apagar sha-* com mais de 30 dias. ECR, GAR, ACR e Harbor têm regras declarativas de ciclo de vida — configure-as **no primeiro dia**, não quando a conta chegar.

17.7 Transferir imagens sem registry

bash

```
# Máquina sem rede (air-gapped)
docker save app:1.0 | gzip > app.tar.gz
# ... transfira por pendrive ...
gunzip -c app.tar.gz | docker load

# Direto por SSH
docker save app:1.0 | ssh usuario@destino 'docker load'
```

```
# skopeo: copia entre registries SEM baixar para o daemon local
skopeo copy docker://docker.io/library/nginx:alpine docker://registry.interno/nginx:alpine
skopeo copy --all docker://origem/app:1.0 docker://destino/app:1.0 # --all = todas as arquiteturas
skopeo inspect docker://nginx:alpine # inspeciona sem baixar
```

skopeo é subutilizado e resolve bem três problemas: espelhar imagens para registry interno, inspecionar sem pull, e copiar preservando digest e assinaturas.

17.8 Boas práticas de distribuição

1. **Deploy por digest**, sempre. Tag serve para humano.
2. **Um registry por ambiente**, ou ao menos projetos separados com permissões distintas.
3. **Espelhe as imagens de terceiros** de que você depende. Elas podem sumir — já sumiram.
4. **Assine no CI e verifique na admissão**. Assinar sem verificar é teatro.
5. **SBOM e proveniência em todo build de produção**. Custa uma flag.
6. **Retenção configurada desde o primeiro dia**.
7. **Escaneie no push e periodicamente** — CVEs novas aparecem em imagens antigas que não mudaram.
8. **Nunca publique imagem construída na máquina de alguém**. Só o CI publica; assim há proveniência.

17.9 Autoteste

1. Quais são os três recursos essenciais da API de um registry OCI?
2. Por que um docker pull de uma imagem sua costuma transferir pouco, mesmo sendo grande?
3. Quais são os limites de pull do Docker Hub hoje, e quais são as quatro mitigações?
4. Por que latest em produção é um problema, e o que usar no lugar?
5. Explique a diferença entre tag e digest no contexto de um ataque de reescrita de tag.
6. Quais são as quatro perguntas de cadeia de suprimentos e o artefato que responde cada uma?
7. Por que a assinatura *keyless* do cosign é operacionalmente melhor que uma chave privada?
8. Qual é o problema de assinar imagens sem um *admission controller* verificando?
9. Por que `--ignore-unfixed` no scanner é pragmatismo e não relaxamento?
10. Você precisa espelhar 50 imagens públicas para um registry interno sem encher o disco local. Qual ferramenta usa e por quê?

17.9.1 Fontes consultadas (11/08/2026)

- [Docker Docs — Usage and rate limits](#) e [GitLab Support — Docker Hub rate limiting](#) — 10/hora sem autenticação, 100/hora com conta gratuita, vigente desde 01/04/2025

CAPÍTULO 18

Segurança de containers

Nível: avançado · Última atualização: 11/08/2026

Container **não** é uma fronteira de segurança forte. Este arquivo explica o que ele protege, o que não protege, e o que fazer a respeito — em ordem de retorno sobre o esforço.

18.1 O modelo de ameaça, explicitado

Antes de qualquer controle, defina de quem você está se defendendo:

Ameaça	Container protege?	O que realmente resolve
Bug da sua aplicação consumindo toda a RAM	✓ com cgroup	Limites de recurso
Sua aplicação lendo arquivos do host	✓	Namespaces (salvo bind mount)
Dependência com CVE conhecida	✗	Escaneamento + atualização
Imagem base maliciosa	✗	Proveniência, assinatura, base confiável
Segredo vazado na imagem	✗	--mount=type=secret, escaneamento de segredo
Escape via bug do kernel	✗	Kernel atualizado, seccomp, gVisor/Kata
Escape via configuração errada (--privileged, socket)	✗	Higiene de configuração
Movimento lateral entre containers	⚠ parcial	Segmentação de rede
Exfiltração de dados	✗	Rede internal, egresso controlado
Tenant hostil no mesmo host	✗	Micro-VM ou máquinas separadas

A frase que resume: container isola **acidente** muito bem e isola **ataque** razoavelmente, desde que bem configurado. Se o adversário é hostil e desconhecido, o container **não é a última barreira** — coloque uma VM.

18.2 As dez medidas, ordenadas por custo-benefício

Se você fizer só as cinco primeiras, já estará acima da média da indústria.

18.2.1 Não rode como root (custo: 3 linhas)

```
RUN addgroup -S app && adduser -S -G app app
USER app
```

Dockerfile

```
user: "1000:1000" # no compose, se a imagem não permitir alterar
```

YAML

bash

```
# Auditoria: quais dos seus containers rodam como root?
docker ps -q | xargs -I{} sh -c 'echo -n "{} "; docker exec {} id -u 2>/dev/null || echo "?"'
```

18.2.2 no-new-privileges (custo: 1 linha)

YAML

```
security_opt: [no-new-privileges:true]
```

Impede que qualquer binário setuid dentro do container eleve privilégio. Não quebra nada em aplicações normais.

18.2.3 Descarte capabilities (custo: 2 linhas)

YAML

```
cap_drop: [ALL]
cap_add: [NET_BIND_SERVICE] # só se precisar de porta < 1024 – e você não deveria
```

18.2.4 Sistema de arquivos raiz somente leitura (custo: 2 linhas)

YAML

```
read_only: true
tmpfs: [/tmp, /run]
```

Elimina a classe inteira de “atacante grava webshell/binário no container”. A maioria das aplicações web funciona sem alteração; as que não funcionam só precisam declarar onde escrevem.

18.2.5 Limites de recurso (custo: 4 linhas)

YAML

```
deploy:
  resources:
    limits: { memory: 512M, cpus: "1.0" }
pids_limit: 200
```

`pids_limit` é a defesa mais barata contra *fork bomb*. Sem limite de memória, um vazamento derruba o host inteiro.

18.2.6 Segmentação de rede

YAML

```
networks:
  interna:
    internal: true
```

Ver [16-redes.md](#) (cap. 14). Uma linha que corta o passo mais comum de uma cadeia de ataque: baixar ferramentas e exfiltrar dados.

18.2.7 Imagem base mínima e atualizada

Menos pacotes = menos CVEs. `distroless/scratch` não têm shell, o que sozinho inutiliza a maior parte dos payloads prontos.

bash

```
docker scout cves minha-app:1.0
docker scout recommendations minha-app:1.0
```

18.2.8 Segredos fora da imagem e fora do ambiente

- **Build:** `--mount=type=secret`, nunca ARG.
- **Runtime:** arquivo em tmpfs (secrets:) em vez de environment: — variáveis aparecem em `docker inspect`, em `/proc/PID/environ`, em despejos de erro e são herdadas por filhos.

bash

```
# Escaneie o repositório e as imagens em busca de segredo vazado
docker run --rm -v "$PWD:/src" zricethezav/gitleaks:latest detect --source=/src
trivy image --scanners secret minha-app:1.0
```

18.2.9 Escaneamento contínuo no CI

Ver [19-registries-e-distribuicao.md](#) (cap. 17). Escaneie **no push e periodicamente** — uma imagem que não mudou pode ter CVE nova amanhã.

18.2.10 Nunca monte o socket do Docker

YAML

```
volumes:
  - /var/run/docker.sock:/var/run/docker.sock # [x] isso é dar root no host
```

Se for inevitável (Portainer, Watchtower, agente de CI), use um **proxy de socket** com `allow-list`:

YAML

```
docker-proxy:
  image: tecnativa/docker-socket-proxy
  environment:
    CONTAINERS: 1 # permite só listar containers
    POST: 0 # nenhuma operação de escrita
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
  networks: [interna]
```

18.3 A demonstração que convence: escape com socket montado

Rode num laboratório descartável, nunca em máquina que importe.

bash

```
# 1) Um container "inocente" com o socket montado
docker run -it --rm -v /var/run/docker.sock:/var/run/docker.sock alpine sh
apk add --no-cache curl >/dev/null

# 2) De dentro dele, crie um container privilegiado com a raiz do host montada
curl -s -X POST --unix-socket /var/run/docker.sock \
-H "Content-Type: application/json" \
-d '{"Image": "alpine", "Cmd": ["sh"], "OpenStdin": true, "Tty": true,
  "HostConfig": {"Binds": ["/:/host"], "Privileged": true}}' \
http://localhost/containers/create
```

```
# 3) Inicie e anexe – você tem a raiz do host em /host, como root.
```

Isso não é um bug. É o comportamento documentado da API. O socket é a interface privilegiada do daemon. Quem o alcança, alcança a máquina.

O mesmo vale para:

- pertencer ao grupo docker;
- `--privileged`;
- `--pid=host` combinado com `nsenter`;
- montar / ou /etc como bind mount gravável.

18.4 As configurações que anulam o isolamento

Configuração	O que quebra	Alternativa
<code>--privileged</code>	Tudo: capabilities, seccomp, LSM, dispositivos	<code>--cap-add</code> específico, <code>--device</code> específico
<code>-v /var/run/docker.sock:...</code>	Root no host	Proxy de socket com allow-list
<code>--pid=host</code>	Vê e sinaliza processos do host	<code>--pid container:outro</code>
<code>--net=host</code>	Sem isolamento de rede; alcança serviços em localhost do host	Rede bridge + <code>-p</code>
<code>--ipc=host</code>	Memória compartilhada do host	<code>--shm-size</code>
<code>--userns=host</code> (com <code>userns-remap</code> ativo)	Anula o remapeamento	—
<code>--security-opt seccomp=unconfined</code>	~350 syscalls expostas	Perfil customizado
<code>--security-opt apparmor=unconfined</code>	Sem MAC	Perfil customizado
<code>-v /:/host</code>	Sistema de arquivos inteiro	Montar só o necessário, <code>:ro</code>
<code>--device=/dev/mem</code>	Memória física	Nunca

bash

```
# Auditoria rápida: quem está privilegiado?
docker ps -q | xargs docker inspect --format \
  '{{.Name}} privileged={{.HostConfig.Privileged}} pid={{.HostConfig.PidMode}} net={{.HostConfig.NetworkMode}}'
```

18.5 Ferramentas de auditoria

bash

```
# Docker Bench for Security – CIS Docker Benchmark automatizado
docker run --rm --net host --pid host --userns host --cap-add audit_control \
  -v /var/lib:/var/lib:ro -v /var/run/docker.sock:/var/run/docker.sock:ro \
  -v /etc:/etc:ro --label docker_bench_security \
  docker/docker-bench-security

# Trivy: vulnerabilidades, segredos, má configuração – em imagem, Dockerfile e Compose
trivy image minha-app:1.0
trivy config .
trivy fs --scanners secret .

# Gype + Syft
syft minha-app:1.0 -o json | gype
```

```
# Dockle: boas práticas de imagem (usuário, healthcheck, segredos)
docker run --rm goodwithtech/dockle minha-app:1.0

# Hadolint: linter de Dockerfile
docker run --rm -i hadolint/hadolint < Dockerfile
```

Deteção em tempo de execução: Falco (CNCF) observa syscalls via eBPF e alerta sobre comportamento anômalo — shell aberto dentro de container, escrita em /etc, conexão de saída inesperada, montagem sensível. É a camada que responde “e se, apesar de tudo, entrarem?”.

```
docker run -d --name falco --privileged \
-v /var/run/docker.sock:/host/var/run/docker.sock \
-v /proc:/host/proc:ro falcosecurity/falco:latest
```

bash

18.6 Isolamento reforçado, quando o padrão não basta

Tecnologia	Como isola	Sobrecarga	Quando usar
runc (padrão)	Namespaces + cgroups	~0	Código confiável, mesmo dono
gVisor (runsc)	Kernel em espaço de usuário intercepta syscalls	10–30% em I/O	Código semiconfiável; reduz drasticamente a superfície de kernel
Kata Containers	Micro-VM com kernel próprio por container	+100–200 ms no start, +RAM	Multi-inquilino, conformidade
Firecracker	Micro-VM minimalista	~125 ms de boot	Base do AWS Lambda e Fargate
Máquinas separadas	Físico	Custo de hardware	Quando o dado justifica

```
docker run --runtime=runsc alpine echo "rodando sob gVisor"
```

bash

Critério de decisão: se você executa código que **você não escreveu e não revisou** — plugins de cliente, execução de código submetido por usuário, build de repositório de terceiro — o container padrão não é suficiente. Use micro-VM.

18.7 Rootless como padrão de servidor

O daemon rodando como root é a maior concentração de privilégio de uma instalação típica.

```
sudo apt install -y docker-ce-rootless-extras uidmap
dockerd-rootless-setupool.sh install
export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock
docker info | grep -i rootless
```

bash

Ganho

Escape não entrega root do host

Perda

Portas < 1024 exigem configuração extra

Ganho	Perda
Cada usuário com seu daemon isolado	Rede em espaço de usuário: mais lenta
Instalação sem privilégio	Alguns drivers indisponíveis; cgroup exige delegação

Podman é rootless e sem daemon por padrão — é o motivo principal de sua adoção em ambientes regulados (bancos, saúde, governo).

18.8 Segredos em tempo de execução

Ordem de qualidade, do pior para o melhor:

Método	Nota	Problema
Embutido na imagem	✗	Qualquer um que baixe a imagem tem o segredo
--build-arg	✗	Fica no histórico da imagem
-e SENHA=...	⚠	Visível em docker inspect, /proc/PID/envIRON, logs de crash, herdado por filhos
--env-file	⚠	Melhor que a linha de comando, mas continua sendo variável de ambiente
secrets: (arquivo em tmpfs)	✓	Bom; fora do Swarm, não há criptografia em repouso
Docker/Swarm secrets	✓	Criptografado em repouso e em trânsito
Vault, AWS/GCP/Azure Secrets Manager	✓✓	Rotação, auditoria, controle de acesso fino
SOPS + age/KMS no Git	✓✓	GitOps com segredo cifrado versionado

bash

```
# Prova de que variável de ambiente vaza
docker run -d --name x -e SENHA=supersecreta alpine sleep 300
docker inspect x --format '{{json .Config.Env}}'      # lá está
docker exec x cat /proc/1/envIRON | tr '\0' '\n'      # e aqui também
docker rm -f x
```

Aplicações modernas frequentemente aceitam o sufixo _FILE:

YAML

```
environment:
  POSTGRES_PASSWORD_FILE: /run/secrets/db_senha
secrets: [db_senha]
```

18.9 Endurecimento do host

O container é tão seguro quanto o host embaixo dele.

bash

```
# Kernel atualizado – a defesa mais importante contra escape
uname -r
sudo apt update && sudo apt upgrade

# Atualizações automáticas de segurança
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades

# Auditoria de quem acessa o Docker
sudo auditctl -w /usr/bin/dockerd -k docker
```

```
sudo auditctl -w /var/run/docker.sock -k docker-socket

# Nunca exponha a API do daemon sem TLS mútuo
ss -tlnp | grep 2375      # se aparecer algo aqui, você tem um incidente
```

A porta 2375 (API sem TLS) exposta à internet é uma das formas mais comuns de servidores serem comprometidos para mineração de criptomoeda. Varreduras automatizadas a encontram em minutos. Se precisa de acesso remoto, use `docker context` sobre **SSH** — que não expõe porta nenhuma.

JSON

```
// /etc/docker/daemon.json – configuração endurecida
{
  "icc": false,                      // containers não conversam na bridge padrão
  "no-new-privileges": true,         // padrão para todos os containers
  "usersns-remap": "default",        // root do container ≠ root do host
  "live-restore": true,
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "default-ulimits": { "nofile": { "Hard": 65535, "Soft": 65535 } }
}
```

18.10 Resposta a incidentes

Quando você suspeita de comprometimento:

bash

```
# 1) NÃO mate o container ainda – preserve as evidências
docker pause CONTAINER

# 2) Capture o estado
docker inspect CONTAINER > incidente-inspect.json
docker logs CONTAINER > incidente-logs.txt 2>&1
docker diff CONTAINER > incidente-diff.txt      # o que foi alterado na camada de escrita
docker export CONTAINER > incidente-fs.tar      # o sistema de arquivos inteiro

# 3) Rede e processos
PID=$(docker inspect -f '{{.State.Pid}}' CONTAINER)
sudo ls -l /proc/$PID/exe /proc/$PID/cwd
sudo ss -tanp | grep $PID
docker top CONTAINER

# 4) Isole
docker network disconnect $(docker inspect -f '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}{{end}}' CONTAINER) CONTAINER

# 5) Só então
docker stop CONTAINER
```

E as perguntas que orientam a investigação: a imagem veio de onde? Foi assinada? O socket estava montado? Havia `--privileged`? Qual credencial esse container conhecia (e portanto precisa ser rotacionada)? Que outros serviços ele alcançava pela rede?

18.11 Checklist de produção

Imagem

- ☐ Base mínima, fixada por digest
- ☐ Multi-stage: nenhum toolchain na imagem final

- ☐ USER não-root
- ☐ Nenhum segredo em ARG, ENV ou camada
- ☐ Escaneada no CI; falha em CRITICAL corrigível
- ☐ SBOM e proveniência gerados
- ☐ Assinada, e a assinatura **verificada** no deploy

Runtime

- ☐ no-new-privileges:true
- ☐ cap_drop: [ALL], com cap_add mínimo
- ☐ read_only: true + tmpfs para os caminhos graváveis
- ☐ Limites de memória, CPU e pids_limit
- ☐ Sem --privileged, sem socket do Docker, sem --pid=host
- ☐ Rede segmentada; internal: true onde não precisa de saída
- ☐ Segredos por arquivo/gerenciador, não por variável de ambiente
- ☐ Limite de tamanho de log configurado

Host

- ☐ Kernel e Docker atualizados; atualizações automáticas de segurança
- ☐ Rootless, ou grupo docker restrito e auditado
- ☐ API do daemon **não** exposta em TCP sem TLS mútuo
- ☐ Auditoria (auditd) no binário e no socket
- ☐ Detecção em execução (Falco) em ambientes sensíveis
- ☐ Backup testado dos volumes

18.12 Autoteste

1. Contra quais três ameaças o container protege bem, e contra quais três não protege?
2. Escreva as cinco linhas de YAML de maior retorno em segurança e explique cada uma.
3. Demonstre, em um comando, por que montar o socket do Docker é dar root no host.
4. Por que read_only: true raramente quebra uma aplicação web, e o que ela precisa declarar?
5. Cite quatro configurações que anulam o isolamento e a alternativa mais restrita de cada.
6. Por que variável de ambiente é um lugar ruim para senha? Cite três formas de vazamento.
7. Quando você escolheria Kata ou gVisor em vez de runc? Qual critério objetivo usa?
8. Por que a porta 2375 exposta é um incidente, e qual é a alternativa correta de acesso remoto?
9. Qual é o primeiro comando ao suspeitar de comprometimento, e por que **não** é docker stop?

10. Sua imagem passa no scanner sem nenhuma CVE. Cite três vetores de ataque que continuam abertos.

Observabilidade e operação

Nível: intermediário → avançado · Última atualização: 11/08/2026

Rodar container é fácil. Operar container é a especialidade. Este arquivo é sobre o que acontece depois do primeiro deploy.

19.1 Logs

19.1.1 A regra fundamental

Container escreve log em stdout/stderr. Ponto.

Não em arquivo. Não em syslog dentro do container. Não com rotação própria. O processo emite; a plataforma decide onde aquilo vai parar. Isso é o *factor XI* dos Twelve-Factor App, e vale porque:

- container é efêmero — arquivo dentro dele some com ele;
- arquivo de log dentro do container **enche o disco do host** sem que ninguém veja;
- a plataforma (Docker, Kubernetes, ECS) já sabe coletar, rotacionar e agregar stdout.

19.1.2 Drivers de log

```
docker system info | grep "Logging Driver" bash
```

Driver	docker logs funciona?	Uso
json-file (padrão)	sim	Desenvolvimento e servidor único. Sem limite por padrão
local	sim	Formato binário, mais eficiente, com rotação por padrão
journald	sim	Integra com journalctl, retenção do systemd
syslog	não	Envio a servidor central
fluentd / gelf	não	Agregação (EFK, Graylog)
awslogs / gcplogs	não	Nuvem
none	não	Quando o app já envia por outro caminho

A configuração que todo servidor deveria ter — sem ela, um container falante enche o disco em semanas, e o sintoma aparece como “aplicação sem relação nenhuma parou de funcionar”:

JSON

```
// /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

bash

```
sudo systemctl restart docker    # vale para containers criados DEPOIS
```

bash

```
# Achar o log que encheu o disco
sudo find /var/lib/docker/containers -name "*.json.log" -size +100M -exec ls -lh {} \;
# Zerar um log sem parar o container (medida de emergência)
sudo truncate -s 0 /var/lib/docker/containers/<ID>/<ID>-json.log
```

19.1.3 Log estruturado

Uma linha JSON por evento, com campos consistentes. Isso transforma log em dado consultável.

JavaScript

```
console.log(JSON.stringify({
  ts: new Date().toISOString(),
  nivel: 'info',
  msg: 'requisição',
  rota: 'GET /api/pedidos',
  status: 200,
  ms: 42,
  trace_id: req.headers['x-trace-id'], // correlação entre serviços
}));
```

Regras que evitam retrabalho:

- **Um evento por linha.** Stack trace multi-linha quebra a correlação; serialize a pilha num campo.
- **Campos com nomes estáveis.** Renomear campo depois quebra dashboards e alertas.
- **Nunca registre segredo, token ou dado pessoal.** Log vaza com muito mais facilidade que banco.
- **Inclua um identificador de correlação** (trace_id) propagado entre serviços.

19.1.4 Agregação

YAML

```
# Loki + Promtail – a opção mais leve para servidor único
loki:
  image: grafana/loki:3.0.0
  command: ["-config.file=/etc/loki/local-config.yaml"]
  volumes: [loki-dados:/loki]

promtail:
  image: grafana/promtail:3.0.0
  volumes:
    - /var/lib/docker/containers:/var/lib/docker/containers:ro
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - ./promtail.yaml:/etc/promtail/config.yaml:ro
```

Solução	Perfil
Loki + Grafana	Leve; indexa rótulos, não o conteúdo. Barato de operar
Elasticsearch + Kibana	Busca completa; caro em memória e operação
OpenSearch	Fork do Elasticsearch, licença Apache 2.0
Vector	Coletor/roteador muito eficiente; bom substituto de Fluentd/Logstash
CloudWatch / Cloud Logging	Gerenciado; simples e caro em volume

19.2 Métricas

19.2.1 O que o Docker já dá

bash

```
docker stats                                # ao vivo
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}"
```

Para coleta automática, **cAdvisor** expõe métricas de container em formato Prometheus:

YAML

```
cadvisor:
  image: gcr.io/cadvisor/cadvisor:v0.49.1
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
  ports: ["8081:8080"]

prometheus:
  image: prom/prometheus:v2.53.0
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prom-dados:/prometheus
  ports: ["9090:9090"]

grafana:
  image: grafana/grafana:11.1.0
  environment:
    GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_SENHA:?}
  volumes: [grafana-dados:/var/lib/grafana]
  ports: ["3001:3000"]
```

prometheus.yml:

YAML

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: cadvisor
    static_configs: [{ targets: ["cadvisor:8080"] }]
  - job_name: aplicacao
    static_configs: [{ targets: ["api:3000"] }]
metrics_path: /metricas
```

19.2.2 As métricas que importam de verdade

Métrica	Por que	Alerta razoável
container_memory_working_set_bytes / limit	Proximidade do OOM	> 85% por 5 min
container_cpu_cfs_throttled_periods_total	Throttling — latência sem CPU aparente	qualquer valor sustentado
Reinícios por hora	Laço de falha	> 3/h
Estado do healthcheck	Serviço degradado	unhealthy por 2 min
Latência p95/p99 da aplicação	O que o usuário sente	depende do SLO
Taxa de erro 5xx	Falha real	> 1% por 5 min
Espaço em /var/lib/docker	Falha iminente e transversal	> 80%
Idade da imagem em produção	Dívida de segurança	> 90 dias

O throttling é a métrica mais subutilizada. Latência alta com CPU “ociosa” quase sempre é throttling de cgroup: o container gasta a cota em 20 ms de um período de 100 ms e fica 80 ms parado. Ver [13-isolamento-namespaces-cgroups.md](#) (cap. 11).

19.2.3 Os quatro sinais de ouro (SRE)

Latência · Tráfego · Erros · Saturação. Se você só puder instrumentar quatro coisas, sejam essas. E meça **latência de requisições com erro separadamente** — erro rápido puxa a média para baixo e esconde o problema.

19.3 Healthchecks

Dockerfile

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=15s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/saude || exit 1
```

bash

```
docker inspect --format '{{json .State.Health}}' NOME | jq
# Status: starting | healthy | unhealthy + os últimos 5 resultados
```

19.3.1 Liveness x readiness — a distinção que evita cascata

Sonda	Pergunta	Ação em falha
Liveness (/vivo)	O processo está de pé e responsivo?	Reiniciar
Readiness (/saude)	Consigo atender agora?	Tirar do balanceamento (sem reiniciar)

O Docker tem apenas um HEALTHCHECK (semântica de readiness); o Kubernetes separa os dois. Implemente **ambos os endpoints** de qualquer forma — a distinção é da aplicação, não da plataforma.

Por que importa: se o banco cair, todas as réplicas ficam “não prontas”. Com liveness apontando para o banco, **todas reiniciam ao mesmo tempo** e o boot em massa piora a situação. Liveness deve testar só o processo; readiness testa as dependências.

Regras de um healthcheck decente:

- Exercite a dependência crítica (uma consulta rápida ao banco, uma escrita no disco). Um 200 fixo é otimismo travestido de monitoramento.
- Seja rápido (< 1 s) e barato: ele roda a cada 30 s, em todas as réplicas, para sempre.
- Use `start_period` generoso para aplicações de boot lento (JVM, migrações).
- Não faça o healthcheck depender de serviço externo de terceiro — você importa a indisponibilidade dele.

19.4 Encerramento gracioso e deploy sem queda

```
docker stop → SIGTERM → espera (10s por padrão) → SIGKILL
```

A sequência correta dentro da aplicação:

1. Recebe SIGTERM.
2. **Marca-se como não pronta** (readiness falha) — o balanceador para de mandar tráfego.
3. Espera as requisições em curso terminarem.
4. Fecha conexões de banco, descarrega buffers.
5. Sai com código 0.

YAML

```
services:
  api:
    stop_grace_period: 30s    # aumente se o app precisa de mais que 10s
    stop_signal: SIGTERM
```

Os três erros que causam 502 no deploy:

1. **Forma shell no CMD** — o `/bin/sh` não repassa o sinal.
2. **App sem tratador de SIGTERM** — como PID 1, o sinal é simplesmente ignorado.
3. **Sair imediatamente ao receber o sinal** — as requisições em curso morrem no meio.

Rolling update manual com Compose:

bash

```
docker compose pull
docker compose up -d --wait --no-deps api # --wait espera ficar healthy
```

Para deploy realmente sem interrupção com um servidor só, é preciso duas instâncias e um proxy que remova a antiga do rodízio antes de derrubá-la — o que o Swarm e o Kubernetes fazem nativamente.

19.5 Diagnóstico de produção — o procedimento

19.5.1 O container reinicia em laço

bash

```
docker ps -a                                # STATUS: Restarting (1) 3 seconds ago
docker logs --tail 100 NOME                 # funciona com o container parado
docker inspect --format '{{.State.ExitCode}} {{.State.OOMKilled}} {{.State.Error}}' NOME
docker inspect --format '{{.RestartCount}}' NOME
```

Exit code	Causa
0	Terminou "normalmente" — o processo não era de longa duração
1	Erro da aplicação; leia os logs
137	SIGKILL — quase sempre OOM ; confira OOMKilled
139	SIGSEGV — falha de segmentação
143	SIGTERM — parada normal
125/126/127	Erro do Docker / não executável / comando não encontrado

19.5.2 O container está lento

bash

```
docker stats --no-stream NOME
# CPU% perto do limite? → throttling
# MEM próximo do limite? → GC agressivo, swap, OOM iminente

# Throttling, o dado que fecha o diagnóstico
CG=/sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}' NOME).scope
cat $CG/cpu.stat | grep -E 'nr_throttled|throttled_usec'

# I/O de disco
docker exec NOME cat /proc/1/io
```

19.5.3 A memória cresce sem parar

bash

```
docker stats --no-stream NOME
docker exec NOME cat /sys/fs/cgroup/memory.current /sys/fs/cgroup/memory.max
docker exec NOME cat /sys/fs/cgroup/memory.stat | head -20
```

Cuidado com a leitura: `container_memory_usage_bytes` **inclui cache de página**, que é recuperável. A métrica correta para "quanto o processo realmente precisa" é `container_memory_working_set_bytes`. Muito alarme falso vem dessa confusão.

19.5.4 Ninguém conecta no serviço

Ver a árvore de diagnóstico em [16-redes.md](#) (cap. 14).

19.6 Atualização de containers

19.6.1 Manual, com Compose (recomendado)

bash

```
docker compose pull
docker compose up -d --wait
docker image prune -af --filter "until=168h"
```

19.6.2 Automática, com Watchtower — e por que eu não recomendaria em produção

YAML

```
watchtower:
  image: containrrr/watchtower
  volumes: [/var/run/docker.sock:/var/run/docker.sock]
  command: --interval 3600 --cleanup --label-enable
```

Opinião profissional: Watchtower é excelente em homelab e perigoso em produção. Ele **puxa a tag** e reinicia — ou seja, faz deploy de código que ninguém testou, no meio da noite, sem rollback. Além disso, exige o socket do Docker montado, com todo o risco descrito em [cap. 18](#).

Em produção: atualização é uma **decisão**, disparada por pipeline, com imagem testada, referenciada por digest e com rollback pronto.

19.7 Backup operacional

bash

```
#!/usr/bin/env bash
# backup-diario.sh
set -euo pipefail

DESTINO=/backups/$(date +%F)
mkdir -p "$DESTINO"

# 1) Dumps lógicos dos bancos (NÃO copie arquivos de banco a quente)
docker compose exec -T db pg_dump -U app --no-owner app | gzip > "$DESTINO/db.sql.gz"

# 2) Volumes de arquivos
for v in $(docker volume ls -q --filter label=com.docker.compose.project=minhaapp); do
  docker run --rm -v "$v":d:ro -v "$DESTINO":/b alpine \
    tar czf "/b/${v}.tgz" -C /d .
done

# 3) A configuração também é dado
tar czf "$DESTINO/config.tgz" compose.yaml nginx/ .env.example

# 4) Verificação — backup não verificado não é backup
for f in "$DESTINO"/*.tgz "$DESTINO"/*.gz; do
  gzip -t "$f" || { echo "CORROMPIDO: $f" >&2; exit 1; }
done

# 5) Fora do local (regra 3-2-1)
rclone sync "$DESTINO" remoto:backups/$(date +%F)

# 6) Retenção
find /backups -maxdepth 1 -type d -mtime +30 -exec rm -rf {} +
```

E, trimestralmente, **restaure num ambiente descartável e verifique**. Backup nunca restaurado é esperança, não backup.

19.8 Uma stack de observabilidade completa

YAML

```

name: observabilidade

services:
  prometheus:
    image: prom/prometheus:v2.53.0
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.retention.time=30d
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prom-dados:/prometheus
    networks: [obs]
    restart: unless-stopped

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:v0.49.1
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro
    networks: [obs]
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:v1.8.1
    command: ["--path.rootfs=/host"]
    volumes: ["/:/host:ro,rslave"]
    networks: [obs]
    restart: unless-stopped

  loki:
    image: grafana/loki:3.0.0
    volumes: [loki-dados:/loki]
    networks: [obs]
    restart: unless-stopped

  promtail:
    image: grafana/promtail:3.0.0
    volumes:
      - /var/lib/docker/containers:/var/lib/docker/containers:ro
      - ./promtail.yml:/etc/promtail/config.yml:ro
    networks: [obs]
    restart: unless-stopped

  grafana:
    image: grafana/grafana:11.1.0
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_SENHA:?}
      GF_USERS_ALLOW_SIGN_UP: "false"
    volumes: [grafana-dados:/var/lib/grafana]
    ports: ["127.0.0.1:3001:3000"]
    networks: [obs]
    restart: unless-stopped

  alertmanager:
    image: prom/alertmanager:v0.27.0
    volumes: ["/.alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro"]
    networks: [obs]
    restart: unless-stopped

volumes: { prom-dados: , loki-dados: , grafana-dados: }
networks: { obs: }

```

Alertas mínimos (alertas.yml do Prometheus):

```
groups:
- name: containers
  rules:
    - alert: ContainerReiniciando
      expr: rate(container_start_time_seconds[15m]) > 0.003
      for: 10m
      annotations: { summary: "{{ $labels.name }}" está reiniciando em laço" }

    - alert: MemoriaPertoDoLimite
      expr: container_memory_working_set_bytes / container_spec_memory_limit_bytes > 0.85
      for: 5m
      annotations: { summary: "{{ $labels.name }}" acima de 85% do limite de memória" }

    - alert: CPUEstrangulada
      expr: rate(container_cpu_cfs_throttled_periods_total[5m]) > 0
      for: 10m
      annotations: { summary: "{{ $labels.name }}" sofrendo throttling de CPU" }

    - alert: DiscoQuaseCheio
      expr: node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{mountpoint="/" } < 0.2
      for: 5m
      annotations: { summary: "menos de 20% de disco livre" }
```

19.9 Rotina operacional

Diária (automatizada): backup com verificação · alertas ativos · espaço em disco.

Semanal: revisar reinícios e containers unhealthy · docker system df · revisar CVEs novas nas imagens em produção.

Mensal: atualizar imagens base · testar restauração de backup · revisar alertas que dispararam sem ação (ruído) e incidentes sem alerta (cobertura).

Trimestral: revisar limites de recurso contra o uso real · revisar configuração de segurança · exercício de recuperação de desastre.

19.10 Autoteste

1. Por que a aplicação deve escrever log em stdout e não em arquivo? Dê duas razões concretas.
2. Qual configuração de log todo servidor deveria ter, e o que acontece sem ela?
3. Diferencie liveness de readiness e explique a cascata que a confusão entre as duas provoca.
4. Um container tem exit code 137. Qual é a causa mais provável e qual comando confirma?
5. Latência alta, CPU aparentemente ociosa. Qual métrica você consulta e onde?
6. Qual é a diferença entre container_memory_usage_bytes e container_memory_working_set_bytes, e qual usar em alerta?
7. Descreva os cinco passos de um encerramento gracioso correto.
8. Cite os três erros que causam 502 durante o deploy.
9. Por que Watchtower é ótimo em homelab e ruim em produção?
10. Cite os quatro sinais de ouro e uma métrica de container para cada um.

Orquestração — quando um servidor não basta

Nível: intermediário → avançado · Última atualização: 11/08/2026

Container resolve empacotamento. Orquestração resolve **onde**, **quantos** e **o que fazer quando falha**. Este arquivo mostra a ponte, sem transformar o material num curso de Kubernetes.

20.1 Os problemas que a orquestração resolve

Com um servidor e o Compose, você já tem restart: unless-stopped. O que **não** tem:

Problema	Compose	Orquestrador
A máquina morre	✗ tudo cai	✓ reagenda em outro nó
Deploy sem interrupção	⚠ manual, com dois containers	✓ rolling update com rollback
Escalar conforme a carga	✗ manual	✓ autoscaling
Balancear entre nós	✗	✓ mesh de serviço embutido
Colocar o serviço onde há recurso	✗	✓ scheduler com afinidade e taints
Segredos distribuídos e cifrados	⚠ arquivo	✓ gerenciado e cifrado
Configuração declarativa reconciliada	⚠ imperativo	✓ loop de reconciliação
Descoberta de serviço entre máquinas	✗	✓ DNS de cluster

A pergunta certa não é "devo usar Kubernetes?", e sim "quantos dos problemas acima eu tenho hoje?". Se a resposta for zero ou um, Compose num servidor é a escolha correta, e adotar Kubernetes é comprar complexidade sem contrapartida.

20.2 As opções, honestamente comparadas

	Compose	Swarm	Nomad	Kubernetes
Máquinas	1	várias	várias	várias
Tempo até produzir	horas	dias	dias	semanas a meses
Complexidade conceitual	baixa	baixa	média	alta
Autoscaling	não	não (só escala manual)	sim	sim
Ecosistema	pequeno	estagnado	médio	enorme
Suporte de nuvem gerenciado	não	não	limitado	todos
Mercado de trabalho	—	pouco	pouco	dominante
Custo operacional	mínimo	baixo	médio	alto
Quando escolher	1 servidor	2-10 servidores, equipe pequena	Cargas mistas (VM + container)	Escala, múltiplas equipes, nuvem

20.2.1 Sobre o Swarm — a avaliação honesta

O Swarm mode continua embutido no Docker Engine, funciona e é **substancialmente** mais simples que o Kubernetes: você reaproveita quase todo o `compose.yaml`.

bash

```
docker swarm init
docker stack deploy -c compose.yaml minha-app
docker service ls
docker service scale minha-app_api=5
docker service update --image ghcr.io/org/api:1.5 minha-app_api # rolling update
docker service rollback minha-app_api
```

O problema não é técnico, é de ecossistema. O desenvolvimento praticamente parou, os fornecedores de nuvem não o oferecem gerenciado, ferramentas novas assumem Kubernetes, e contratar quem conheça Swarm é difícil.

Opinião profissional, com discordância legítima: para 2 a 10 servidores, com uma equipe pequena e sem exigência de autoscaling, **Swarm ainda é a decisão tecnicamente mais racional** — entrega 80% do valor com 15% da complexidade. Mas é uma aposta em tecnologia estagnada, e você deve tomá-la com os olhos abertos. Se houver perspectiva de crescer o time ou migrar para nuvem, vá direto ao Kubernetes.

20.3 Swarm em 20 minutos

bash

```
# No primeiro nó (manager)
docker swarm init --advertise-addr 192.168.1.10
# ele imprime um comando 'docker swarm join --token ...'

# Nos demais nós (workers)
docker swarm join --token SWMTKN-1-xxx 192.168.1.10:2377

docker node ls
```

`compose.yaml` para stack (a seção `deploy`: passa a ser respeitada):

YAML

```
services:
  api:
    image: ghcr.io/org/api@sha256:abc...
    networks: [interna]
    deploy:
      replicas: 3
      update_config:
        parallelism: 1 # uma réplica por vez
        delay: 10s
        order: start-first # sobe a nova antes de derrubar a antiga → sem queda
        failure_action: rollback
      rollback_config:
        parallelism: 0 # rollback de todas de uma vez
      restart_policy:
        condition: on-failure
        max_attempts: 3
      resources:
        limits: { cpus: "1.0", memory: 512M }
      placement:
        constraints: [node.role == worker]
        preferences: [spread: node.labels.zona]
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://127.0.0.1:3000/saude"]
      interval: 10s
      start_period: 30s
```

```
secrets: [db_senha]

secrets:
  db_senha:
    external: true      # criado com: docker secret create db_senha arquivo.txt

networks:
  interna:
    driver: overlay
    attachable: true
```

bash

```
docker secret create db_senha ./senha.txt
docker stack deploy -c compose.yaml minha-app
docker service ps minha-app_api      # onde cada réplica está rodando
docker service logs -f minha-app_api
```

O que o Swarm dá de graça: *routing mesh* (qualquer nó aceita a conexão e a encaminha à réplica correta), rede overlay cifrada opcional, segredos cifrados em repouso e em trânsito, rolling update com rollback automático em falha.

O que ele não dá: autoscaling, controle de admissão, CRDs, ecossistema.

20.4 Kubernetes — o mínimo para atravessar a ponte

O que muda no vocabulário:

Docker/Compose	Kubernetes	Observação
container	container (dentro de um Pod)	
—	Pod	Menor unidade agendável: 1+ containers compartilhando rede e volumes
serviço do Compose	Deployment	Gerencia réplicas de Pods e o rolling update
deploy.replicas	ReplicaSet	Gerenciado pelo Deployment
nome DNS do serviço	Service	IP e DNS estáveis para um conjunto de Pods
ports:	Service + Ingress	Ingress termina TLS e roteia por host/caminho
volume nomeado	PersistentVolumeClaim	Provisionado por uma StorageClass
environment:	ConfigMap	Configuração não sensível
secrets:	Secret	⚠ base64, não cifrado por padrão em etcd
healthcheck	livenessProbe + readinessProbe + startupProbe	Três, não uma
deploy.resources	resources.requests / .limits	requests guia o agendamento
restart:	restartPolicy (sempre, num Deployment)	
rede do projeto	Namespace + NetworkPolicy	Namespace não isola rede por si só
docker compose up	kubectl apply -f	Declarativo e reconciliado

Equivalente mínimo do nosso serviço:

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
```

```

replicas: 3
selector:
  matchLabels: { app: api }
template:
  metadata:
    labels: { app: api }
  spec:
    securityContext:
      runAsNonRoot: true
      runAsUser: 1000
      seccompProfile: { type: RuntimeDefault }
    containers:
      - name: api
        image: ghcr.io/org/api@sha256:abc...
        ports: [{ containerPort: 3000 }]
        env:
          - name: DATABASE_URL
            valueFrom:
              secretKeyRef: { name: db, key: url }
        resources:
          requests: { memory: "128Mi", cpu: "100m" }
          limits: { memory: "512Mi", cpu: "1000m" }
        livenessProbe:
          httpGet: { path: /vivo, port: 3000 }
          initialDelaySeconds: 15
        readinessProbe:
          httpGet: { path: /saude, port: 3000 }
          periodSeconds: 5
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities: { drop: ["ALL"] }
          volumeMounts: [{ name: tmp, mountPath: /tmp }]
        volumes: [{ name: tmp, emptyDir: {} }]
    ---
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  selector: { app: api }
  ports: [{ port: 80, targetPort: 3000 }]

```

Note quantos dos conceitos são **os mesmos** que você já aprendeu: usuário não-root, capabilities descartadas, raiz somente leitura, limites de recurso, sondas. O Kubernetes muda a sintaxe e o escopo, não os fundamentos.

20.4.1 requests × limits — a distinção que causa incidentes

- **requests**: o que o scheduler reserva. Se você pedir menos do que usa, o nó fica superprovisionado e tudo degrada junto.
- **limits**: o teto. Memória acima do limite = OOM kill. CPU acima = **throttling**, não morte.

Prática de campo consolidada: defina requests de memória **iguais** aos limits (memória não é compressível; ficar sem ela mata) e **não defina limite de CPU** em cargas latência-sensíveis (CPU é compressível; o limite só introduz throttling desnecessário quando há folga na máquina). Isso é debatido, mas a experiência com throttling em produção pesa a favor.

20.4.2 Distribuições leves, para aprender e para a borda

bash

```
# k3s – Kubernetes completo em um binário, ideal para homelab e edge
curl -sL https://get.k3s.io | sh -
sudo k3s kubectl get nodes

# kind – cluster dentro de containers Docker, ótimo para CI e estudo
kind create cluster --name estudo
kubectl get nodes

# minikube – VM local
minikube start
```

20.5 O caminho de migração, na ordem que funciona

1. **Containerize corretamente.** Sem estado na camada de escrita, configuração por ambiente, sinais tratados, healthchecks reais, log em stdout. **Este material inteiro é este passo.**
2. **Opere com Compose num servidor.** Descubra os problemas reais de operação com uma ferramenta simples.
3. **Automatize o build e a publicação.** CI produzindo imagem assinada, referenciada por digest.
4. **Externalize o estado.** Banco gerenciado ou com replicação própria, sessão em Redis, arquivos em armazenamento de objetos.
5. **Só então** avalie orquestração — e escolha com base em quantos dos problemas da seção 1 você realmente tem.

Pular direto do passo 1 para o 5 é o erro clássico, e é caro: você acaba depurando Kubernetes e a aplicação ao mesmo tempo, sem saber qual dos dois está errado.

Opinião profissional: uma aplicação bem containerizada roda em qualquer orquestrador com esforço pequeno. Uma aplicação mal containerizada não é salva por orquestrador nenhum — só fica mal orquestrada, em escala.

20.6 Sinais de que chegou a hora (e de que não chegou)

Chegou a hora quando:

- Uma máquina não aguenta mais a carga, e escalar verticalmente ficou caro demais.
- A indisponibilidade da máquina única passou a ser inaceitável para o negócio.
- Há várias equipes disputando a mesma infraestrutura e você precisa de isolamento e quota.
- A carga varia muito e pagar pelo pico o tempo todo saiu caro.
- Você já opera dezenas de serviços e o `compose.yaml` virou incontrolável.

Não chegou quando:

- “Todo mundo usa.” (Todo mundo tem problemas que você não tem.)
- “Fica bem no currículo.” (Legítimo como motivação pessoal; ruim como decisão de arquitetura — e vale dizer isso em voz alta na reunião.)

- Você tem 3 serviços e 100 usuários.
- Ninguém na equipe conhece Kubernetes e não há orçamento para aprender direito.

O custo escondido: um cluster Kubernetes exige alguém cuidando de atualizações, CNI, CSI, ingress, certificados, RBAC, políticas, observabilidade e do próprio cluster. Em nuvem gerenciada (EKS/GKE/AKS) a fatia de controle sai por volta de US\$ 70–100/mês por cluster, **além** dos nós — e a maior parte do custo continua sendo o tempo de gente.

20.7 Alternativas que dispensam orquestrador

Nem toda escala precisa de Kubernetes:

Opção	Modelo	Bom para
Fly.io, Railway, Render	PaaS sobre container	Aplicações pequenas e médias, sem operação
AWS App Runner / Google Cloud Run	Container serverless	Cargas HTTP com tráfego variável; escala a zero
AWS ECS + Fargate	Orquestrador gerenciado, sem Kubernetes	Quem já está na AWS e quer simplicidade
Compose + vários serviços + proxy	Manual	Cargas previsíveis, equipe pequena
Nomad	Orquestrador simples	Cargas mistas (containers, binários, VMs)

Cloud Run e ECS/Fargate merecem consideração séria antes do Kubernetes. Você entrega a mesma imagem OCI, paga por uso e não opera plano de controle nenhum. Para muitas equipes, essa é a resposta certa — e é frequentemente pulada por inércia cultural.

20.8 Autoteste

1. Cite quatro problemas que a orquestração resolve e o Compose não.
2. Qual é a “pergunta certa” antes de adotar Kubernetes, e por que ela é melhor que “devo usar Kubernetes?”
3. Por que o Swarm é tecnicamente adequado para 2–10 servidores, e por que ainda assim pode ser má escolha?
4. Traduza para Kubernetes: serviço do Compose, volume nomeado, healthcheck, rede do projeto.
5. Qual é a diferença entre requests e limits, e por que a recomendação difere entre memória e CPU?
6. Descreva os cinco passos de migração na ordem correta, e explique o custo de pular o 2 e o 4.
7. Cite três sinais de que **não** chegou a hora de adotar Kubernetes.
8. Por que uma aplicação mal containerizada não é salva por orquestrador nenhum?
9. Quando Cloud Run ou ECS/Fargate seriam a resposta certa em vez de um cluster?
10. O que o Swarm dá de graça que você teria de montar à mão com Compose?

Teoria avançada — limites, provas e resultados formais

Nível: pesquisa · Última atualização: 11/08/2026

Este arquivo trata do que **não pode** ser feito e do que **custa quanto**, com o rigor que a literatura permite. Containers são engenharia de sistemas, não um campo com teoremas próprios — mas se apoiam em resultados formais de três áreas: **teoria do escalonamento, isolamento de desempenho e segurança de sistemas**.

Onde não há resultado formal, o texto diz isso explicitamente, em vez de emprestar autoridade matemática a uma prática de engenharia.

21.1 Agendamento de containers é NP-difícil

O problema: dado um conjunto de containers, cada um com uma demanda de recursos em várias dimensões (CPU, memória, disco, rede), e um conjunto de máquinas com capacidades, aloque os containers minimizando o número de máquinas usadas.

Isso é **empacotamento vetorial** (*vector bin packing*), a generalização multidimensional do *bin packing*.

Resultado 1 — o problema é NP-difícil. Já o *bin packing* unidimensional está entre os 21 problemas NP-completos de Karp (1972), por redução a partir do *partition*. A versão vetorial o contém como caso particular.

Resultado 2 — nem aproximar é fácil. Para *bin packing* unidimensional, não existe algoritmo de aproximação com razão melhor que $3/2$ a menos que $P = NP$ (segue diretamente da NP-completude do *partition*: decidir se 2 caixas bastam já é NP-completo). Para a versão d -dimensional, os melhores limites conhecidos de inaproximabilidade crescem com a dimensão — a dificuldade **aumenta** com o número de recursos considerados simultaneamente.

Resultado 3 — heurísticas simples são boas na prática. *First Fit Decreasing* garante, no caso unidimensional, no máximo $11/9 \cdot OPT + 6/9$ caixas (Dósa, 2007, refinando o limite clássico de Johnson). É por isso que schedulers reais usam heurísticas gulosas com pontuação, não solução ótima.

Consequência prática: o scheduler do Kubernetes não busca o ótimo. Ele filtra nós viáveis (*predicates*) e pontua os que sobraram (*priorities*), escolhendo o melhor da amostra. É uma decisão consciente de trocar otimalidade por latência de decisão — em um cluster de milhares de nós, tomar a decisão em milissegundos vale mais que economizar 3% de máquinas.

E é por isso que **requests importa mais que limits**: o requests é a entrada do empacotamento. Pedir menos do que se usa quebra a premissa do algoritmo e produz nós sobrecarregados enquanto o scheduler ainda os considera livres.

21.2 Isolamento de desempenho: o resultado negativo

A pergunta formal: cgroups garantem que um container não afete o desempenho de outro?

A resposta é não, e é demonstrável. cgroups particionam recursos **contabilizáveis** (ciclos de CPU, páginas de memória, IOPS). Não particionam recursos **compartilhados e não contabilizados**:

Recurso comparti- lhado	Por que não é particionável	Efeito observável
Cache L3 da CPU	Compartilhado entre núcleos; sem contabilidade por processo	<i>Noisy neighbor</i> : um container que varre memória despeja o cache dos outros
Largura de banda de memória	Barramento único	Vazão cai para todos
TLB	Compartilhado	Mais <i>page walks</i>
Filas do controlador de disco	Reordenação no dispositivo	Latência de I/O imprevisível
Buffers do NIC, cache ARP	Compartilhados no kernel	Perda de pacote sob rajada
Locks e estruturas do kernel	Globais por definição	Contenção em <code>mmap_sem</code> , <code>inode locks</code>
Entropia (<code>/dev/random</code>)	Pool compartilhado	Bloqueio em geração de chave

Mitigações parciais existem, e são de hardware: Intel RDT/CAT (*Cache Allocation Technology*) e AMD QoS permitem particionar o cache L3 por classe de serviço. Isso é raramente usado fora de operadores de nuvem, porque exige suporte de hardware, configuração de BIOS e integração no orquestrador.

A afirmação correta, portanto, é: *cgroups fornecem isolamento de desempenho para recursos contabilizáveis, sob a hipótese de que os recursos não contabilizados não estão saturados*. Essa hipótese é rotineiramente violada em produção — e é a origem da maior parte das anomalias de latência que “não aparecem nas métricas”.

21.3 O throttling de CFS: um problema quantificável

O Linux implementa o limite de CPU via *CFS bandwidth control*: uma **cota** (`cpu.max`) a cada **período** (100 ms por padrão). Esgotada a cota, todas as threads do cgroup são suspensas até o próximo período.

A consequência formal. Seja Q a cota por período P , e n o número de threads executáveis. Se as n threads rodam em paralelo, a cota é consumida em Q/n unidades de tempo de parede. O container fica suspenso por:

$$\text{tempo_suspenso} = P - Q/n$$

Para $Q = 100$ ms, $P = 100$ ms (`--cpus 1.0`) e $n = 8$ threads: a cota se esgota em 12,5 ms e o container fica **87,5 ms parado** em cada período de 100 ms.

O resultado é contraintuitivo: um container com `--cpus 1.0` e 8 threads pode ter latência de cauda **muito pior** que o mesmo container com 1 thread, mesmo consumindo a mesma CPU total. Foi esse fenômeno que motivou uma série de correções no kernel (a partir da 5.14, com melhor distribuição das fatias entre CPUs) e a recomendação, hoje comum, de **não impor limite de CPU** em cargas latência-sensíveis.

bash

```
# Meça no seu ambiente
CG=/sys/fs/cgroup/system.slice/docker-${docker inspect -f '{{.Id}}' NOME}.scope
cat $CG/cpu.stat
# nr_periods, nr_throttled, throttled_usec
# throttled_usec / (nr_periods * 100000) = fração do tempo suspenso
```

A **mitigação estrutural** é alinhar o paralelismo interno da aplicação à cota: GOMAXPROCS (Go), -XX:ActiveProcessorCount (JVM), UV_THREADPOOL_SIZE (Node), OMP_NUM_THREADS (OpenMP). Bibliotecas como automaxprocs fazem isso lendo o cgroup.

21.4 Limites de isolamento de segurança

21.4.1 O argumento de superfície de ataque

Formalize a superfície como o conjunto de pontos de entrada acessíveis ao adversário:

Modelo	Superfície aproximada
Container (runc)	~350 syscalls do Linux, menos as ~40 bloqueadas por seccomp → ~310
gVisor	~20 syscalls do host (o Sentry implementa o resto em espaço de usuário)
Micro-VM (Firecracker)	~5 dispositivos virtio + a interface KVM
Processo comum sem container	~350 syscalls, sem restrição adicional

Isso não é um teorema: número de syscalls **não** é medida rigorosa de risco (uma syscall explorável pesa mais que cem seguras). Mas é a métrica que a literatura de segurança de sistemas usa como proxy, e a diferença de ordem de grandeza entre container e micro-VM é real e consistente com o histórico de CVEs.

21.4.2 O que a história das vulnerabilidades mostra

Escapes documentados de container concentram-se em três categorias, e a distribuição é informativa:

1. **Bugs de kernel** — waitid (CVE-2017-5123), Dirty COW (CVE-2016-5195), io_uring em diversos anos. **Container não protege:** o kernel é a fronteira, e ela é compartilhada.
2. **Bugs do runtime** — o caso canônico é **CVE-2019-5736**, no runc: um container conseguia sobrescrever o binário /proc/self/exe do próprio runc no host, obtendo execução como root na próxima invocação. A correção foi clonar o binário para memória a cada execução.
3. **Configuração errada** — --privileged, socket montado, bind mount de /. **É a categoria mais comum na prática**, e é inteiramente evitável.

A **conclusão defensável:** a categoria 3 domina a estatística de incidentes reais. Investir em higiene de configuração tem retorno maior que investir em runtime exótico — até o ponto em que o modelo de ameaça inclua adversário hostil com código arbitrário, quando (1) passa a dominar e a micro-VM se torna necessária.

21.4.3 O limite fundamental

Teorema informal do isolamento compartilhado: se dois domínios de execução compartilham um componente de confiança (o kernel), então o isolamento entre eles é no máximo tão forte quanto a corretude desse componente.

Isso não é um resultado profundo — é quase uma tautologia. Mas é a formulação que torna a decisão de arquitetura óbvia: **se você não confia na corretude do kernel para o seu modelo de ameaça, containers não são a resposta.** É exatamente por isso que AWS Lambda, Google Cloud Run e Fly.io colocam containers dentro de micro-VMs: eles executam código arbitrário de terceiros desconhecidos.

21.5 Custo teórico do copy-on-write

O OverlayFS faz CoW **em granularidade de arquivo**, não de bloco.

Seja S o tamanho do arquivo e k o número de bytes escritos. O custo da primeira escrita é:

$C_{primeira}(S, k) = O(S)$	– copia o arquivo inteiro para a camada superior
$C_{seguintes}(k) = O(k)$	– escreve direto na cópia

Consequência quantificada: escrever 1 byte no meio de um arquivo de 1 GB custa 1 GB de I/O. Para um banco de dados que atualiza páginas de 8 KB dentro de arquivos de gigabytes, o custo é proibitivo — e é a razão formal, não folclórica, para “nunca coloque banco de dados na camada de escrita”.

Sistemas com CoW em granularidade de bloco (btrfs, ZFS) não têm esse problema: o custo é $O(\text{tamanho_do_bloco})$. É por isso que os drivers btrfs e zfs do Docker existem, e por que alguns operadores os preferem apesar da complexidade adicional.

Custo de leitura em função da profundidade. Numa pilha de L camadas, resolver um caminho exige, no pior caso, L consultas de metadados até encontrar o arquivo (ou o whiteout que o esconde). O OverlayFS mitiga com cache de dentry, mas o pior caso permanece $O(L)$ — o que sustenta a recomendação empírica de manter as imagens abaixo de algumas dezenas de camadas.

21.6 Deduplicação: o ganho é combinatório

Seja um conjunto de n imagens, cada uma com m camadas, sobre um universo de U camadas distintas. O armazenamento com endereçamento por conteúdo é:

$\text{Espaço} = \sum_{c \in U} \text{tamanho}(c)$	– cada camada guardada UMA vez
--	--------------------------------

em vez de $\sum_{i=1..n} \sum_{j=1..m} \text{tamanho}(c_{ij})$.

O fator de economia é $(n \cdot m) / |U|$ — o **grau médio de compartilhamento**. Concretamente: 50 micros serviços sobre a mesma node:22-alpine armazenam essa base uma vez em vez de 50 vezes. Com 50 MB de base, são 2,45 GB economizados **por host**, e o mesmo fator na banda de rede a cada pull.

O corolário operacional é forte: padronizar a imagem base numa organização não é preferência estética; é uma decisão com efeito multiplicativo mensurável em disco,

banda e tempo de deploy. E é o argumento formal a favor das *golden base images* corporativas.

21.7 Tempo de partida: a análise

$$T_{\text{total}} = T_{\text{pull}} + T_{\text{extração}} + T_{\text{setup}} + T_{\text{processo}}$$

Termo	Ordem de grandeza	Depende de
T_{pull}	0 (cache local) a dezenas de segundos	Tamanho e banda
$T_{\text{extração}}$	100 ms - vários segundos	Tamanho, CPU (descompressão), I/O
T_{setup}	1-10 ms	Criação de namespaces, cgroups, rede
T_{processo}	ms a minutos	A aplicação (JVM é o caso patológico)

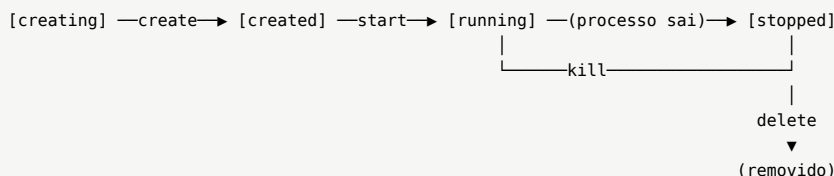
O termo que o container controla é T_{setup} , e ele é minúsculo. "Containers iniciam em milissegundos" é verdade sobre T_{setup} e falso sobre T_{total} na primeira execução.

Isso explica a direção da pesquisa:

- **Lazy pulling** (eStargz, SOCI, Nydus): iniciar o container **antes** de a imagem estar inteira no disco, buscando blocos sob demanda. Reduz $T_{\text{pull}} + T_{\text{extração}}$ de dezenas de segundos para segundos em imagens grandes — o que se tornou crítico com imagens de IA de vários gigabytes.
- **Snapshot/restore de VM** (Firecracker): restaurar um processo já inicializado, eliminando T_{processo} . É como o AWS Lambda ataca o *cold start*.
- **Pré-aquecimento**: manter um conjunto de containers ociosos prontos. Troca custo por latência.

21.8 O modelo formal da OCI Runtime Spec

A especificação define um **bundle** como o par (config.json, rootfs/) e uma máquina de estados com cinco estados e transições bem definidas:



A especificação impõe obrigações verificáveis:

- create **deve** ser idempotente quanto ao ID (dois create com o mesmo ID falham).
- O estado **deve** ser consultável por state, retornando JSON com um esquema fixo.
- Hooks (prestart, createRuntime, createContainer, startContainer, poststart, poststop) **devem** executar em pontos definidos, e falha em hook **deve** abortar a operação.

Por que isso é teoricamente relevante: é um contrato suficientemente preciso para permitir implementações independentes intercambiáveis. runc, crun, youki, runsc e

kata-runtime foram escritos por equipes diferentes, em linguagens diferentes, com modelos de isolamento radicalmente diferentes — e são substituíveis sem alterar as camadas superiores. Essa é a mesma propriedade que fez o TCP/IP e o POSIX terem sucesso: **a interface, não a implementação, é o ativo duradouro.**

21.9 Problemas em aberto

Problema	Estado em ago/2026
Isolamento de desempenho previsível sem hardware dedicado	Aberto. RDT/CAT ajuda parcialmente; contenção de barramento de memória continua sem solução
Verificação formal de perfis seccomp	Pesquisa incipiente. Não há método prático para provar que um perfil é suficiente e mínimo para um dado binário
Cold start abaixo de 1 ms com isolamento forte	Firecracker chega a ~125 ms; isolados de V8 chegam a <1 ms mas só para JS/Wasm. A lacuna persiste
Deduplicação semântica entre imagens	Camadas deduplicam por hash exato. Dois <code>apt install curl</code> em bases distintas não compartilham nada, embora o conteúdo seja quase idêntico
Ataques de canal lateral entre containers	Mitigação em software é cara; separação física continua sendo a única resposta forte
Reprodutibilidade bit a bit de builds	SOURCE_DATE_EPOCH resolve timestamps; ordenação de arquivos, aleatoriedade em compiladores e resolução dinâmica de dependências continuam abertos
Migração ao vivo de containers	CRIU funciona em casos restritos; estado de rede, descritores abertos e memória compartilhada permanecem difíceis

21.10 Leituras primárias

Trabalho	Por que ler
Karp (1972), <i>Reducibility Among Combinatorial Problems</i>	A origem da NP-completude do bin packing
Johnson (1973), tese sobre <i>bin packing</i> ; Dósa (2007)	Os limites de aproximação de First Fit Decreasing
Verma et al. (2015), <i>Large-scale cluster management at Google with Borg</i> , EuroSys	O sistema do qual o Kubernetes descende; a fonte primária sobre escalonamento em escala
Agache et al. (2020), <i>Firecracker: Lightweight Virtualization for Serverless Applications</i> , NSDI	O trade-off isolamento × cold start, quantificado
Young et al. (2019), <i>The True Cost of Containing: A gVisor Case Study</i> , HotCloud	Medição honesta da sobrecarga de gVisor
Harter et al. (2016), <i>Slacker: Fast Distribution with Lazy Docker Containers</i> , FAST	Mostrou que 76% do tempo de start é pull/extração e apenas 6,4% dos dados são lidos — a origem do lazy pulling
Menage (2007), <i>Adding Generic Process Containers to the Linux Kernel</i> , Linux Symposium	O artigo original dos cgroups
OCI Runtime Spec e Image Spec	As especificações; leitura curta e esclarecedora

Referências completas em [cap. 28](#).

21.11 Autoteste

1. Por que agendar containers é NP-difícil, e por que schedulers reais não buscam o ótimo?
2. Qual é a razão de aproximação de First Fit Decreasing, e por que isso justifica heurísticas gulosas?
3. Enuncie o resultado negativo sobre isolamento de desempenho e cite três recursos não particionáveis.
4. Deduza a fórmula do tempo suspenso por throttling e calcule para `--cpus 1.0` com 4 threads.
5. Por que "número de syscalls" é um proxy imperfeito de superfície de ataque, e por que ainda assim é usado?
6. Enuncie o teorema informal do isolamento compartilhado e derive dele uma decisão de arquitetura.
7. Qual é o custo assintótico da primeira escrita em CoW por arquivo, e como btrfs muda isso?
8. Deduza o fator de economia da deduplicação por conteúdo e aplique a 50 serviços com base comum de 50 MB.
9. Decomponha T_{total} de partida e diga qual termo o container realmente otimiza.
10. Cite três problemas em aberto e o que impede a solução de cada um.

Estado da arte — agosto de 2026

Nível: pesquisa · Data de referência: 11/08/2026 · **Este é o arquivo que envelhece mais rápido.**

O que está estabelecido, o que está em disputa e o que ainda é aposta. Onde uma afirmação vem de fonte secundária ou é minha leitura, o texto diz isso.

22.1 O panorama em uma tabela

Camada	Situação em ago/2026
Formato de imagem	OCI, indiscutido. O registry virou depósito genérico de artefatos
Runtime de baixo nível	runc domina; crun avança; youki (Rust) amadurecendo
Runtime de alto nível	containerd 2.x é o padrão de fato; CRI-O em OpenShift
Engine de desenvolvimento	Docker Engine 29; Podman 5, colima, OrbStack, Rancher Desktop disputando o desktop
Orquestração	Kubernetes venceu. A disputa migrou para o que roda em cima dele
Build	BuildKit padrão; cache remoto e multi-arch como higiene básica
Segurança	Deslocou-se do isolamento para a cadeia de suprimentos
Fronteira ativa	WebAssembly, isolamento confidencial, imagens "zero CVE", cargas de IA

22.2 O que a Docker Engine 29 mudou

Duas mudanças estruturais, ambas verificadas na documentação oficial:

1. O containerd image store passou a ser o padrão em instalações novas. O armazenamento de camadas e conteúdo deixa de usar o backend histórico do Docker e passa ao containerd. Efeitos: suporte nativo a multi-plataforma local (`--load` de várias arquiteturas), snapshotters plugáveis (o que destrava *lazy pulling*) e convergência com o que roda no Kubernetes.

Detalhe que importa na prática: a mudança vale para **instalações novas**. Atualizar da 28 para a 29 não migra automaticamente o seu store existente. Duas máquinas da mesma equipe podem ter comportamentos diferentes conforme tenham sido instaladas antes ou depois.

2. Suporte experimental a nftables como backend de firewall, ativável com `"firewall-backend": "nftables"` no `daemon.json`. As regras são funcionalmente equivalentes às de iptables, **mas quem usa a cadeia DOCKER-USER** para restringir acesso precisa reescrevê-la. O iptables está em substituição gradual em todas as distribuições Linux há anos; o Docker é dos últimos grandes projetos a migrar.

A versão corrente na data de referência é a **29.7.1**, publicada em **04/08/2026**.

22.3 WebAssembly: a aposta que amadureceu parcialmente

A promessa: módulos Wasm iniciam em microssegundos, ocupam kilobytes, rodam com isolamento por design (sem syscalls — só o que a plataforma expõe via WASI) e são portáteis entre arquiteturas sem recompilar.

Onde está em agosto de 2026:

- O **shim runwasi** integra runtimes Wasm ao containerd. Fontes secundárias reportam que a Docker Engine 29 estabilizou esse shim, permitindo executar cargas Wasm com pegada de memória muito menor que a de um container Alpine equivalente. *Não confirmei essa estabilização na documentação oficial da Docker na data desta escrita; trate como reportado, não como verificado.*
- **WASI Preview 2** e o **Component Model** destravaram a composição de módulos e o acesso a I/O de forma padronizada — o que era o principal bloqueio para cargas de servidor.
- Kubernetes executa Wasm via RuntimeClass apontando para o shim, sem alteração no restante do cluster.

O que Wasm resolve bem hoje: funções sem estado, plugins e extensões (Envoy, proxies), edge computing, execução de código não confiável de usuários, cold start crítico.

O que ainda não resolve: threads e I/O bloqueante maduros, ecossistema de bibliotecas comparável, depuração e observabilidade, cargas com estado.

Minha leitura, não consenso: Wasm **não** substituirá containers para aplicações de servidor convencionais nesta década. Ele ocupa a faixa onde container é pesado demais: funções muito curtas, plugins e execução de código de terceiros. A previsão de 2019, de que “Wasm mataria o Docker”, não se realizou e não parece caminho provável — os dois coexistem sob o mesmo runtime, o que é uma vitória do modelo OCI, não uma substituição.

22.4 Cadeia de suprimentos: de boa prática a obrigação legal

Este é o deslocamento do biênio, e ele deixou de ser técnico.

22.4.1 Sigstore graduou; a regulação chegou

- **Sigstore atingiu o status de projeto graduado da CNCF em outubro de 2025**, o que consolidou a assinatura *keyless* como padrão da indústria.
- O **EU Cyber Resilience Act (CRA)** entrou na fase de aplicação. Conforme reportado por fontes do setor: as **obrigações de notificação de vulnerabilidade passam a valer em 11/09/2026**, e a regulação **entra em vigor plenamente em 11/12/2027**. O CRA exige que fabricantes de produtos com elementos digitais **mantenham SBOM** e a disponibilizem a reguladores mediante solicitação.

A consequência para quem publica software na Europa: SBOM e gestão de vulnerabilidade deixaram de ser diferencial e passaram a ser requisito de conformidade, com prazo. Isso muda a economia de todo o ecossistema de imagens.

22.4.2 O efeito prático: "zero CVE" virou expectativa de linha de base

Surgiu uma categoria comercial de **imagens base endurecidas com CVEs próximas de zero** e SLA de correção — Docker Hardened Images, Chainguard Images, Minimus e concorrentes. O argumento de venda é direto: em vez de sua equipe caçar e corrigir CVEs em imagens base, você paga por imagens mantidas nesse estado.

Avaliação profissional: a proposta é legítima e resolve um problema real de custo de manutenção. Duas ressalvas honestas: (1) "zero CVE" refere-se a **CVEs conhecidas na data do escaneamento** em uma base minimalista — não é uma propriedade de segurança absoluta; (2) é uma nova forma de aprisionamento a fornecedor na camada mais básica da sua pilha. A alternativa gratuita continua sendo distroless + escaneamento próprio + disciplina de atualização, que custa tempo de equipe em vez de licença.

22.4.3 O que se tornou higiene mínima em 2026

```
docker buildx build --sbom=true --provenance=true -t org/app:1.0 --push .
cosign sign --yes org/app@sha256:...
trivy image --severity CRITICAL,HIGH --ignore-unfixed org/app:1.0
```

bash

E, do lado da execução, uma política de admissão que **verifica** — Kyverno, Sigstore Policy Controller ou OPA Gatekeeper ingerindo atestados SLSA e **VEX** (o formato que declara "esta CVE não é explorável no meu contexto", reduzindo o ruído dos scanners).

22.5 Isolamento: o retorno da máquina virtual

O ciclo se fechou. Depois de uma década vendendo "containers substituem VMs", a indústria convergiu para **containers dentro de VMs**:

Tecnologia	Onde está em ago/2026
Firecracker	Base do AWS Lambda e Fargate; ~125 ms de boot; usado também por Fly.io
Kata Containers	Maduro; RuntimeClass no Kubernetes; adoção em nuvens públicas e ambientes regulados
gVisor	Base do Google Cloud Run e do GKE Sandbox; sobrecarga de 10-30% em I/O
Confidential Containers	Projeto CNCF; usa AMD SEV-SNP, Intel TDX e ARM CCA para cifrar memória em uso

Confidential computing é a fronteira relevante: a memória do container permanece cifrada mesmo para o hipervisor e o operador da nuvem, com atestação remota do estado. Casos de uso — processar dado de saúde ou financeiro em nuvem pública sem confiar no provedor; inferência de modelo proprietário em infraestrutura de terceiro. Hoje ainda é caro, imaturo em ferramental e dependente de hardware específico. *Aposta pessoal:* será commodity até o fim da década, movida pela regulação de dados mais do que pela demanda técnica.

22.6 IA mudou os requisitos de container

Cargas de IA quebraram três premissas do modelo tradicional:

Premissa antiga	Realidade com IA
Imagens de dezenas a centenas de MB	Dezenas de GB (CUDA, cuDNN, PyTorch, pesos)
Cold start de segundos é aceitável	Carregar um modelo pode levar minutos
Container não precisa de acesso a hardware especial	GPU é obrigatória e precisa ser particionada
Estado fica em banco	Pesos de modelo são artefatos gigantes e imutáveis

As respostas em desenvolvimento:

- **Lazy pulling** (eStargz, SOCI, Nydus) — iniciar antes de a imagem inteira estar baixada. O *containerd image store* como padrão na Engine 29 é pré-requisito para isso ficar acessível no Docker.
- **Modelos como artefatos OCI** — distribuir pesos pelo registry, com as mesmas garantias de digest, assinatura e cache das imagens.
- **Particionamento de GPU** — NVIDIA MIG e time-slicing expostos como recurso agendável.
- **Docker Model Runner** — a Docker Inc. passou a oferecer execução local de LLMs com APIs compatíveis com OpenAI e Ollama, empacotadas no fluxo de container.
- **MCP Catalog e MCP Toolkit** — catálogo curado de servidores MCP com um gateway que faz proxy, log e controle das conexões usadas por agentes de IA.

Leitura da estratégia: a Docker Inc. está se posicionando como **camada de governança e isolamento para agentes de IA** — sandbox, catálogo curado, imagens endurecidas. Faz sentido: executar código gerado por agente é exatamente o problema de “rodar código não confiável” que containers atacam, e é um mercado novo onde a marca tem tração. Se isso se sustenta comercialmente, ainda não dá para dizer.

22.7 eBPF: a camada que está reescrevendo a rede e a observabilidade

eBPF permite executar programas verificados no kernel sem módulos. Aplicado a containers:

Ferramenta	O que faz
Cilium	CNI que substitui kube-proxy e iptables; política de rede em L3-L7
Tetragon	Observação e aplicação de política em nível de syscall, com contexto de processo
Falco	Deteção de comportamento anômalo em execução
Pixie / Parca / Coroot	Observabilidade sem instrumentar a aplicação

Por que importa: o iptables não escala para milhares de serviços (as regras são avaliadas linearmente); eBPF usa mapas de hash. Além disso, correlacionar evento de kernel com o container e o processo de origem é exatamente o que faltava para a segurança em execução ser acionável.

A migração do Docker para nftables (seção 2) é parte do mesmo movimento maior: **o plano de dados de rede está saindo do iptables.**

22.8 Debates abertos, com os dois lados

22.8.1 "Docker Desktop vale a assinatura?"

A favor: integração superior em macOS e Windows, atualização automática, Scout, funcionalidades de segurança gerenciada, suporte. Para uma empresa, um desenvolvedor perdendo duas horas com colima já custa mais que um ano de licença.

Contra: o Engine é livre; as alternativas (Podman Desktop, Rancher Desktop, colima) cobrem a maior parte dos casos; e é aprisionamento numa camada que deveria ser commodity.

Minha posição: em Linux, não há motivo para Docker Desktop. Em macOS e Windows, pague se a empresa pode — o custo de suporte interno das alternativas costuma exceder a licença. Em organização com restrição orçamentária ou de conformidade, Podman Desktop é uma escolha madura e sem ressalva técnica relevante.

22.8.2 "Kubernetes é complexo demais para a maioria?"

A favor da crítica: a maioria das equipes que o adota tem menos de dez serviços e um pico de tráfego previsível. A complexidade operacional é real e recorrente.

Contra: a alternativa costuma ser um conjunto de scripts caseiros que reimplementa mal um subconjunto do Kubernetes, sem documentação nem comunidade. E o mercado de trabalho o exige.

Minha posição: a crítica é válida para **um** servidor e questionável a partir de cinco. O erro mais comum não é escolher Kubernetes; é escolhê-lo antes de resolver containerização, CI e externalização de estado. Ver [cap. 20](#).

22.8.3 "Imagens 'zero CVE' pagas resolvem o problema?"

Tratado na seção 4. Em resumo: resolvem um custo real, criam uma dependência nova, e o "zero" é relativo à data e ao escopo do escaneamento.

22.9 Previsões para 2027-2028 — e o que as invalidaria

Explicitamente **especulação**, com o critério de falsificação junto:

Previsão	Confiança	O que a invalidaria
SBOM obrigatório em produção regulada na UE	alta	Adiamento da aplicação do CRA
Lazy pulling padrão para imagens grandes	alta	Custo de armazenamento cair a ponto de tornar o pull irrelevante
Wasm consolidado em nicho, sem substituir containers	média-alta	WASI resolver threads e I/O e o ecossistema de bibliotecas fechar a lacuna
Confidential containers como commodity em nuvem	média	Custo de hardware ou falha de confiança nas TEEs (já houve ataques a SGX)
Docker Desktop com mais recursos exclusivos de assinatura	alta	Pressão competitiva de OrbStack e Podman
Assinatura verificada na admissão como padrão	média	Atrito operacional maior que o benefício percebido
Consolidação dos fornecedores de imagem endurecida	média	Uma alternativa aberta boa o bastante surgir

22.10 O que não mudou, e provavelmente não vai mudar

Vale terminar por aqui, porque é o que sustenta o resto do material:

- **O formato OCI.** Uma imagem de 2016 roda hoje. Aposte nele.
- **Namespaces e cgroups.** A base do isolamento no Linux não vai ser substituída.
- **Multi-stage e imagem enxuta.** Continua sendo a maior alavanca de tamanho e segurança.
- **“Container é processo.”** O modelo mental permanece o mesmo.
- **Estado vive em volume ou fora do container.** Verdade estrutural, não moda.
- **Tratar sinais, healthcheck honesto, log em stdout, configuração por ambiente.** Continuam separando quem opera de quem sofre.

Ferramenta muda; fundamento não. É por isso que os arquivos 10 a 21 deste material envelhecem em década, e este aqui envelhece em meses.

22.11 Autoteste

1. Quais foram as duas mudanças estruturais da Engine 29, e por que a primeira só vale para instalações novas?
2. Onde Wasm ganha de container hoje, e onde ainda não ganha?
3. Que obrigação legal passa a valer em 11/09/2026 e o que ela exige na prática?
4. Por que “zero CVE” é uma afirmação relativa? Cite duas ressalvas.
5. Por que a indústria voltou a colocar containers dentro de VMs? Cite três produtos que fazem isso.
6. Quais três premissas do modelo de container as cargas de IA quebraram, e qual é a resposta a cada uma?
7. Como eBPF muda a rede de containers, e o que isso tem a ver com a migração para nftables?
8. Apresente os dois lados do debate sobre a assinatura do Docker Desktop e defenda uma posição.
9. Escolha uma previsão da seção 9 e descreva um evento concreto que a invalidaria.
10. Cite três fundamentos que não mudaram e explique por que são estáveis.

22.11.1 Fontes consultadas (11/08/2026)

- [Docker Engine v29 Release](#) — Docker Blog · [Docker Engine 29 release notes](#) · [Linuxiac](#) — [Containerd Becomes Default, Experimental nftables Support](#)
- [endoflife.date](#) — Docker Engine — 29.7.1, 04/08/2026
- [Container Runtime Alternatives 2026 Deep Dive](#) — [youngju.dev](#) — panorama de runtimes; **fonte secundária** para a estabilização do shim runwasi na Engine 29
- [Docker Docs](#) — [Model Runner](#) · [Bret Fisher](#) — [Docker AI, Model Runner e MCP Toolkit](#) · [Collabnix](#) — [What’s New in Docker in 2026](#)
- [Docker](#) — [EU Cyber Resilience Act \(CRA\): Overview](#) · [RapidFort](#) — [EU CRA for Containers & Kubernetes](#) — prazos de 11/09/2026 e 11/12/2027

- [Minimus — Container Image Governance Guide 2026](#) — Sigstore graduado na CNCF em out/2025; VEX e políticas de admissão
- [Help Net Security — eBPF com Cilium, Tetragon e SBOMs](#)

PARTE III

Prática e erros

Laboratórios progressivos e o catálogo de armadilhas

CAPÍTULO 23

Prática — laboratórios progressivos

Nível: todos · Última atualização: 11/08/2026

Aprender Docker é como aprender a nadar: não dá para ler o caminho todo. Cada laboratório tem **objetivo**, **passos**, **critério de aprovação** e o **conceito que ele consolida**.

Ambiente: qualquer um do [cap. 3](#), inclusive [Play with Docker](#). Onde um laboratório exige Linux, está marcado.

23.1 Lab 1 — Sobrevivência (30 min)

Objetivo: rodar, inspecionar, entrar e limpar. O ciclo básico.

```
docker run -d --name web -p 8080:80 nginx:alpine
curl -sI localhost:8080 | head -1
docker ps
docker logs web
docker exec -it web sh -c 'ls /usr/share/nginx/html && hostname'
docker stop web && docker rm web
docker ps -a | grep web || echo "removido"
```

bash

Critério de aprovação:

- ☐ curl devolveu HTTP/1.1 200 OK.
- ☐ Você explicou, em `-p 8080:80`, qual porta é do host.
- ☐ O hostname de dentro do container era diferente do da sua máquina.
- ☐ Nenhum container web sobrou.

Consolida: container é processo; `-p` é `host:container`; `exec` entra no que já roda.

23.2 Lab 2 — A primeira imagem sua (45 min)

Objetivo: escrever um Dockerfile e construir.

Crie `app.py`:

```
import http.server, os, socketserver
PORTA = int(os.environ.get("PORT", "8000"))
class H(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200); self.end_headers()
        self.wfile.write(f"01a de {socketserver.socket.gethostname()}\n".encode())
    def log_message(self, *a): pass
with socketserver.TCPServer(("0.0.0.0", PORTA), H) as s:
    print(f"ouvindo na {PORTA}", flush=True); s.serve_forever()
```

Python

Dockerfile:

Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.13-slim
WORKDIR /app
COPY app.py .
RUN useradd -m app
USER app
EXPOSE 8000
CMD ["python", "app.py"]
```

bash

```
docker build -t meu-py:1.0 .
docker run --rm -d --name py -p 8000:8000 meu-py:1.0
curl localhost:8000
docker rm -f py
```

Critério:

- ☐ A imagem foi construída sem erro.
- ☐ curl devolveu "Ola de ...".
- ☐ docker image inspect meu-py:1.0 --format '{{.Config.User}}' mostra app, não vazio.
- ☐ Você entende por que 0.0.0.0 e não 127.0.0.1.

Consolida: Dockerfile, contexto de build, usuário não-root, bind em 0.0.0.0.

23.3 Lab 3 — Persistência (45 min)

Objetivo: provar a diferença entre camada de escrita e volume.

bash

```
# Sem volume: o dado morre
docker run --rm --name t alpine sh -c 'echo importante > /d.txt; cat /d.txt'
docker run --rm alpine cat /d.txt 2>&1 || echo "morreu, como esperado"

# Com volume: sobrevive
docker volume create meus
docker run --rm -v meus:/dados alpine sh -c 'echo importante > /dados/d.txt'
docker run --rm -v meus:/dados alpine cat /dados/d.txt

# Postgres com e sem volume
docker run -d --name pg -e POSTGRES_PASSWORD=x -v pgdata:/var/lib/postgresql/data postgres:16-alpine
sleep 8
docker exec pg psql -U postgres -c "CREATE TABLE t(id int); INSERT INTO t VALUES (42);"
docker rm -f pg
docker run -d --name pg -e POSTGRES_PASSWORD=x -v pgdata:/var/lib/postgresql/data postgres:16-alpine
sleep 8
docker exec pg psql -U postgres -c "SELECT * FROM t;" # o 42 continua lá
docker rm -f pg && docker volume rm pgdata meus
```

Critério:

- ☐ O arquivo sem volume não sobreviveu à recriação.
- ☐ O 42 sobreviveu à destruição e recriação do container Postgres.
- ☐ Você sabe dizer onde o volume fica no host (docker volume inspect).

Consolida: efemeridade da camada de escrita; volume nomeado para estado.

23.4 Lab 4 — Multi-stage e o tamanho da imagem (1 h)

Objetivo: reduzir uma imagem em 80% com multi-stage.

Construa **primeiro** a versão ingênua e meça; depois a multi-stage e compare.

Ingênua (Dockerfile.ruim):

```
FROM node:22
WORKDIR /app
COPY . .
RUN npm ci && npm run build
CMD ["node", "dist/main.js"]
```

Dockerfile

Multi-stage (Dockerfile): use o modelo do [06-exemplos.md exemplo 4](#) (cap. 6).

```
docker build -f Dockerfile.ruim -t app:ruim .
docker build -t app:bom .
docker images | grep app
docker history app:bom
```

bash

Critério:

- ☐ app:bom é pelo menos 60% menor que app:ruim.
- ☐ Você identifica, no `docker history`, a maior camada de cada uma.
- ☐ Alterar uma linha de código e reconstruir app:bom **não** reexecuta o `npm ci`.

Consolida: multi-stage, cache de camadas, ordem das instruções.

23.5 Lab 5 — Compose com dependências (1 h)

Objetivo: subir app + banco + cache com ordem correta.

Use o `compose.yaml` do [04-como-comecar.md, Passo 5](#) (cap. 4) e evolua para incluir `healthcheck` com `condition: service_healthy`.

```
docker compose up -d
docker compose ps
docker compose exec app sh -c 'nslookup cache'
docker compose logs -f app
docker compose down
```

bash

Critério:

- ☐ O app só subiu depois de o cache ficar healthy.
- ☐ O app alcança o cache pelo **nome**, não por IP.
- ☐ `docker compose config` mostra o YAML resolvido sem erro.

Consolida: Compose, DNS interno, `depends_on` com condição.

23.6 Lab 6 — Depuração de rede (Linux, 1 h)

Objetivo: diagnosticar três falhas de conectividade plantadas.

bash

```
# Falha A: app em 127.0.0.1
docker run -d --name fa -p 9001:9001 python:3.13-slim \
  python -c "import http.server,socketserver;
  ↪ socketserver.TCPServer(('127.0.0.1',9001),http.server.BaseHTTPRequestHandler).serve_forever()"
curl localhost:9001 # falha – descubra por quê e conserte usando 0.0.0.0

# Falha B: bridge padrão sem DNS
docker run -d --name fb1 nginx:alpine
docker run --rm alpine ping -c1 fb1 # falha – conserte criando uma rede

# Falha C: porta já em uso
docker run -d --name fc1 -p 9003:80 nginx:alpine
docker run -d --name fc2 -p 9003:80 nginx:alpine # falha – leia o erro e resolva

docker rm -f fa fb1 fc1 fc2 2>/dev/null
```

Critério:

- ☐ Você explicou e corrigiu cada uma das três falhas.
- ☐ Usou `docker logs`, `docker port` e `ss -tlnp` no diagnóstico.
- ☐ Sabe entrar no namespace de rede com `--network container:NOME`.

Consolida: a árvore de diagnóstico de [cap. 14](#).

23.7 Lab 7 — Segurança: endurecer um container (1 h)

Objetivo: transformar um container permissivo em um endurecido, sem quebrá-lo.

bash

```
# Antes: permissivo
docker run -d --name antes nginx:alpine
docker exec antes id # root
docker exec antes sh -c 'echo x > /etc/y' # escreve na raiz

# Depois: endurecido
docker run -d --name depois \
  --user 101:101 \
  --cap-drop=ALL --cap-add=NET_BIND_SERVICE \
  --security-opt no-new-privileges:true \
  --read-only \
  --tmpfs /var/cache/nginx --tmpfs /var/run \
  --memory 128m --pids-limit 100 \
  nginx:alpine
sleep 2
docker exec depois sh -c 'echo x > /etc/y' 2>&1 || echo "escrita bloqueada – correto"
curl -sI localhost 2>/dev/null | head -1 # o nginx ainda funciona?
docker rm -f antes depois
```

Critério:

- ☐ O container endurecido **não** consegue escrever na raiz.
- ☐ Ele **não** roda como root.
- ☐ E ainda assim serve HTTP.
- ☐ Você aplicou o mesmo endurecimento no `compose.yaml` do projeto-modelo.

Consolida: o checklist de [cap. 18](#).

23.8 Lab 8 — Segredo que não vaza (Linux, 45 min)

Objetivo: provar o vazamento por ARG e corrigir com `--mount=type=secret`.

bash

```
# Vazamento
echo "token-secreto-123" > segredo.txt
cat > Dockerfile.vaza <<'EOF'
# syntax=docker/dockerfile:1
FROM alpine
ARG TOKEN
RUN echo "$TOKEN" > /tmp/t && cat /tmp/t && rm /tmp/t
EOF
docker build -f Dockerfile.vaza --build-arg TOKEN=$(cat segredo.txt) -t vaza .
docker history --no-trunc vaza | grep -c token    # > 0: vazou

# Correção
cat > Dockerfile.ok <<'EOF'
# syntax=docker/dockerfile:1
FROM alpine
RUN --mount=type=secret,id=tok cat /run/secrets/tok > /dev/null && echo "usei o segredo"
EOF
docker build -f Dockerfile.ok --secret id=tok,src=segredo.txt -t ok .
docker history --no-trunc ok | grep -c token      # 0: não vazou

rm segredo.txt Dockerfile.vaza Dockerfile.ok
```

Critério:

- ☐ Você viu o token no histórico da imagem vaza.
- ☐ Você confirmou que ele **não** aparece na imagem ok.

Consolida: [12-imagens-e-camadas.md](#), seção 6 (cap. 10).

23.9 Lab 9 — Limites e OOM (Linux, 45 min)

Objetivo: ver o OOM killer agir e ler o exit code.

bash

```
docker run --rm --memory 64m --memory-swap 64m python:3.13-slim \
  python -c "a=[]
while True: a.append(bytearray(5*1024*1024))" ; echo "exit: $?"
# esperado: Killed, exit 137

# Confirme que foi OOM
docker run -d --name oom --memory 64m --memory-swap 64m python:3.13-slim \
  python -c "a=[]
while True: a.append(bytearray(5*1024*1024))"
sleep 3
docker inspect oom --format '{{.State.OOMKilled}} {{.State.ExitCode}}'
docker rm -f oom

# Throttling de CPU
docker run -d --name cpu --cpus 0.2 alpine sh -c 'while ;; do ;; done'
sleep 5
CG=/sys/fs/cgroup/system.slice/docker-${(docker inspect -f '{{.Id}}' cpu).scope}
cat $CG/cpu.stat | grep throttled
docker rm -f cpu
```

Critério:

- ☐ Exit code 137 e `OOMKilled=true`.
- ☐ `nr_throttled` maior que zero no container limitado a 0.2 CPU.

Consolida: cap. 11.

23.10 Lab 10 — Do zero à produção (projeto integrador, 3 h)

Objetivo: pegar uma aplicação sua (ou o [projeto-modelo](#) (cap. 7)) e levá-la a um estado publicável.

Etapas, cada uma com seu critério:

1. **Containerizar** com multi-stage, não-root, healthcheck, sinais tratados.
 - ☐ `docker build` produz imagem < 200 MB, roda como não-root, para em < 2 s.
2. **Compose** com banco, rede segmentada, limites e segredos.
 - ☐ Sobe com um comando; o banco não tem `ports::; internal: true` na rede interna.
3. **CI** que constrói, testa, escaneia e publica (use o [exemplo 12](#) (cap. 6)).
 - ☐ O pipeline falha se um teste quebrar ou se houver CVE crítica corrigível.
4. **Publicar** no GHCR, com tag semântica e digest.
 - ☐ A imagem está no registry, referenciada por digest no deploy.
5. **Observar:** healthcheck, log estruturado, limite de log.
 - ☐ `docker inspect` mostra healthy; os logs são JSON de uma linha.
6. **Backup** de volume testado.
 - ☐ Você apagou o volume, restaurou do backup e os dados voltaram.

Critério de aprovação final:

- ☐ Outra pessoa consegue subir tudo seguindo só o seu `README.md`.
- ☐ Você consegue explicar cada decisão do Dockerfile e do Compose.
- ☐ Nenhum segredo no Git, na imagem ou em variável de ambiente inspecionável.

Consolida: tudo. Este laboratório é o exame prático do assunto.

23.11 Autoavaliação final

Se você consegue fazer o Lab 10 sem consultar, você **sabe Docker** no nível de operar em produção num servidor. O que falta a partir daí é escala (orquestração, [cap. 20](#)) e profundidade interna ([13](#) (cap. 11), [14](#) (cap. 12), [60](#) (cap. 21)).

Erros que você deveria conseguir diagnosticar em menos de um minuto, ao final:

1. Container sai imediatamente com `Exited (0)`.
2. `connection refused` num container aparentemente saudável.
3. Exit code 137.
4. "A imagem está com 1,5 GB."

5. `ping outro-container` não resolve o nome.
6. Build refaz o `npm ci` a cada mudança de código.
7. Arquivos criados pelo container pertencem ao root no host.
8. O disco encheu de `/var/lib/docker`.

Se algum desses ainda te trava, volte ao arquivo correspondente. As respostas estão todas em [cap. 24](#).

Armadilhas, mitos e más práticas

Nível: todos · Última atualização: 11/08/2026

O catálogo dos erros que todo mundo comete, por que persistem e como sair de cada um. Organizado por frequência com que aparecem na vida real.

24.1 Parte 1 — Os oito erros que travam o iniciante

24.1.1 -p invertido → "connection refused"

```
docker run -p 80:8080 nginx # [x] nginx escuta na 80
docker run -p 8080:80 nginx # [ok]
```

bash

-p é **host:container**, de fora para dentro. Diagnóstico: docker port NOME mostra o mapeamento real; docker logs mostra em que porta o app subiu.

24.1.2 App em 127.0.0.1 dentro do container → "connection refused" com container saudável

Dentro do container, 127.0.0.1 é o loopback **dele**, não a sua máquina. O app precisa escutar em 0.0.0.0.

Framework	Correção
Flask	app.run(host="0.0.0.0")
Express/Node	app.listen(3000, "0.0.0.0")
Vite	--host (ou server.host: true)
Rails	-b 0.0.0.0
.NET	ASPNETCORE_URLS=http://0.0.0.0:8080
Django	runserver 0.0.0.0:8000

Diagnóstico: docker exec NOME ss -tlnp — se aparece 127.0.0.1:porta, achou.

24.1.3 Container sai na hora com Exited (0)

```
docker run -d ubuntu # STATUS: Exited (0)
```

bash

O container vive enquanto o PID 1 viver. ubuntu tem CMD ["bash"]; sem terminal, o bash lê EOF e termina. Dê a ele um processo que fique de pé, ou -it para um terminal. Diagnóstico: docker logs NOME; docker inspect --format '{{.State.ExitCode}}' NOME.

24.1.4 Editar arquivo dentro do container e perder tudo

bash

```
docker exec -it app vi /app/config.json # some no docker rm
```

Container não é servidor. Edite no host e reconstrua, ou use bind mount no desenvolvimento.

24.1.5 docker compose down -v apagou o banco

O -v remove volumes. Sem confirmação. Use `docker compose down` no dia a dia; reserve -v para quando você **quer** zerar. E faça backup antes.

24.1.6 Build refaz tudo a cada mudança de código

Ordem errada no Dockerfile. `COPY package*.json` **antes** de `COPY . .` e do `npm ci`. Ver [17-dockerfile-e-build.md](#) (cap. 15).

24.1.7 Arquivos do container pertencem ao root no host

O processo rodou como root e escreveu num bind mount. Rode com `--user "$(id -u):$(id -g)"`, ou crie um usuário com o UID certo na imagem. Ver [15-armazenamento-e-volumes.md](#) (cap. 13).

24.1.8 docker stop demora exatos 10 segundos

Sinal de que o SIGTERM está sendo ignorado. Duas causas: forma shell no CMD (CMD `node app.js` em vez de CMD `["node", "app.js"]`), ou o app não trata SIGTERM sendo PID 1. Use a forma `exec` e `tini/--init`.

24.2 Parte 2 — Os mitos, e a realidade

Mito	Realidade
"Container é uma VM leve"	Não há virtualização nem kernel próprio. É processo restrito. Ver 10 (cap. 8)
"Container é seguro por padrão"	O padrão é razoável, não é isolamento forte. Kernel compartilhado
"Preciso de microserviços para usar Docker"	Container empacota; microserviço é decisão organizacional. Independentes
"latest é a versão mais recente"	É só a tag padrão, e é reescrita a cada push. Perigosa em produção
"Alpine é sempre melhor por ser menor"	Para Python quebra wheels e força compilar; use <code>slim</code> . Ver 12 (cap. 10)
"Rodei <code>rm</code> do segredo, então sumiu"	A camada anterior o mantém. Recuperável do registry
"cgroups garantem isolamento de desempenho"	Só de recursos contabilizáveis. Cache L3 e barramento vazam. Ver 60 (cap. 21)
"Docker vai morrer por causa do Wasm/Kubernetes"	Wasm ocupa nicho; K8s usa containers OCI por baixo. Coexistem
"Container não tem estado"	Pode ter — em volume. Não deve ter na camada de escrita
"Se passa no scanner, está seguro"	Scanner vê CVE de versão, não configuração errada nem lógica

Mito	Realidade
"EXPOSE publica a porta"	É documentação. Quem publica é -p
"Preciso de Kubernetes para produção"	Para um servidor, Compose é adequado. Ver 25 (cap. 20)

24.3 Parte 3 — Más práticas que persistem (e por quê)

24.3.1 Rodar como root

Por que persiste: funciona, e as imagens oficiais frequentemente vêm assim por compatibilidade. **Por que é ruim:** amplia o impacto de qualquer escape; cria arquivos com dono errado. **Correção:** USER não-root, sempre.

24.3.2 :latest em produção

Por que persiste: é o padrão, e "funciona na minha máquina". **Por que é ruim:** deploy imprevisível, rollback impossível, ataque de reescrita de tag. **Correção:** tag semântica + digest.

24.3.3 Um RUN por linha

```
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*
```

Dockerfile

Por que persiste: parece mais legível. **Por que é ruim:** camadas extras, cache do update desatualizado (erro 404), o rm não reduz nada. **Correção:** encadeie com && na mesma camada.

24.3.4 Instalar mais do que precisa

Por que persiste: conveniência ("vou precisar de vim para depurar"). **Por que é ruim:** cada pacote é superfície de ataque e CVE. **Correção:** base mínima; depure com nsenter ou container efêmero anexado, não com ferramentas embutidas.

24.3.5 --privileged como muleta

Por que persiste: resolve "permission denied" na hora. **Por que é ruim:** é root no host sem barreira. **Correção:** --cap-add específico, --device específico. Ver [cap. 18](#).

24.3.6 Montar o socket do Docker

Por que persiste: Portainer, Watchtower e agentes de CI pedem. **Por que é ruim:** root no host. **Correção:** proxy de socket com allow-list, ou API por TLS/SSH.

24.3.7 Segredo em variável de ambiente

Por que persiste: é o jeito mais fácil e todo tutorial faz. **Por que é ruim:** aparece em inspect, /proc/PID/envIRON, logs de crash, e é herdado por filhos. **Correção:** arquivo em tmpfs (secrets:) ou gerenciador de segredos.

24.3.8 Não limitar recurso

Por que persiste: funciona até o dia em que não funciona. **Por que é ruim:** um vazamento derruba o host inteiro. **Correção:** `--memory` e `--pids-limit` em todo container de produção.

24.3.9 Não limitar log

Por que persiste: o padrão não avisa. **Por que é ruim:** enche o disco em semanas, e o sintoma parece não ter relação. **Correção:** `max-size/max-file` no `daemon.json`.

24.3.10 Um processo de banco em bind mount

Por que persiste: parece prático poder ver os arquivos. **Por que é ruim:** `copy-on-write` e semântica de `fsync` degradam ou corrompem. **Correção:** volume nomeado; backup por `pg_dump`.

24.4 Parte 4 — Armadilhas de plataforma

24.4.1 macOS / Windows

Armadilha	Correção
I/O de disco lento em bind mount	VirtioFS (mac); no WSL, projeto em ~, nunca em <code>/mnt/c</code>
RAM "sumindo"	A VM do Docker reserva RAM fixa; ajuste nas Settings
<code>host.docker.internal</code> não existe no Linux	<code>--add-host=host.docker.internal:host-gateway</code>
Imagem ARM64 não roda / roda emulada e lenta	<code>--platform linux/amd64</code> , ou build multi-arch
<code>--network host</code> não funciona como esperado	"host" é a VM, não a sua máquina

24.4.2 Linux

Armadilha	Correção
permission denied no socket	Entrar no grupo docker e relogar
Bind mount bloqueado no Fedora/RHEL	SELinux: use <code>:z/:Z</code>
Sub-rede do Docker colide com a VPN	Mudar <code>bip/default-address-pools</code> no <code>daemon.json</code>
ufw não protege porta publicada	<code>-p 127.0.0.1:...</code> , ou cadeia <code>DOCKER-USER</code>
CRLF quebra script no container	<code>dos2unix</code> ; <code>.gitattributes</code> com <code>eol=lf</code>

24.5 Parte 5 — Referência rápida de diagnóstico

bash

```
# Por que o container morreu?
docker inspect --format '{{.State.ExitCode}} {{.State.OOMKilled}} {{.State.Error}}' NOME
docker logs --tail 100 NOME

# Códigos de saída
# 0=terminou (não era serviço) · 1=erro do app · 125=erro do Docker
# 126=não executável · 127=comando não encontrado
# 137=SIGKILL (quase sempre OOM) · 139=SIGSEGV · 143=SIGTERM (parada normal)

# A imagem está gorda?
```



```

docker history IMAGEM
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock wagoodman/dive IMAGEM

# O disco encheu?
docker system df
sudo du -sh /var/lib/docker/* | sort -hr | head

# A rede não conecta?
docker port NOME
docker exec NOME ss -tlnp
docker run --rm --network container:NOME nicolaka/netshoot

# O que este container pode fazer (segurança)?
docker inspect --format 'priv={{.HostConfig.Privileged}} user={{.Config.User}}' NOME
    
```

24.6 Parte 6 — O anti-checklist

Se você faz **qualquer** um destes, pare e corrija:

- ☐ :latest em produção
- ☐ Container rodando como root sem necessidade
- ☐ --privileged fora de um caso justificado e documentado
- ☐ Socket do Docker montado sem proxy
- ☐ Segredo em ARG, ENV ou -e
- ☐ Banco de dados em bind mount
- ☐ Nenhum limite de memória em produção
- ☐ Nenhum limite de tamanho de log
- ☐ Deploy por tag, não por digest
- ☐ .dockerignore ausente
- ☐ Imagem nunca escaneada
- ☐ Backup nunca testado com restauração real
- ☐ docker stop demorando 10 s (sinais não tratados)
- ☐ Porta de banco publicada em 0.0.0.0

24.7 Autoteste

1. Um container está de pé e mesmo assim dá "connection refused". Cite as duas causas mais prováveis e como distinguir.
2. Por que "container é uma VM leve" é falso, e qual é a consequência prática do erro?
3. Explique por que apagar um segredo num RUN posterior não o remove da imagem.
4. Por que Alpine é boa escolha para Go e má escolha para Python?
5. Cite três más práticas que persistem e a razão de cada uma persistir.
6. Por que ufw deny não protege uma porta publicada, e quais são as duas correções?
7. Exit code 137: o que é e qual comando confirma a causa?
8. Por que segredo em variável de ambiente é ruim? Cite três formas de vazamento.
9. Um docker stop demora 10 segundos. Quais são as duas causas e as correções?
10. Passe pelo anti-checklist e identifique quais itens se aplicam a algo que você já fez.

PARTE IV

Economia e ecossistema

Quanto custa de verdade e onde estudar mais

CAPÍTULO 25

Custos e licenças

Nível: todos · **Preços consultados em 11/08/2026** · Câmbio usado: US\$ 1 ≈ R\$ 5,40 (ordem de grandeza)

Aviso de validade: preço sem data é desinformação. Todos os valores abaixo têm a data de consulta explícita. Preços de nuvem e de assinatura mudam com frequência — **confirme na fonte** antes de decidir. Onde não confirmei um número exato, o texto diz isso.

25.1 A resposta curta

O Docker Engine é gratuito e software livre (Apache 2.0). Rodar containers não custa licença.

O que custa dinheiro é:

- o **Docker Desktop** em empresas grandes (o app, não o Engine);
- **pulls** acima do limite no Docker Hub;
- **armazenamento e tráfego** em registries de nuvem;
- **imagens base endurecidas** comerciais (opcional);
- e, sempre, **o tempo das pessoas** — que é o maior custo real, e o que ninguém coloca na planilha.

25.2 Docker Engine — o que é grátis

Componente	Licença	Custo
Docker Engine (dockerd, CLI)	Apache 2.0	Grátis, para qualquer uso, inclusive comercial
BuildKit, buildx	Apache 2.0	Grátis
Docker Compose v2	Apache 2.0	Grátis
containerd, runc	Apache 2.0 (CNCF)	Grátis

Quem paga a conta do que é grátis? A Docker Inc. (via assinaturas do Desktop e do Hub), os provedores de nuvem e a CNCF (mantenedora do containerd e do runc, financiada por membros corporativos). O Engine é o “produto de entrada” que sustenta a venda do resto.

25.3 Docker Desktop — onde entra a cobrança

Docker Desktop é o **aplicativo gráfico** para macOS, Windows e Linux. É o que muda de preço, não o Engine.

25.3.1 Planos (consultados em 11/08/2026)

Plano	Preço (anual)	Para quem
Personal	US\$ 0	Uso pessoal, educação, open source não comercial, e empresas pequenas
Pro	~US\$ 9/usuário/mês	Desenvolvedor individual profissional
Team	~US\$ 15/usuário/mês	Times, com gestão centralizada
Business	~US\$ 24/usuário/mês	Empresa: SSO, SCIM, endurecimento, política

25.3.2 A regra que define se você paga

Docker Desktop é gratuito para:

- uso pessoal;
- educação;
- projetos open source não comerciais;
- **empresas com menos de 250 funcionários E menos de US\$ 10 milhões de receita anual.**

É **pago** (assinatura Pro/Team/Business por usuário) para empresas que ultrapassem **qualquer** dos dois limites acima.

Note o "E": os dois critérios precisam ser satisfeitos para o uso gratuito. Uma startup de 30 pessoas com US\$ 15 milhões de receita **precisa pagar**; uma ONG de 300 pessoas com US\$ 2 milhões de receita **também** — o limite de funcionários basta para exigir licença.

25.3.3 Custos ocultos do Desktop

- **Consumo de recursos:** a VM reserva RAM (2–8 GB) e CPU mesmo ociosa.
- **Auditoria de conformidade:** provar quem usa Desktop numa empresa grande dá trabalho — é um custo administrativo real.
- **Migração** se decidir trocar: retreinar a equipe nas alternativas.

25.3.4 Como evitar o custo, legalmente

O Engine e as alternativas cobrem quase tudo, com licença livre:

Alternativa	Plataforma	Licença	Ressalva
Docker Engine	Linux	Apache 2.0	Sem GUI; é o próprio Docker
Podman + Podman Desktop	Linux, mac, Win	Apache 2.0	alias docker=podman cobre 90%
Rancher Desktop	Linux, mac, Win	Apache 2.0	Traz Kubernetes junto
colima	mac	Apache 2.0	Só CLI; leve
OrbStack	mac	Proprietário	Pago para uso comercial; o mais rápido

Alternativa	Plataforma	Licença	Ressalva
Docker Engine no WSL2	Win	Apache 2.0	Sem o app; ver 03 (cap. 3)

Recomendação: em Linux, nunca pague — use o Engine. Em empresa grande que já usa macOS/Windows, avalie honestamente: o custo de suporte interno das alternativas pode exceder a licença. Em organização com restrição orçamentária, Podman Desktop é maduro e sem ressalva técnica relevante.

25.4 Docker Hub — armazenamento e o limite de pull

25.4.1 Planos (consultados em 11/08/2026)

Plano	Preço	Repositórios privados	Pulls
Personal	US\$ 0	1	100/hora (autenticado)
Pro	~US\$ 9/mês	ilimitados	ilimitado (uso justo)
Team	~US\$ 15/usuário/mês	ilimitados	ilimitado
Business	~US\$ 24/usuário/mês	ilimitados	ilimitado

25.4.2 Os limites de pull — o custo que quebra CI (vigente desde 01/04/2025)

Situação	Limite
Sem autenticação	10 pulls/hora , por IP
Conta gratuita autenticada	100 pulls/hora
Pro / Team / Business	Ilimitado, uso justo

Impacto real: um runner de CI em IP compartilhado esgota os 10 pulls em minutos e o pipeline quebra com `toomanyrequests`. Isso é custo — de tempo e de confiabilidade — mesmo sendo “grátis”.

Mitigações (custo baixo ou zero):

- autenticar no CI (grátis, sobe para 100/h);
- espelho puxador na sua rede (`registry:2` como pull-through cache);
- registries alternativos: **GHCR** e **Quay** têm público ilimitado.

25.5 Registries de nuvem — o custo por GB e por egress

Você paga por **armazenamento** e, sobretudo, por **transferência de saída (egress)**. Valores são **ordens de grandeza aproximadas, consultados em 11/08/2026** — confirme na calculadora de cada provedor.

Registry	Armazenamento	Egress	Grátis
GHCR	Na cota do GitHub	Generoso	Público ilimitado
Quay.io	Plano gratuito para público	—	Público ilimitado

Registry	Armazenamento	Egress	Grátis
AWS ECR	~US\$ 0,10/GB/mês	Egress padrão da AWS (~US\$ 0,09/GB saindo)	500 MB/mês (privado), 12 meses
AWS ECR Public	Grátis até certo limite	Grátis para o mundo	Sim
Google Artifact Registry	~US\$ 0,10/GB/mês	Egress padrão do GCP	0,5 GB
Azure ACR	Por nível (Basic/Std/ Premium)	Egress padrão	Camada limitada

O egress é o custo que surpreende. Uma imagem de 1 GB puxada 1.000 vezes por dia por nós em outra rede = ~1 TB/dia de saída. A ~US\$ 0,09/GB, são ~US\$ 90/dia só de transferência. **Correção:** registry na mesma região dos nós (egress intra-região costuma ser grátis), imagens menores, cache/espelho local. É onde imagem enxuta vira economia direta.

25.6 O custo de rodar containers na nuvem

Container não tem licença, mas a **infraestrutura** que o executa, sim. Comparação de modelos (ordens de grandeza, 11/08/2026 — confirme):

Modelo	Cobrança	Bom para	Cuidado
VM própria + Docker	Por hora da VM	Carga previsível	Você opera tudo
AWS ECS + Fargate	vCPU-hora + GB-hora	Sem gerenciar servidor	Mais caro por unidade que EC2
Google Cloud Run	Por requisição + CPU/RAM usados; escala a zero	Tráfego variável	Cold start; caro sob tráfego alto constante
AWS Lambda (container)	Por invocação + duração	Picos, eventos	Limite de imagem; cold start
Kubernetes gerenciado (EKS/GKE/AKS)	~US\$ 70-100/mês pelo <i>control plane</i> + nós	Escala, multi-serviço	Complexidade operacional (a maior fatia é tempo de gente)
Kubernetes próprio (k3s numa VM)	Só a VM	Homelab, aprendizado	Você é o SRE

A regra de bolso: para tráfego constante, VM ou nós reservados são mais baratos por unidade. Para tráfego intermitente, serverless (Cloud Run/Lambda) evita pagar pela ociosidade. O erro caro é rodar carga constante em serverless caro por unidade, ou provisionar cluster grande para carga pequena.

25.7 Imagens base endurecidas — o custo novo de 2026

Surgiu uma categoria comercial: **imagens base com CVEs próximas de zero e SLA de correção.**

Produto	Modelo	Licença/preço
Docker Hardened Images	Assinatura (Docker)	Comercial; consulte

Produto	Modelo	Licença/preço
Chainguard Images	Algumas grátis, catálogo completo pago	Comercial
Minimus distroless (Google)	Comercial Grátis	Consulte Apache 2.0 — você faz o escaneamento

A conta a fazer: você paga por imagens mantidas sem CVE, ou paga o tempo da sua equipe para manter distroless + escaneamento + atualização disciplinada. Para times pequenos sem especialista em segurança, a assinatura pode sair mais barata que o tempo. Para times com essa competência, distroless grátis resolve. Ver [65-estado-da-arte.md](#) (cap. 22).

25.8 O custo que ninguém coloca na planilha

O maior custo do Docker **não é licença nem nuvem — é tempo de gente:**

- **Aprendizado:** 60-100 horas até competência produtiva (ver [cap. 2](#)).
- **Operação:** alguém cuida de atualizações, backups, segurança, disco cheio às 3 da manhã.
- **Depuração:** a curva de "por que não conecta / por que está lento / por que encheu o disco".
- **A dívida de complexidade** de adotar orquestração cedo demais — frequentemente o custo invisível mais alto de todos.

Opinião profissional: a decisão financeira mais impactante em containers raramente é "Desktop pago ou grátis?". É "**orquestração agora ou depois?**". Adotar Kubernetes cedo demais custa, em tempo de equipe, muito mais que qualquer licença de Desktop. Ver [cap. 20](#).

25.9 Cenários de custo, do zero ao corporativo

Cenário	Configuração	Custo mensal aproximado
Aprendiz	Play with Docker / Engine no Linux / Codespaces	US\$ 0
Dev solo (Linux)	Docker Engine + GHCR + VPS	US\$ 5
Dev solo (Mac)	Docker Desktop Personal (grátis) ou colima	US\$ 0
Homelab	k3s ou Compose numa máquina própria	custo de energia
Startup < 250 pessoas, < US\$ 10M	Desktop grátis + GHCR + Cloud Run	baixo, escala com uso
Empresa média	Desktop Team (20 devs) + ECR + EKS	~US\$ 300 (Desktop) + ~US\$ 100 (control plane) + nós + tempo
Empresa grande	Desktop Business + Hardened Images + registry corporativo	licenças por usuário + assinaturas + equipe de plataforma

25.10 Como não gastar à toa

1. **Em Linux, use o Engine.** Nunca pague Desktop onde ele não agrega.
2. **Autentique no Docker Hub** — os 100 pulls/h grátis resolvem a maioria dos CI.
3. **Imagens menores = menos egress e menos armazenamento.** Multi-stage se paga.
4. **Registry na mesma região dos nós.** Egress intra-região costuma ser grátis.
5. **Escala a zero** (Cloud Run/Lambda) para carga intermitente; **nós reservados** para carga constante.
6. **Não adote Kubernetes antes de precisar.** É o gasto de tempo mais alto e mais evitável.
7. **Configure retenção no registry desde o primeiro dia**, antes de a conta de armazenamento crescer.

25.11 Autoteste

1. O Docker Engine custa licença? E o Docker Desktop? Qual é a diferença entre os dois?
2. Uma empresa de 30 pessoas com US\$ 15 milhões de receita pode usar Docker Desktop de graça? Justifique com a regra do "E".
3. Quais são os limites de pull do Docker Hub hoje, e por que isso é um custo mesmo sendo "grátis"?
4. Por que o egress é o custo de registry que mais surpreende, e como reduzi-lo?
5. Quando serverless (Cloud Run) sai mais barato que VM, e quando sai mais caro?
6. Qual é a decisão financeira de maior impacto em containers, na opinião do autor? Por quê?
7. Você quer evitar pagar Docker Desktop numa empresa. Cite três alternativas livres e a ressalva de cada.
8. Imagem endurecida paga ou distroless grátis: qual é a conta que decide?
9. Qual é o custo que "ninguém coloca na planilha", e por que ele domina?
10. Liste três formas concretas de reduzir o custo de rodar containers na nuvem.

25.11.1 Fontes consultadas (11/08/2026)

- [Docker Pricing](#) — página oficial — planos Personal/Pro/Team/Business (valores aproximados; confirme na fonte)
- [Docker Docs — Usage and rate limits](#) — limites de pull vigentes desde 01/04/2025
- [Docker Subscription Service Agreement / FAQ](#) — regra dos 250 funcionários e US\$ 10M de receita
- Calculadoras de preço da AWS, Google Cloud e Azure — **ordens de grandeza; sempre confirmar na fonte antes de decidir**

CAPÍTULO 26

Cursos gratuitos e certificações

Nível: todos · **Pesquisado na web em 11/08/2026** · Este arquivo envelhece em ~1 ano.

Metodologia: links e existência dos cursos verificados por busca em 11/08/2026. **Não assisti aos cursos;** a avaliação de qualidade combina reputação conhecida do autor/canal com o que a busca retornou. Trate as recomendações como ponto de partida, não como resenha. Datas de publicação e disponibilidade mudam — confirme antes de investir tempo. Onde não confirmei um detalhe, o texto diz isso.

Ordem de prioridade do preset: **português → inglês → francês.**

26.1 Português (Brasil e Portugal)

26.1.1 Vídeo — gratuito

Curso	Autor / Canal	Onde	Nível	Vale?
Descomplicando o Docker	Jefferson Fernando / LINUXtips	YouTube	iniciante→intermediário	Sim. A referência em PT-BR. Didático, prático, foco em quem vai operar de verdade. Há versões de anos diferentes; prefira a mais recente
Docker do Zero	Fabício Veronez	YouTube	iniciante→intermediário	Sim. Um dos melhores comunicadores de DevOps em português; conecta bem Docker a CI/CD e nuvem
Intensivão Docker (2h)	vários (busca 2025)	YouTube	iniciante	Bom para uma visão geral rápida antes de aprofundar
Curso de Docker Completo	vários	YouTube (playlists)	iniciante	Qualidade varia por playlist; cheque data e conteúdo

LINUXtips também tem treinamentos pagos aprofundados (Docker Essentials, “Descomplicando” em plataforma própria). O material do YouTube é o gratuito e já leva longe. As **anotações da comunidade** sobre o “Descomplicando o Docker” estão em repositórios públicos no GitHub — úteis como resumo consultável.

26.1.2 Texto e documentação em português

Recurso	Observação
free-programming-books (PT-BR)	Índice comunitário; seção de Docker/DevOps com links atualizados pela comunidade
Documentação oficial	A docs.docker.com não tem tradução oficial confiável em PT; use o original em inglês

26.2 Inglês

26.2.1 Vídeo — gratuito

Curso	Autor / Plataforma	Nível	Vale?
Docker Tutorial for Beginners (Full DevOps Course)	freeCodeCamp (YouTube)	iniciante	Sim. Completo, com laboratórios acessíveis no navegador. Um dos melhores pontos de partida gratuitos
Docker for the Absolute Beginner	KodeKloud (Mumshad Mannambeth)	iniciante	Sim. A parte introdutória costuma ser gratuita; laboratórios interativos excelentes. É a origem do estilo "hands-on" da área
Docker Full Course for Beginners	freeCodeCamp / vários (2026)	iniciante	Atualizado; bom para ver os padrões atuais
Free Learning Week	KodeKloud	todos	Janelas periódicas de acesso gratuito a cursos + laboratórios pagos

26.2.2 Laboratórios interativos — gratuitos

Recurso	O que é
Play with Docker	Terminal real no navegador; sessões de 4h. Comece por aqui
Killercodea	Cenários guiados com terminal real
KodeKloud labs	Laboratórios interativos; alguns gratuitos
Docker's own "Getting Started"	Tutorial oficial, prático

26.2.3 Texto — a fonte definitiva

A **documentação oficial** ([docs.docker.com](#)) é, honestamente, melhor que a maioria dos cursos pagos: bem escrita, com trilha de aprendizado, e sempre atualizada. Combine-a com prática no Play with Docker.

26.3 Francês

Curso	Autor / Canal	Onde	Nível	Vale?
Docker formation de A à Z	Xavki (Xavier Pestel)	YouTube	iniciante→avançado	Sim. ~30 vídeos, progressão clara. A melhor referência gratuita em francês; o canal tem trilha DevOps inteira

Curso	Autor / Canal	Onde	Nível	Vale?
Tutoriels Docker-Compose (FR)	Xavki	YouTube	intermediário	Continuação natural, focada em Compose
Tutoriel Docker pour débutants (1h)	vários	YouTube	iniciante	Panorama rápido
Apprendre Docker — cours complet	devopssec.fr	texto	iniciante→intermediário	Curso escrito, bem estruturado
5 ressources gratuites (2026)	learnthings.fr	agregador	—	Lista curada de recursos gratuitos

26.4 Certificações — o mapa honesto

26.4.1 Docker Certified Associate (DCA) — atenção ao estado

Situação em 11/08/2026: confusa, e é preciso ser franco. As fontes divergem:

- Parte relata que a **DCA foi descontinuada em 2026**, sem substituto direto anunciado pela Docker.
- Parte relata que a certificação **passou à Mirantis** (que adquiriu a Docker Enterprise em

2019. e continuaria disponível para registro.

O que fazer com isso: confirme o estado atual diretamente com a Mirantis/Docker antes de estudar para a DCA. Independentemente do estado administrativo, o mercado migrou: a competência de container em produção hoje se sobrepõe fortemente a **Kubernetes**, e as certificações de Kubernetes têm valor de mercado muito mais claro.

26.4.2 As certificações que valem em 2026

Todas são da **Linux Foundation / CNCF**, práticas (você opera um cluster real na prova), e têm reconhecimento de mercado sólido.

Certificação	Foco	Formato	Preço (consultar)
KCNA — Kubernetes and Cloud Native Associate	Fundamentos (inclui containers)	Múltipla escolha, 90 min	~US\$ 250 (relatado; confirme)
KCSA — Kubernetes and Cloud Native Security Associate	Segurança cloud-native	Múltipla escolha	consultar
CKA — Certified Kubernetes Administrator	Operar clusters	Prática , 2h	consultar (tipicamente mais que KCNA)
CKAD — Certified Kubernetes Application Developer	Desenvolver para K8s	Prática , 2h	consultar
CKS — Certified Kubernetes Security Specialist	Segurança avançada (exige CKA)	Prática , 2h	consultar

Preços das certificações mudam e há promoções frequentes (Cyber Monday, bundles com treinamento, vouchers de eventos). O valor de ~US\$ 250 para a KCNA foi o relatado na busca de 11/08/2026 — **confirme na training.linuxfoundation.org**. Todas incluem uma retentativa gratuita e um período de acesso a ambiente de treino.

26.4.3 Trilha recomendada

Docker (este material) → KCNA → CKA (ou CKAD, conforme o papel) → CKS
 fundamento porta competência de mercado especialização
 de container de entrada real e reconhecida em segurança

- **Começa em Docker** porque containers são o pré-requisito conceitual do resto.
- **KCNA** é a porta de entrada barata e de baixo risco (múltipla escolha), boa para validar fundamentos.
- **CKA/CKAD** são as que abrem porta no mercado, e são **práticas** — não se passa decorando.
- **CKS** é especialização, exige CKA vigente.

26.4.4 Preparação gratuita para as certificações

Recurso	Para qual	Custo
KodeKloud (cursos + labs)	KCNA, CKA, CKAD, CKS	Parte gratuita; assinatura para o completo
Kubernetes docs / tutoriais oficiais	Todas	Grátis
killer.sh	CKA, CKAD, CKS	Duas sessões incluídas ao comprar o exame — o simulador mais próximo da prova real
CNCF curriculum (GitHub)	Todas	Grátis; o programa oficial de cada exame
"Kubernetes the Hard Way" (Kelsey Hightower)	CKA e entendimento profundo	Grátis; monta um cluster do zero, à mão

26.5 Trilha de estudo sugerida, do zero ao empregável

Fase	O que fazer	Tempo	Custo
1. Fundamentos	Este material (blocos A e B) + Play with Docker	2-4 semanas	US\$ 0
2. Prática	Os 10 laboratórios do cap. 23 + um projeto seu	2-4 semanas	US\$ 0
3. Curso guiado	LINUXtips ou Fabrício Veronez (PT) · freeCodeCamp ou KodeKloud (EN)	1-2 semanas	US\$ 0
4. Validação	KCNA	2-4 semanas de estudo	~US\$ 250
5. Mercado	CKA/CKAD + orquestração (25 (cap. 20))	2-3 meses	preço do exame

Realismo: só faça certificação se ela tiver função concreta (exigência de vaga, de cliente, de promoção). Para *aprender*, o material gratuito + prática vale mais que qualquer

certificado. Para *sinalizar* competência no mercado, CKA/CKAD são as que o RH reconhece. KCNA é boa relação custo-benefício como primeiro selo; DCA, no estado atual incerto, eu não priorizaria.

26.6 Certificadores gratuitos — existe algo de valor real?

Resposta franca: certificados verdadeiramente gratuitos e com valor de mercado real praticamente não existem em container/Kubernetes. O que há:

Tipo	Valor de mercado	Observação
Certificados de conclusão do YouTube/plataforma grátis	Simbólico	Servem para portfólio, não abrem porta sozinhos
Badges de “free learning week”	Simbólico a baixo	Marketing das plataformas
CKA/CKAD/CKS/KCNA (pagos)	Real	São os que o mercado reconhece
Cloud Skills Boost (Google), AWS Skill Builder	Baixo a médio	Bons para aprender; o certificado grátis vale pouco

Conclusão honesta: o conhecimento é gratuito e abundante; a **sinalização reconhecida** é paga. Não confunda “fiz um curso grátis com certificado” com “tenho uma credencial de mercado” — são coisas diferentes, e o RH sabe a diferença.

26.7 Autoteste

1. Qual é a melhor referência gratuita em vídeo para Docker em português, e por quê?
2. Por que a documentação oficial em inglês é recomendada mesmo para quem prefere português?
3. Qual é o estado da certificação DCA em agosto de 2026, e o que fazer diante dessa incerteza?
4. Ordene KCNA, CKA e CKS numa trilha e justifique a ordem.
5. Por que CKA/CKAD “não se passa decorando”?
6. Qual é a melhor referência gratuita em francês, e o que ela cobre?
7. Certificado gratuito de conclusão tem valor de mercado? Distinga “aprender” de “sinalizar”.
8. Que ferramenta acompanha a compra do exame CKA e é o melhor simulador?
9. Monte uma trilha do zero ao empregável com tempo e custo por fase.
10. Por que começar por Docker antes de Kubernetes, mesmo mirando certificação de Kubernetes?

26.7.1 Fontes consultadas (11/08/2026)

- [LINUXtips — Docker Essentials](#) e anotações comunitárias do “Descomplicando o Docker” (diogo-alves, Pacheco95)
- [free-programming-books — cursos gratuitos PT-BR](#)
- [freeCodeCamp — Docker Full Course](#) · [KodeKloud — Free Courses](#) e [Free Learning Week](#)

- [Xavki](#) — Docker formation de A à Z · [devopssec.fr](#) — Apprendre Docker · [learnthings.fr](#) — 5 ressources gratuites 2026
- [KodeKloud](#) — Docker Certification guide · [TrueCert](#) — DCA Discontinued · [Sailor.sh](#) — [KCNA Exam Guide 2026](#) — **fontes divergem sobre o estado da DCA; preço da KCNA (~US\$ 250) a confirmar em training.linuxfoundation.org**

PARTE V

Fontes

Bibliografia, referências e glossário

Bibliografia comentada

Nível: todos · Última atualização: 11/08/2026

Livros, com autor, edição e para que nível servem. **Nada aqui foi inventado:** onde não tenho certeza de uma edição ou ISBN, cito só autor e título e digo que é aproximado. Confirme a edição mais recente antes de comprar — livros de tecnologia envelhecem, e alguns destes já têm parte datada.

27.1 Para aprender Docker

27.1.1 Docker Deep Dive — Nigel Poulton

- **Autor:** Nigel Poulton (Docker Captain)
- **Edição:** atualizada em 2025 (o autor revisa o livro periodicamente; confirme a mais recente)
- **Nível:** iniciante → intermediário
- **Onde:** Leanpub, Amazon, Apple Books, Google Books
- **Por que ler:** é o livro mais popular e mais bem avaliado sobre Docker, e com razão. Poulton escreve com clareza incomum, começa do zero e chega a Swarm, redes e segurança. A edição de 2025 acrescentou capítulos sobre **Wasm containers** e **Docker Model Runner** (LLMs locais).
- **Ressalva honesta:** cobre Swarm com peso que o mercado não dá mais; a parte de orquestração envelheceu em relação ao domínio do Kubernetes. O núcleo (imagens, containers, rede, armazenamento) continua excelente e atual.
- **Gratuito?** Não. É pago em todas as plataformas. Preço acessível no Leanpub.

27.1.2 Docker in Action, 2ª edição — Jeff Nickoloff e Stephen Kuenzli

- **Editora:** Manning, 2ª ed. (2019)
- **Nível:** intermediário
- **Por que ler:** mais profundo que o Poulton em operação e em detalhes de runtime. Bom segundo livro, para quem já roda containers e quer entender melhor o que acontece por baixo.
- **Ressalva:** 2019 — anterior ao BuildKit padrão, aos avanços de cadeia de suprimentos e às mudanças recentes da Engine. O conceitual resiste; ferramentas e comandos, verifique.

27.1.3 The Docker Book — James Turnbull

- **Nível:** iniciante → intermediário
- **Por que ler:** foi o livro de referência da primeira geração; didático. Turnbull é um bom professor.

- **Ressalva: datado.** Útil historicamente e para os fundamentos, mas muita coisa mudou. Não seria minha primeira escolha em 2026.

27.2 Para ir além do Docker (containers e Kubernetes)

27.2.1 Container Security — Liz Rice

- **Editora:** O'Reilly (2020)
- **Nível:** intermediário → avançado
- **Por que ler:** a melhor referência sobre segurança de containers. Liz Rice explica namespaces, cgroups, capabilities, seccomp e escapes com uma clareza que este material tentou imitar. Se você só for ler um livro além do básico, que seja este.
- **Ressalva:** 2020 — cadeia de suprimentos (SBOM, assinatura, SLSA) evoluiu muito depois; complemente com [cap. 22](#) e a documentação do Sigstore.
- **Gratuito?** A O'Reilly às vezes libera este título em PDF via patrocínio (Aqua Security já distribuiu). Procure "Container Security Liz Rice free PDF" — **confirme a legitimidade** antes de baixar; o download oficial patrocinado é legal, cópias piratas não.

27.2.2 Kubernetes Up & Running — Hightower, Burns, Beda

- **Editora:** O'Reilly, 3ª ed. (2022)
- **Nível:** intermediário
- **Por que ler:** escrito por gente que construiu o Kubernetes (Brendan Burns e Joe Beda estão entre os criadores). A ponte natural depois deste material, se você for para orquestração.
- **Ressalva:** Kubernetes muda rápido; confira contra a documentação da versão que você usa.

27.2.3 Kubernetes Patterns — Bilgin Ibryam e Roland Huß

- **Editora:** O'Reilly, 2ª ed. (2023)
- **Nível:** intermediário → avançado
- **Por que ler:** cataloga padrões de projeto para cargas em container. Ótimo para quem já opera e quer parar de reinventar.

27.2.4 Cloud Native DevOps with Kubernetes — Arundel e Domingus

- **Editora:** O'Reilly, 2ª ed. (2022)
- **Nível:** intermediário
- **Por que ler:** conecta container e Kubernetes à prática de DevOps (CI/CD, observabilidade, custo). Boa visão de fim a fim.

27.3 Fundamentos de sistemas (o "por baixo" dos containers)

27.3.1 The Linux Programming Interface — Michael Kerrisk

- **Editora:** No Starch Press (2010)
- **Nível:** avançado → referência
- **Por que ler:** a referência sobre a interface de programação do Linux. Namespaces, processos, sinais, memória — tudo que faz um container ser um container está explicado aqui, na fonte. Não é sobre Docker; é sobre o que o Docker usa.
- **Ressalva:** 2010, mas o núcleo de syscalls mudou pouco. É um tijolo de 1.500 páginas — use como referência, não como leitura linear.
- **Gratuito?** Não, mas o autor mantém material complementar e as páginas de manual (man) que ele escreveu são gratuitas e excelentes.

27.3.2 How Linux Works, 3ª ed. — Brian Ward

- **Editora:** No Starch Press (2021)
- **Nível:** intermediário
- **Por que ler:** contexto de sistema para quem não vem de Linux profundo. Ajuda a entender cgroups, processos e o que o container está mentindo para o processo.

27.3.3 Systems Performance, 2ª ed. — Brendan Gregg

- **Editora:** Addison-Wesley (2020)
- **Nível:** avançado
- **Por que ler:** o livro definitivo sobre desempenho de sistemas Linux, com **um capítulo dedicado a containers** (cgroups, throttling, observação). Se você chegou ao ponto de depurar latência e throttling de container (13 (cap. 11), 60 (cap. 21)), é aqui que se aprofunda.

27.4 Para arquitetura e produção

27.4.1 Designing Data-Intensive Applications — Martin Kleppmann

- **Editora:** O'Reilly (2017)
- **Nível:** intermediário → avançado
- **Por que ler:** não é sobre containers, mas é sobre o problema que os containers **não** resolvem: estado, consistência, replicação. Quando você externalizar o estado (passo 4 da migração em [cap. 20](#)), este livro é o mapa. Um dos melhores livros técnicos desta geração.
- **Nota:** há uma 2ª edição em preparação; confira se já saiu.

27.4.2 Site Reliability Engineering — Google (Beyer et al.)

- **Editora:** O'Reilly (2016)
- **Nível:** intermediário
- **Por que ler:** os conceitos de SLO, monitoramento e os quatro sinais de ouro que aparecem em [cap. 19](#) vêm daqui.
- **Gratuito?** **Sim, legalmente.** O Google disponibiliza o texto completo em sre.google/books. Também há o *SRE Workbook*, igualmente gratuito.

27.5 O que é legalmente gratuito

Obra	Onde	Observação
Site Reliability Engineering + SRE Workbook	sre.google/books	Texto completo, oficial
Documentação oficial do Docker	docs.docker.com	Melhor que muitos livros pagos
Kubernetes the Hard Way — Kelsey Hightower	GitHub	Tutorial gratuito, clássico
Páginas de manual do Linux (man 7 namespaces, man 7 cgroups)	qualquer Linux	Fonte primária, de Michael Kerrisk
Especificações OCI	github.com/opencontainers	Curtas e esclarecedoras
Container Security — Liz Rice	via patrocínio O'Reilly (confirme a legitimidade)	Frequentemente liberado em PDF

27.6 Recomendação por perfil

Você é...	Leia, nesta ordem
Iniciante total	Docker Deep Dive (Poulton) + docs oficiais + prática
Dev que quer produção	Poulton → Container Security (Rice) → SRE (grátis)
Quem quer entender por baixo	The Linux Programming Interface (Kerrisk) + Systems Performance (Gregg)
Indo para Kubernetes	Kubernetes Up & Running → Kubernetes Patterns
Arquiteto	Designing Data-Intensive Applications + Kubernetes Patterns + SRE
Foco em segurança	Container Security (Rice) → CKS curriculum → docs Sigstore

27.7 O que não recomendo

Sem citar título específico injustamente, um alerta de categoria:

- **Livros de "Docker em 24 horas / para leigos" genéricos** — costumam estar desatualizados e raramente vão além do que a documentação oficial gratuita cobre melhor.
- **Qualquer livro de Docker anterior a 2019** como fonte principal — perde BuildKit, Compose v2, cadeia de suprimentos e as mudanças recentes de Engine. Bom para história, ruim para prática.
- **Cursos/livros que ensinam docker-compose (com hífen) e version:** — sintaxe obsoleta; sinal de material velho.

27.8 Autoteste

1. Qual livro você recomendaria a um iniciante total, e qual a ressalva dele?
2. Qual é a melhor referência sobre segurança de containers, e o que a data dela exige que você complemente?
3. Cite dois livros legalmente gratuitos e onde obtê-los.
4. Por que "Designing Data-Intensive Applications" aparece numa bibliografia de Docker, se não fala de containers?

5. Qual livro consultar para depurar throttling de CPU em container?
6. Por que um livro de Docker de 2018 é problemático como fonte principal em 2026?
7. Qual é a fonte primária, gratuita e sempre atualizada sobre namespaces e cgroups?
8. Monte uma trilha de leitura para quem quer entender containers "por baixo".
9. Por que a documentação oficial pode ser melhor que um livro pago?
10. Que sinais indicam que um material de Docker está desatualizado, só pela sintaxe usada?

CAPÍTULO 28

Referências — specs, docs, código, pessoas

Nível: todos · Última atualização: 11/08/2026

Fontes primárias e verificáveis. Preferência por documentação oficial, especificações e código sobre artigos de blog. Onde cito blog ou fonte secundária, digo que é isso.

28.1 Especificações (as fontes de verdade)

Spec	Onde	O que define
OCI Image Spec	github.com/opencontainers/image-spec	Formato da imagem: manifesto, camadas, config, digests
OCI Runtime Spec	github.com/opencontainers/runtime-spec	Bundle, config.json, ciclo de vida, hooks
OCI Distribution Spec	github.com/opencontainers/distribution-spec	API HTTP do registry
CNI (Container Network Interface)	github.com/containernetworking/cni	Plugins de rede
CSI (Container Storage Interface)	github.com/container-storage-interface/spec	Plugins de armazenamento
CRI (Container Runtime Interface)	parte do Kubernetes	Contrato kubelet ↔ runtime
SLSA	slsa.dev	Níveis de garantia de proveniência
SPDX / CycloneDX	spdx.dev · cyclonedx.org	Formatos de SBOM

São curtas e esclarecedoras. Ler a Runtime Spec uma vez dissolve mais dúvidas que dez tutoriais.

28.2 Documentação oficial

Recurso	URL
Docker Docs	docs.docker.com
Referência do Dockerfile	docs.docker.com/reference/dockerfile
Compose Specification	github.com/compose-spec/compose-spec
Docker Engine release notes	docs.docker.com/engine/release-notes
BuildKit	github.com/moby/buildkit
Best practices de Dockerfile	docs.docker.com/build/building/best-practices
Rate limits do Docker Hub	docs.docker.com/docker-hub/usage

28.3 Código-fonte (para quem quer o fundo do poço)

Projeto	Repositório	O que é
Moby	github.com/moby/moby	O upstream do Docker Engine
containerd	github.com/containerd/containerd	Runtime de alto nível (CNCF, graduado)
runc	github.com/opencontainers/runc	Runtime de referência da OCI
crun	github.com/containers/crun	Runtime em C, mais rápido
youki	github.com/youki-dev/youki	Runtime em Rust
Podman	github.com/containers/podman	Engine sem daemon
BuildKit	github.com/moby/buildkit	Motor de build
gVisor	github.com/google/gvisor	Kernel em espaço de usuário
Firecracker	github.com/firecracker-microvm/firecracker	Micro-VM
Kata Containers	github.com/kata-containers/kata-containers	Container em micro-VM
runwasi	github.com/containerd/runwasi	Shim Wasm para containerd

28.4 Ferramentas mencionadas neste material

Ferramenta	O que faz	URL
dive	Analisa camadas e desperdício de imagem	github.com/wagoodman/dive
hadolint	Linter de Dockerfile	github.com/hadolint/hadolint
Trivy	Vulnerabilidades, segredos, má config	github.com/aquasecurity/trivy
Grype / Syft	Escaneamento / SBOM	github.com/anchore
Dockle	Boas práticas de imagem	github.com/goodwithtech/dockle
cosign	Assinatura de imagem	github.com/sigstore/cosign
skopeo	Copiar/inspecionar imagens sem daemon	github.com/containers/skopeo
netshoot	Caixa de ferramentas de rede	github.com/nicolaka/netshoot
Docker Bench	Auditoria CIS	github.com/docker/docker-bench-security
Falco	Deteção em runtime	github.com/falcosecurity/falco
cAdvisor	Métricas de container	github.com/google/cadvisor
Watchtower	Atualização automática (cuidado em produção)	github.com/containrrr/watchtower
gitleaks	Deteção de segredo	github.com/gitleaks/gitleaks

28.5 Artigos acadêmicos (fundamentos e teoria)

Referenciados em [cap. 21](#):

- **Menage, P.** (2007). *Adding Generic Process Containers to the Linux Kernel*. Linux Symposium. — a origem dos cgroups.
- **Verma, A. et al.** (2015). *Large-scale cluster management at Google with Borg*. EuroSys. — o ancestral do Kubernetes.
- **Agache, A. et al.** (2020). *Firecracker: Lightweight Virtualization for Serverless Applications*. USENIX NSDI. — o trade-off isolamento × cold start.

- **Young, E. G. et al.** (2019). *The True Cost of Containing: A gVisor Case Study*. USENIX HotCloud. — sobrecarga de gVisor medida.
- **Harter, T. et al.** (2016). *Slacker: Fast Distribution with Lazy Docker Containers*. USENIX FAST. — 76% do tempo de start é pull/extração; a origem do lazy pulling.
- **Karp, R.** (1972). *Reducibility Among Combinatorial Problems*. — NP-completude do bin packing.
- **Merkel, D.** (2014). *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal. — o artigo de divulgação original.

Muitos estão em [usenix.org](https://www.usenix.org) (acesso aberto) e nos anais das conferências.

28.6 Páginas de manual do Linux (fonte primária, gratuita)

```
man 7 namespaces
man 7 cgroups
man 7 capabilities
man 2 clone
man 2 unshare
man 2 seccomp
man 8 runc
man 5 dockerd    # se instalado
```

bash

Escritas em grande parte por Michael Kerrisk. São a verdade sobre o que o kernel faz — acima de qualquer blog.

28.7 Blogs e fontes secundárias (marcados como tais)

Úteis, mas **secundários** — verifique contra a documentação oficial:

Fonte	Perfil
Docker Blog	Oficial, mas é marketing além de técnica
Julia Evans (jvns.ca)	Explicações excelentes de containers e Linux; os <i>zines</i> são ótimos
Ivan Velichko (iximiuz.com)	Laboratórios interativos e artigos profundos sobre containers
Brendan Gregg (brendan-gregg.com)	Desempenho de sistemas, referência
LWN.net	Cobertura profunda de kernel; a melhor fonte sobre mudanças no Linux
endoflife.date	Datas de fim de vida e versões — usei para confirmar a Engine 29.7.1

28.8 Pessoas a seguir

Nomes que ajudam a acompanhar o campo (fatos públicos; sem inferir nada além do papel profissional conhecido):

Pessoa	Por quê
Liz Rice	Autoridade em segurança de containers e eBPF; autora e mantenedora
Nigel Poulton	Educador; autor de <i>Docker Deep Dive</i>
Kelsey Hightower	Referência em Kubernetes e cloud native; <i>Kubernetes the Hard Way</i>
Brendan Gregg	Desempenho de sistemas Linux
Michael Kerrisk	Autor de <i>TLPI</i> e mantenedor das man pages do Linux

Pessoa	Por quê
Ivan Velichko	Conteúdo técnico profundo e prático sobre containers
Solomon Hykes	Criador do Docker (contexto histórico)
Jérôme Petazzoni	Um dos primeiros engenheiros do Docker; material didático clássico

28.9 Comunidades

Comunidade	Onde
CNCF Slack	slack.cncf.io
Docker Community	docker.com/community
r/docker	reddit.com/r/docker
Stack Overflow	tags docker, docker-compose
Server Fault	operação e produção

28.10 Como este material foi verificado

Transparência sobre a produção:

- **Versões e datas** (Engine 29.7.1, limites do Hub, prazos do CRA): confirmadas por busca em **11/08/2026**, com as fontes citadas no rodapé de **03** (cap. 3), **11** (cap. 9), **19** (cap. 17), **65** (cap. 22), **80** (cap. 25) e **85** (cap. 26).
- **O projeto-modelo foi executado**: a suíte de 22 testes passou em Node v24.18.0 e a aplicação foi verificada de ponta a ponta fora de container. O `docker build/docker compose up` **não** puderam ser executados no ambiente de escrita (sem acesso ao socket do daemon), o que está declarado no [README do projeto](#) (cap. 7).
- **Comandos de exemplo** seguem a documentação oficial vigente na data. Ainda assim: **execute na sua máquina**. Versões mudam, e um comando que “deveria funcionar” é uma hipótese até você rodá-lo.
- **Onde uma afirmação vem de fonte secundária** (ex.: estabilização do shim runwasi na Engine 29, estado da certificação DCA), o texto marca isso explicitamente em vez de apresentá-la como verificada.

28.11 Autoteste

1. Quais são as três especificações da OCI e o que cada uma define?
2. Onde está o código-fonte do Docker Engine, e por que ele não se chama “docker”?
3. Qual página de manual você leria para entender namespaces na fonte?
4. Cite três ferramentas de segurança de imagem e o que cada uma faz.
5. Qual artigo mostrou que a maior parte do tempo de início de um container é pull/extração?
6. Por que blogs são marcados como fonte secundária aqui?
7. Qual fonte foi usada para confirmar a versão 29.7.1 da Engine?
8. O que exatamente foi verificado no projeto-modelo, e o que não pôde ser?
9. Cite duas pessoas a seguir e o motivo de cada uma.

10. Se um comando deste material falhar na sua máquina, o que a seção 10 sugere que você conclua?

Glossário — Docker e containers

Todo termo técnico do material, definido. · Última atualização: 11/08/2026

Termos em inglês são mantidos quando é assim que o campo os usa, com a tradução na primeira menção. Ordenado alfabeticamente.

AppArmor — *Linux Security Module* usado em Debian/Ubuntu que aplica perfis de controle de acesso obrigatório a processos, inclusive containers. Ver também [LSM], [seccomp].

Attestation (atestado) — declaração assinada sobre uma imagem: como foi construída (proveniência SLSA), o que contém (SBOM) ou se uma CVE se aplica (VEX). Fica no registry, ligada ao [digest].

Base image (imagem base) — a imagem declarada no primeiro FROM, sobre a qual as demais camadas são empilhadas. Ex.: alpine, node:22-alpine, scratch.

Bind mount — montagem de um diretório do host dentro do container, por caminho absoluto. Usado para código em desenvolvimento. Difere de [volume] por não ser gerenciado pelo Docker.

Bridge — driver de rede padrão: uma rede virtual isolada com [NAT]. A bridge padrão não tem [DNS] interno; redes bridge criadas pelo usuário têm.

BuildKit — motor de build moderno do Docker (padrão desde a Engine 23): grafo de dependências, build paralelo, cache remoto, --mount=type=secret e type=cache.

Bundle (OCI) — o par config.json + rootfs/ que o [runtime] de baixo nível executa.

Capability — fração do poder do root concedida individualmente (ex.: CAP_NET_BIND_SERVICE para porta < 1024). O Docker concede 14 das ~40 por padrão.

cgroup (control group) — recurso do kernel Linux que limita e contabiliza o consumo de recursos (CPU, memória, I/O, número de processos) de um grupo de processos. A **v2** unifica os controladores e é o padrão atual.

CNI (Container Network Interface) — especificação de plugins de rede, usada por orquestradores.

Compose — ferramenta que define e opera aplicações multi-container a partir de um arquivo YAML (compose.yaml). A **v2** (docker compose, com espaço) substituiu a v1 (docker-compose, com hífen).

Confidential container — container cuja memória é cifrada em uso por uma [TEE] (AMD SEV-SNP, Intel TDX, ARM CCA), protegendo-o até do hipervisor e do operador da nuvem.

containerd — [runtime] de alto nível (projeto graduado da CNCF) que gerencia o ciclo de vida dos containers, o pull de imagens e os snapshots. Usado pelo Docker e diretamente pelo Kubernetes. Desde a Engine 29, seu *image store* é o padrão em instalações novas.

Container — um ou mais processos isolados por [namespaces], limitados por [cgroups] e restringidos por políticas de segurança, executando a partir de uma [imagem].

Copy-on-write (CoW) — estratégia em que um arquivo de uma camada inferior só é copiado para a camada de escrita quando modificado. No [OverlayFS] é por arquivo; em btrfs/zfs, por bloco.

CRI (*Container Runtime Interface*) — contrato entre o kubelet do Kubernetes e o runtime. Implementado por containerd e CRI-O; **não** pelo Docker Engine (origem do episódio do [dockershim]).

crun — [runtime] de baixo nível escrito em C, mais rápido e leve que o [runc]. Padrão do Podman no Fedora.

CVE (*Common Vulnerabilities and Exposures*) — identificador padronizado de vulnerabilidade conhecida. [Scanners] reportam CVEs por versão de pacote.

Digest — hash SHA-256 do manifesto de uma imagem (sha256:...). Imutável e verificável, ao contrário da [tag]. Deploys de produção devem referenciar o digest.

distroless — imagens mínimas do Google, sem shell nem gerenciador de pacotes, contendo só o runtime da linguagem. Menor superfície de ataque; a variante :debug traz shell.

DNS interno — o resolvidor em 127.0.0.11 que o Docker embute em redes definidas pelo usuário, resolvendo nomes de container/serviço para IP. Ausente na bridge padrão.

Dockerfile — arquivo de texto com a receita de construção de uma [imagem].

dockershim — adaptador (removido do Kubernetes em 2022) que permitia ao kubelet falar com o Docker Engine. Sua remoção foi o “Kubernetes está removendo o Docker” de 2020.

dockerd — o daemon do Docker: gerencia imagens, containers, redes, volumes e builds; expõe a API.

eBPF — mecanismo do kernel Linux para executar programas verificados sem módulos. Base de Cilium (rede), Tetragon e Falco (segurança), e de observabilidade sem instrumentação.

Egress — tráfego de saída. O principal custo variável de registries e nuvens; imagens menores reduzem-no.

ENTRYPOINT — instrução do Dockerfile que define o executável fixo do container; o [CMD] vira seu argumento padrão.

EXPOSE — instrução que **documenta** a porta do container. Não publica nada — quem publica é -p ou ports:.

Firecracker — [micro-VM] minimalista da AWS, base do Lambda e do Fargate; boot em ~125 ms.

gVisor — [runtime] que implementa um kernel em espaço de usuário (runsc), interceptando syscalls e reduzindo a superfície de kernel. Base do Google Cloud Run.

HEALTHCHECK — instrução/definição que informa como verificar se o container está saudável. Habilita condition: service_healthy no Compose.

Image store — o subsistema que armazena camadas e conteúdo de imagens. O *containerd image store* é o padrão em instalações novas desde a Engine 29, substituindo o backend histórico.

Imagem — pacote imutável, em [camadas], com o sistema de arquivos e a configuração de uma aplicação. Identificada por [digest].

Init (PID 1) — o primeiro processo de um [namespace] de PID. Recebe tratamento especial do kernel: sinais sem tratador são ignorados, e sua morte encerra o namespace. [tini] é um init mínimo para containers.

Kata Containers — [runtime] que executa cada container numa [micro-VM] com kernel próprio, oferecendo isolamento de VM com interface de container.

Layer (camada) — conjunto de alterações no sistema de arquivos, endereçado por hash de conteúdo. Imagens são pilhas de camadas reaproveitáveis.

Lazy pulling — iniciar um container antes de a [imagem] estar inteiramente baixada, buscando blocos sob demanda (eStargz, SOCI, Nydus). Relevante para imagens grandes, como as de IA.

Liveness — sonda que responde "o processo está de pé?". Falha → **reiniciar**. Distinta de [readiness].

LSM (Linux Security Module) — arcabouço do kernel para controle de acesso obrigatório. Implementações: [AppArmor], [SELinux].

Manifest (manifesto) — documento JSON que lista as camadas e a configuração de uma imagem para uma arquitetura.

Manifest list / índice — manifesto que aponta para vários manifestos, um por arquitetura (base do multi-arch).

Micro-VM — máquina virtual minimalista, de boot rápido e footprint reduzido, usada para dar isolamento de VM a containers ([Firecracker], [Kata]).

Moby — o projeto upstream de código aberto do qual o Docker Engine é montado.

Multi-stage build — Dockerfile com vários FROM, em que artefatos de um estágio são copiados para outro, mantendo o toolchain fora da imagem final.

Namespace — recurso do kernel Linux que dá a um conjunto de processos uma visão própria de um recurso global (PIDs, montagens, rede, hostname, IPC, usuários, cgroups, tempo).

NAT (Network Address Translation) — tradução de endereços que permite aos containers na [bridge] alcançarem a rede externa e receberem portas publicadas.

nftables — sucessor do iptables para firewall no Linux. Backend **experimental** no Docker desde a Engine 29.

OCI (Open Container Initiative) — organização sob a Linux Foundation que mantém as especificações de [imagem], [runtime] e distribuição. A razão pela qual imagens são portáteis entre runtimes.

OOM killer (Out Of Memory) — mecanismo do kernel que mata um processo que excede o limite de memória do [cgroup]. Resulta em exit code 137.

Orquestrador — sistema que agenda e gerencia containers em várias máquinas (Kubernetes, Swarm, Nomad).

OverlayFS — sistema de arquivos de união do kernel usado pelo driver overlay2 para apresentar as camadas empilhadas como um único sistema de arquivos.

PID namespace — [namespace] que isola a tabela de processos; dentro dele, o processo principal é [PID 1].

Podman — engine de containers sem daemon e rootless por padrão, com CLI compatível com a do Docker.

Privileged (--privileged) — modo que concede todas as [capabilities], desliga [sec-comp] e [LSM] e dá acesso a todos os dispositivos. Equivale a root no host.

Provenance (proveniência) — atestado de como, onde e a partir de quê uma imagem foi construída. Formalizado pelo [SLSA].

Readiness — sonda que responde "consigo atender agora?". Falha → **tirar do balanceamento**, sem reiniciar. Distinta de [liveness].

Registry — servidor que armazena e distribui imagens (Docker Hub, GHCR, Quay, ECR). Hoje, também depósito de artefatos ([SBOM], assinaturas).

Rootless — modo em que o daemon e os containers rodam como usuário sem privilégio, via [user namespace]. Reduz o dano de um escape.

runc — [runtime] de baixo nível de referência da [OCI]: cria namespaces e cgroups, aplica segurança, faz execve e sai.

Runtime — programa que executa containers. De **baixo nível**: runc, crun, youki, runsc, kata. De **alto nível**: containerd, CRI-O.

SBOM (Software Bill of Materials) — inventário de todos os componentes e versões de uma imagem. Formatos: [SPDX], [CycloneDX]. Exigido pelo [CRA] da UE.

Scanner — ferramenta que analisa uma imagem em busca de [CVEs], segredos ou má configuração (Trivy, Gype, Docker Scout).

scratch — imagem base vazia (0 byte). Usada para binários estáticos; sem shell, sem libc.

seccomp (secure computing mode) — filtro de chamadas de sistema. O perfil padrão do Docker bloqueia ~40 das ~350 syscalls.

SELinux — [LSM] usado em Fedora/RHEL. Exige os sufixos :z/:Z em [bind mounts].

Shim — processo intermediário (containerd-shim-runc-v2), um por container, que mantém o container vivo e reporta seu término, permitindo reiniciar o daemon sem derrubá-lo.

SLSA (Supply-chain Levels for Software Artifacts) — arcabouço de níveis de garantia de integridade da cadeia de suprimentos.

Swarm — modo de orquestração embutido no Docker Engine. Funcional e simples, mas com desenvolvimento estagnado; perdeu mercado para o Kubernetes.

tag — rótulo humano e **mutável** que aponta para um [digest] (ex.: nginx:alpine). latest é apenas a tag padrão, não "a mais recente".

TEE (Trusted Execution Environment) — ambiente de hardware que cifra memória em uso, base dos [confidential containers].

tini — init mínimo (~10 KB) que, como [PID 1], repassa sinais ao processo real e colhe zumbis. Disponível também via docker run --init.

tmpfs — sistema de arquivos em RAM. Usado para dados efêmeros e segredos, que somem ao parar o container.

Twelve-Factor App — conjunto de práticas para aplicações em nuvem (config por ambiente, log em stdout, processos sem estado) fortemente alinhado ao modelo de containers.

User namespace — [namespace] que mapeia UIDs/GIDs, permitindo que o root dentro do container seja um usuário sem privilégio fora. Base do [rootless].

VEX (*Vulnerability Exploitability eXchange*) — atestado que declara se uma [CVE] é explorável em um contexto, reduzindo o ruído dos [scanners].

veth pair — par de interfaces virtuais ligadas como um cabo; uma ponta no container (eth0), a outra na [bridge] do host.

Volume — armazenamento gerenciado pelo Docker, fora das [camadas], que sobrevive ao container. Forma correta de guardar estado.

WASI / WebAssembly (Wasm) — formato de bytecode portátil e isolado por design, executável em containers via shim [runwasi]. Ocupa o nicho de funções muito curtas e código não confiável.

Whiteout — marcador criado na camada de escrita do [OverlayFS] para “apagar” um arquivo de camada inferior. O arquivo original continua ocupando espaço na imagem.

youki — [runtime] de baixo nível escrito em Rust, com segurança de memória por construção.

29.0.1 Termos de comando frequentes

Termo	Significado
-d / detached	Rodar em segundo plano
-it	Interativo com terminal (-i + -t)
--rm	Remover o container ao terminar
Build context	A pasta enviada ao daemon no docker build (o . final)
.dockerignore	Arquivo que exclui itens do build context
Exit code 137	SIGKILL — quase sempre [OOM]
Exit code 143	SIGTERM — parada normal
docker compose config	Mostra o YAML final resolvido — o melhor depurador de Compose

APÊNDICE A

Código-fonte do projeto modelo

Listagem integral dos arquivos de 07-projeto-modelo/, na íntegra e sem cortes, para leitura em papel. O projeto roda de verdade: veja o capítulo correspondente para as instruções de execução.

A.1 07-projeto-modelo/.dockerignore

```
# .dockerignore – tão importante quanto o Dockerfile.
#
# Tudo que está aqui NÃO é enviado ao daemon como contexto de build. Isso:
# 1. acelera o build (contexto menor viaja mais rápido);
# 2. evita invalidar o cache por causa de arquivo irrelevante;
# 3. impede que segredo entre na imagem por um COPY . . distraído.
#
# Regra de ouro: se você não sabe se um arquivo deve entrar na imagem, ele não deve.

# Segredos – o motivo nº 1 deste arquivo existir
.env
.env.*
!.env.example
*.pem
*.key
id_rsa*
.npmrc
.netrc

# Controle de versão e metadados
.git
.gitignore
.github
.gitattributes

# Dependências e artefatos: são reconstruídos DENTRO da imagem.
# Copiar node_modules do host quebra pacotes com binário nativo quando o host não é Linux.
node_modules
dist
build
coverage
*.tsbuildinfo

# Dados locais – jamais devem ir para dentro da imagem
dados/
backups/
*.sqlite
*.db

# Ruído
*.log
npm-debug.log*
.DS_Store
Thumbs.db
.vscode
.idea
*.swp

# Arquivos de infraestrutura: não são usados em tempo de execução da imagem
Dockerfile*
.dockerignore
```

```
compose*.yaml
compose*.yml
Makefile
README.md
docs/
```

A.2 07-projeto-modelo/.env.example

INI

```
# Copie para .env antes de subir: cp .env.example .env
#
# O arquivo .env NÃO vai para o Git (está no .gitignore) nem para a imagem (está no
# .dockerignore). Isso é intencional: segredo não pertence a nenhum dos dois.
#
# Em produção de verdade, nem .env se usa – o valor vem do gerenciador de segredos da
# plataforma (Docker Secrets, Vault, AWS Secrets Manager, Kubernetes Secret).

# Obrigatória: sem ela, `docker compose up` aborta com mensagem explícita.
NOME_DO_MURAL=Mural do Homelab

# Porta em que o proxy escuta NA SUA MÁQUINA (só no loopback).
PORTA_PUBLICA=8080

# Quantos recados o mural guarda antes de descartar os mais antigos.
MAX_RECADOS=500
```

A.3 07-projeto-modelo/app/package.json

JSON

```
{
  "name": "mural-de-recados",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "Projeto-modelo do curso de Docker: API + página, com persistência em volume.",
  "engines": {
    "node": ">=22"
  },
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "test": "node --test"
  },
  "dependencies": {},
  "devDependencies": {}
}
```

A.4 07-projeto-modelo/app/src/config.js

JavaScript

```
// config.js – configuração vem do ambiente, e o processo FALHA RÁPIDO se algo faltar.
//
// Por que assim: um container que sobe com configuração inválida e só quebra na primeira
// requisição é muito pior que um container que se recusa a subir. O orquestrador sabe lidar
// com "não subiu"; ele não sabe lidar com "subiu errado".

function obrigatoria(nome) {
  const valor = process.env[nome];
  if (valor === undefined || valor === '') {
    throw new Error(`Variável de ambiente obrigatória ausente: ${nome}`);
  }
}
```

```

    return valor;
  }

function inteiro(nome, padrao) {
  const bruto = process.env[nome];
  if (bruto === undefined || bruto === '') return padrao;
  const n = Number.parseInt(bruto, 10);
  if (!Number.isInteger(n) || n <= 0) {
    throw new Error(`Variável ${nome} deve ser um inteiro positivo, recebi: "${bruto}"`);
  }
  return n;
}

export const config = {
  porta: inteiro('PORT', 3000),

  // 0.0.0.0, nunca 127.0.0.1: dentro do container, o loopback não é alcançável pelo -p do host.
  host: process.env.HOST || '0.0.0.0',

  // Caminho do arquivo de dados. Em produção ele aponta para dentro de um VOLUME.
  arquivoDados: process.env.ARQUIVO_DADOS || '/dados/recados.json',

  // Só para demonstrar uma variável obrigatória de verdade: sem ela, o container não sobe.
  nomeDoMural: obrigatoria('NOME_DO_MURAL'),

  maxRecados: inteiro('MAX_RECADOS', 500),
  tamanhoMaxTexto: inteiro('TAMANHO_MAX_TEXTO', 280),

  // Prazo para o encerramento gracioso. Precisa ser MENOR que o timeout do `docker stop` (10s),
  // senão o SIGKILL chega antes de terminarmos por conta própria.
  prazoEncerramentoMs: inteiro('PRAZO_ENCERRAMENTO_MS', 8000),

  ambiente: process.env.NODE_ENV || 'development',
};

```

A.5 07-projeto-modelo/app/src/log.js

JavaScript

```

// log.js – log estruturado em JSON, uma linha por evento, direto no stdout.
//
// Por que JSON em uma linha: o driver de log do Docker captura stdout/stderr como texto.
// Ferramentas de agregação (Loki, Elasticsearch, CloudWatch) fazem parse de JSON por linha
// sem configuração. Log multi-linha (stack trace solto) quebra essa correlação.
//
// Por que stdout e não arquivo: em container, o processo NÃO gerencia seus próprios arquivos
// de log. Ele escreve em stdout e a plataforma decide onde aquilo vai parar. Escrever em
// arquivo dentro do container é o caminho garantido para encher o disco em silêncio.

function emitir(nivel, mensagem, extras = {}) {
  const linha = {
    ts: new Date().toISOString(),
    nivel,
    msg: mensagem,
    ...extras,
  };
  // Erros não serializam bem por padrão: name/message/stack não são propriedades enumeráveis.
  if (extras.erro instanceof Error) {
    linha.erro = {
      nome: extras.erro.name,
      mensagem: extras.erro.message,
      pilha: extras.erro.stack,
    };
  }
  const destino = nivel === 'erro' ? process.stderr : process.stdout;
  destino.write(JSON.stringify(linha) + '\n');
}

export const log = {

```

```

info: (msg, extras) => emitir('info', msg, extras),
aviso: (msg, extras) => emitir('aviso', msg, extras),
erro: (msg, extras) => emitir('erro', msg, extras),
};

```

A.6 07-projeto-modelo/app/src/repositorio.js

JavaScript

```

// repositorio.js – persistência em arquivo JSON, com escrita atômica.
//
// Este é o ponto do projeto que ensina VOLUME. O arquivo vive em /dados, que no compose.yaml
// é um volume nomeado. Sem o volume, tudo aqui morre junto com o container – e essa é
// exatamente a demonstração pedida no laboratório 2 do README.
//
// Por que arquivo JSON e não um banco de verdade: para o projeto rodar com ZERO dependências
// de npm, o que o torna reproduzível sem rede no momento do build. O README explica como
// trocar por Postgres, que é o próximo passo natural.

import { randomUUID } from 'node:crypto';
import { readFile, writeFile, rename, mkdir } from 'node:fs/promises';
import { dirname } from 'node:path';

export class ErroValidacao extends Error {
  constructor(mensagem) {
    super(mensagem);
    this.name = 'ErroValidacao';
  }
}

export class Repositorio {
  #caminho;
  #limite;
  #tamanhoMaxTexto;
  #cache = null;

  // Serializa as escritas: sem isto, duas requisições simultâneas leem o mesmo estado,
  // cada uma acrescenta o seu recado e uma sobrescreve a outra (lost update).
  #fila = Promise.resolve();

  constructor({ caminho, limite = 500, tamanhoMaxTexto = 280 }) {
    this.#caminho = caminho;
    this.#limite = limite;
    this.#tamanhoMaxTexto = tamanhoMaxTexto;
  }

  async iniciar() {
    await mkdir(dirname(this.#caminho), { recursive: true });
    this.#cache = await this.#lerDoDisco();
    return this.#cache.length;
  }

  async #lerDoDisco() {
    try {
      const bruto = await readFile(this.#caminho, 'utf8');
      const dados = JSON.parse(bruto);
      if (!Array.isArray(dados)) throw new Error('conteúdo não é uma lista');
      return dados;
    } catch (e) {
      // Arquivo ainda não existe: primeira execução. Estado inicial vazio é legítimo.
      if (e.code === 'ENOENT') return [];
      // Arquivo corrompido: NÃO apagamos silenciosamente. Propagar é mais honesto que
      // fingir que o mural sempre esteve vazio.
      throw new Error(`arquivo de dados ilegível em ${this.#caminho}: ${e.message}`);
    }
  }

  // Escrita atômica: grava num temporário e renomeia. `rename` no mesmo sistema de arquivos
  // é atômico no Linux – ou o arquivo antigo está lá, ou o novo, nunca metade dos dois.

```



```

// Se o container morrer no meio de um writeFile direto, o JSON fica truncado e o app não
// sobe mais. Esta é a diferença entre um exemplo de tutorial e código que aguenta um kill.
async #gravar(dados) {
  const temporario = `${this.#caminho}.tmp`;
  await writeFile(temporario, JSON.stringify(dados, null, 2), 'utf8');
  await rename(temporario, this.#caminho);
}

#enfileirar(tarefa) {
  const resultado = this.#fila.then(tarefa, tarefa);
  // A fila não pode "quebrar" por causa de uma falha; ela segue para a próxima tarefa.
  this.#fila = resultado.then(
    () => undefined,
    () => undefined,
  );
  return resultado;
}

listar({ limite = 50 } = {}) {
  return this.#cache.slice(-limite).reverse();
}

total() {
  return this.#cache.length;
}

validar({ autor, texto }) {
  if (typeof autor !== 'string' || autor.trim().length === 0) {
    throw new ErroValidacao('campo "autor" é obrigatório');
  }
  if (autor.trim().length > 60) {
    throw new ErroValidacao('campo "autor" excede 60 caracteres');
  }
  if (typeof texto !== 'string' || texto.trim().length === 0) {
    throw new ErroValidacao('campo "texto" é obrigatório');
  }
  if (texto.trim().length > this.#tamanhoMaxTexto) {
    throw new ErroValidacao(`campo "texto" excede ${this.#tamanhoMaxTexto} caracteres`);
  }
  return { autor: autor.trim(), texto: texto.trim() };
}

async adicionar(entrada) {
  const limpo = this.validar(entrada);
  return this.#enfileirar(async () => {
    const recado = {
      id: randomUUID(),
      autor: limpo.autor,
      texto: limpo.texto,
      criadoEm: new Date().toISOString(),
    };
    const proximo = [...this.#cache, recado];
    // Limite de crescimento: sem isto, o volume enche até o disco acabar.
    const podado = proximo.slice(-this.#limite);
    await this.#gravar(podado);
    this.#cache = podado;
    return recado;
  });
}

async remover(id) {
  return this.#enfileirar(async () => {
    const proximo = this.#cache.filter((r) => r.id !== id);
    if (proximo.length === this.#cache.length) return false;
    await this.#gravar(proximo);
    this.#cache = proximo;
    return true;
  });
}

// Usado pelo healthcheck: confirma que o disco ainda aceita escrita. Um healthcheck que só
// responde "ok" sem tocar na dependência crítica não informa nada.

```

```

async verificarSaude() {
  const sonda = `${this.#caminho}.saude`;
  await writeFile(sonda, String(Date.now()), 'utf8');
  return true;
}
}

```

A.7 07-projeto-modelo/app/src/rotas.js

JavaScript

```

// rotas.js – roteamento e tratamento de erro, sem framework.
//
// Sem Express de propósito: mantém o projeto com ZERO dependências de npm, o que faz o build
// da imagem ser reproduzível offline e o Dockerfile ficar legível sem ruído de lockfile.

import { ErroValidacao } from './repositorio.js';
import { log } from './log.js';

const PAGINA = (nome) => `<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>${nome}</title>
  <style>
    :root { color-scheme: light dark; }
    body { font-family: system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1rem; line-height: 1.5; }
    form { display: grid; gap: .5rem; margin-bottom: 2rem; }
    input, textarea, button { font: inherit; padding: .5rem; }
    li { border-left: 3px solid currentColor; padding: .25rem .75rem; margin: .75rem 0; opacity: .9; }
    small { opacity: .6; }
    ul { list-style: none; padding: 0; }
  </style>
</head>
<body>
  <h1>${nome}</h1>
  <form id="f">
    <input name="autor" placeholder="Seu nome" required maxlength="60">
    <textarea name="texto" placeholder="Seu recado" required maxlength="280" rows="3"></textarea>
    <button>Publicar</button>
  </form>
  <ul id="lista"></ul>
  <script>
    const lista = document.getElementById('lista');
    async function carregar() {
      const r = await fetch('/api/recados');
      const { recados } = await r.json();
      lista.innerHTML = recados.map(x =>
        '<li><strong>' + escapar(x.autor) + '</strong><br>' + escapar(x.texto) +
        '<br><small>' + new Date(x.criadoEm).toLocaleString('pt-BR') + '</small></li>').join('');
    }
    function escapar(s) { const d = document.createElement('div'); d.textContent = s; return d.innerHTML; }
    document.getElementById('f').addEventListener('submit', async (ev) => {
      ev.preventDefault();
      const dados = Object.fromEntries(new FormData(ev.target));
      const r = await fetch('/api/recados', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(dados),
      });
      if (!r.ok) { const e = await r.json(); alert(e.erro); return; }
      ev.target.reset();
      carregar();
    });
    carregar();
  </script>
</body>
</html>`;

```

```

function responderJson(res, status, corpo) {
  const texto = JSON.stringify(corpo);
  res.writeHead(status, {
    'Content-Type': 'application/json; charset=utf-8',
    'Content-Length': Buffer.byteLength(texto),
  });
  res.end(texto);
}

async function lerCorpo(req, limiteBytes = 64 * 1024) {
  const pedacos = [];
  let total = 0;
  for await (const pedaco of req) {
    total += pedaco.length;
    // Sem este limite, um cliente malicioso derruba o processo por consumo de memória.
    if (total > limiteBytes) throw new ErroValidacao('corpo da requisição grande demais');
    pedacos.push(pedaco);
  }
  if (total === 0) return {};
  try {
    return JSON.parse(Buffer.concat(pedacos).toString('utf8'));
  } catch {
    throw new ErroValidacao('corpo não é um JSON válido');
  }
}

export function criarManipulador({ repositorio, config }) {
  return async function manipular(req, res) {
    const inicio = process.hrtime.bigint();
    const url = new URL(req.url, `http://${req.headers.host} || 'localhost'`);
    const rota = `${req.method} ${url.pathname}`;

    try {
      // --- Healthcheck: usado pelo HEALTHCHECK do Dockerfile e pelo depends_on do Compose ---
      if (rota === 'GET /saude') {
        await repositorio.verificarSaude();
        return responderJson(res, 200, {
          status: 'ok',
          recados: repositorio.total(),
          ambiente: config.ambiente,
        });
      }

      // --- Liveness x readiness: sondas diferentes respondem perguntas diferentes ---
      // /vivo = "o processo está de pé?" (se falhar, reinicie)
      // /saude = "consigo atender?" (se falhar, tire do balanceamento)
      if (rota === 'GET /vivo') {
        return responderJson(res, 200, { status: 'vivo', pid: process.pid });
      }

      if (rota === 'GET /api/recados') {
        const limite = Math.min(Number(url.searchParams.get('limite')) || 50, 200);
        return responderJson(res, 200, {
          total: repositorio.total(),
          recados: repositorio.listar({ limite }),
        });
      }

      if (rota === 'POST /api/recados') {
        const corpo = await lerCorpo(req);
        const recado = await repositorio.adicionar(corpo);
        log.info('recado publicado', { id: recado.id, autor: recado.autor });
        return responderJson(res, 201, recado);
      }

      if (req.method === 'DELETE' && url.pathname.startsWith('/api/recados/')) {
        const id = decodeURIComponent(url.pathname.slice('/api/recados/'.length));
        const removido = await repositorio.remover(id);
        if (!removido) return responderJson(res, 404, { erro: 'recado não encontrado' });
        return responderJson(res, 204, {});
      }
    }
  };
}

```

```

if (rota === 'GET /') {
  const html = PAGINA(config.nomeDoMural);
  res.writeHead(200, {
    'Content-Type': 'text/html; charset=utf-8',
    'Content-Length': Buffer.byteLength(html),
  });
  return res.end(html);
}

return responderJson(res, 404, { erro: 'rota não encontrada', rota });
} catch (e) {
  if (e instanceof ErroValidacao) {
    return responderJson(res, 400, { erro: e.message });
  }
  // Erro inesperado: registre COM contexto e devolva uma mensagem genérica.
  // Vazar a stack trace na resposta HTTP entrega detalhes internos a quem sondar a API.
  log.erro('falha ao processar requisição', { rota, erro: e });
  return responderJson(res, 500, { erro: 'erro interno' });
} finally {
  const ms = Number(process.hrtime.bigint() - inicio) / 1e6;
  if (url.pathname !== '/saude' && url.pathname !== '/vivo') {
    log.info('requisição', { rota, status: res.statusCode, ms: Number(ms.toFixed(2)) });
  }
}
};
}

```

A.8 07-projeto-modelo/app/src/server.js

JavaScript

```

// server.js – ponto de entrada. PID 1 dentro do container.
//
// As três responsabilidades deste arquivo, na ordem em que importam num container:
// 1. Falhar rápido e alto se a configuração estiver errada.
// 2. Subir o servidor.
// 3. Encerrar com elegância quando chegar SIGTERM – porque `docker stop` manda SIGTERM,
//     espera 10 segundos e então manda SIGKILL. Quem não trata SIGTERM perde requisições
//     em curso a cada deploy.

import http from 'node:http';
import { config } from './config.js';
import { log } from './log.js';
import { Repositorio } from './repositorio.js';
import { criarManipulador } from './rotas.js';

async function principal() {
  const repositorio = new Repositorio({
    caminho: config.arquivoDados,
    limite: config.maxRecados,
    tamanhoMaxTexto: config.tamanhoMaxTexto,
  });

  const carregados = await repositorio.iniciar();
  log.info('repositório pronto', { arquivo: config.arquivoDados, recados: carregados });

  const servidor = http.createServer(criarManipulador({ repositorio, config }));

  // Conexões ociosas seguram o encerramento. Este limite garante que o processo consiga sair.
  servidor.keepAliveTimeout = 5000;
  servidor.headersTimeout = 6000;

  await new Promise((resolve, reject) => {
    servidor.once('error', reject);
    servidor.listen(config.porta, config.host, resolve);
  });

  log.info('servidor ouvindo', {

```

```

host: config.host,
porta: config.porta,
ambiente: config.ambiente,
pid: process.pid,
node: process.version,
});

// ----- Encerramento gracioso -----
let encerrando = false;
async function encerrar(sinal) {
  if (encerrando) return; // um segundo Ctrl+C não deve reentrar aqui
  encerrando = true;
  log.info('sinal recebido, encerrando', { sinal });

  // Rede de segurança: se as conexões não fecharem no prazo, saia mesmo assim.
  // Sem isto, um cliente com keep-alive impediria a saída até o SIGKILL do Docker.
  const prazo = setTimeout(() => {
    log.avelo('prazo de encerramento estourado, saindo à força');
    process.exit(1);
  }, config.prazoEncerramentoMs);
  prazo.unref();

  servidor.close((erro) => {
    if (erro) {
      log.erro('falha ao fechar o servidor', { erro });
      process.exit(1);
    }
    log.info('encerrado com elegância');
    process.exit(0);
  });
  servidor.closeIdleConnections?.();
}

process.on('SIGTERM', () => encerrar('SIGTERM')); // docker stop / orquestrador
process.on('SIGINT', () => encerrar('SIGINT')); // Ctrl+C

// Um erro não tratado deixa o processo em estado desconhecido. Registrar e sair é mais
// correto que seguir servindo: o supervisor (restart policy) sobe uma instância limpa.
process.on('uncaughtException', (erro) => {
  log.erro('exceção não tratada', { erro });
  process.exit(1);
});
process.on('unhandledRejection', (motivo) => {
  log.erro('promessa rejeitada sem tratamento', { erro: motivo });
  process.exit(1);
});
}

principal().catch((erro) => {
  // Configuração inválida cai aqui. Sair com código != 0 faz o Docker/Compose acusar a falha.
  log.erro('falha na inicialização', { erro });
  process.exit(1);
});

```

A.9 07-projeto-modelo/app/test/repositorio.test.js

JavaScript

```

// Testes do repositório, com o test runner embutido do Node (node:test) – sem jest, sem vitest.
// Rode com: npm test      (ou) docker compose --profile testes run --rm testes

import { test, describe, beforeEach, after } from 'node:test';
import assert from 'node:assert/strict';
import { mkdtemp, rm, readFile } from 'node:fs/promises';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

import { Repositorio, ErroValidacao } from '../src/repositorio.js';

```

```
let pastaTemp;
let repo;

async function novoRepo(opcoes = {}) {
  pastaTemp = await mkdtemp(join(tmpdir(), 'mural-'));
  const r = new Repositorio({ caminho: join(pastaTemp, 'recados.json'), ...opcoes });
  await r.iniciar();
  return r;
}

describe('Repositorio', () => {
  beforeEach(async () => {
    repo = await novoRepo();
  });

  after(async () => {
    if (pastaTemp) await rm(pastaTemp, { recursive: true, force: true });
  });

  test('começa vazio quando o arquivo não existe', () => {
    assert.equal(repo.total(), 0);
    assert.deepEqual(repo.listar(), []);
  });

  test('adiciona um recado e devolve id e timestamp', async () => {
    const r = await repo.adicionar({ autor: 'Ana', texto: 'Olá' });
    assert.match(r.id, /^[0-9a-f-]{36}$/);
    assert.equal(r.autor, 'Ana');
    assert.ok(Date.parse(r.criadoEm));
    assert.equal(repo.total(), 1);
  });

  test('remove espaços em branco nas pontas', async () => {
    const r = await repo.adicionar({ autor: ' Ana ', texto: ' Olá ' });
    assert.equal(r.autor, 'Ana');
    assert.equal(r.texto, 'Olá');
  });

  test('rejeita autor vazio', async () => {
    await assert.rejects(() => repo.adicionar({ autor: ' ', texto: 'x' }), ErroValidacao);
  });

  test('rejeita texto acima do limite', async () => {
    const curto = await novoRepo({ tamanhoMaxTexto: 5 });
    await assert.rejects(() => curto.adicionar({ autor: 'A', texto: '123456' }), ErroValidacao);
  });

  test('rejeita corpo sem os campos esperados', async () => {
    await assert.rejects(() => repo.adicionar({}), ErroValidacao);
    await assert.rejects(() => repo.adicionar({ autor: 'A' }), ErroValidacao);
  });

  test('lista do mais novo para o mais antigo', async () => {
    await repo.adicionar({ autor: 'A', texto: 'primeiro' });
    await repo.adicionar({ autor: 'B', texto: 'segundo' });
    const lista = repo.listar();
    assert.equal(lista[0].texto, 'segundo');
    assert.equal(lista[1].texto, 'primeiro');
  });

  test('respeita o limite máximo, descartando os mais antigos', async () => {
    const limitado = await novoRepo({ limite: 3 });
    for (const t of ['1', '2', '3', '4', '5']) {
      await limitado.adicionar({ autor: 'A', texto: t });
    }
    assert.equal(limitado.total(), 3);
    assert.deepEqual(
      limitado.listar().map((r) => r.texto),
      ['5', '4', '3'],
    );
  });
});
```

```

test('remove por id e devolve false para id inexistente', async () => {
  const r = await repo.adicionar({ autor: 'A', texto: 'x' });
  assert.equal(await repo.remover(r.id), true);
  assert.equal(repo.total(), 0);
  assert.equal(await repo.remover('nao-existe'), false);
});

test('persiste no disco e é relido por uma instância nova', async () => {
  await repo.adicionar({ autor: 'Ana', texto: 'persistido' });

  const outro = new Repositorio({ caminho: join(pastaTemp, 'recados.json' )});
  await outro.iniciar();

  assert.equal(outro.total(), 1);
  assert.equal(outro.listar()[0].texto, 'persistido');
});

test('escritas concorrentes não se perdem (a fila serializa)', async () => {
  // Sem a fila em Repositorio#enqueue, este teste falha: as gravações se sobrepõem.
  await Promise.all(
    Array.from({ length: 20 }, (_, i) => repo.adicionar({ autor: 'A', texto: `n${i}` })),
  );
  assert.equal(repo.total(), 20);

  const noDisco = JSON.parse(await readFile(join(pastaTemp, 'recados.json'), 'utf8'));
  assert.equal(noDisco.length, 20);
});

test('arquivo corrompido falha alto em vez de fingir estar vazio', async () => {
  const { writeFile } = await import('node:fs/promises');
  const caminho = join(pastaTemp, 'quebrado.json');
  await writeFile(caminho, '{ isto não é json', 'utf8');

  const quebrado = new Repositorio({ caminho });
  await assert.rejects(() => quebrado.iniciar(), /ilegível/);
});

```

A.10 07-projeto-modelo/app/test/rotas.test.js

JavaScript

```

// Teste de integração HTTP: sobe o servidor de verdade numa porta efêmera e faz requisições.
// Sem mock – é o comportamento observável que interessa.

import { test, describe, before, after } from 'node:test';
import assert from 'node:assert/strict';
import http from 'node:http';
import { mkdtemp, rm } from 'node:fs/promises';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

import { Repositorio } from '../src/repositorio.js';
import { criarManipulador } from '../src/rotas.js';

let servidor;
let base;
let pastaTemp;

async function pedir(caminho, opcoes = {}) {
  const resposta = await fetch(base + caminho, opcoes);
  const texto = await resposta.text();
  return {
    status: resposta.status,
    corpo: texto ? JSON.parse(texto) : null,
    tipo: resposta.headers.get('content-type'),
    texto,
  };
}

```

```
describe('rotas HTTP', () => {
  before(async () => {
    pastaTemp = await mkdtemp(join(tmpdir(), 'mural-http-'));
    const repositario = new Repositorio({ caminho: join(pastaTemp, 'r.json') });
    await repositario.iniciar();

    const config = { nomeDoMural: 'Mural de Teste', ambiente: 'test' };
    servidor = http.createServer(criarManipulador({ repositario, config }));

    await new Promise((r) => servidor.listen(0, '127.0.0.1', r));
    base = `http://127.0.0.1:${servidor.address().port}`;
  });

  after(async () => {
    await new Promise((r) => servidor.close(r));
    await rm(pastaTemp, { recursive: true, force: true });
  });

  test('GET /saude responde 200 com status ok', async () => {
    const r = await pedir('/saude');
    assert.equal(r.status, 200);
    assert.equal(r.corpo.status, 'ok');
  });

  test('GET /vivo responde 200', async () => {
    const r = await pedir('/vivo');
    assert.equal(r.status, 200);
    assert.equal(r.corpo.status, 'vivo');
  });

  test('GET / devolve HTML com o nome configurado', async () => {
    const resposta = await fetch(base + '/');
    const html = await resposta.text();
    assert.equal(resposta.status, 200);
    assert.match(resposta.headers.get('content-type'), /text\/html/);
    assert.match(html, /Mural de Teste/);
  });

  test('POST /api/recados cria e devolve 201', async () => {
    const r = await pedir('/api/recados', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ autor: 'Ana', texto: 'Olá container' }),
    });
    assert.equal(r.status, 201);
    assert.equal(r.corpo.autor, 'Ana');
    assert.ok(r.corpo.id);
  });

  test('GET /api/recados lista o que foi criado', async () => {
    const r = await pedir('/api/recados');
    assert.equal(r.status, 200);
    assert.ok(r.corpo.total >= 1);
    assert.equal(r.corpo.recados[0].texto, 'Olá container');
  });

  test('POST com corpo inválido devolve 400 e mensagem legível', async () => {
    const r = await pedir('/api/recados', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ autor: '', texto: '' }),
    });
    assert.equal(r.status, 400);
    assert.match(r.corpo.erro, /autor/);
  });

  test('POST com JSON malformado devolve 400, não 500', async () => {
    const r = await pedir('/api/recados', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: '{ quebrado',
    });
  });
});
```



```

});
assert.equal(r.status, 400);
});

test('DELETE de id inexistente devolve 404', async () => {
  const r = await pedir('/api/recados/nao-existe', { method: 'DELETE' });
  assert.equal(r.status, 404);
});

test('rota desconhecida devolve 404 em JSON', async () => {
  const r = await pedir('/nada/aqui');
  assert.equal(r.status, 404);
  assert.match(r.tipo, /application\/json/);
});

test('a resposta de erro não vaza stack trace', async () => {
  const r = await pedir('/api/recados', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ autor: 'A' }),
  });
  assert.equal(r.status, 400);
  assert.ok(!r.texto.includes('at '), 'a resposta não deve conter stack trace');
});
});

```

A.11 07-projeto-modelo/compose.yaml

YAML

```

# compose.yaml – a stack completa do projeto-modelo.
#
# Não há campo `version:` – ele é obsoleto na Compose Specification atual e só gera aviso.
#
# Perfis (`profiles`) permitem que um único arquivo sirva a três usos:
#   docker compose up -d           → padrão: proxy + api (produção local)
#   docker compose --profile dev up → api com recarga automática, sem proxy
#   docker compose --profile testes run --rm testes → roda a suíte e sai

name: mural

services:

  # -----
  # proxy – única porta exposta ao mundo. A api NÃO publica porta nenhuma.
  # -----
  proxy:
    image: nginx:1.27-alpine
    ports:
      # 127.0.0.1 no lado esquerdo: escuta SÓ no loopback. Sem isso, o serviço fica
      # visível para toda a rede local – e as regras de iptables que o Docker escreve
      # passam por cima do ufw, então o firewall do host não te salva.
      - "127.0.0.1:${PORTA_PUBLICA:-8080}:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      api:
        condition: service_healthy # só sobe quando a api estiver REALMENTE pronta
    networks: [borda, interna]
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://127.0.0.1/saude"]
      interval: 15s
      timeout: 3s
      retries: 3

  # -----
  # api – a aplicação
  # -----

```

```

api:
  build:
    context: .
    target: producao
  image: mural-de-recados:1.0
  environment:
    # `?:mensagem` faz o Compose ABORTAR se a variável não estiver definida, em vez de
    # subir com valor vazío. Falhar aqui é barato; falhar em produção, não.
    NOME_DO_MURAL: ${NOME_DO_MURAL:?defina NOME_DO_MURAL no arquivo .env}
    MAX_RECADOS: ${MAX_RECADOS:-500}
    NODE_ENV: production
  volumes:
    - dados-mural:/dados          # volume NOMEADO: os recados sobrevivem ao container
  networks: [interna]            # não está em `borda`: inalcançável de fora do proxy
  restart: unless-stopped
  stop_grace_period: 15s         # tempo para o SIGTERM ser tratado antes do SIGKILL
  deploy:
    resources:
      limits:
        memory: 256M             # sem limite, um vazamento derruba o host inteiro
        cpus: "0.5"
      reservations:
        memory: 64M
  security_opt:
    - no-new-privileges:true     # impede escalada por binário setuid dentro do container
  read_only: true                # sistema de arquivos raiz somente leitura...
  tmpfs:
    - /tmp:size=16m              # ...com um /tmp em RAM, porque o Node precisa escrever ali
  cap_drop: [ALL]                # nenhuma capability: o app só faz bind numa porta > 1024
  logging:
    driver: json-file
    options:
      max-size: "10m"            # sem isso, o log cresce até o disco acabar
      max-file: "3"

# -----
# dev – mesma aplicação, com recarga automática e código vindo do host
# -----
dev:
  profiles: [dev]
  build:
    context: .
    target: dev
  environment:
    NOME_DO_MURAL: ${NOME_DO_MURAL:-Mural (dev)}
    NODE_ENV: development
  ports:
    - "127.0.0.1:3000:3000"
  volumes:
    - ./app/src:/app/src:ro      # edite no host, veja no container. :ro porque o app não escreve aqui
    - ./app/test:/app/test:ro
    - dados-dev:/dados
  networks: [interna]

# -----
# testes – container efêmero que roda a suíte
# -----
testes:
  profiles: [testes]
  build:
    context: .
    target: testes
  command: ["node", "--test"]
  volumes:
    - ./app/src:/app/src:ro
    - ./app/test:/app/test:ro
  networks: [interna]

volumes:
  dados-mural:
  dados-dev:

```

```
networks:
  borda:
  interna:
    internal: true    # esta rede não tem rota de saída para a internet
```

A.12 07-projeto-modelo/Dockerfile

Dockerfile

```
# syntax=docker/dockerfile:1
#
# Dockerfile do projeto-modelo – multi-stage com três estágios:
# base        camada comum, para não repetir configuração
# dev         com --watch, usado pelo perfil de desenvolvimento
# testes      roda a suíte e falha o build se algum teste quebrar
# producao    o que vai para o servidor: mínimo, sem root, com healthcheck
#
# Construir só um estágio: docker build --target dev -t mural:dev .

# -----
# base – tudo que os outros estágios compartilham
# -----
FROM node:22-alpine AS base

# tini é um init de 10 KB: vira PID 1, repassa sinais ao processo real e colhe zumbis.
# Sem ele (ou sem '--init'), um processo filho órfão nunca é colhido e a tabela de processos
# do container acumula zumbis até estourar o pids-limit.
RUN apk add --no-cache tini

WORKDIR /app

# O diretório de dados é criado NA IMAGEM, já com o dono certo. Isso importa: quando um volume
# nomeado vazio é montado sobre um caminho que existe na imagem, o Docker copia o conteúdo E as
# permissões daquele caminho para o volume. É assim que um container sem root consegue escrever
# num volume – sem isso, o volume nasceria pertencendo ao root e o app tomaria EACCES.
RUN mkdir -p /dados && chown -R node:node /dados /app

ENV NODE_ENV=production \
    ARQUIVO_DADOS=/dados/recados.json \
    PORT=3000 \
    HOST=0.0.0.0

# -----
# dev – recarga automática, código vindo por bind mount
# -----
FROM base AS dev
ENV NODE_ENV=development
COPY --chown=node:node app/package.json ./package.json
COPY --chown=node:node app/src ./src
COPY --chown=node:node app/test ./test
USER node
EXPOSE 3000
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "--watch", "src/server.js"]

# -----
# testes – estágio dedicado. `docker build --target testes .` falha se um teste falhar,
# o que transforma o próprio build num portão de qualidade.
# -----
FROM base AS testes
ENV NODE_ENV=test
COPY --chown=node:node app/package.json ./package.json
COPY --chown=node:node app/src ./src
COPY --chown=node:node app/test ./test
USER node
# `node --test` sem argumento descobre os arquivos *.test.js sozinho – funciona igual no
# Node 22 e no 24. Passar um diretório mudou de comportamento entre versões; não faça isso.
RUN node --test
```

```
# -----
# producao – imagem final
# -----
FROM base AS producao

# Metadados no padrão OCI: aparecem no `docker inspect` e em ferramentas de inventário.
LABEL org.opencontainers.image.title="mural-de-recados" \
      org.opencontainers.image.description="Projeto-modelo do curso de Docker" \
      org.opencontainers.image.source="https://github.com/exemplo/mural" \
      org.opencontainers.image.licenses="MIT"

# Somente o necessário para EXECUTAR. Testes e ferramental ficam de fora – menos tamanho
# e menos superfície de ataque.
COPY --chown=node:node app/package.json ./package.json
COPY --chown=node:node app/src ./src

# Trocar de usuário DEPOIS dos COPY: as camadas de cópia precisam de permissão de escrita.
USER node

EXPOSE 3000

# O healthcheck bate na rota que realmente exercita o disco. Um healthcheck que só devolve 200
# sem tocar na dependência crítica declara "saudável" um container incapaz de trabalhar.
# start-period dá carência ao boot: falhas ali não contam como unhealthy.
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/saude || exit 1

# tini como PID 1; o node é o filho que recebe os sinais corretamente.
ENTRYPOINT ["/sbin/tini", "--"]
# Forma exec (lista JSON), nunca a forma shell – é o que garante que o SIGTERM chegue ao node.
CMD ["node", "src/server.js"]
```

A.13 07-projeto-modelo/Makefile

Makefile

```
# Makefile – atalhos para os comandos do projeto.
# Rode `make` sem argumento para ver a lista.

.DEFAULT_GOAL := ajuda
.PHONY: ajuda subir derrubar logs testes dev shell saude backup restaurar limpar reconstruir tamanho

ajuda: ## Mostra esta ajuda
    @grep -E '^[a-z-]+:.*?## .*$$' $(MAKEFILE_LIST) | awk 'BEGIN {FS = ":.*?## ";} {printf " \033[36m%-14s\033[0m %s\n", $$1, \
    ↪  $$2}'

subir: ## Sobe a stack (proxy + api) em segundo plano e espera ficar saudável
    @test -f .env || (echo "crie o .env primeiro: cp .env.example .env" && exit 1)
    docker compose up -d --build --wait
    @echo "pronto: http://localhost:${PORTA_PUBLICA:-8080}"

derrubar: ## Para e remove containers e redes (os volumes SÃO PRESERVADOS)
    docker compose down

logs: ## Acompanha os logs de todos os serviços
    docker compose logs -f

testes: ## Roda a suíte de testes num container efêmero
    docker compose --profile testes run --rm testes

dev: ## Sobe o modo de desenvolvimento com recarga automática (porta 3000)
    docker compose --profile dev up --build

shell: ## Abre um shell dentro do container da api
    docker compose exec api sh

saude: ## Mostra o estado do healthcheck da api
    @docker inspect --format '{{json .State.Health}}' $(docker compose ps -q api) | \
    python3 -m json.tool 2>/dev/null || \
```

```

    docker inspect --format '{{.State.Health.Status}}' $(docker compose ps -q api)

backup: ## Salva o volume de dados em ./backups/
    @mkdir -p backups
    docker run --rm -v mural_dados-mural:/dados:ro -v "$PWD/backups:/backup" alpine \
        tar czf /backup/dados-$(date +%F_%H%M%S).tgz -C /dados .
    @ls -lh backups | tail -1

restaurar: ## Restaura o backup mais recente de ./backups/ (ARQUIVO=... para escolher outro)
    @ARQ=${ARQUIVO:-$(ls -lt backups/*.tgz | head -1)}; \
    echo "restaurando $$ARQ"; \
    docker compose stop api; \
    docker run --rm -v mural_dados-mural:/dados -v "$PWD/backups:/backup" alpine \
        sh -c "rm -rf /dados/* && tar xzf /backup/$(basename $$ARQ) -C /dados"; \
    docker compose start api

tamanho: ## Mostra o tamanho da imagem e as camadas que mais pesam
    @docker images mural-de-recados:1.0 --format "imagem: {{.Size}}"
    @docker history mural-de-recados:1.0 --format "table {{.Size}}\t{{.CreatedBy}}" | head -12

reconstruir: ## Reconstrói do zero, sem cache
    docker compose build --no-cache

limpar: ## ⚠ Derruba tudo E APAGA OS VOLUMES (perde os recados)
    docker compose down -v

```

A.14 07-projeto-modelo/nginx/default.conf

nginx

```

# nginx/default.conf – proxy reverso na frente da api.
#
# Por que existe um proxy num projeto tão pequeno: é o padrão que você vai reencontrar em
# qualquer implantação real. Ele termina a conexão do cliente, esconde a topologia interna,
# comprime, aplica cabeçalhos de segurança e permite trocar/escalar a api sem que o cliente perceba.

upstream api {
    # 'api' é o NOME DO SERVIÇO no compose.yaml. O DNS embutido do Docker (127.0.0.11)
    # resolve esse nome para o IP do container. Nunca use IP fixo aqui: ele muda a cada recriação.
    # Com múltiplas réplicas, o DNS devolve vários IPs e o nginx faz round-robin.
    server api:3000;
    keepalive 16;
}

server {
    listen 80;
    server_name _;

    # Não anuncie a versão do nginx: informação gratuita para quem está sondando.
    server_tokens off;

    client_max_body_size 1m;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    gzip on;
    gzip_types text/plain text/css application/json application/javascript;
    gzip_min_length 512;

    location / {
        proxy_pass http://api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

```
    proxy_connect_timeout 3s;
    proxy_read_timeout    30s;
}

# Healthcheck do proxy não polui o log de acesso.
location /saude {
    access_log off;
    proxy_pass http://api/saude;
}
}
```